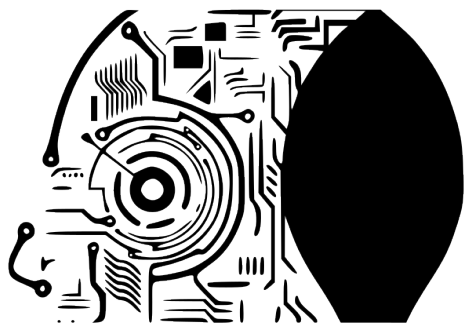




MODELOS DE I.A.

<https://martinezpenya.es/ModelosIA/>

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Modelos de IA (CE Inteligencia Artificial y Big Data)

IES Eduardo Primo Marques (Carlet)

David Martinez Peña

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Índice de contenidos

1. 🚀 Información importante	9
1.1 🚫 Denominación del curso	9
1.2 📄 Contenidos y temporalización	9
1.3 🎯 Resultados de aprendizaje, pesos y calificación	10
1.4 📝 Evaluación	11
1.5 📖 Legislación vigente	11
2. UD00	12
2.1 UD00 — Presentación del módulo y curso rápido de Docker	12
1. Introducción	12
2. Resultados de aprendizaje y objetivos	12
3. Presentación del curso y del módulo (RA7-i)	13
4. Cómo se evalúa el módulo	14
5. Entorno de trabajo del curso	14
6. Curso rápido de Docker: conceptos (RA7-i)	19
7. Curso rápido de Docker: uso básico (RA7-i)	23
8. El Dockerfile: construir imágenes reproducibles (RA7-i)	24
9. Docker Compose: varios servicios a la vez (RA7-i)	26
10. El contenedor de prácticas de IA (RA7-i)	26
11. Copias de seguridad de imágenes y contenedores (RA7-i)	29
12. Puntos clave de la unidad	30
13. Glosario	30
14. FAQ	30
15. Sesiones	31
A. Anexo · chuleta de comandos	31
B. Anexo · catálogo de contenedores para experimentar	33
16. Recursos	34
17. Evaluación	34
18. Recuperación	34
2.2 Diapositivas de la UD00	36
2.3 UD00 — Ejercicios	37
A. Sobre el módulo y la evaluación	37
B. Conceptos de Docker	37
C. Dockerfile	37
D. Docker Compose	37
E. Prácticos	37

F. Instalación, versiones y mantenimiento	38
Soluciones	38
2.4 Talleres	39
UD00 · Taller 1 — Verificación del entorno y primer contenedor	39
UD00 · Taller 2 — Contenedor de prácticas de IA	42
2.5 Notebooks	44
UD00 · Notebook 1 — Bienvenida al entorno Python para IA¶	44
3. UD01	47
3.1 Caracterización de sistemas y utilización de modelos de Inteligencia Artificial	47
Fundamentos de los Sistemas Inteligentes	47
Tipos de Inteligencia Artificial. Escuelas y clasificaciones	60
Utilización de modelos de Inteligencia Artificial	65
Técnicas de la Inteligencia Artificial	67
Campos de Aplicaciones de la Inteligencia Artificial	73
Nuevas Formas de Interacción	85
Mapa conceptual	88
3.2 Diapositivas de la UD01	89
3.3 Actividades	90
Comparativa Java vs Python para Robocode	90
Robocode TankRoyale	94
Robocode TankRoyale en Python	107
Entregable de Robocode Tankroyale	125
3.4 Talleres	128
Taller UD01_T01: Preparar entorno para Java	128
Taller UD01_T02: Crear cuenta en GitHub	135
Taller UD01_T03: Markdown	142
4. UD05	151
4.1 UD05 — Sistemas expertos y controladores inteligentes	151
1. Introducción	151
2. Resultado de aprendizaje y criterios de evaluación	151
3. Objetivos de la unidad	152
4. Arquitectura y dinámica de los sistemas expertos (RA5-a)	152
5. Estructuras elementales de representación (RA5-a/b)	155
6. Representar y simular comportamientos con experta (RA5-b)	156
7. Sistemas híbridos reglas/datos (RA5-b)	158
8. Sistemas de razonamiento impreciso: lógica difusa (RA5-b/d)	159
9. Variación de características y dinámica (RA5-c)	0
10. Estrategias de control de un sistema experto (RA5-d)	0

11. Controladores inteligentes (RA5-e)	0
12. Aplicaciones y tendencias (contenidos del RA5)	0
13. Puntos clave de la unidad	0
14. Glosario	0
15. FAQ	0
16. Sesiones	0
17. Recursos	0
18. Evaluación	0
19. Recuperación	0
4.2 Diapositivas de la UD05	0
4.3 UD05 · Ejercicios	0
A. Del conocimiento a la arquitectura (RA5-a)	0
B. Representación del conocimiento (RA5-a/b)	0
C. Simular comportamientos con experta (RA5-b)	0
D. Sistemas híbridos reglas/datos (RA5-b)	0
E. Lógica difusa (RA5-b/d)	0
F. Variación de características y dinámica (RA5-c)	0
G. Estrategias de control (RA5-d)	0
H. Controladores inteligentes (RA5-e)	0
I. Aplicaciones y tendencias (contenidos RA5)	0
J. Caso práctico	0
4.4 Talleres	0
UD05 · Taller 1 — Simular un sistema experto con experta	0
UD05 · Taller 2 — Lógica difusa: el problema de las propinas	0
UD05 · Taller 3 — Sistema experto como controlador de un proceso	0
4.5 UD05 · Actividades guiadas	0
Referencia: un sistema experto grande, resuelto	0
4.6 UD05 · Actividades entregables	0
Rúbricas	0
4.7 Notebooks	0
Sistemas expertos con Python y Expertal	0
Fuentes de informaciónl	0
0 Piedra, papel o tijera	0
Adivina el animall	0
Pruebas del sistemas expertol	0
¿Quién sobrevivió al Titanic?l	0
Simulador sistema de riego	0
¿Quién jugará el próximo partido?l	0

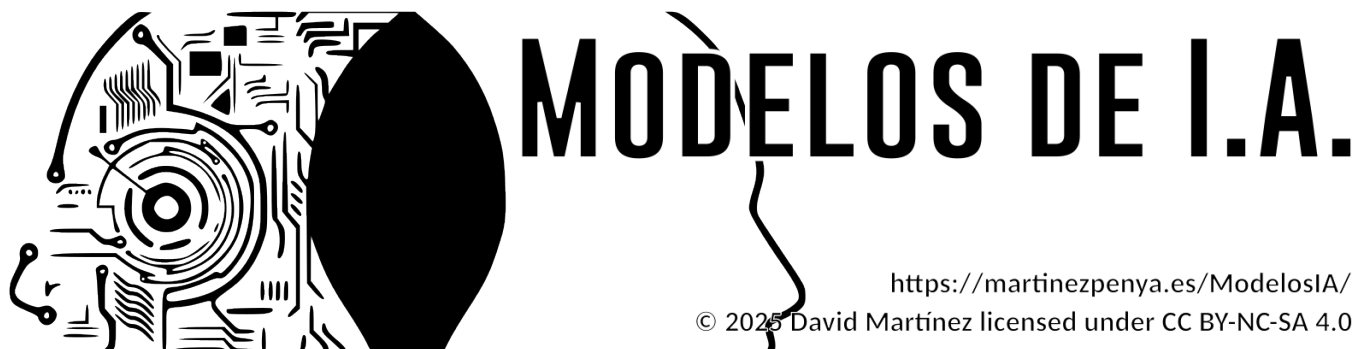
Practica Experta¶	0
Fuentes de información¶	0
Esta es la práctica original en CLIPS¶	0
Esta es la adaptación/traducción a Experta¶	0
Lesiones de rodilla¶	0
Entrega¶	0
Valor de mercado previsto para jugadores emergentes¶	0
Entrega¶	0
Sistema de lógica difusa para encontrar centrocampistas jóvenes con un buen potencial¶	0
Entrega¶	0
Simulador de quemador de gas¶	0
Implementación simple¶	0
Implementación en lógica difusa¶	0
Enfoque alternativo¶	0
Entrega¶	0
5. UD03	0
5.1 Procesamiento del Lenguaje Natural	0
Introducción al PLN	0
Técnicas de procesamiento de lenguaje. Limitaciones	0
Formación del investigador en PLN	0
Elaboración de un sistema de procesamiento de lenguaje orientado a una tarea específica	0
5.2 Diapositivas de la UD03	0
5.3 Actividades guiadas de la UD03	0
5.4 Actividades entregables de la UD03	0
6. UD04	0
6.1 UD04 — Análisis de sistemas robotizados	0
1. Introducción	0
2. Resultado de aprendizaje y criterios de evaluación	0
3. Objetivos de la unidad	0
4. Métodos y aplicaciones de la robótica (RA4-a)	0
5. Qué clase de problema resuelve la robótica	0
6. Modelado y control cinemático (RA4-a)	0
7. Los problemas y sus soluciones (RA4-b)	0
8. Percepción robótica (RA4-b)	0
9. Incertidumbre y aprendizaje (RA4-b)	0
10. Técnicas de programación de robots (RA4-c)	0
11. Humanos y robots (RA4-c, RA4-d)	0
12. Diseño e implementación de sistemas robotizados (RA4-d)	0

13. Practicar sin robot: los dos simuladores de la unidad	0
14. Puntos clave de la unidad	0
15. Glosario	0
16. FAQ	0
17. Sesiones	0
18. Recursos	0
19. Evaluación	0
20. Recuperación	0
6.2 UD04 · Ejercicios	0
A. Métodos y aplicaciones de la robótica (RA4-a)	0
B. Sensores y actuadores (RA4-a)	0
C. Qué problema resuelve la robótica (RA4-a)	0
D. Modelado y control cinemático (RA4-a)	0
E. Problemas y soluciones (RA4-b)	0
F. Espacio de configuración y planificación (RA4-b)	0
G. Percepción (RA4-b)	0
H. Aprendizaje en robótica (RA4-b)	0
I. Técnicas de programación de robots (RA4-c)	0
J. Humanos y robots (RA4-c, RA4-d)	0
K. Diseño e implementación (RA4-d)	0
L. Simulación con Python (RA4-a, RA4-b)	0
6.3 Talleres	0
UD04 · Taller 1 — Cinemática de un manipulador	0
UD04 · Taller 2 — Diseño de un sistema robotizado	0
6.4 Diapositivas de la UD04	0
6.5 UD04 · Actividades guiadas	0
1 · Introducción a OpenCV	0
2 · Vehículos de Braitenberg	0
3 · Ejemplos de robots en AITK	0
6.6 UD04 · Actividades entregables	0
EX1 · Ejercicios de OpenCV	0
EX2 · Navegar con cámara, por reglas	0
EX3 · Navegar con cámara, con lógica difusa	0
EX4 · Generar los datos de entrenamiento	0
EX5 · Controlar el robot con una red neuronal	0
EX6 · Aprendizaje por refuerzo con NEAT	0
Rúbricas	0
6.7 Notebooks	0

Introducción a OpenCV¶	0
2 Vehículos de Braitenberg	0
3 Ejemplos de robots en AITK	0
Ejercicios de OpenCV¶	0
Hacemos un seguidor de línea usando la cámara robot¶	0
Hacemos un seguidor de línea usando la cámara del robot y lógica difusa¶	0
EX4: Generación de datos de entrenamiento¶	0
EX5: Controlar el robot con una red neuronal¶	0
Cargar datos de la Red Neuronal "guardada" previamente¶	0
EX6: Entrenar un robot usando Aprendizaje por Refuerzo (NEAT)¶	0
UD04 · Taller 1 — Cinemática de un manipulador¶	0
UD04 · Taller 2 — Diseño de un sistema robotizado¶	0
7. UD06	0
7.1 UD06 — Aplicación de principios legales y éticos de la IA	0
1. Introducción	0
2. Resultado de aprendizaje y criterios de evaluación	0
3. Objetivos de la unidad	0
4. Riesgos y deontología de la IA (RA6-a)	0
5. Privacidad y protección de datos (RA6-b)	0
6. Cumplimiento estricto de la legalidad (RA6-c)	0
7. Security by design (RA6-d)	0
8. Privacy by design (RA6-e)	0
9. Sesgos de género y algorítmicos (RA6-f)	0
10. Puntos clave de la unidad	0
11. Glosario	0
12. FAQ	0
13. Sesiones	0
14. Recursos	0
15. Evaluación	0
16. Recuperación	0
7.2 Diapositivas de la UD06	0
7.3 UD06 · Ejercicios	0
A. Riesgos y deontología (RA6-a)	0
B. Privacidad y protección de datos (RA6-b)	0
C. Cumplimiento legal (RA6-c)	0
D. Security by design (RA6-d)	0
E. Privacy by design (RA6-e)	0
F. Sesgos de género y algorítmicos (RA6-f)	0

G. Casos prácticos	0
7.4 Talleres	0
UD06 · Taller 1 — Análisis de un caso ético	0
UD06 · Taller 2 — Auditoría de sesgos con Fairlearn	0
7.5 Debates	0
UD06 · Debate 1 — Límites éticos de la Inteligencia Artificial	0
UD06 · Debate 2 — El algoritmo contra el crimen	0
7.6 Notebooks	0
UD06 · Notebook 1 — Detección y corrección de sesgos¶	0
7.7 UD06 · Actividades guiadas	0
Antes de cada debate	0
Para ampliar	0
Actualidad: veinticuatro noticias para el debate	0
7.8 UD06 · Actividades entregables	0
Resumen de entregas	0
Rúbrica de los debates	0
Tertulia de ciencia-ficción	0
8. 🙋 Sobre mí...	0
8.1 🧑 David Martínez Peña	0
8.2 📧 Contacto:	0
9. Fuentes de información	0

1. 🚀 Información importante



1.1 🏠 Denominación del curso

📖 **Curso de especialización de Inteligencia Artificial y Big Data**

🏠 Modelos de Inteligencia Artificial (MIA) · Código **5071** · **90 h** · 4 ECTS

i **Curso 2026-2027**

La docencia empieza el **1 de octubre de 2026** y el módulo termina el **28 de mayo de 2027**; junio se reserva a la **convocatoria ordinaria**. Las clases son de **lunes a jueves**: los viernes no hay clase y se aprovechan para las tareas y los talleres en casa.

Vacaciones: Navidad del 22 de diciembre al 6 de enero · Pascua del 25 de marzo al 5 de abril.

Festivos que caen en día de clase: 12 de octubre, 7 y 8 de diciembre, y **Fallas, 17 y 18 de marzo**. El 9 de octubre (Día de la Comunitat Valenciana) y el 19 de marzo caen en viernes, así que no afectan.

Entre Fallas y Pascua, **marzo es el mes más interrumpido del curso**: tenlo en cuenta para planificar entregas.

1.2 📋 Contenidos y temporalización

Las unidades se imparten en este orden. El identificador **UDxx** va ligado a su resultado de aprendizaje (UD05 es siempre RA5), no al orden en que se imparte.

UD	Título	RA	Horas	Semanas	Fechas
UD00	Presentación y curso rápido de Docker	—	6	1-2	1 oct - 8 oct
UD01	Caracterización de sistemas de IA	RA1	12	3-6	12 oct - 6 nov
UD02	Modelos de IA y resolución de problemas	RA2	12	7-10	9 nov - 3 dic
UD05	Sistemas expertos	RA5	12	11-15	7 dic - 14 ene
UD03	Procesamiento del Lenguaje Natural	RA3	12	16-19	18 ene - 11 feb
UD04		RA4	12	20-23	15 feb - 11 mar

UD	Título	RA	Horas	Semanas	Fechas
	Análisis de sistemas robotizados				
UD06	Principios legales y éticos de la IA	RA6	6	24-28	15 mar - 23 abr
UD07	Proyecto integrador (común al curso)	RA7	15	29-33	26 abr - 28 may
Cierre	Presentaciones RA7 y prueba ordinaria (solo RA no superados)	—	3	34-35	1 jun - 18 jun
Total			90	30 nominales 33 con clase	

**Por qué este orden**

La UD05 (sistemas expertos) se imparte justo después de la UD02 porque se construye sobre las reglas y la lógica difusa que se ven allí. Y la UD06 (ética) ocupa el tramo de Fallas y Pascua, el más fragmentado del curso, porque es la unidad con menos carga de laboratorio.

El **contenido del módulo acaba a finales de abril**. Después viene el **proyecto intermodular (RA7)**, común a todo el curso de especialización: durante unas cinco semanas tu equipo trabaja el proyecto en las horas de todos los módulos.

1.3 Resultados de aprendizaje, pesos y calificación

RA	CE	Descripción	Nota mínima	Peso
RA1	4	Caracteriza sistemas de Inteligencia Artificial relacionándolos con la mejora de la eficiencia operativa de las organizaciones y empresas.	5	13,33 %
RA2	6	Utiliza modelos de sistemas de Inteligencia Artificial implementando sistemas de resolución de problemas.	5	13,33 %
RA3	7	Relaciona el procesamiento de lenguaje natural con sus aplicaciones determinando su potencial e identificando sus limitaciones.	5	13,33 %
RA4	4	Analiza sistemas robotizados, evaluando opciones de diseño e implementación.	5	13,33 %
RA5	5	Aplica sistemas expertos evaluando la influencia de los controladores inteligentes en el comportamiento del sistema.	5	13,33 %
RA6	6	Aplica principios legales y éticos al desarrollo de la Inteligencia Artificial integrándolos como parte del proceso.	5	13,33 %
RA7	9	Desarrolla un proyecto integrador que combine técnicas de IA y análisis	5	20 %

RA	CE	Descripción	Nota mínima	Peso
		de datos masivos para resolver un problema real o simulado, gestionando todo el ciclo de vida del proyecto.		
				100 %

Cada RA se califica **1 a 10, sin decimales** (Orden 8/2025, art. 5.1): **40 %** tareas, talleres y ejercicios + **60 %** prueba escrita. Para superar el módulo hace falta **5 o más en cada RA**.

1.4 Evaluación

-  La evaluación del módulo se realiza sobre los **Resultados de Aprendizaje (RA)** del currículo. Cada RA tiene asociados sus **criterios de evaluación (CE)**, que son los que determinan el grado de adquisición de las competencias del módulo.
-  La nota final del módulo sale de la ponderación de los RA. Cada RA se evalúa de forma independiente, con **calificación de 1 a 10, sin decimales** (Orden 8/2025, art. 5.1).
-  Cada RA tiene **una prueba en Moodle** al cerrar su unidad, con preguntas de test de cuatro opciones y preguntas de desarrollo. En las de test, una respuesta incorrecta **descuenta un tercio** del valor de la pregunta, así que responder al azar penaliza. Las preguntas se extraen aleatoriamente del banco: **cada alumno recibe un examen distinto**.
-  Hay que obtener al menos un **5 en cada RA** para aprobar el módulo.
-  Si un RA queda por debajo de 5, se recupera con las actividades y pruebas diseñadas para **ese** resultado de aprendizaje. La convocatoria ordinaria de junio se dirige **solo a los RA no superados**: no hay prueba global del módulo.
-  La **UD00** (presentación y Docker) **no se califica**: el entorno de trabajo es un requisito. Sus tres entregables —los dos talleres y el notebook— se marcan como **hecho / no hecho**, sin nota pero **obligatorios**: son la prueba de que tienes el entorno operativo, y sin ellos no se pueden hacer las prácticas de la UD02.

Importante

-  Aprobar las evaluaciones parciales **no garantiza** aprobar el módulo.
-  Puedes aprobar las dos evaluaciones con buena nota, tener **un solo RA suspendido** y suspender el módulo.

1.5 Legislación vigente

-  [RD 279/2021, de 20 de abril](#) — establece el curso de especialización y fija los **RA y CE** del módulo.
-  [RD 497/2024, de 21 de mayo](#) — modifica determinados reales decretos de cursos de especialización.
-  [Decreto 95/2026, de 9 de junio \(DOGV\)](#) — currículo en la Comunitat Valenciana: fija el módulo en **90 h**.
-  [Horario y organización del curso \(GVA\)](#)

 21 de agosto de 2026

2. UD00

2.1 UD00 — Presentación del módulo y curso rápido de Docker

Unidad 0 · 6 h · semanas 1-2 (1 al 8 de octubre)

Unidad transversal de presentación y arranque del curso. **No tiene prueba escrita:** el entorno de trabajo es **requisito** para el resto del módulo, y el entregable del Taller 1 se marca como **hecho / no hecho**.

1. Introducción

Esta primera unidad tiene un doble objetivo. Por una parte, **situar el curso:** qué es el curso de especialización en Inteligencia Artificial y Big Data, qué se va a aprender en el módulo 5071 *Modelos de Inteligencia Artificial* y cómo se va a evaluar. Por otra, **dejar listo el entorno de trabajo:** sin herramientas fiables y reproducibles, cualquier práctica de IA se convierte en una lucha contra configuraciones que no funcionan. Por eso el núcleo técnico de la unidad es un **curso rápido de Docker**, la tecnología que nos permitirá ejecutar el mismo entorno de Python y Jupyter en cualquier equipo.

La idea que sostiene la unidad

La IA se hace con **entornos reproducibles:** si cada alumno tiene una configuración distinta, los resultados no son comparables. Docker resuelve ese problema empaquetando el entorno completo en un contenedor.

2. Resultados de aprendizaje y objetivos

La UD00 es **transversal** (no desarrolla un RA propio del módulo): sirve de nivelador tecnológico y de presentación. Las competencias que trabaja se asocian a la **ética y el cuidado del entorno de trabajo** (RA7-i) y a los **hábitos de trabajo** del curso.

Objetivo	Descripción
O1	Describir el curso de especialización IA y Big Data y el papel del módulo 5071 en él.
O2	Explicar cómo se evalúa el módulo (criterios, calificación, recuperación).
O3	Identificar el entorno de trabajo del curso: Aules, web, Python, Jupyter y Docker.
O4	Diferenciar contenedor, imagen, volumen y red, y explicar qué ventajas aporta Docker frente a las máquinas virtuales.
O5	Crear, ejecutar, detener y eliminar contenedores con <code>docker run</code> .
O6	Escribir un <code>Dockerfile</code> sencillo y levantarlo con <code>docker build</code> .
O7	Orquestar varios servicios con Docker Compose y conservar datos con volúmenes.
O8	Levantar un contenedor de prácticas con Jupyter y las bibliotecas del curso.

¿Por qué no tiene RA propio?

El currículo del RD 279/2021 desarrolla 6 resultados de aprendizaje para el módulo 5071 (RA1-RA6). La UD00 es una **unidad de presentación y puesta en marcha** que prepara el terreno para esas unidades: sin ella, los RA1-RA6 perderían horas en configurar entornos.

3. Presentación del curso y del módulo (RA7-i)

3.1 El curso de especialización en IA y Big Data

El **curso de especialización en Inteligencia Artificial y Big Data** es un título de **Grado Superior** (familia Informática y Comunicaciones) de **600 horas y 36 ECTS** establecido por el RD 279/2021. Su competencia general es *programar y aplicar sistemas inteligentes que optimizan la gestión de la información y la explotación de datos masivos, garantizando la seguridad y la ética*.

Se organiza en **cinco módulos**:

Código	Módulo	Horas en CV
5071	Modelos de Inteligencia Artificial	90
5072	Sistemas de aprendizaje automático	90
5073	Programación de Inteligencia Artificial	210
5074	Sistemas de Big Data	90
5075	Big Data aplicado	120

El acceso se realiza desde titulaciones de grado superior de la familia de informática y otras relacionadas (ASIR, DAM, DAW, telecomunicaciones, mecatrónica y robótica industrial).



Una frase para recordar

Este curso enseña a **construir sistemas inteligentes** (5071, 5072, 5073) y a **tratar datos a gran escala** (5074, 5075). El módulo 5071 es la base conceptual de los sistemas de IA.

3.2 El módulo 5071 *Modelos de Inteligencia Artificial*

El módulo 5071 se imparte a lo largo de todo el curso (**90 horas, 3 horas semanales**, 4 ECTS) y desarrolla **6 resultados de aprendizaje** (RA1-RA6), más el **RA7 de proyecto integrador transversal**:

RA	Qué aprenderás
RA1	Caracterizar sistemas de IA y relacionarlos con la eficiencia operativa
RA2	Utilizar modelos de IA para implementar sistemas de resolución de problemas
RA3	Relacionar el procesamiento del lenguaje natural (PLN) con sus aplicaciones
RA4	Analizar sistemas robotizados y evaluar su diseño e implementación
RA5	Aplicar sistemas expertos y valorar los controladores inteligentes
RA6	Aplicar principios legales y éticos (privacidad, sesgos, normativa)
RA7	Proyecto integrador transversal de todo el curso (definición → datos → modelos → Big Data → comunicación)

3.3 El proyecto integrador (RA7)

El **RA7** es un **proyecto común a todo el curso de especialización**: se desarrolla en **equipos** durante el bloque final (semanas 25-29) y en él **confluyen todos los módulos** del título. El alumnado define un problema, obtiene datos, aplica modelos de IA, integra soluciones de Big Data, comunica resultados y evalúa críticamente el trabajo realizado.

Es transversal por dos razones: 1. Lo **imparten todos los docentes** del curso, no solo el del 5071. 2. La **nota se consensúa** entre el equipo docente, valorando la contribución de cada miembro.

**Implicación para este módulo**

El contenido del 5071 (UD00-UD06) **finaliza a finales de abril** porque las últimas semanas del curso se dedican por completo al proyecto integrador. Desde el primer día, el proyecto te permitirá aplicar lo aprendido en este módulo.

4. Cómo se evalúa el módulo

La evaluación se rige por la **Orden 8/2025** de la Comunitat Valenciana (modificada por la **Orden 5/2026**). Lo que debes saber desde el primer día:

Los **pesos de cada RA, la nota mínima y el reparto 40/60** están en la [página de información importante](#), que es la referencia única: si algún día cambian, cambian allí. Aquí van las reglas que conviene tener claras desde el primer día y que no dependen de los pesos:

Aspecto	Regla
Calificación por RA	De 1 a 10, sin decimales (art. 5.1)
Evaluación continua	Se pierde al superar el 15 % de inasistencia (asistencia $\geq$ 85 %)
Evaluaciones parciales	Dos parciales informativos (fin del 1.er y 2.º trimestre)
Recuperación	Programa individual por cada RA no superado ; en junio, solo los RA pendientes
Esta unidad (UD00)	No se califica : es requisito. Sus tres entregables (los dos talleres y el notebook) se marcan hecho / no hecho

**Asistencia**

Si superas el **15 % de faltas** perderás la evaluación continua del módulo (art. 7.3 Orden 8/2025). Es un criterio objetivo, no una decisión del profesor.

**Evaluación inicial**

Durante el primer mes realizaremos una **evaluación inicial o diagnóstica** (obligatoria antes de finalizar el segundo mes lectivo). No cuenta para la nota: sirve para conocer tus competencias previas y adaptar el ritmo del curso.

5. Entorno de trabajo del curso

Herramienta	Para qué la usaremos
Aules (Moodle de la GVA)	Entrega de tareas, pruebas, avisos y calificaciones
Web del curso (mkdocs)	Teoría, ejercicios, talleres y notebooks en el navegador
Python + Jupyter	Notebooks de prácticas con las bibliotecas de IA
Docker	Entornos reproducibles y el contenedor de prácticas
Git (opcional)	Control de versiones de tus proyectos

**Sobre Aules**

Aules es la **plataforma oficial de e-learning de la Conselleria de Educación**, basada en **Moodle**. Accederás con tu usuario (NIA) y desde ahí se gestionan las tareas del módulo.

El stack de IA del curso

Las prácticas se basan en **Python** con estas bibliotecas:

numpy, pandas, matplotlib, scikit-learn, scikit-fuzzy, nltk, spaCy, torch y experta.

Este stack se ejecutará en un **contenedor de prácticas de IA** que montaremos en el taller 2 de esta unidad.

5.2 Instalar Docker

Antes de nada, Docker tiene que funcionar en tu máquina. Estos pasos están probados en el aula.

INSTALACIÓN EN UBUNTU

<https://www.digitalocean.com/community/tutorials/how-to-install-and-use-docker-on-ubuntu-20-04-es>

Requisitos previos:

- **apt-transport-https:** permite que el administrador de paquetes transfiera datos a través de https
- **ca-certificates:** permite que el navegador web y el sistema verifiquen los certificados de seguridad
- **curl:** transfiere datos (similar a wget)
- **software-properties-common:** agrega scripts para administrar el software

```
1 sudo apt-get install curl apt-transport-https ca-certificates software-properties-common
```

Agregamos repositorio

```
1 # Primero clave GPG
2 curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -
3
4 sudo add-apt-repository "deb [arch=amd64] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable"
5
6 sudo apt update
7
8 sudo apt install docker-ce
9
10 sudo systemctl status docker
```

Por defecto, el comando docker solo puede ser ejecutado por el usuario root o un usuario del grupo docker, que se crea automáticamente durante el proceso de instalación de Docker.

Para evitar escribir sudo al ejecutar el comando docker, agregue su nombre de usuario al grupo docker:

```
1 sudo usermod -aG docker ${USER}
2
3 # Cerramos y abrimos sesión de nuevo o ejecutamos
4 su - ${USER}
5
6 # Confirmamos los grupos de nuestro usuario
7 id -nG
```

DOCKER DESKTOP

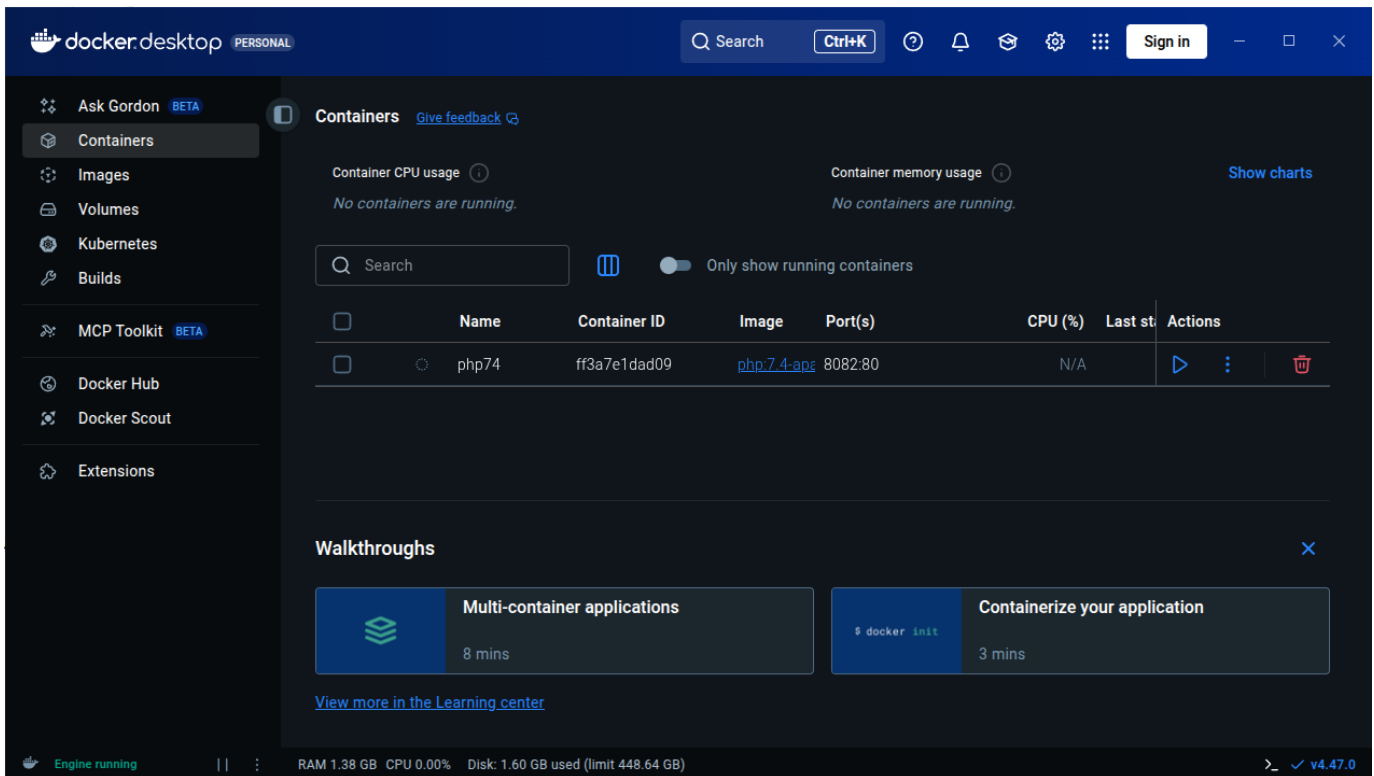
¿Qué es Docker Desktop?



Es la aplicación oficial de Docker que te da una **interfaz gráfica (GUI)** para manejar contenedores, además de la línea de comandos.

¿Para qué sirve?

- **Gestión visual:** Ver contenedores, imágenes y volúmenes de forma gráfica
- **Configuración fácil:** Ajustar recursos (CPU, RAM) con sliders
- **Monitorización:** Ver en tiempo tiempo real qué está pasando



Compatibilidad por Sistema Operativo

Windows

- **Windows 10/11** 64-bit (versiones Home, Pro, Enterprise, Education)
- **Requisitos importantes:**
 - Habilitar **WSL 2** (Windows Subsystem for Linux)
 - Virtualización activada en BIOS/UEFI
- **Windows Home** necesita WSL 2, **Pro/Enterprise** puede usar Hyper-V

macOS

- **macOS 12 Monterey** o superior
- **Tipos de chip:**
 - **Apple Silicon** (M1, M2, M3, etc.)
 - **Intel** con procesador de 2010 o más nuevo
- Necesita **macOS actualizado**

Linux (versión nativa)

- **Distribuciones compatibles:**
 - Ubuntu 20.04 LTS o superior
 - Debian 11 o superior
 - Fedora 36 o superior
 - Arch Linux (y derivados)
- **Requisitos:** kernel 5.10+, systemd, 64-bit

Guía Rápida de Instalación

Windows:

1. Descarga desde docker.com/products/docker-desktop
2. Ejecuta el instalador `.exe`
3. Sigue el asistente (marca "Use WSL 2" si tienes Windows Home)
4. Reinicia cuando termine

5. ¡Listo! Docker se inicia automáticamente

macOS:

1. Descarga desde la web oficial
2. Arrastra Docker.app a la carpeta Applications
3. Ejecuta desde Launchpad
4. Autoriza con contraseña del sistema
5. Espera a que configure todo (puede tardar unos minutos)

Linux (Ubuntu/Debian ejemplo):

```
1 # Paso 1: Descargar el .deb oficial (siempre la última versión)
2 wget https://desktop.docker.com/linux/main/amd64/docker-desktop-amd64.deb
3
4 # Paso 2: Instalar
5 sudo apt install ./docker-desktop-*.deb
6
7 # Paso 3: Iniciar
8 systemctl --user start docker-desktop
```

⚠ Problemas con Ubuntu 24.04 ▾

Si tienes problemas con Ubuntu 24.04, yo me he encontrado dos:

1. Problema de permisos

```
1 ...
2 S'estan processant els activadors per a desktop-file-utils (0.27-2build1)...
3 N: La baixada es duu a terme fora de l'entorn segur com a root ja que el fitxer «/home/ubuntu/docker-desktop-4.27.2-amd64.deb» no és accessible per l'usuari «_apt». - pkgAcquire::Run (13: Permission denied)
```

Lo he resuelto cambiando los permisos del deb:

```
1 # Change ownership of the file to make it accessible
2 sudo chown _apt:root /home/ubuntu/docker-desktop-4.27.2-amd64.deb
3 # Or alternatively, change permissions to make it readable
4 sudo chmod 644 /home/ubuntu/docker-desktop-4.27.2-amd64.deb
```

2. Lanzas Docker-desktop, aparece el icono, pero desaparece y la aplicación no inicia: Parece un problema con un cambio en Ubuntu 24.04 que he resuelto con la información de este [post](#):

```
1 sudo systemctl -w kernel.apparmor_restrict_unprivileged_userns=0
2 systemctl --user restart docker-desktop
```

En algunos casos parece que la solución anterior solo sirve hasta que reinicias, si es así, prueba esto también:

Crea un nuevo fichero:

```
1 sudo nano /etc/apparmor.d/opt.docker-desktop.bin.com.docker.backend
```

Escribe dentro el siguiente contenido:

```
1 abi <abi/4.0>,
2
3 include <tunables/global>
4
5 /opt/docker-desktop/bin/com.docker.backend flags=(default_allow) {
6   userns,
7
8   # Site-specific additions and overrides. See local/README for details.
9   include if exists <local/opt.docker-desktop.bin.com.docker.backend>
10 }
```

Reinicia el servicio `apparmor.service`:

```
1 sudo systemctl restart apparmor.service
```

Ventajas docker desktop (GUI) vs Línea de Comandos (CLI)

✅ Ventajas de Docker Desktop:

- **Más fácil para empezar** - Ideal para principiantes
- **Todo integrado** - No necesitas instalar nada más
- **Debugging visual** - Ves los logs y estados de un vistazo
- **Gestión de recursos** - Controlas CPU/RAM fácilmente

❌ Desventajas:

- **Más pesado** - Consume más recursos de tu PC
- **Menos flexible** - Algunas opciones avanzadas solo por comandos
- **Dependes de la GUI** - Si se cierra la app, pierdes la interfaz

🎯 Conclusión:

- **Empezad con Docker Desktop** para aprender sin frustraciones
- **Aprended también los comandos básicos** para ser más versátiles
- Usad **ambos**: la GUI para lo cotidiano y la terminal para lo avanzado

⚠ Comprueba la instalación antes de seguir

`docker run hello-world` debe terminar sin errores. Si te responde `permission denied while trying to connect to the Docker daemon socket`, te falta el paso de añadir tu usuario al grupo `docker` y **cerrar y abrir sesión**.

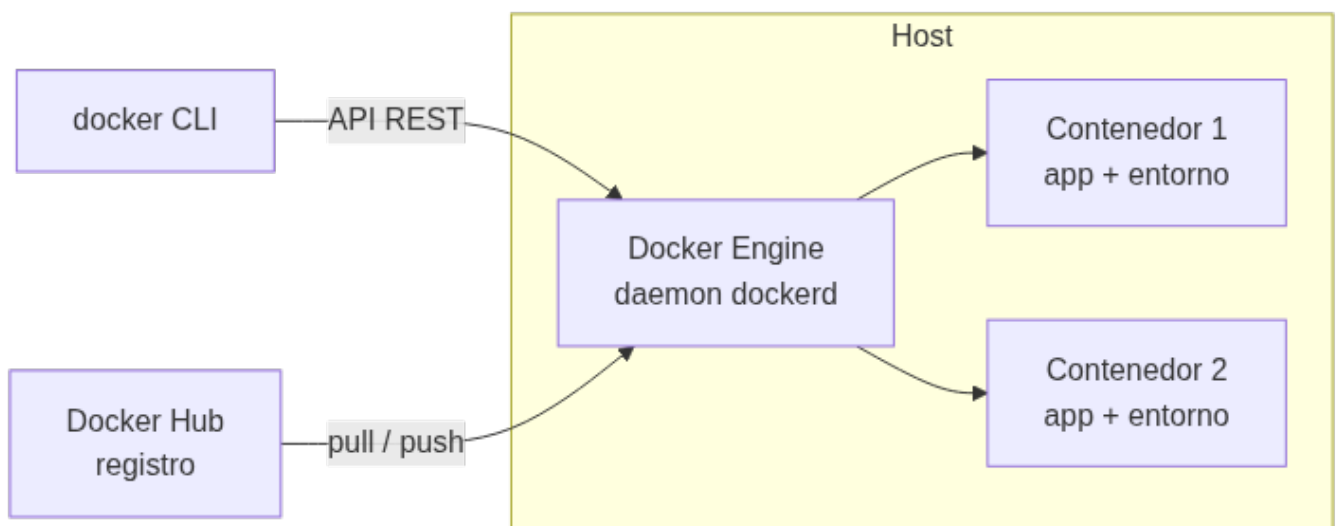
6. Curso rápido de Docker: conceptos (RA7-i)

6.1 El problema: entornos que no se reproducen

¿Cuántas veces funciona "en mi ordenador" y no en el del compañero? La causa es que los entornos difieren: versiones de Python, bibliotecas, sistema operativo, configuraciones. **Docker resuelve este problema** empaquetando la aplicación y todo lo que necesita en un **contenedor**.

✎ Definición

Docker es una plataforma abierta para desarrollar, distribuir y ejecutar aplicaciones. Permite separar la aplicación de la infraestructura mediante contenedores: entornos aislados y ligeros que incluyen todo lo necesario para ejecutar la aplicación.



6.2 Contenedores frente a máquinas virtuales

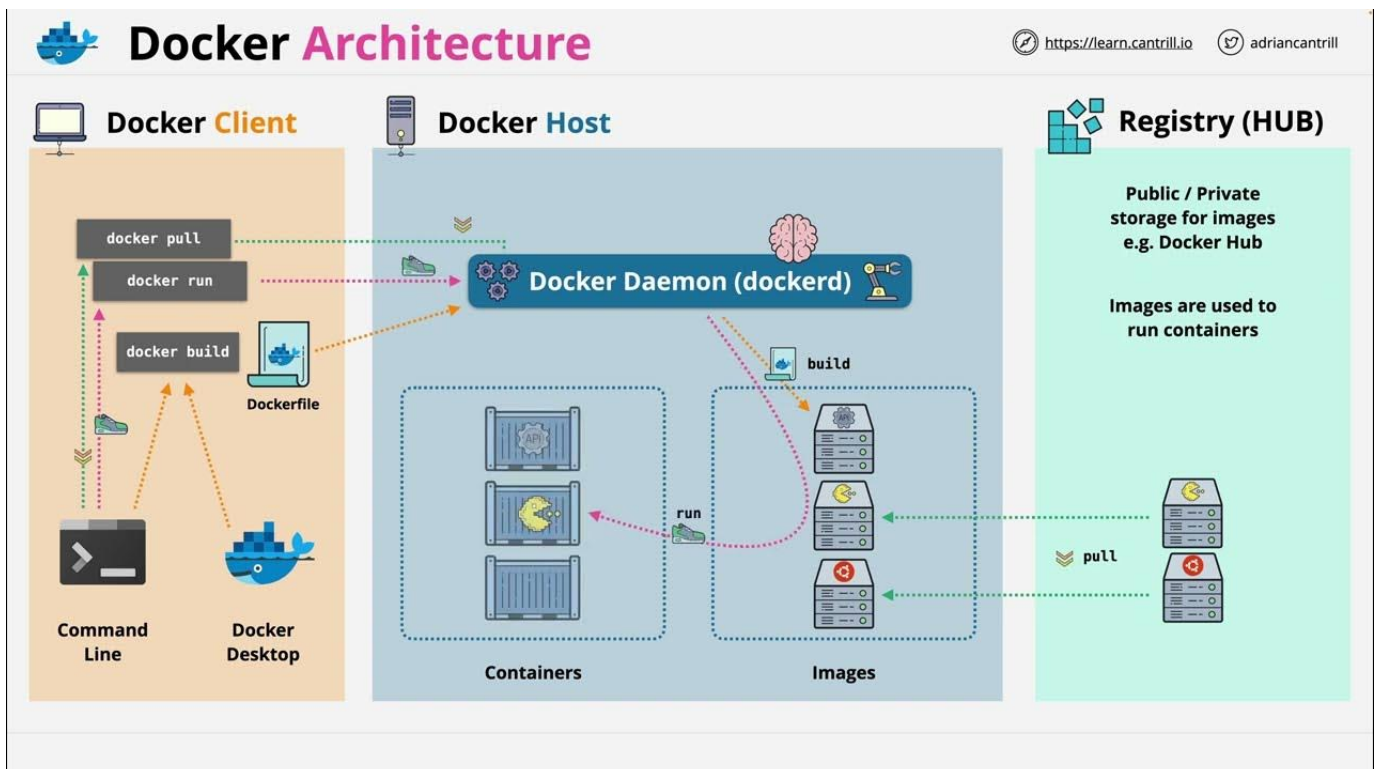
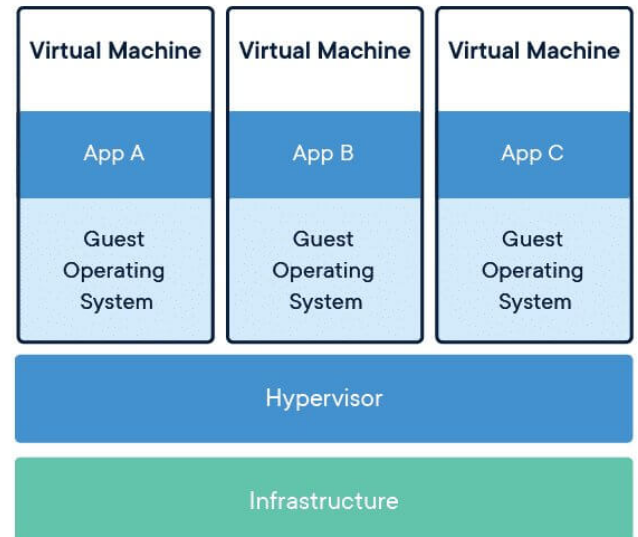
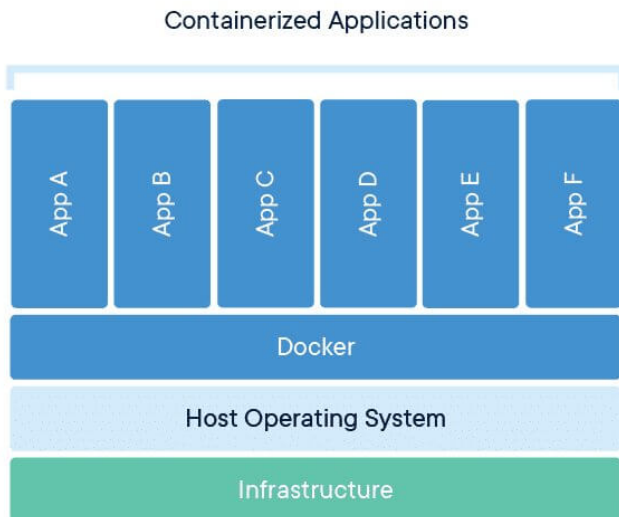
Característica	Máquina virtual	Contenedor
Aislamiento	Por hardware (hipervisor)	Por procesos (namespaces del kernel)
Sistema operativo	SO completo con su propio kernel	Comparte el kernel del host
Arranque	Minutos	Segundos
Tamaño	Gigabytes	Megabytes
Carga en un servidor	Decenas	Cientos/miles
Reproducibilidad	Media	Alta (imagen inmutable + capas)



¿Y entonces las VMs son inútiles?

No. Las VMs siguen siendo necesarias cuando se requiere **aislamiento total de kernel** o se ejecutan sistemas operativos distintos. La práctica habitual es combinarlas: una VM con Docker encima. En este curso trabajaremos con contenedores porque son **ligeros y reproducibles**.

LA ARQUITECTURA DE DOCKER EN IMÁGENES



Imágenes

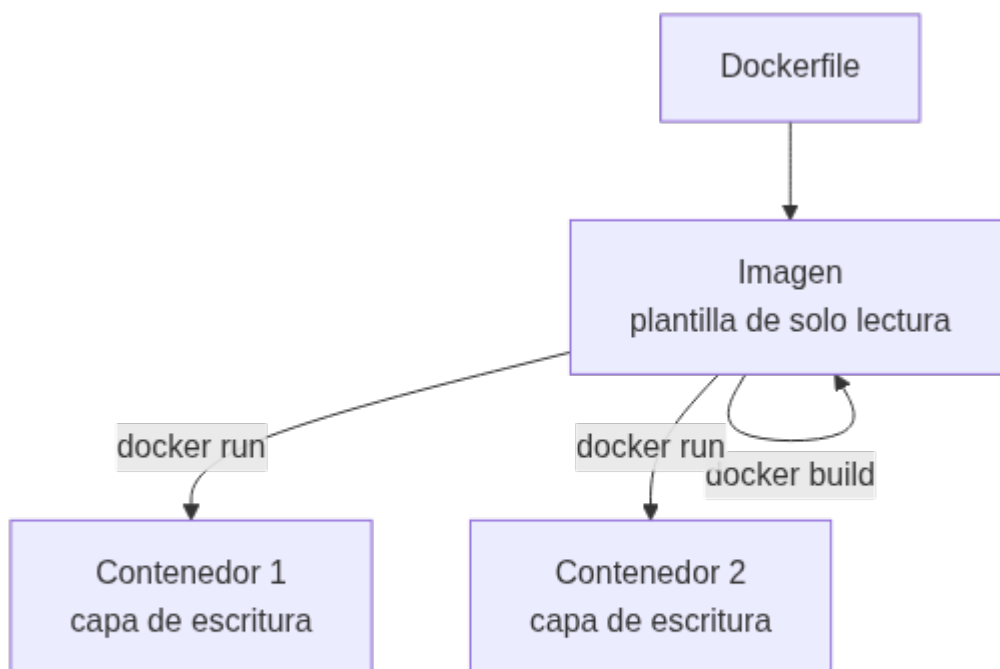
- ▶ Plantilla de solo lectura
- ▶ Sistema de ficheros y parámetros listos para ejecutar
- ▶ Basadas en sistemas operativos Linux

Contenedores

- ▶ Instancia de una imagen
- ▶ Es un directorio dentro del sistema, similar a los "jail root"
- ▶ Pueden ser ejecutados, reiniciados, parados...

6.3 Imágenes y contenedores

- **Imagen:** plantilla **de solo lectura** (un paquete inmutable con el SO base, la aplicación y las dependencias). Se construye por **capas**.
- **Contenedor:** **instancia ejecutable** creada a partir de una imagen. Añade una capa de escritura por encima de la imagen.



La regla de las capas

Cada instrucción del `Dockerfile` crea una **capa**. Al reconstruir una imagen, solo se regeneran las capas que han cambiado: por eso **el orden de las instrucciones importa** para aprovechar la caché.

6.4 El registro y el primer contenedor

Docker Hub es el registro público por defecto: se usa `pull` para **descargar** imágenes y `push` para **subirlas**. Nuestro primer comando será:

```
1 docker run hello-world
```

La primera vez, Docker descarga la imagen `hello-world`, crea un contenedor a partir de ella, ejecuta un mensaje de confirmación y lo detiene. Ese es exactamente el ciclo completo de Docker en una sola línea.



Comprueba tu instalación

Antes de seguir, verifica que Docker está instalado y el demonio en marcha:

```
1 docker --version
2 docker run hello-world
```

6.5 Mismo código, dos entornos: el caso de `experta`

El argumento definitivo a favor de los contenedores se ve mejor con una biblioteca que usaremos de verdad en este módulo: `experta`, el motor de reglas de la UD02 y la UD05.

Dato	Valor
Última versión	1.9.4 , del 16/11/2019
Python que declara soportar	3.5 a 3.8
Mantenimiento	abandonado ; fallo abierto como <i>issue</i> #34 desde julio de 2023
Causa	su dependencia <code>frozendict==1.2</code> usa <code>collections.Mapping</code> , que desapareció en Python 3.10

Monta dos contenedores con **el mismo código** y compara:

```
1 # Contenedor A: Python 3.9, experta funciona tal cual
2 docker run --rm python:3.9-slim bash -c \
3   "pip install -q experta && python -c 'from experta import KnowledgeEngine; print(1)'"
4
5 # Contenedor B: Python 3.12, el mismo código revienta
6 docker run --rm python:3.12-slim bash -c \
7   "pip install -q experta && python -c 'from experta import KnowledgeEngine; print(1)'"
```

El contenedor B falla con `AttributeError: module 'collections' has no attribute 'Mapping'`, y se arregla con tres líneas **antes** del `import`, las mismas que verás en los notebooks de la UD02:

```
1 import collections, collections.abc
2 for nombre in ("Mapping", "Iterable", "MutableMapping"):
3     if not hasattr(collections, nombre):
4         setattr(collections, nombre, getattr(collections.abc, nombre))
5
6 from experta import KnowledgeEngine # ahora sí
```



Tres lecciones en un solo ejemplo

- El entorno importa:** mismo código, misma biblioteca, dos resultados. Sin fijar la versión de Python, «en mi máquina funciona» no significa nada.
- Las dependencias se abandonan:** `experta` no publica nada desde 2019. Elegir una biblioteca es apostar también por quien la mantiene.
- Se puede convivir con ello:** un parche documentado y versionado, y sigues adelante. Lo que no vale es descubrirlo el día de la entrega.

7. Curso rápido de Docker: uso básico (RA7-i)

7.1 El ciclo de vida de un contenedor

Comando	Qué hace
<code>docker run</code>	Crea y arranca un contenedor nuevo (descarga la imagen si falta)
<code>docker ps</code>	Lista los contenedores en marcha
<code>docker ps -a</code>	Lista también los detenidos
<code>docker stop</code>	Detiene un contenedor (envía SIGTERM)
<code>docker start</code>	Reactiva un contenedor parado conservando sus cambios
<code>docker rm</code>	Elimina un contenedor (parado)
<code>docker rmi</code>	Elimina una imagen



`docker run` no borra el contenedor

Cuando el proceso principal termina, el contenedor **se detiene pero no se elimina**. Cada `docker run` crea un contenedor nuevo; por eso conviene usar `--rm` en los contenedores desechables y limpiar con `docker rm` los que ya no se usen.

7.2 Flags clave de `docker run`

Flag	Significado	Ejemplo
<code>-it</code>	Interactivo + terminal (para entrar a un shell)	<code>docker run -it ubuntu bash</code>
<code>-d</code>	En segundo plano (detached)	<code>docker run -d -p 8080:80 nginx</code>
<code>-p</code>	Publica un puerto (host:contenedor)	<code>docker run -p 8888:8888 ...</code>
<code>-v</code>	Monta un volumen o bind mount	<code>docker run -v "\$PWD":/app ...</code>
<code>--rm</code>	Elimina el contenedor al terminar	<code>docker run --rm hello-world</code>
<code>--name</code>	Asigna un nombre al contenedor	<code>docker run --name mi-python ...</code>

7.3 Qué ocurre dentro de `docker run`

Cuando ejecutas `docker run`, el demonio hace lo siguiente:

1. Comprueba si la imagen está en la caché local; si no, la **descarga** (`pull`).
2. **Crea el contenedor**: añade una capa de escritura sobre las capas de solo lectura.
3. **Monta el filesystem** (y los volúmenes o binds indicados) y crea la **interfaz de red**.
4. **Arranca el proceso principal** (el comando indicado, o el `CMD` de la imagen).
5. Cuando ese proceso termina, el contenedor **se detiene** (no se elimina).

7.4 Volúmenes y persistencia

Los contenedores son **efímeros**: al eliminarlos se pierden sus datos. Para conservarlos:

- **Volúmenes gestionados** (*named volumes*): los gestiona Docker (en `/var/lib/docker/volumes`). Son la opción recomendada para datos que no necesitas ver desde el host (cachés, datasets, bases de datos).
- **Bind mounts**: montan una **carpeta real del host** dentro del contenedor. Son ideales para el código y los notebooks del curso: editas en tu equipo y el contenedor ve los cambios al instante.

```
1 # bind mount: la carpeta practicas del host se ve en /app dentro del contenedor
2 docker run --rm -v "$PWD/practicas":/app -w /app python:3.12 python app.py
```

**Bind mount vs volumen**

Para **notebooks y código** del curso usa un **bind mount** (`-v ./practicass:/home/jovyan/work`): son ficheros reales de tu equipo. Los **volúmenes nombrados** quedan para datos que no necesitas tocar desde el host.

8. El Dockerfile: construir imágenes reproducibles (RA7-i)

8.1 Instrucciones esenciales

Un `Dockerfile` es un **guion** que describe cómo construir la imagen. Siempre empieza por `FROM` (la imagen base).

Instrucción	Qué hace
<code>FROM</code>	Define la imagen base (obligatoria)
<code>WORKDIR</code>	Establece el directorio de trabajo
<code>COPY</code>	Copia ficheros del contexto de build a la imagen
<code>RUN</code>	Ejecuta un comando durante el build (instalar, compilar)
<code>ENV</code>	Define variables de entorno
<code>EXPOSE</code>	Documenta el puerto (no lo publica; hace falta <code>-p</code>)
<code>CMD</code>	Comando por defecto en runtime (sobrescribible en <code>docker run</code>)
<code>ENTRYPOINT</code>	Ejecutable fijo; se puede combinar con <code>CMD</code> para los argumentos

```

1 FROM python:3.12-slim
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install --no-cache-dir -r requirements.txt
5 COPY app.py .
6 CMD ["python", "app.py"]

```

8.2 La caché de capas

Cada instrucción crea una capa. Docker **reutiliza las capas en caché** mientras las instrucciones y su contenido no cambien. Por eso se copia `requirements.txt` **antes** que el código: así las dependencias se instalan una vez y se cachean.

**Buenas prácticas para un curso rápido**

- Anclar la versión de la imagen base (`FROM python:3.12-slim`, no `FROM python`).
- Usar `.dockerignore` para excluir del build context `.git`, `*.pyc`, `.env`, datasets grandes.
- Unir `apt-get update` && `apt-get install` en un solo `RUN`.
- Un **proceso por contenedor** y contenedores **efímeros**.

8.3 Construir y ejecutar

```

1 docker build -t mi-app .          # construye la imagen desde el Dockerfile del directorio actual
2 docker run -p 8000:8000 mi-app    # ejecuta la imagen publicando el puerto 8000

```

8.4 Gestionar las imágenes que acumulas

Es un instalador donde podemos incorporar nuestra aplicación. Es el punto de inicio para crear contenedores. Hay imágenes oficiales de por ejemplo Ubuntu, Apache, etc, que fueron creadas por sus creadores oficiales.

Página oficial para imágenes: <https://hub.docker.com>

Vamos a utilizar la siguiente imagen para pruebas:

https://hub.docker.com/_/hello-world

Para ejecutar este contenedor “hello-word” escribimos en la terminal:

```
1 docker run hello-world
```

Una vez ejecutado, ya dispondremos de la imagen descargada, podemos ver todas las imágenes que tenemos descargadas con:

```
1 docker images
2
3 REPOSITORY          TAG          IMAGE ID      CREATED        SIZE
4 hello-world         latest      ee301c921b8a  9 months ago  9.14kB
```

Desde la página web de docker hub, podemos ver diferentes versiones de la misma imagen en la pestaña “TAGS”

Overview **Tags**

Sort by: Newest Filter Tags

TAG	DIGEST	OS/ARCH	VULNERABILITIES	COMPRESSED SIZE
latest	dbbd3cf66631	linux/386	None found	2.66 KB
	245fe15fbb8f	windows/amd64	None found	115.14 MB
	088bdbea94d5	windows/amd64	None found	99.78 MB
+8 more...				

docker pull hello-world:latest

TAG	DIGEST	OS/ARCH	VULNERABILITIES	COMPRESSED SIZE
nanoserver-ltsc2022	245fe15fbb8f	windows/amd64	None found	115.14 MB

docker pull hello-world:nanoserv...

Podemos descargar una imagen específica y ejecutarla:

```
1 docker run hello-world:linux
2
3 docker images
4 REPOSITORY          TAG          IMAGE ID      CREATED        SIZE
5 hello-world         latest      ee301c921b8a  9 months ago  9.14kB
6 hello-world         linux       ee301c921b8a  9 months ago  9.14kB
7
8 docker run hello-world:linux
```

Para eliminar una imagen utilizamos el parámetro `rmi` por ejemplo:

```
1 docker pull alpine
2
3 docker images
4 REPOSITORY          TAG          IMAGE ID      CREATED        SIZE
5 alpine              latest      ace17d5d883e  3 weeks ago   7.73MB
6 hello-world         latest      ee301c921b8a  9 months ago  9.14kB
7 hello-world         linux       ee301c921b8a  9 months ago  9.14kB
8
9 # Eliminamos, opción 1
10 docker rmi ace17d5d883e
11
12 # Eliminamos, opción 2
13 docker rmi alpine
14
15 # Eliminamos, opción 3
16 docker rmi ace1
```

También se pueden buscar imágenes desde consola.

```

1  docker search ubuntu
2
3  NAME                                DESCRIPTION                                STARS    OFFICIAL
4  ubuntu                             Ubuntu is a Debian-based Linux operating sys... 16888    [OK]
5  websphere-liberty                   WebSphere Liberty multi-architecture images ... 298      [OK]
6  open-liberty                        Open Liberty multi-architecture images based... 64       [OK]
7  neurodebian                         NeuroDebian provides neuroscience research s... 106      [OK]
8  ubuntu-debootstrap                 DEPRECATED; use "ubuntu" instead            52       [OK]
9  ubuntu-upstart                     DEPRECATED, as is Upstart (find other proces... 115      [OK]
10 ubuntu/nginx                       Nginx, a high-performance reverse proxy & we... 112
11 ubuntu/squid                       Squid is a caching proxy for the Web. Long-t... 83
12 ubuntu/cortex                       Cortex provides storage for Prometheus. Long... 4
13 ubuntu/prometheus                  Prometheus is a systems and service monitori... 56
14 ubuntu/apache2                     Apache, a secure & extensible open-source HT... 70
15  ...

```

9. Docker Compose: varios servicios a la vez (RA7-i)

Compose orquesta **múltiples contenedores** (p. ej. la aplicación + una base de datos) mediante un fichero `compose.yaml`.

```

1  services:
2    jupyter:
3      image: jupyter/scipy-notebook
4      ports:
5        - "8888:8888"
6      volumes:
7        - ./practicas:/home/jovyan/work
8      environment:
9        - JUPYTER_TOKEN=cursoia

```

Comando	Qué hace
<code>docker compose up -d</code>	Crea y arranca los servicios en segundo plano
<code>docker compose down</code>	Detiene y elimina contenedores (y la red por defecto)
<code>docker compose down -v</code>	Además borra los volúmenes nombrados (¡borra datos!)
<code>docker compose config</code>	Muestra la configuración resuelta sin arrancar nada
<code>docker compose logs -f</code>	Sigue los logs de los servicios
<code>docker compose exec</code>	Ejecuta un comando dentro de un contenedor en marcha

Para ordenar el arranque se usa `depends_on` (con `condition: service_healthy` y un `healthcheck` para esperar a que un servicio esté listo).



`down -v` borra datos

`docker compose down -v` elimina también los **volúmenes nombrados**. Úsalo solo cuando quieras empezar de cero; tus notebooks se perderían.

10. El contenedor de prácticas de IA (RA7-i)

El entorno del curso se levantará con la imagen `jupyter/scipy-notebook`, que ya incluye `numpy`, `scipy`, `scikit-learn`, `pandas`, `matplotlib` y otras bibliotecas científicas.



¿Por qué python:3.12-slim y no Alpine?

La variante `-slim` usa **Debian (glibc)**; la `-alpine` usa **musl libc**. Bibliotecas como scikit-learn y torch **solo publican wheels para glibc**: en Alpine habría que compilar desde fuente (torch puede tardar 30-60 min). Con `-slim` todo el stack se instala con wheels precompilados. Los Jupyter Docker Stacks científicos también se construyen sobre Ubuntu (glibc).

El contenedor de prácticas:

- Publica el puerto **8888** (Jupyter).
- Monta tu carpeta de prácticas en `/home/jovyan/work` (bind mount) para que los notebooks sean ficheros reales de tu equipo.
- Fija el token con `JUPYTER_TOKEN` para que la URL sea determinista.

```
1 docker compose up -d
2 # abre http://localhost:8888 y usa el token cursoia
3 docker compose exec jupyter bash # para entrar dentro
4 docker compose down # cuando termines
```

Lo pondrás en marcha paso a paso en el **Taller 2** de esta unidad.

10.1 Construir la imagen con un Dockerfile propio

Otra forma de levantar un contenedor con docker-compose es utilizar un Dockerfile para generar la imagen (en lugar de usar una de DockerHub)

Necesitamos una carpeta con la siguiente estructura:

```
1 .
2 |— docker-compose.yml
3 |— Dockerfile
4 |— notebooks
5   |— rockpaperscissors.ipynb
```

El fichero `Dockerfile` tiene el siguiente contenido:

```
1 #versión de python compatible con experta
2 FROM python:3.6
3 #librería de python para sistemas expertos
4 RUN pip3 install experta
5 #librería para usar notebooks en python
6 RUN pip3 install notebook
7 #creamos la carpeta de trabajo
8 WORKDIR /home/MIA2526
9 #ejecutamos el jupyter notebook en el puerto 8888
10 CMD jupyter notebook --allow-root --ip=0.0.0.0 --port=8888 --no-browser
```

Ahora, para el `docker-compose.yml` tendremos:

```
1 services:
2   experta:
3     container_name: experta #bautizamos a nuestro contenedor
4     build: . #con esta linea le indicamos que busque el Dockerfile, en lugar de buscar una imagen en un repositorio
5     ports:
6       - "8888:8888" #publicamos el puerto para que sea accesible desde fuera
7     volumes:
8       - ./notebooks:/home/MIA2526 #aquí asignamos la carpeta local notebooks con la carpeta de trabajo del contenedor
```

Y por último necesitamos el notebook para hacer pruebas, en nuestro caso es un juego de piedra, papel o tijeras:

`rockpaperscissors.ipynb` este fichero debemos guardarlo en la carpeta `notebooks`

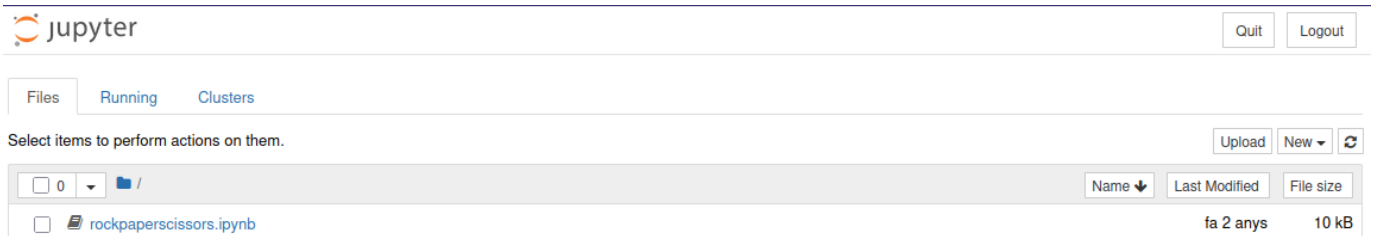
Ahora con toda la estructura lista y desde la raíz de la carpeta (donde estan el `docker-compose.yml` y el `Dockerfile`) ejecutamos:

```

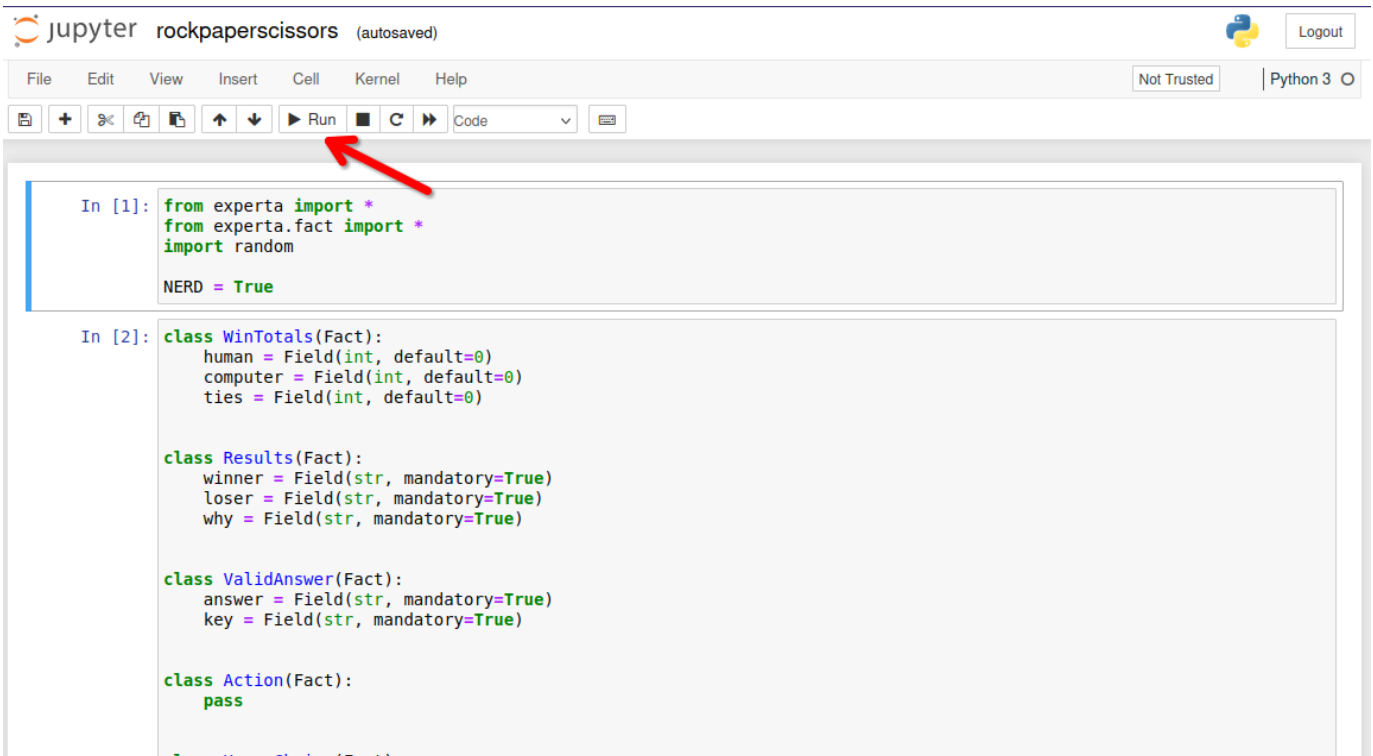
1 $ docker-compose up
2 [+] Running 1/1
3 ✓ Container experta Recreated 0.1s
4 Attaching to experta
5 experta | [I 16:30:08.568 NotebookApp] Writing notebook server cookie secret to /root/.local/share/jupyter/runtime/
6 notebook_cookie_secret
7 experta | [I 16:30:08.784 NotebookApp] Serving notebooks from local directory: /home/MIA2324
8 experta | [I 16:30:08.784 NotebookApp] Jupyter Notebook 6.4.10 is running at:
9 experta | [I 16:30:08.784 NotebookApp] http://e0ec65ac1d84:8888/?token=825b1ba76502787821bc045496e3429bca02ba09720a835
10 b
11 experta | [I 16:30:08.784 NotebookApp] or http://127.0.0.1:8888/?token=825b1ba76502787821bc045496e3429bca02ba09720a83
12 5b
13 experta | [I 16:30:08.784 NotebookApp] Use Control-C to stop this server and shut down all kernels (twice to skip conf
14 irmation).
15 experta | [C 16:30:08.787 NotebookApp]
16 experta |
17 experta | To access the notebook, open this file in a browser:
18 experta | file:///root/.local/share/jupyter/runtime/nbserver-7-open.html
19 experta | Or copy and paste one of these URLs:
20 experta | http://e0ec65ac1d84:8888/?token=825b1ba76502787821bc045496e3429bca02ba09720a835b
21 experta | or http://127.0.0.1:8888/?token=825b1ba76502787821bc045496e3429bca02ba09720a835b

```

Ahora podemos hacer click directamente sobre el enlace a <http://127.0.0.1:8888/?token=bebf660273e8e168c7fec90978ed56fb50db6b08d915cb14> donde veremos nuestro jupyter notebook:



Si hacemos click sobre el notebook `rockpaperscissors.ipynb` podremos ver:



Después de pulsar varias veces (para ir ejecutando todas las celdas) podremos ver como evoluciona nuestro juego:


```
In [*]: rps.reset()
rps.run()

Lets play a game!
You choose rock, paper, or scissors,
and I'll do the same.
Scissors (S), Paper (P), Rock (R)? R
Computer wins! Paper covers rock
Play again?Y
Scissors (S), Paper (P), Rock (R)? S
Tie! Ha-ha!
Play again?Y
Scissors (S), Paper (P), Rock (R)? P
You win! Paper covers rock

Play again? |
```

Para detener el contenedor que hemos lanzado con `docker-compose`, solo hemos de pulsar `^Ctrl` + `C`

Actualizado respecto al material antiguo

El `Dockerfile` de partida fijaba `python:3.6`, fuera de soporte desde 2021. La versión del curso usa `python:3.12-slim` e instala las dependencias desde `requirements-ia.txt`. Los ficheros listos para usar están en `entorno/`, junto a esta unidad. Con Python 3.12, `experta` necesita el parche del apartado 6.5.

11. Copias de seguridad de imágenes y contenedores (RA7-i)

Copias de contenedores

Ya estén encendidos o apagados, podemos realizar respaldos de seguridad de los contenedores. Utilizando la opción `"export"` empaquetará el contenido, generando un fichero con extensión `".tar"` de la siguiente manera:

```
1 docker export -o fichero-resultante.tar nombre-contenedor
```

o

```
1 docker export nombre-contenedor > fichero-resultante.tar
```

Restauración de copias de seguridad de contenedores

Hay que tener en cuenta, antes de nada, que no es posible restaurar el contenedor directamente, de forma automática. En cambio, sí podemos crear una imagen, a partir de un respaldo de un contenedor, mediante el parámetro `"import"` de la siguiente manera:

```
1 docker import fichero-backup.tar nombre-nueva-imagen
```

Copias de imágenes

Aunque no tiene mucho sentido por que se bajan muy rápido, también tenemos la posibilidad de realizar copias de seguridad de imágenes. El proceso se realiza al utilizar el parámetro `'save'`, que empaquetará el contenido y generará un fichero con extensión `"tar"`, así:

```
1 docker save nombre_imagen > imagen.tar
```

o

```
1 docker save -o imagen.tar nombre_imagen
```

Restaurar copias de seguridad de imágenes

Con el parámetro 'load', podemos restaurar copias de seguridad en formato '.tar' y de esta manera recuperar la imagen.

```
1 docker load -i fichero.tar
```

12. Puntos clave de la unidad

- El curso de especialización IA y Big Data dura **600 horas / 36 ECTS** y el módulo 5071 se imparte en **90 horas** en la Comunitat Valenciana.
- El módulo 5071 desarrolla **6 RA** más el **RA7 de proyecto integrador transversal** (común a todo el curso, semanas 25-29, nota consensuada).
- **La normativa** exige alcanzar **todos los RA** del módulo; el centro lo concreta en **cada RA ≥ 5** . La nota de cada RA combina **40 % tareas + 60 % prueba escrita**; se pierde la evaluación continua superando el **15 % de inasistencia**.
- El entorno de trabajo usa **Aules** (Moodle), la web del curso, **Python + Jupyter** y **Docker**.
- Un **contenedor** es una instancia ejecutable de una **imagen**; Docker aporta aislamiento por procesos, ligereza y reproducibilidad frente a las máquinas virtuales.
- Los contenedores son **efímeros**: los datos se conservan con **volúmenes** o **bind mounts**.
- Un **Dockerfile** define la imagen por capas; el **orden de las instrucciones** aprovecha la caché.
- **Compose** orquesta varios servicios con un solo fichero.

13. Glosario

Término	Definición
Contenedor	Instancia ejecutable de una imagen con su propia capa de escritura
Imagen	Plantilla de solo lectura que define el entorno (SO base + app + dependencias)
Capa	Cada instrucción del Dockerfile; las capas se reutilizan en caché
Dockerfile	Fichero que describe cómo construir una imagen
Registro	Repositorio de imágenes (p. ej. Docker Hub)
Daemon (dockerd)	Proceso de Docker que hace el trabajo pesado
CLI (docker)	Cliente de línea de comandos que envía órdenes al daemon
Bind mount	Montaje de una carpeta real del host dentro del contenedor
Volumen	Almacenamiento gestionado por Docker para persistir datos
Compose	Herramienta para orquestar varios contenedores con un fichero YAML
Servicio	Un contenedor definido dentro de un <code>compose.yaml</code>
Healthcheck	Comprobación del estado de un servicio para ordenar el arranque
RA (resultado de aprendizaje)	Capacidad que el alumnado debe demostrar al terminar un módulo
CE (criterio de evaluación)	Evidencia observable que permite comprobar un RA
FE (formación en empresa)	Fase en empresa del alumnado (opcional en este curso, según centro)

14. FAQ

? ¿Docker es una máquina virtual? ✓

No. Las VMs virtualizan **hardware** (cada una con su kernel); los contenedores virtualizan el **sistema operativo** (comparten el kernel del host) y aíslan por procesos. Por eso son más ligeros y arrancan en segundos.

? ¿Pierdo mis datos si elimino un contenedor? ▾

Sí, a menos que los guardes fuera: en un **volumen** o en un **bind mount**. Por eso en las prácticas montamos tu carpeta de trabajo dentro del contenedor.

? ¿Por qué `EXPOSE` no me deja abrir el navegador? ▾

Porque `EXPOSE` solo **documenta** el puerto. Para que sea accesible hay que **publicarlo** con `-p` (o `ports:` en Compose).

? ¿Qué pasa si `docker run` me pide descargar una imagen muy grande? ▾

Solo ocurre la **primera vez**. Después, Docker reutiliza la imagen desde la caché local. Si la imagen es demasiado grande, se puede empezar con variantes más ligeras (`-slim`) y anclar la versión.

? ¿El RA7 cuenta para la nota del módulo 5071? ▾

Sí. El RA7 aporta el **20 %** de la nota final del módulo (RA1-RA6 el 80 %). Además, como es transversal, se evalúa de forma **consensuada** por todo el equipo docente del curso.

? ¿Puedo instalar Docker en cualquier equipo? ▾

Docker funciona en Linux, Windows y macOS. En este curso lo usaremos en el aula y para los talleres; si tu equipo no lo soporta, el entorno gestionado del aula será el plan B.

15. Sesiones

La unidad son **6 h en dos semanas** (1-8 de octubre), a 3 h por semana.

Semana	Fechas	Horas	Contenido	Evidencia
1	1-2 oct	3	Presentación del curso, del módulo y de la evaluación · instalación de Docker · conceptos: imagen, contenedor, registro	Docker funcionando (<code>docker run hello-world</code>)
2	5-8 oct	3	<code>docker run</code> y volúmenes · Dockerfile y Compose · contenedor de prácticas de IA	Taller 1 (hecho / no hecho) y entorno del curso levantado

Si el horario del módulo son 2 h + 1 h

Cada semana se parte en dos sesiones. El **bloque de 2 h** es el único que admite trabajo con contenedores (descargas, construcción de imágenes, resolución de errores); la sesión de **1 h** se dedica a los conceptos, al repaso y a cerrar el taller. La primera semana es corta: el curso empieza el jueves 1 de octubre.

A. Anexo · chuleta de comandos

Referencia rápida para consultar en clase, no para memorizar.

GESTIÓN DE IMAGENES

```

1  docker image
2  docker history
3  docker inspect
4  docker save/load
5  docker rmi

```

GESTIÓN DE CONTENEDORES

```

1  docker attach
2  docker exec
3  docker inspect
4  docker kill
5  docker logs
6  docker pause/unpause
7  docker port
8  docker ps
9  docker rename
10 docker start/stop/restart
11 docker rm
12 docker run
13 docker stats
14 docker top
15 docker update

```

EJEMPLO

```

1  # Ver los contenedores que tenemos
2  docker ps
3  CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
4
5  # Ver las imagenes que tenemos
6  docker images
7  REPOSITORY          TAG          IMAGE ID      CREATED       SIZE
8
9  # Crear un contenedor con una imagen básica de debian
10 # Como no tenemos ninguna imagen de debian, la descarga y la ejecuta
11 docker run debian
12 # Intentamos ver el contenedor en ejecución, no aparece nada porque ya se ha cerrado
13 docker ps
14 # Podemos verlo con
15 docker ps -a
16 CONTAINER ID   IMAGE     COMMAND   CREATED          STATUS          PORTS     NAMES
17 09b14daab800   debian    "bash"    2 seconds ago    Exited (0) 1 second ago             pensive_wozniak
18
19 # Ejecutar un comando en un contenedor
20 docker run debian /bin/echo "Hello World"
21 Hello World
22
23 # Información
24 docker inspect debian

```

Crear contenedor interactivo y con nombre

<https://jonthgs.wordpress.com/2019/09/25/create-a-debian-container-in-docker-for-development/>

Para que docker no se invente un nombre como “pensive_wozniak” (comando anterior) podemos definir el nombre que queremos.

Utilizaremos una de las imágenes de: https://hub.docker.com/_/debian/tags

```

1  # Obtenemos la imagen, en el apartado anterior la hemos ejecutado directamente con "run", esto
2  # la obtiene implícitamente. En este caso la vamos a descargar.
3  $ docker pull debian:13-slim
4
5  # --name
6  # -h hostname que tendrá el contenedor
7  # -e codificación de caracteres
8  # -it modo interactivo
9  # /bin/bash -l la shell que se ejecutará
10 $ docker run --name debian-mini -h equipo1 -e LANG=C.UTF-8 -it debian:13-slim /bin/bash -l
11
12 --- Estamos dentro del contenedor ---
13 # Una vez dentro del contenedor podemos actualizarlo e instalar los paquetes que creamos necesarios
14 apt update && apt upgrade --yes && apt install sudo locales --yes
15 # Configurar timezone
16 dpkg-reconfigure tzdata
17
18 # Vamos a nuestra home y creamos un archivo
19 cd
20 echo "hola" > prueba.txt
21
22 # Salimos del contenedor
23 exit (o control + d)
24
25 --- Ahora volvemos a la consola de nuestro equipo ---
26 $ docker ps
27 CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
28
29 $ docker ps -a
30 CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
31 8c6b29c818ee   debian:13-slim   "/bin/bash -l"   44 seconds ago   Exited (0) 11 seconds ago   debian-mini

```

B. Anexo · catálogo de contenedores para experimentar

MONITORIZACIÓN Y GESTIÓN

- **Netdata** - Monitorización de sistemas en tiempo real con dashboard web completo
- **Glances** - Monitorización del sistema via web que muestra información de CPU, memoria, red y procesos
- **Portainer** - Interfaz web para gestionar y administrar contenedores Docker
- **Uptime Kuma** - Monitor de disponibilidad para verificar el estado de contenedores y servicios

PROXY Y RED

- **Nginx Proxy Manager** - Proxy inverso con interfaz web para gestionar hosts virtuales y certificados SSL/TLS
- **Acestream** - Motor P2P para streaming de video, útil para integrar canales en Jellyfin
- **Libreddit** - Interfaz alternativa para Reddit libre de publicidad y tracking (modo lectura)
- **Invidious** - Interfaz alternativa para YouTube que respeta la privacidad

MULTIMEDIA Y ENTRETENIMIENTO

- **Jellyfin** - Servidor multimedia libre para organizar y streaming de películas, series y música
- **Soulseek** - Cliente para la red P2P Soulseek especializada en compartir música
- **youtube-dl** - Herramienta para descargar videos y audio de YouTube y otros sitios
- **Photoprism** - Servicio de gestión y visualización de fotos personales con funciones de IA

PRODUCTIVIDAD Y ORGANIZACIÓN

- **Nextcloud** - Plataforma de colaboración y almacenamiento en la nube auto-alojado
- **Linkding** - Marcador social para guardar y organizar enlaces web
- **GitBucket** - Plataforma Git auto-alojada similar a GitHub
- **SearXNG** - Metabuscador privado que agrega resultados de múltiples motores de búsqueda

SEGURIDAD Y CONTRASEÑAS

- **Vaultwarden** - Implementación alternativa del gestor de contraseñas Bitwarden, más ligera y eficiente

AUTOMATIZACIÓN DEL HOGAR

- **Home Assistant** - Plataforma de domótica de código abierto para automatizar y controlar dispositivos del hogar
- **Homebridge** - Puente que permite integrar dispositivos no compatibles con el ecosistema Apple HomeKit
- **Camera.UI** - Aplicación para gestionar y visualizar cámaras de seguridad con integración HomeKit

UTILIDADES Y MANTENIMIENTO

- **Homepage** - Dashboard personalizable como página de inicio para acceder a todos los servicios
- **Watchtower** - Servicio que actualiza automáticamente los contenedores Docker cuando hay nuevas versiones disponibles



Uso de este catálogo

Sirve para elegir imagen en la fase 4 del Taller 1. Antes de proponer una, comprueba en Docker Hub que sigue mantenida: hay imágenes populares con años sin actualizar.

16. Recursos

- [Ejercicios de la unidad](#)
- Talleres:
 - [T01 · Verificación del entorno y primer contenedor](#) — entregable, se marca **hecho / no hecho**
 - [T02 · Contenedor de prácticas de IA](#) — entregable, se marca **hecho / no hecho**
- **Notebooks de la unidad** — entregable, se marca **hecho / no hecho**. El nombre lo abre aquí mismo ya renderizado, el badge azul lo descarga y el de Colab lo ejecuta en el navegador sin instalar nada:

Notebook	Descargar	Ejecutar
N01 · Entorno Python para IA	 Descargar .ipynb	 Open in Colab



Referencias de la unidad ▾

Documentación oficial de Docker:

- [What is Docker?](#)
- [What is a container?](#)
- [Dockerfile reference](#)
- [Compose Quickstart](#)
- [Building best practices](#)

Normativa y plataforma:

- [RD 279/2021](#) — currículo del curso de especialización
- [Orden 8/2025 de la Comunitat Valenciana](#) — evaluación
- [Aules GVA](#) — plataforma del centro

17. Evaluación

- **Tarea de la unidad** (40 % de la nota del RA al que se asocia): entregar el informe de los talleres 1 y 2 en Moodle.
- **Evaluación inicial:** diagnóstica, sin nota, antes del segundo mes lectivo.
- **La normativa exige alcanzar todos los RA** del módulo para superarlo (art. 5.1 de la Orden 8/2025: la calificación del módulo está «en función de la consecución de los RA»; y las Instrucciones 26-27, que impiden calificar positivamente un módulo con RA no superados). El centro concreta ese mandato exigiendo **≥ 5 en cada RA**.

18. Recuperación

Si no superas la tarea de esta unidad, se activará un **programa de recuperación individual** (art. 14.4 Orden 8/2025) con actividades y criterios de evaluación específicos para el RA correspondiente.

[Volver al índice](#)

 23 de agosto de 2026

2.2 Diapositivas de la UD00



Diapositivas PDF



Cómo se manejan

Flechas o clic para avanzar · **F** para verlas a pantalla completa · **P** para las notas del ponente.

🕒 22 de agosto de 2026

2.3 UD00 — Ejercicios



Cómo se trabajan

Resuélvelos en tu cuaderno o en un documento Markdown, a tu ritmo. Si te atascas en alguno, pregunta en clase o por Moodle: el profesor te da la solución. (El **notebook** de la unidad es un entregable aparte, con su propia entrega — ver [Recursos](#).)

A. Sobre el módulo y la evaluación

1. Cita los **7 RA** del módulo y asocia cada uno con su número.
2. ¿Cuántas horas tiene el módulo? ¿Cuántas semanas y a qué ritmo semanal?
3. Explica cómo se calcula la nota final del módulo (pesos de RA1-RA6 y RA7).
4. ¿Es el RA7 un caso especial a la hora de superar el módulo? Razona tu respuesta.
5. ¿Por qué el contenido del módulo debe **finalizar a finales de abril**?
6. Indica cuándo se hacen las **evaluaciones parciales** y qué RAs cubren cada una.
7. ¿Qué herramientas usaremos como entorno de trabajo durante el curso? Describe brevemente cada una.

B. Conceptos de Docker

1. Diferencia **imagen** y **contenedor** con un ejemplo.
2. Explica el papel del **daemon**, del **cliente** y de los **registros** en la arquitectura de Docker.
3. Enumera dos ventajas de los contenedores frente a las máquinas virtuales y una situación en la que convenga usar ambas.
4. ¿Qué hace exactamente `docker run` cuando la imagen aún no está descargada? Describe los pasos.
5. ¿Qué ocurre con los datos de un contenedor cuando lo eliminas? ¿Cómo los conservas?
6. ¿Cuál es la diferencia entre `docker stop`, `docker rm` y `docker rmi`?
7. Explica con tus palabras qué ocurre al ejecutar `docker run hello-world` la primera vez.

C. Dockerfile

1. ¿Por qué un Dockerfile debe empezar por `FROM`?
2. Diferencia `RUN`, `CMD` y `ENTRYPOINT`.
3. Escribe un Dockerfile que parta de `python:3.12-slim`, copie `requirements.txt` y `app.py`, instale las dependencias y ejecute la app.
4. ¿Por qué se recomienda copiar `requirements.txt` antes que el resto del código? ¿Qué relación tiene con las capas?
5. ¿Qué es un `.dockerignore` y para qué sirve?
6. Dado el Dockerfile anterior, escribe los comandos para **construir** la imagen y para **ejecutarla** publicando el puerto 8000.

D. Docker Compose

1. ¿Qué ventaja aporta `docker-compose.yml` frente a varios `docker run`?
2. Escribe un `compose.yml` con un servicio `jupyter` que publique el puerto 8888, monte la carpeta `./practicas` y use el token `cursoia`.
3. Explica la diferencia entre `docker compose up -d`, `docker compose down` y `docker compose down -v`.
4. ¿Para qué sirven los comandos `docker compose config`, `docker compose logs -f` y `docker compose exec`?
5. ¿Cómo se ordena el arranque de servicios en Compose? ¿Qué papel juegan los **healthchecks**?
6. ¿Cómo se conservan los datos con Compose? ¿Qué hace la clave `volumes` de nivel superior?

E. Prácticos

1. Levanta un contenedor `python:3.12` interactivo y escribe un programa que imprima `Hola desde Docker`. ¿Qué comando usas para entrar y para salir?

2. Levanta `nginx` en segundo plano en el puerto `8080` y comprueba en tu navegador que responde. ¿Cómo lo detienes y lo eliminas?
3. Crea una carpeta `practicas`, monta esa carpeta en un contenedor `python:3.12` y ejecuta desde dentro un archivo `app.py` que hayas escrito en el host.
4. Escribe el `docker-compose.yml` del taller 2, levanta Jupyter y abre la URL en el navegador.

F. Instalación, versiones y mantenimiento

1. Tras ejecutar `sudo usermod -aG docker $USER`, ¿por qué hay que **cerrar y volver a abrir la sesión**? ¿Qué mensaje de error aparece si no lo haces?
2. El mismo `pip install experta` funciona en `python:3.9-slim` y falla al **importar** en `python:3.12-slim`. Explica la causa, qué hace el parche de tres líneas y por qué el problema no aparece al instalar sino al importar.
3. Diferencia `docker save`/`docker load` de `docker export`/`docker import`. ¿Cuál conserva el historial de capas de la imagen y por qué importa?
4. Tienes 12 GB ocupados en imágenes. Explica qué hace `docker image prune -a`, en qué se diferencia de borrar por identificador y qué riesgo tiene ejecutarlo sin mirar.

Soluciones

Soluciones

Las soluciones no se publican: se corrigen y comentan en clase.

[Volver a la UD00](#) · [Taller 1](#) · [Taller 2](#)

 21 de agosto de 2026

2.4 Talleres

UD00 · Taller 1 — Verificación del entorno y primer contenedor



Unidad no calificable, pero requisito

El Taller 1 se entrega y se marca como **hecho / no hecho**: no lleva nota, pero se tiene en cuenta y **es requisito para las prácticas de la UD02** (sin entorno funcionando no se puede hacer Robocode). Los viernes no hay clase: son el momento de recopilar capturas y redactar la memoria.

Objetivo: dejar Docker funcionando en **tu** máquina y demostrarlo con evidencias.

Entrega: una memoria en PDF con las capturas y las explicaciones de las seis fases. Se marca como **hecho / no hecho**.

FASE 1 — INSTALA DOCKER Y DEMUÉSTRALO

1. Instala Docker en tu equipo siguiendo el apartado 5.2 de la teoría (Ubuntu o Docker Desktop, según tu sistema).
2. Ejecuta y captura la salida de:

```
1 docker --version
2 docker run hello-world
```



El error más común del primer día

Si aparece `permission denied while trying to connect to the Docker daemon socket`, te falta añadir tu usuario al grupo `docker` y **cerrar y volver a abrir la sesión**. Explica en la memoria por qué hace falta reiniciar la sesión.

FASE 2 — PRIMER CONTENEDOR INTERACTIVO

```
1 docker run -it --name mi-python python:3.12 bash
```

Dentro del contenedor:

```
1 python --version
2 python -c "print('Hola desde Docker')"
3 exit
```

Comprueba que el contenedor **sigue existiendo** aunque esté parado, y vuelve a entrar:

```
1 docker ps -a
2 docker start mi-python
3 docker attach mi-python
```

En la memoria: explica la diferencia entre `docker run`, `docker start` y `docker attach`.

FASE 3 — UN SERVICIO EN SEGUNDO PLANO

```
1 docker run -d --name mi-nginx -p 8080:80 nginx
2 docker ps
```

Abre `http://localhost:8080` y captura la página de bienvenida. Después inventaría lo que llevas acumulado:

```
1 docker images      # ¿cuántas imágenes y de qué tamaño?
2 docker ps -a       # ¿cuántos contenedores y en qué estado?
```

FASE 4 — EL MISMO CÓDIGO EN DOS VERSIONES DE PYTHON

Esta fase reproduce el caso del apartado 6.5 con tus propias manos:

```
1 # Contenedor A
2 docker run --rm python:3.9-slim bash -c \
3   "pip install -q experta && python -c 'from experta import KnowledgeEngine; print(\"OK\")'"
4
5 # Contenedor B
6 docker run --rm python:3.12-slim bash -c \
7   "pip install -q experta && python -c 'from experta import KnowledgeEngine; print(\"OK\")'"
```

Captura **las dos salidas** y responde en la memoria:

1. ¿Qué error exacto da el contenedor B y por qué?
2. ¿Qué versión de `experta` se ha instalado en cada caso? ¿Y de `frozendict`?
3. Si tuvieras que entregar hoy un proyecto con `experta`, ¿qué imagen base elegirías y por qué?

FASE 5 — ELIGE UNA IMAGEN Y ESCRIBE SU COMPOSE

1. Elige una imagen del **anexo B** de la teoría (o cualquiera de [Docker Hub](#)).
2. Comprueba que está **mantenida**: mira la fecha de la última actualización.
3. Escribe su `docker-compose.yml` con puertos y, si lo necesita, un volumen para sus datos.
4. Levántalo con `docker compose up -d` y captura el servicio funcionando en el navegador.

FASE 6 — VOLÚMENES Y LIMPIEZA

Crea en tu equipo una carpeta `practicas` con un `app.py`:

```
1 print("Mi primera app en un contenedor")
```

Ejecútala **sin instalar Python en el host**:

```
1 cd practicas
2 docker run --rm -v "$PWD":/app -w /app python:3.12 python app.py
```

Y deja la máquina limpia:

```
1 docker compose down          # si dejaste el servicio de la fase 5
2 docker stop mi-nginx
3 docker rm mi-nginx mi-python
4 docker ps -a                 # ya no deben aparecer
```

ENTREGA DEL TALLER 1

Recopila las capturas y las explicaciones de las seis fases en **una memoria en PDF**. Se valora que expliques lo que ocurre, no solo que pegues pantallazos: una captura sin explicación no demuestra que hayas entendido nada.

Fase	Evidencia mínima
1	<code>docker run hello-world</code> correcto
2	Contenedor interactivo, con la versión de Python impresa desde dentro
3	nginx respondiendo en el navegador + inventario de imágenes y contenedores
4	Las dos salidas del caso <code>experta</code> y el error del contenedor B
5	<code>docker-compose.yml</code> propio y su servicio funcionando
6	La app del host ejecutada en el contenedor y la máquina limpia



Soluciones

Las soluciones no se publican: se corrigen y comentan en clase.

[Volver a la UD00](#) · [Ejercicios](#) · [Taller 2](#)

 21 de agosto de 2026

UD00 · Taller 2 — Contenedor de prácticas de IA

**Entregable · se marca hecho / no hecho**

Se trabaja en clase, en el bloque largo de la semana 2, y **se entrega en Moodle**. No lleva nota, pero **es requisito**: al terminar tendrás el **entorno del curso levantado**, que es el que usarás en el resto de las unidades. Sin él no se pueden hacer las prácticas de la UD02.

Objetivo: levantar un entorno **Jupyter** reproducible con las bibliotecas del curso.

FASE 1 — ESTRUCTURA DE TRABAJO

```
1 practicas/
2   └─ docker/
3       ├── docker-compose.yml
4       └─ requirements-ia.txt
```

FASE 2 — requirements-ia.txt

```
1 numpy
2 pandas
3 matplotlib
4 scikit-learn
5 scikit-fuzzy
6 nltk
7 spacy
8 torch
9 experta
```

FASE 3 — docker-compose.yml

```
1 services:
2   jupyter:
3     image: jupyter/scipy-notebook
4     ports:
5       - "8888:8888"
6     volumes:
7       - ./practicas:/home/jovyan/work
8     environment:
9       - JUPYTER_TOKEN=cursoia
```

FASE 4 — VERIFICA LA CONFIGURACIÓN ANTES DE ARRANCAR

```
1 docker compose config
```

Comprueba que el puerto 8888:8888 y el token quedan resueltos correctamente.

FASE 5 — ARRANCA JUPYTER

```
1 docker compose up -d
2 docker compose ps
```

Abre en el navegador `http://localhost:8888` y usa el token `cursoia`.

Dentro de Jupyter, crea un notebook en la carpeta `work` y verifica las bibliotecas:

```
1 import numpy, pandas, matplotlib, sklearn, skfuzzy
2 import nltk, spacy, torch
3 print("Entorno de IA listo:", numpy.__version__)
```

**Nota para spacy**

Algunos modelos de `spacy` se descargan aparte (`python -m spacy download es_core_news_sm`). Lo veremos en la UD03.

FASE 6 — GESTIONA EL SERVICIO

```
1 docker compose logs -f jupyter # sigue los logs (Ctrl+C para salir)
2 docker compose exec jupyter bash # entra dentro del contenedor en marcha
3 docker compose down           # detiene y elimina
```

FASE 7 — PERSISTENCIA (VOLÚMENES NOMBRADOS)

Modifica el `docker-compose.yml` para que la carpeta de Jupyter use un **volumen nombrado** y comprueba que tus notebooks sobreviven a un `down + up`:

```
1 services:
2   jupyter:
3     image: jupyter/scipy-notebook
4     ports:
5       - "8888:8888"
6     volumes:
7       - jupyter-work:/home/jovyan/work
8     environment:
9       - JUPYTER_TOKEN=cursoia
10
11 volumes:
12   jupyter-work:
```

ENTREGA

Sube a Moodle un **informe breve**, que se marca como **hecho / no hecho**, con:

1. Captura de la **fase 5 de este taller** (Jupyter abierto con el import correcto).
2. Salida de `docker compose ps` mostrando el servicio activo.
3. Respuesta a: *¿por qué este entorno es reproducible en cualquier equipo?*
4. Captura de la **fase 7** mostrando que un notebook sobrevive a `docker compose down`.

**Soluciones**

Las soluciones no se publican: se corrigen y comentan en clase.

[Volver a la UD00](#) · [Ejercicios](#) · [Taller 1](#)

21 de agosto de 2026

2.5 Notebooks

UD00 · Notebook 1 — Bienvenida al entorno Python para IA

Este notebook comprueba que tu entorno de trabajo funciona y hace un primer contacto con las bibliotecas que usaremos durante todo el curso: **numpy**, **pandas**, **matplotlib**, **scikit-learn**, **scikit-fuzzy**, **nltk**, **spaCy**, **torch** y **experta**.

Puedes ejecutarlo en el contenedor de prácticas de la UD00 (taller 2) o en Google Colab.

Objetivos

1. Comprobar que el entorno (local o Docker) tiene las bibliotecas del curso.
2. Hacer un primer ejemplo con **arrays** (numpy), **tablas** (pandas) y **gráficas** (matplotlib).
3. Entrenar un clasificador simple con **scikit-learn**.
4. Guardar el notebook resuelto y entregarlo en Moodle.

```
In [ ... import sys
print("Python", sys.version.split()[0])

import numpy
import pandas
import matplotlib
import sklearn
import skfuzzy
import nltk
import spacy
import torch

print("numpy", numpy.__version__)
print("pandas", pandas.__version__)
print("matplotlib", matplotlib.__version__)
print("scikit-learn", sklearn.__version__)
print("scikit-fuzzy", skfuzzy.__version__)
print("nltk", nltk.__version__)
print("spacy", spacy.__version__)
print("torch", torch.__version__)
```



1. Arrays con numpy

Un **array** es una estructura eficiente para trabajar con datos numéricos. Es la base de casi todas las librerías de IA.


```
In [ ... import numpy as np

temperaturas = np.array([21.0, 22.5, 19.0, 24.0, 23.0])
print("Temperaturas:", temperaturas)
print("Media:", temperaturas.mean())
print("Máximo:", temperaturas.max())

cuadrados = np.arange(1, 6) ** 2
print("Cuadrados:", cuadrados)
```

2. Tablas con pandas

Un **DataFrame** es una tabla con filas y columnas. Es la forma estándar de manejar datos en IA (veremos la *DataFrame* en todas las unidades).

```
In [ ... import pandas as pd

datos = pd.DataFrame({
    "día": ["lun", "mar", "mié", "jue", "vie"],
    "temperatura": [21.0, 22.5, 19.0, 24.0, 23.0],
    "ventas": [120, 135, 98, 152, 141],
})
datos
```

```
In [ ... print(datos.describe())
print()
print("Ventas medias por día par/impar:")
print(datos.groupby(datos["día"].str.len()).ventas.mean())
```

3. Gráfica con matplotlib

Visualizar los datos es el primer paso de cualquier análisis. La gráfica debe llevar **título y etiquetas**.

```
In [ ... import matplotlib.pyplot as plt

plt.figure(figsize=(6, 3))
plt.plot(datos["día"], datos["temperatura"], marker="o")
plt.title("Temperatura de la semana")
plt.xlabel("Día")
plt.ylabel("Temperatura (°C)")
plt.grid(True)
plt.show()
```

4. Primer modelo con scikit-learn

Entrenamos un **clasificador** sencillo: dado un punto (x, y), predecir si está por encima o por debajo de la línea $y = x$. Es un ejemplo de **aprendizaje supervisado**, que veremos con detalle en la UD02.

```
In [ ... import numpy as np
from sklearn.tree import DecisionTreeClassifier

rng = np.random.default_rng(42)
X = rng.uniform(-2, 2, size=(200, 2))
y = (X[:, 1] > X[:, 0]).astype(int)

# Sin limitar la profundidad: el árbol crece hasta memorizar los datos
# de entrenamiento
modelo = DecisionTreeClassifier(random_state=42)
modelo.fit(X, y)
print("Precisión:", modelo.score(X, y))
print("Predicción para (0.5, 1.0):", modelo.predict([[0.5, 1.0]]))
```

5. Actividad

Responde en celdas Markdown del propio notebook:

1. ¿Qué bibliotecas faltaban en tu entorno (si alguna)? ¿Cómo las instalarías?
2. Añade a la tabla `datos` una columna `humedad` con valores inventados y pinta la temperatura y la humedad en la misma gráfica con dos ejes.
3. ¿Qué significa que el modelo anterior tiene precisión 1.0 sobre los mismos datos con los que se entrenó? ¿Es una buena medida del rendimiento? (Pista: en la UD02 hablaremos de *sobreajuste*.)

Entrega: guarda este notebook con tus respuestas y súbelo a Moodle. Se marca como **hecho / no hecho**: no lleva nota, pero es la prueba de que tu entorno funciona, y es **requisito** para las prácticas de las unidades siguientes.

```
In [ ... print("¡Entorno de IA listo!")
```

Volver al material de la unidad

- [Teoría de la UD00 — presentación y curso rápido de Docker](#)
- [Ejercicios de la unidad](#)
- [Taller 1 — Verificación del entorno y primer contenedor](#)
- [Taller 2 — Contenedor de prácticas de IA](#)

Los enlaces son absolutos a propósito: este notebook también se abre en Colab y se descarga, donde las rutas relativas no funcionarían.

🕒 21 de agosto de 2026

3. UD01

3.1 Caracterización de sistemas y utilización de modelos de Inteligencia Artificial

Fundamentos de los Sistemas Inteligentes

Definición de Inteligencia Artificial (IA)

La Inteligencia Artificial es un campo de la informática y la ciencia de la computación que se enfoca en la creación de sistemas y programas capaces de realizar tareas que normalmente requieren inteligencia humana.

Una primera definición bastante común que podemos encontrar para la Inteligencia Artificial es:



Definición

"Habilidad para aprender y resolver problemas, llevada a cabo por una máquina o software"

En general, la mayoría de los expertos coinciden en que es la simulación de procesos de inteligencia humana por parte de máquinas, especialmente sistemas informáticos. Estos procesos incluyen:

1. El **aprendizaje** a través de la adquisición de información y reglas para el uso de la información.
2. El **razonamiento** usando las reglas para llegar a conclusiones aproximadas o definitivas.
3. La **autocorrección**.

Una definición más concreta y consensuada podría ser:



Definición

"La inteligencia artificial es la inteligencia llevada a cabo por máquinas. En ciencias de la computación, una máquina «inteligente» ideal es un agente flexible que percibe su entorno y lleva a cabo acciones que maximicen sus posibilidades de éxito en algún objetivo o tarea".

En realidad, cada generación de hardware y software ha asignado este término a las arquitecturas y técnicas de vanguardia en ese momento. Es por esto que la propia definición puede ir cambiando y evolucionando a medida que se van alcanzando metas más ambiciosas. Podríamos decir que cada nueva oleada de avance tecnológico en este ámbito pasa a conformar una nueva definición de inteligencia artificial, o, al menos, añade un matiz propio a ésta.

La realidad actual es que la IA despierta tanta fascinación como desconfianza. En la gran mayoría de los casos, no se conoce bien la técnica con la que se desarrolla. Ese desconocimiento es el que favorece que se mezcle la realidad con las influencias y fantasías de lo leído y visto en novelas, series y películas.

¿Qué es realmente posible con la tecnología actual y qué sigue perteneciendo al campo de la ciencia ficción? Una habilidad que debe tener cualquier profesional del campo de la IA es, precisamente, ser capaz de explicar de forma sencilla las verdaderas amenazas que esta tecnología puede representar, y saber transmitir una imagen de responsabilidad al respecto.



Recuerda

Los ordenadores (y con ellos la inteligencia artificial) no son ni buenos ni malos. Hacen lo que los humanos programamos que hagan. La inteligencia y, sobre todo, la intencionalidad que pueda tener un programa o aplicación la proporciona el humano (o equipo de humanos) que lo definen y desarrollan.

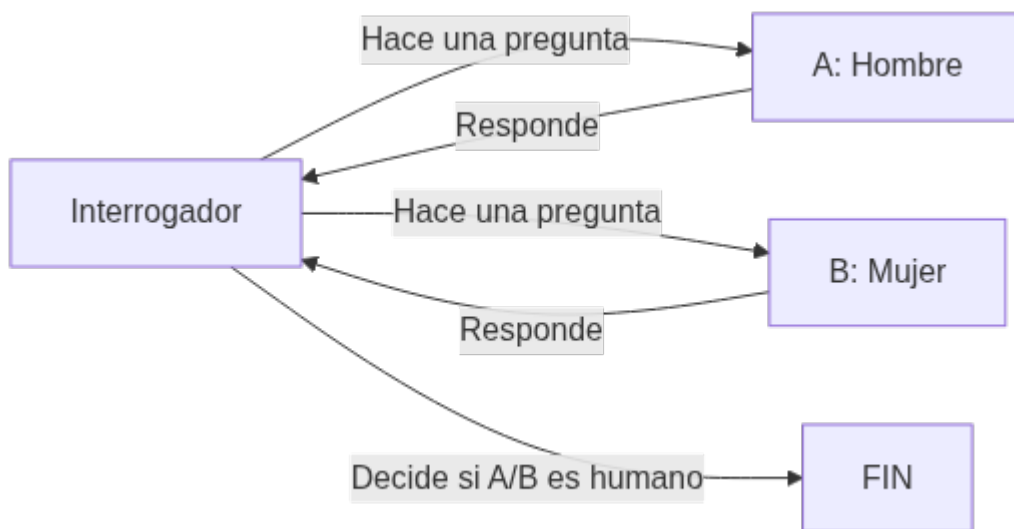
Historia de la IA

A lo largo de la historia, la IA ha pasado por diferentes etapas de desarrollo, con avances y desafíos significativos.

Prehistoria de la IA o proto-IA (Antes del 1950): En realidad, antes de 1956 ya hubo una serie de hitos científicos que podríamos considerar parte del nacimiento de la Inteligencia Artificial:

En 1943 McCulloch y Pitts presentaron un primer modelo de lo que podría ser una neurona artificial, publicándose en el Boletín de Biofísica Matemática con el título: "A logical calculus of the ideas immanent in nervous activity". Partieron de tres fuentes: conocimientos sobre la fisiología básica y funcionamiento de las neuronas en el cerebro, el análisis formal de la lógica proposicional de Russell y Whitehead y la teoría de la computación de Turing. Es por esto que se consideran los eventos más importantes en el origen de la IA (Russell, S., y Norvig, P. 2008)

En 1950, en el trabajo "Computing Machinery and Intelligence", Alan Turing define la conducta inteligente de la máquina como la capacidad de lograr eficiencia a nivel humano en todas las actividades de tipo cognoscitivo, suficiente para engañar a un evaluador humano, y da forma al famoso "Test de Turing". En este histórico artículo Turing propuso que la pregunta «¿puede pensar una máquina?» era demasiado filosófica para tener valor y, para hacerlo más concreto, propuso un «juego de imitación». En la prueba de Turing intervienen dos personas y una computadora. Una persona, el interrogador, se sienta en una sala y teclea preguntas en la terminal de una computadora. Cuando aparecen las respuestas en la terminal, el interrogador intenta determinar si fueron hechas por otra persona o por una computadora. Si la computadora actúa de manera inteligente, según Turing, es inteligente. Turing continúa, "Ahora podemos preguntar: '¿Qué sucederá cuando una máquina tome el papel de A en este juego?' ¿Decidirá el interrogador erróneamente tan a menudo cuando se juegue de esta manera como lo hace cuando el juego se juega entre un hombre y una mujer? Estas preguntas reemplazan nuestra pregunta original: '¿Pueden las máquinas pensar?' "(Turing, 1950) La configuración de la prueba puede representarse de esta manera:



Este test puede servir, como señala Turing, no solo para probar una destreza verbal superficial, sino también el conocimiento de fondo y la capacidad de razonamiento subyacente, ya que los interrogadores pueden hacer cualquier pregunta o plantear cualquier desafío verbal que elijan. Con respecto a este test, Turing predijo famosamente que "dentro de unos cincuenta años [para el año 2000] será posible programar computadoras... para hacer que jueguen el juego de imitación tan bien que un interrogador promedio tendrá no más del 70 por ciento de probabilidad de hacer la identificación correcta después de cinco minutos de interrogación" (Turing 1950); una predicción que ha fallado notoriamente. En el año 2000, las máquinas en la competición del Premio Loebner jugaron tan mal el juego que el interrogador promedio tuvo un 100 por ciento de probabilidad de hacer la identificación correcta después de cinco minutos de interrogación (ver Moor 2001).

Primeros Conceptos de IA (Décadas de 1950 y 1960): El término "Inteligencia Artificial" fue acuñado por John McCarthy en 1956, durante una conferencia (Dartmouth Summer Research Conference on Artificial Intelligence) que se considera el punto de inicio oficial del campo. En dicho encuentro, John McCarthy, Marvin Minsky, Nathaniel Rochester, Claude Shannon, Ray Solomonoff, Oliver Selfridge, Trenchard More, Arthur Samuel, Herbert Simon y Allen Newell, crearon la conjetura inicial "Every aspect of learning or any other feature of intelligence can be so precisely described that a machine can be made to simulate it". En esa época, los investigadores estaban entusiasmados por la idea de que las computadoras pudieran ser programadas para simular la capacidad de razonar y resolver problemas como lo hace el ser humano. Se realizaron esfuerzos iniciales para desarrollar programas que pudieran jugar al ajedrez, realizar cálculos matemáticos y comprender lenguaje natural. En esta conferencia se hicieron previsiones triunfalistas a diez años que jamás se cumplieron (como diríamos ahora "**se vinieron muy arriba**").

1956 Dartmouth Conference: The Founding Fathers of AI



John McCarthy



Marvin Minsky



Claude Shannon



Ray Solomonoff



Alan Newell



Herbert Simon



Arthur Samuel



Oliver Selfridge



Nathaniel Rochester



Trenchard More

A partir de esta conferencia, los siguientes años, se dieron sucesivos éxitos y avances (teniendo en cuenta que entonces se contaba con computadores y herramientas de computación bastante rudimentarios) fruto de investigaciones y multitud de proyectos con grandes expectativas. Los más importantes fueron:

- Creación del LISP en 1958 por John McCarthy (será el lenguaje de programación predominante en posteriores desarrollos de Inteligencia Artificial).
- Desarrollo de Micromundos (mundo de bloques) en 1959 por Minsky y Papert en el MIT.
- Construcción del demostrador de Teoremas de Geometría en 1959 por Herbert Gelernter.
- Investigación de los "sistemas expertos" en 1965 por Santford.
- Lanzamiento de ELIZA en 1966 (será el primer chatbot que implementa lenguaje natural).

La Década del Estancamiento (1970): A finales de la década de 1960 y principios de la década de 1970, la IA experimentó una etapa de estancamiento conocida como el "invierno de la IA". Los avances prometidos no se materializaron, y las expectativas sobre lo que la IA podría lograr superaron las capacidades tecnológicas y computacionales de la época. Esto llevó a una disminución en la financiación y el interés en el campo.

El Renacimiento de la IA (Décadas de 1980 y 1990): En la década de 1980, la IA experimentó un resurgimiento significativo debido a avances en la teoría y la computación. Se desarrollaron nuevas técnicas de razonamiento, representación del conocimiento y búsqueda heurística. Los sistemas expertos, que utilizan reglas y conocimiento específico de dominio para resolver problemas, se convirtieron en una aplicación exitosa de la IA en campos como la medicina y la ingeniería. Se produjo una rivalidad entre Estados Unidos y Japón para ver quién investigaba y desarrollaba más aplicaciones de IA. El principal gran hito en esta etapa fue la creación de R1, el primer sistema experto comercial, en 1982 gracias a McDermott (en 4 años de implantación supuso un ahorro de 40 millones de dólares al año).

Aprendizaje Automático y el auge de la IA moderna (finales de los 90 y década de 2000 en adelante): El siglo XXI ha sido testigo de una revolución en la IA, impulsada en gran parte por el auge del Aprendizaje Automático. Con la disponibilidad masiva de datos y avances en algoritmos, las máquinas ahora pueden aprender patrones complejos a partir de datos y mejorar su rendimiento a través de la experiencia. Esto ha llevado a avances significativos en campos como la visión por computadora, el procesamiento del lenguaje natural y el reconocimiento de voz.

La consagración definitiva de la Inteligencia Artificial llegó a finales de los 90. Con los siguientes grandes hitos, que veremos con algo más de detalle:

- El programa **Deep Blue** desarrollado por **IBM** logró vencer en **1997** al campeón del mundo en ajedrez, **Gari Kaspárov**.



Deep Blue fue una "supercomputadora" que desarrolló la empresa IBM en los años 90 del S.XX para jugar al ajedrez.

En febrero de 1996 se enfrentaron el entonces campeón del mundo, Gary Kaspárov contra la máquina. La primera partida la ganó Deep Blue, otras tres las ganó Kaspárov y dos más quedaron en tablas. Fue la primera máquina que derrotó a un experto jugador.

En realidad, siendo fieles al concepto de Inteligencia Artificial, esta máquina no se puede considerar exactamente como tal, pues "solo" era capaz de calcular a gran velocidad millones de posiciones posibles por segundo, pero le faltaba "intuición". Es decir, contaba con una tremenda base de datos (posibles jugadas, movimientos, etc), pero sus programadores no fueron jugadores de ajedrez expertos, por lo que la máquina no siempre elegía la jugada óptima.

i Curiosidad

¿Sabes cuántas jugadas posibles hay en el ajedrez? Son cerca de 10^{120} (diez elevado a ciento veinte... 100000000000000000000... así hasta 120 ceros)

Para hacerte una idea más real de la tremenda capacidad de cálculo de Deep Blue, se estima que la cantidad de átomos que existen en el Universo es de entre 10^{80} y 10^{82}

El ajedrez es un juego complejo... ¿no crees?

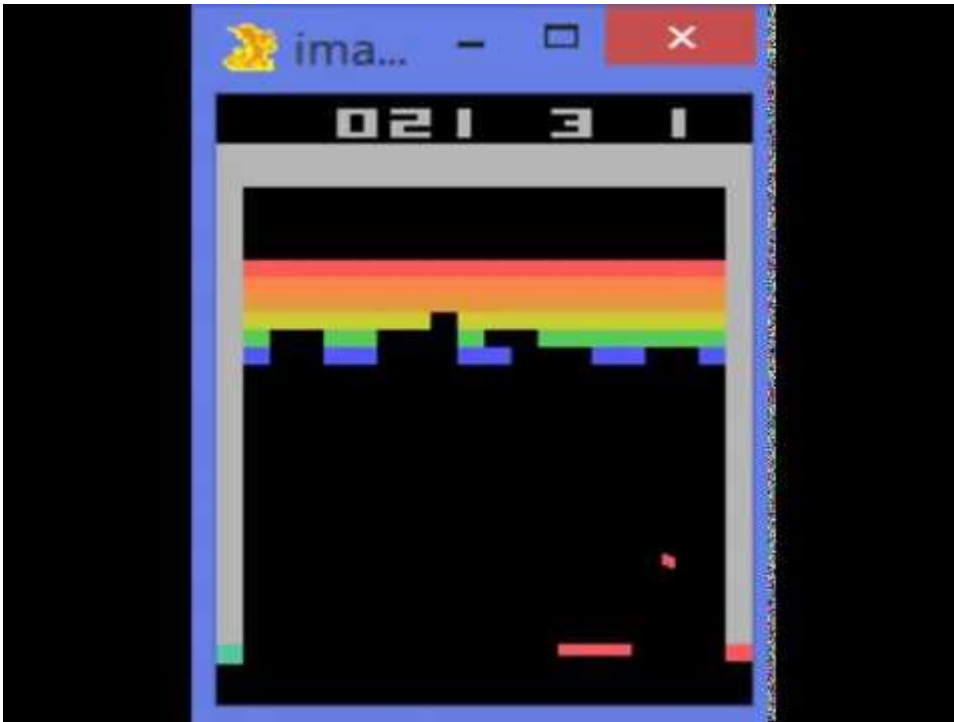
- El sistema **Watson**, también de **IBM**, logró ganar en **2011** el popular concurso televisivo **Jeopardy!** frente a los dos máximos campeones de este programa.



IBM continuó investigando en el campo de la Inteligencia Artificial (a la vez que sus ordenadores aumentaban su capacidad de cómputo) y, con lo aprendido con Deep Blue y otros desarrollos, creó Watson, una computadora que logró en 2011 ganar en Jeopardy!, uno de los concursos de conocimiento más famosos en Estados Unidos.

Watson era un sistema capaz de comprender y responder preguntas, en lenguaje natural (es decir, expresadas como habla cualquier humano, con variaciones y diferentes formas de expresar la misma idea). Contaba con una base de datos almacenada localmente (sin conexión a Internet) compuesta de enciclopedias, diccionarios, tesauros, artículos de noticias, obras literarias y otras bases de datos complementarias).

- La empresa **DeepMind** publicó "el vídeo de los 500 millones de dólares". En dicho vídeo mostraba cómo la red neuronal que había desarrollado aprende a jugar al **Arkanoid** de manera autónoma. Google acabó comprando esta empresa en **2014** por **500 millones** de dólares (de ahí el nombre del vídeo).



DeepMind, una compañía inglesa creada en 2010, publicó a los pocos años de su creación el que se denominó "El vídeo de los 500 millones de dólares". En dicho vídeo (que puedes ver más arriba) mostraba cómo su Inteligencia Artificial había aprendido a jugar gracias a la técnica de entrenamiento autónomo (Machine Learning) por refuerzo al "Arkanoid" (un juego arcade del S.XX).

La red neuronal que desarrolló "aprendía" a jugar como un humano (con memoria a corto plazo, aplicando lo aprendido en cada partida a las siguientes). Así, en el vídeo podemos ver cómo en las primeras partidas descubre cómo debe mover para evitar perder. Mas adelante aprende a ganar puntos destruyendo ladrillos, y a "ganar" la partida alcanzando la máxima puntuación. Y finalmente "descubre" que si logra colar la pelota por un lateral hasta la parte superior de la pantalla gana en mucho menos tiempo.

Al poco tiempo de publicarse este vídeo recibió oferta de compra de Facebook, que no se materializó, y acto seguido de Google, que la compró por 500 millones de dólares (de ahí el nombre del vídeo).

Ya dentro de la matriz de Google continuaron profundizando en las técnicas de Aprendizaje Automático, logrando otro hito, como veremos más adelante.

- **Google** liberó **Tensor Flow**, su librería para Machine Learning, en **2015**, permitiendo que cualquier persona pudiera acceder a sus servidores y crear su propio equipo con capacidad de autoprogramación y de aprender de forma autónoma.

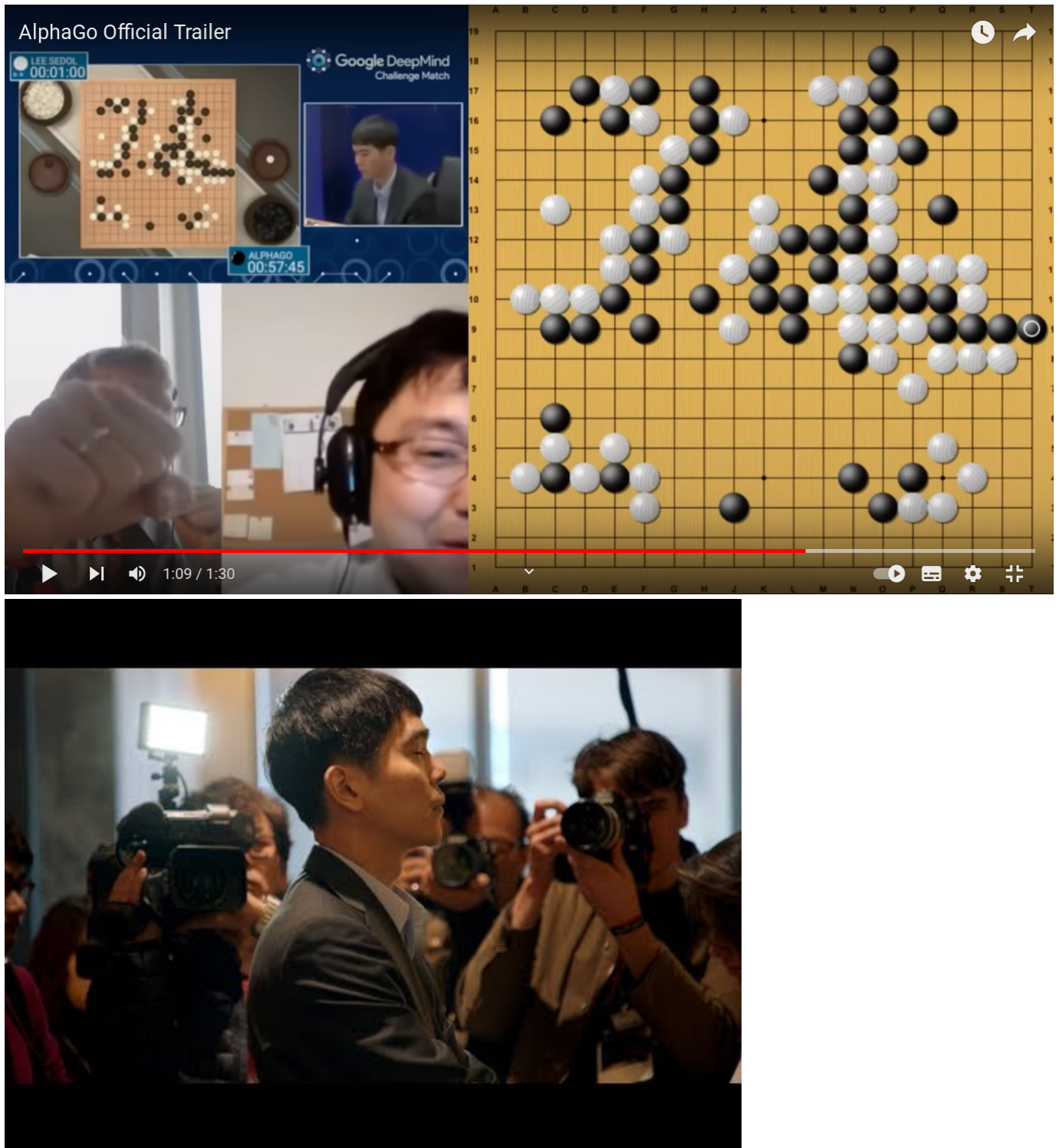


En noviembre de 2015 Google liberó TensorFlow, el programa de Inteligencia Artificial que había desarrollado mejorando un sistema de aprendizaje automático anterior conocido como DistBelief.

Fue la primera vez que se ponía a disposición de cualquier usuario, investigador o empresa interesados en realizar sus propios experimentos de Inteligencia Artificial. Supuso un gran impulso tanto en el campo de la investigación (la propia comunidad de desarrolladores ayudó y sigue ayudando a mejorar y perfeccionar la herramienta) como en el de la democratización de la Inteligencia Artificial, haciéndola accesible para todos.

En la actualidad sigue utilizándose tanto para primeras aproximaciones al universo de la IA, como para desarrollar prototipos o ejercicios más complejos.

- La IA de **AlphaGo** de **Google** sorprendió a todos proponiendo en una partida de Go una jugada que nunca hubiera hecho un experto jugador humano... que en pocos movimientos más le dio la victoria.



Trailer: https://www.youtube.com/watch?v=8tq1C8spV_g

Documental completo: <https://www.youtube.com/watch?v=WXuK6gekU1Y>

Con subtítulos en español: <https://www.youtube.com/watch?v=GfJ7zr4sYx4>

Si jugar al ajedrez es complicado... el juego Go (de origen también oriental) lo es todavía más. La división de Google Deep Mind desarrolló una Inteligencia Artificial capaz de jugar a este juego. Pero la diferencia respecto a logros previos alcanzados por programas de IA capaces de jugar (al ajedrez, a un concurso de preguntas y respuestas...) es que lo hacía "con intuición".

En marzo de **2016** AlphaGo se enfrentó a **Lee Sedol**, que era uno de los mejores jugadores del momento. Ganó AlphaGo. Pero además lo hizo con una jugada (el movimiento 37 de esa partida) completamente inesperado y que ningún experto jugador humano hubiera hecho nunca. Es decir, que aunque la máquina estaba entrenada con registros de partidas reales jugadas por

expertos, no siguió ninguna estrategia observada en su base de datos (contaba con 50 millones de partidas de Go entre humanos). Había aprendido a jugar "con intuición": aprendió de los ejemplos "humanos" pero descubrió formas de jugar nuevas (¡y eficaces!). La cara que se le puso a Lee Sedol al observar y analizar el movimiento de la máquina en ese turno 37 de la partida se hizo famosa.

- **Ian Goodfellow** presentó en **2014** su generador de imágenes basado en lo que conocemos como red **GAN**, logrando que un humano no sepa distinguir si se trata de imágenes reales o inventadas.

Las redes GAN (Generative Adversarial Networks) o generativas antagónicas presentadas por Ian Goodfellow en 2014 han permitido generar fotografías que parecen auténticas a cualquier observador humano.

Posteriormente este tipo de técnica (Aprendizaje Automático Supervisado, que veremos más adelante) también se ha aplicado a la generación de textos tal y como los escribiría un humano.

En esencia las redes GAN se componen de una red generadora (que crea la imagen, texto o diseño) y una red discriminadora (que determina si el resultado de la red generadora es aceptable o no). Ambas redes "compiten" entre ellas (la primera para "engañar a la segunda, y la segunda para detectar fallos en lo generado por la primera). El sistema se retroalimenta y perfecciona con cada iteración.



Curiosidad

En esta web se muestra, cada vez que actualizas la página, la imagen de un rostro humano generado por IA: <https://thispersondoesnotexist.com/>. En algunos casos se notan cosas raras (en las pupilas, orejas, o fondos), pero en general suelen salir rostros que bien podrían corresponder con personas reales ¡Pero en realidad esas personas no existen!

- Desarrollo de **GPT3** por **OpenAI** a través de técnicas de Deep Learning.

GPT-3 es la tercera generación del Modelo de Predicción del Lenguaje que ha sido presentada en mayo de **2020**. Se trata de una Inteligencia Artificial "educada" para escribir cualquier tipo de texto, con cualquier tipo de estilo. A partir de unas pocas palabras que le proporcionas explicando qué es lo que quieres te devuelve un texto complejo que trata sobre lo que le hayas pedido.

Lo más importante de esta tecnología son los **175 Billones de parámetros que utiliza** la para conseguir dar textos naturales (con aspecto de haber sido escritos por humanos).

- **GPT-4** representa la cuarta generación del Modelo de Predicción del Lenguaje, presentado en septiembre de **2022**. Esta versión ha experimentado avances significativos en la capacidad de generación de texto. Como una Inteligencia Artificial avanzada, GPT-4 ha sido entrenado para producir textos aún más naturales y coherentes, adaptándose a cualquier estilo y contenido requerido. Sorprendentemente, cuenta con una impresionante cantidad de **250 Billones de parámetros**, lo que le permite alcanzar un nivel de sofisticación sin precedentes en la creación de textos que parecen ser obra de escritores humanos expertos.

Aprendizaje Profundo (Deep Learning): El Aprendizaje Profundo, una rama del Aprendizaje Automático, ha sido uno de los desarrollos más destacados en la IA moderna. Las redes neuronales profundas, inspiradas en la organización del cerebro humano, han demostrado ser altamente efectivas en tareas como el reconocimiento de imágenes, la traducción automática y el juego de estrategia. La escalabilidad de los algoritmos de Aprendizaje Profundo, junto con la disponibilidad de potentes unidades de procesamiento gráfico (GPU), ha impulsado el rápido progreso en el campo.

IA en la Sociedad Actual: En la actualidad, la IA ha permeado en diversas áreas de nuestra vida cotidiana. Está presente en aplicaciones como motores de búsqueda en línea, asistentes virtuales, recomendaciones de productos y servicios, sistemas de navegación, automóviles autónomos y mucho más. La IA también está siendo utilizada en campos como la medicina para el diagnóstico y tratamiento, en finanzas para la detección de fraudes y en la industria para optimizar procesos de producción.

Desafíos Actuales y Futuros: Aunque la IA ha logrado avances impresionantes, todavía enfrenta desafíos significativos. Uno de ellos es la interpretabilidad y explicabilidad de los modelos de IA, especialmente en aplicaciones críticas donde las decisiones pueden tener un impacto importante en las vidas humanas. Otro desafío es el sesgo en los datos y la falta de diversidad, lo que puede llevar a resultados injustos o discriminatorios. A medida que la IA continúa avanzando, es esencial abordar estos desafíos y asegurar que su desarrollo y aplicación se realicen de manera ética y responsable.

**Curiosidad****¿La Inteligencia Artificial es buena o mala?**

Piensa en diferentes momentos históricos en los que la humanidad ha desarrollado alguna tecnología: El dominio del fuego, la rueda, el hormigón, la pólvora, la imprenta, la radio, Internet...

La tecnología en sí misma no es ni buena ni mala. Son las personas que la conocen y controlan quienes pueden hacer un uso beneficioso o dañino de ellas.

¿Te suena la frase "Un gran poder exige una gran responsabilidad"? La IA nos da un poder tan grande (o mayor) que el Spiderman... Hemos de ser responsables al utilizarla.

El futuro de la IA

Los campos en los que más se ha desarrollado y aplicado la IA en estos últimos años son:

- Sistemas autónomos
- Aprendizaje Autónomo (Machine Learning)
- Aprendizaje Profundo (Deep Learning)
- Redes neuronales.
- Reconocimiento de patrones
- Procesado del lenguaje natural
- Desarrollo de chatbots
- Reconocimiento de emociones

En la actualidad se está trabajando (y se esperan mejoras en los próximos años) en campos como:

- Asistentes virtuales
- Traducción simultánea universal.
- Control de juegos con el pensamiento.

Y a medio plazo se prevé que la Inteligencia Artificial proporcione soluciones y mejoras en los siguientes ámbitos:

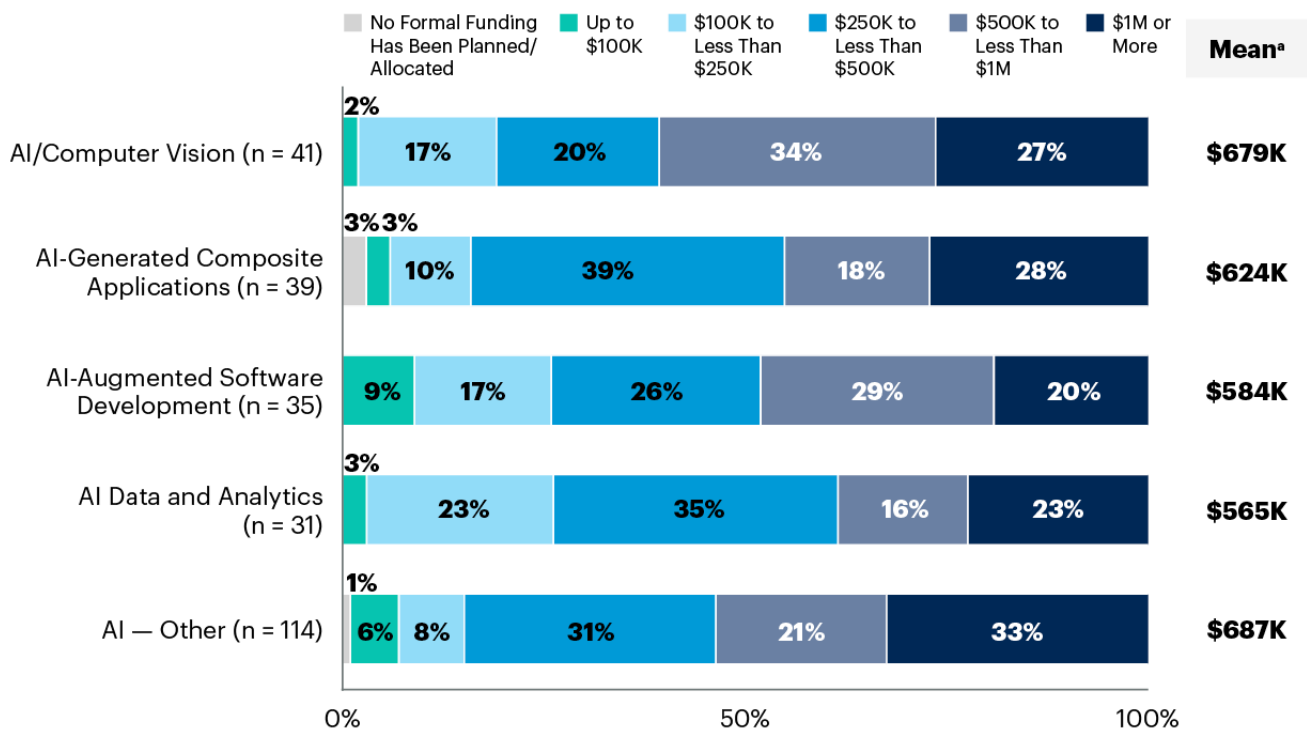
- Nueva generación de robots interconectados con la nube.
- Robots médicos autónomos.
- Asistentes personales robóticos.
- Ciber-Seguridad cognitiva.

Y a largo plazo se vislumbra que puedan llegar a desarrollarse computadoras robóticas con forma y comportamiento humano.

Inversión en proyectos de IA

Como hemos visto la Inteligencia Artificial ya está demostrando de manera práctica los beneficios que puede proporcionar a empresas e instituciones. Las empresas tecnológicas fueron pioneras hace pocos años al incorporar en sus procesos y productos estas aplicaciones. Y en la medida en que todo el entramado empresarial y social se está digitalizando, la posibilidad de incorporar tecnologías inteligentes en cualquier sector está al alcance de casi cualquiera.

De hecho, en los últimos años la Inteligencia Artificial es el primer o segundo ámbito en el que más dinero están dispuestas a invertir las empresas (por delante de otras tecnologías emergentes como Internet de las Cosas, o los datos en la nube). En el siguiente gráfico puedes ver el nivel de financiación que se preveía dedicar en 2022 a incorporar y desarrollar Inteligencia Artificial en un grupo de empresas analizado por Gartner.



fuelle: <https://www.gartner.com/en/newsroom/press-releases/2021-09-29-gartner-finds-33-percent-of-technology-providers-plan-to-invest-1-million-or-more-in-ai-within-two-years>

Limitaciones Prácticas Actuales

Aunque muchos de los argumentos filosóficos y científicos en contra de la inteligencia artificial pueden abordarse con respuestas lógicas, teóricas o basadas en la evidencia, es importante reconocer que existen limitaciones prácticas actuales en el campo de la IA que aún no se han superado por completo. Algunas de estas limitaciones incluyen:

- Complejidad de la mente humana:** La mente humana es increíblemente compleja, y aún no comprendemos completamente todos los aspectos de cómo funciona. La IA actual está lejos de replicar la complejidad y la plasticidad del cerebro humano.
- Memoria y recursos limitados:** Aunque las capacidades de almacenamiento y procesamiento de las computadoras han aumentado drásticamente, todavía estamos lejos de igualar la capacidad de almacenamiento y la eficiencia de procesamiento del cerebro humano.
- Falta de comprensión de la conciencia:** A pesar de los avances en neurociencia y cognición, aún no entendemos completamente la naturaleza de la conciencia y cómo emerge en el cerebro. Sin esta comprensión, es difícil replicarla en una máquina.
- Ética y responsabilidad:** La creación de inteligencia artificial plantea cuestiones éticas y de responsabilidad importantes, como el temor a la toma de decisiones no éticas o la falta de responsabilidad en caso de errores graves.
- IA en entornos no controlados:** La IA puede funcionar bien en entornos controlados y con datos bien estructurados, pero tiene dificultades para adaptarse a situaciones inesperadas o entornos no controlados.
- Creatividad e intuición:** Aunque se han logrado avances en la generación de contenido creativo por parte de las máquinas, la verdadera creatividad e intuición humana siguen siendo difíciles de replicar.
- Emociones y empatía:** La comprensión y expresión emocional, así como la empatía, son aspectos desafiantes de la inteligencia humana que no se han logrado de manera completa en la IA.
- Interacción social humana:** La comprensión del lenguaje natural, la interacción social y la percepción de matices emocionales en el contexto de la comunicación humana son desafíos actuales para la IA.

⚠ Importante

Es importante tener en cuenta estas limitaciones y reconocer que la IA actual está lejos de igualar la inteligencia humana en todos sus aspectos. Sin embargo, esto no implica que no haya avances significativos en el campo de la IA, ni que no se puedan superar algunas de estas limitaciones en el futuro.

Principios de Sistemas Inteligentes

Así pues, puede entenderse por sistema informático el conjunto de cosas (hardware y software) y al conjunto de reglas (procedimientos) que de manera conjunta se emplean para el fin último de adquirir, almacenar, procesar y representar la información de manera automatizada.

Definición

El **hardware** incluye computadoras o cualquier tipo de dispositivo electrónico, principalmente constituido alrededor de semiconductores como memoria, procesadores, sistemas de almacenamiento externo, etc.

El **software** incluye al sistema operativo, firmware y aplicaciones. También se puede considerar parte del sistema a quien hace uso del hardware: los programadores y los usuarios.

La potencia y eficacia de un sistema de la información radica en la correcta correlación de una gran cantidad de datos ingresados mediante procesos específicos para cada campo o tarea, con el objetivo de producir información para la posterior toma de decisiones. Un sistema informático se destaca por su diseño, facilidad de uso, flexibilidad, mantenimiento automático de los registros, apoyo en la toma de decisiones críticas y la conservación del anonimato en informaciones irrelevantes.

Por todo ello, si la inteligencia artificial es un subconjunto de la informática (en el sentido de computación o ciencia de las tecnologías de la información), un sistema de inteligencia artificial ha de ser por extensión un subconjunto dentro de los sistemas informáticos.

Definición

Se denominará, por tanto, sistema inteligente a un programa o conjunto de programas de computación que reúne características y comportamientos asimilables al de la inteligencia humana o animal.

Para que un sistema informático pueda ser considerado un sistema inteligente, habrá de tener las características que se enumeran a continuación:

1. En primer lugar y como clara obviedad el sistema debe poseer inteligencia, entendiendo como ello el **ser un sistema inteligente**.
2. El hecho de ser un sistema implica que ha de disponer de sistematización entre sus componentes, es decir, **las partes del sistema han de tener correlaciones con otros elementos del mismo sistema**.
3. El sistema ha de ser capaz de **cumplir uno o varios objetivos**, o sea, una cierta situación, condición o estado que el sistema inteligente busca lograr.
4. El sistema ha de disponer de **capacidad sensorial**, para ello ha de tener una parte que sea capaz de recibir comunicaciones del entorno: el sistema inteligente ha de poder reaccionar ante el entorno y sus variaciones, lo que se consigue normalmente mediante sensórica.
5. El sistema ha de exhibir capacidad de conceptualización (entendiendo como concepto al elemento básico del pensamiento), lo que implica la **necesidad de poder almacenar información**. Para poder conceptualizar es necesario el desarrollo de niveles de abstracción.
6. El sistema ha de disponer de **procedimientos y métodos con reglas de actuación**, por tanto, el sistema ha de ser capaz de relacionar situaciones y consecuencias de acciones.
7. El aprendizaje es la capacidad más importante y exclusiva de un sistema inteligente. **El sistema aprende nuevos conceptos a partir de la información recibida de los sentidos, las reglas conocidas y la experiencia**. El aprendizaje también es la capacidad de detectar relaciones (patrones) entre la «situación» y la parte «situación futura» de una regla de actuación.

Definición

Los primeros sistemas inteligentes, como los sistemas de expertos, no cumplen todas estas características por lo que reciben el nombre de sistemas de inteligencia incompletos.

La inteligencia artificial (IA) es un área de cambio social, que transforma rápidamente los hábitos y costumbres de las sociedades, y por ello ha de prestarse mucha atención a sus posibilidades, y marcar una serie de límites éticos. Los principios fundamentales o empleos éticos de la IA son los siguientes:

1. **La IA debe estar libre de prejuicios**, tanto en el caso inferencia como en el entrenamiento. Entrenar datos de manera equivocada puede generar discriminantes negativas que alejan la toma de decisiones de la realidad. Además, en conjunto, toda IA debe programarse de manera que no se usen conjuntos sesgados, y evitar discriminaciones algorítmicas por medio de métricas imparciales avaladas por expertos humanos. Aunque parece algo lejano, se introducen continuamente sesgos en el aprendizaje de las IA, muchas veces de manera involuntaria e inadvertida.



Ejemplo

Sistemas de ayuda médica entrenados en varones blancos de entre 30 y 50 años de edad podrán dar resultados buenos sobre el grupo para el que se ha desarrollado, pero pueden actuar no tan correctamente en otras categorías.

2. **Ayudar a ayudar.** Se debe de identificar de forma clara la responsabilidad de las decisiones tomadas por los sistemas autónomos. Los usuarios de las IA, especialmente en caso de grandes corporaciones y gobiernos, han de aprender a estimar y evaluar las consecuencias positivas y negativas de la implantación de sistemas de inteligencia artificial, en la sociedad en su conjunto, dado el gran poder de cambio que generan.
3. **Uso de algoritmos abiertos.** Para poder confiar en la respuesta de una IA es preciso tener acceso limpio al algoritmo de entrenamiento y de toma de decisiones que posee, lo que comporta acceso a todo su modelo matemático. Supone la única manera de poder explicar el funcionamiento que tendrá la misma.
4. **Seguridad, privacidad y confiabilidad.** Dado que las IA hacen uso de gran cantidad de datos, se ha de velar por la transparencia y la privacidad en el uso de los mismos.



Importante

Un asistente virtual ha de garantizar que las conversaciones escuchadas no se filtrarán ni difundirán a terceros.

5. **Bien común.** Ningún sistema de IA debería ser desplegado si al hacerlo se atenta contra el bien común.

Los agentes inteligentes son entidades capaces de percibir su entorno a través de sensores y actuar en él mediante efectores para alcanzar objetivos específicos. Un agente puede ser tan simple como un programa que juega ajedrez o tan complejo como un vehículo autónomo que navega por las calles de una ciudad. Los agentes inteligentes se basan en la idea de que una "inteligencia" puede emerger de la interacción entre un sistema y su entorno, sin necesidad de una planificación o conocimiento exhaustivo previo.

Componentes de un Agente Inteligente:

1. **Sensores (S):** Los sensores son dispositivos que permiten al agente percibir información sobre su entorno. Pueden incluir cámaras, micrófonos, sensores de temperatura, GPS, entre otros. La información que los sensores recopilan se utiliza para representar el estado actual del entorno.
2. **Actuadores (A):** Los actuadores son los medios mediante los cuales el agente interactúa con su entorno. Pueden ser ruedas en un robot, motores en un brazo robótico o simplemente salidas de datos en un sistema de software. Los actuadores permiten que el agente tome decisiones y realice acciones para alcanzar sus objetivos.
3. **Función del Agente (f):** La función del agente representa el comportamiento del agente en función de las percepciones que recibe. Toma como entrada el estado actual del entorno y devuelve una acción que el agente debe ejecutar. Esta función puede ser simple o compleja, dependiendo de la complejidad de la tarea que el agente debe realizar.
4. **Arquitectura (A):** La arquitectura del agente se refiere a cómo se organiza el agente en términos de sus componentes y cómo interactúan entre sí. Puede haber diferentes arquitecturas según la complejidad de la tarea y los requisitos de rendimiento.



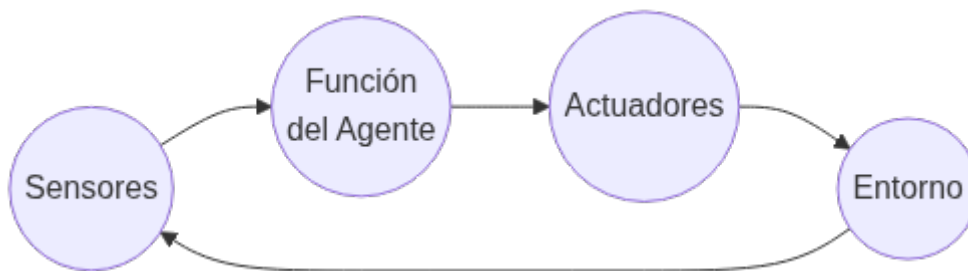
Ejemplo:

Agente Inteligente:

Un ejemplo sencillo de un agente inteligente es un sistema de navegación GPS en un automóvil. En este caso:

- **Sensores (S):** El sistema de navegación utiliza sensores GPS para recibir información sobre la ubicación actual del automóvil y sensores de velocidad para conocer su velocidad y dirección.
- **Actuadores (A):** Los actuadores son los mecanismos que permiten al sistema de navegación proporcionar instrucciones al conductor para alcanzar el destino deseado, como la pantalla de navegación o las indicaciones de voz.
- **Función del Agente (f):** La función del agente en este caso podría ser bastante simple: recibir la ubicación actual y el destino deseado, calcular la ruta más rápida y segura y guiar al conductor a lo largo del camino.
- **Arquitectura (A):** La arquitectura del sistema de navegación podría ser una combinación de algoritmos de planificación de rutas, sistemas de reconocimiento de voz para recibir comandos del conductor y sistemas de visualización para mostrar las indicaciones.

A continuación, un diagrama para ilustrar la interacción de un agente inteligente con su entorno:



En el diagrama, los sensores recopilan información del entorno, que se utiliza como entrada para la función del agente. La función del agente procesa la información y toma una decisión sobre qué acción ejecutar. Luego, los actuadores implementan la acción en el entorno, lo que puede cambiar su estado. El ciclo se repite continuamente, permitiendo al agente inteligente interactuar y adaptarse a su entorno para lograr sus objetivos.

Tipos de Inteligencia Artificial. Escuelas y clasificaciones

En la actualidad la evolución de la Inteligencia Artificial comprende un campo tan amplio, con tantas ramas, que es complicado poder atender a una única clasificación. De hecho encontramos diferentes clasificaciones según la diferente visión con la que se aborda dicha tecnología. Las hay más filosóficas, más técnicas, y según su aplicación.

Dentro de las distintas clasificaciones que vamos a ver, hay algunas tipologías que están definidas "en teoría" aunque aún no exista ninguna aplicación IA de esa clase.

Según tareas a resolver

La primera clasificación de la Inteligencia Artificial que vamos a ver se basa en qué tipo de tareas nos ayuda a resolver o ejecutar.

Cuando hablamos de tarea a resolver nos referimos, por ejemplo, a si pretendemos que la IA sea capaz de jugar al ajedrez en un ordenador o si pretendemos que sea capaz de gestionar una cocina sin intervención humana (desde el abastecimiento de alimentos, procesado, decisión de qué cocinar en cada momento, limpieza y mantenimiento de utensilios, resolver imprevistos o accidentes...).

Hay tareas sencillas, concretas, y puntuales que son relativamente sencillas de programar, mientras que hay tareas complejas en las que, además influyen muchos factores exteriores (sentimientos, contexto, moral, ética, creencia religiosa...).

Seguro que eres capaz de ir intuyendo que nuestros programas de Inteligencia Artificial actuales son más bien de los que resuelven tareas en un entorno muy concreto y acotado, mientras que hay que irse a las películas o series de ficción para poder hablar de algún caso de Inteligencia Artificial capaz de actuar en escenarios complejos, cambiantes y con "conciencia".

Esta clasificación según tareas es una aproximación bastante simple, con dos opciones:



	IA Débil	IA Fuerte
CAPACIDADES	Realiza tareas específicas .	Puede realizar cualquier tarea intelectual que un humano puede hacer.
EJEMPLOS	Siri, Alexa , Google Translate.	IA consciente de sí misma (Aún no existe).
ESTADO ACTUAL	Existe y se utiliza ampliamente .	Aún no existe y es objeto de investigación y debate .

Como veremos en la explicación más detallada de cada una de estas dos posibles categorías de Inteligencia Artificial, en la actualidad aún no hemos sido capaces de desarrollar ninguna IA Fuerte. Todo lo que conocemos en este momento quedaría dentro de la clasificación de IA Débil.

INTELIGENCIA ARTIFICIAL DÉBIL (O ESTRECHA)

Definición

IA Débil también conocida como **IA estrecha**, se define como la inteligencia artificial racional que se centra típicamente en una tarea estrecha. Es decir orientada a resolver problemas muy concretos, en un entorno perfectamente acotado.

Por tanto consideramos que este tipo de Inteligencia Artificial débil es limitada, pues no es capaz de adaptarse o asumir cambios respecto a lo que se le ha programado.

Los asistentes virtuales (Siri, Ok Google, Alexa, etc.) son un ejemplo bastante ilustrativo de hasta dónde es capaz de llegar la Inteligencia Artificial débil. Cualquiera de ellos opera dentro de un rango de respuestas limitado, definido en su base de datos. En realidad "la máquina" no tiene inteligencia genuina. No es capaz de aprender, ni de tener en cuenta el entorno o el contexto en el que se le realizan las preguntas. No tiene conciencia, ni mucho menos vida propia. Sin duda es un tipo bastante sofisticado de IA débil, pero llega un punto en el que no es capaz de responder cierta clase de preguntas, y, salvo que cambiáramos su base de datos y programación nunca sería capaz de llegar a encontrar por sí misma respuestas adecuadas a dichas preguntas.

De hecho, uno de los principales entretenimientos más habituales cuando nos ponemos "a charlar" con un asistente virtual es intentar llevarla al límite... A ver en qué tipo de pregunta, cada cual más compleja o absurda, es incapaz de responder. O, sin llegar al límite de no encontrar respuesta, puede llegar a dar respuestas molestas o inadecuadas.

Desde el punto de vista de esta clasificación (por tarea a resolver), **las características** de la Inteligencia Artificial débil son:

- **Ya existen en la vida real:** Como hemos comentado, los asistentes virtuales, programas como Watson o Alpha Go que vimos en la unidad anterior.
- Se orientan a **resolver problemas muy concretos:** El programa que "sabe" jugar al Go, no sabe hacer otra cosa. Ni tiene posibilidades de aprender a jugar a otra cosa, por muy similar que sea.
- Son **reactivas:** No tienen iniciativa, es necesario que se desencadene la acción que tienen programada para que se inicie su rutina. En el ejemplo del asistente virtual, tiene programado responder cuando le preguntas, y por tanto nunca tomará la iniciativa de ofrecerte nada sin que tú lo actives previamente.
- **No son flexibles:** Colapsan si se encuentran en un caso no previsto en su programación.
- Quedan **limitadas por lo que programa un humano:** Es el humano quien programa lo que "tiene que pensar" la máquina. Si el humano no programa deja sin considerar ciertas opciones o posibles situaciones, la IA nunca será capaz de suplirlo o aprenderlo sobre la marcha por sí misma.
- Se **programan con pocas redes neuronales:** Hablaremos más adelante sobre las redes neuronales. Por el momento es suficiente entender que el nivel de computación compleja que requieren este tipo de Inteligencias Artificiales es menor que otros casos.
- **No razonan, solo computan:** No tienen en cuenta ningún factor moral, contextual, circunstancial, emocional... que a un humano le haría reaccionar de manera diferente.
- La máquina está programada para alcanzar tal objetivo o funcionar de tal manera, y así lo hará sin "entender" lo que está haciendo. Por tanto: **no tiene conciencia**.

- **Aprenden a base de ejemplos:** Necesita conocer muchos ejemplos de lo que tiene que hacer (la base de datos), con todas las variantes posibles. Por ejemplo, en la máquina que juega al Go, se la "entrenó" con 50 millones de partidas de dicho juego.
- **Son repetitivas:** No se cansan nunca, son implacables, siempre la misma rutina. No salen de su marco de trabajo: Y esto supone que pueden ocuparse de tareas mecánicas, repetitivas, "aburridas" para sustituir al humano mejorando rendimiento y precisión. Pero necesitará siempre una supervisión humana que vaya decidiendo cómo adaptar el programa a las cambiantes circunstancias.

Según se mire la Inteligencia Artificial débil podría llegar a ser peligrosa. Porque este tipo de IA, al no tomar en consideración todo un contexto amplio, ni seguir las reglas sociales, éticas,... ejecuta las tareas para las que se le ha entrenado con eficacia y contundencia. No evalúa las consecuencias como lo hacemos los humanos, considerando un espectro amplio de efectos y relaciones. Por eso, es una opción incompleta, inestable y peligrosa si no se utiliza con prudencia, o si quien la programa pretende, precisamente, causar mal.

INTELIGENCIA ARTIFICIAL FUERTE



Definición

La **Inteligencia Artificial fuerte (IAF)** o general o (IAG), desde el punto de vista de la tarea a resolver, sería aquella que iguala o excede la inteligencia humana promedio. Sería capaz de realizar con éxito cualquier tarea intelectual del ser humano, teniendo en cuenta todos los factores y matices que pueden intervenir cuando una persona toma decisiones en cada momento mientras realiza una tarea.

En comparación con la Inteligencia Artificial débil, las características de la fuerte son:

- **No existe en la realidad:** Si quieres "ver" cómo sería puedes recurrir a personajes de ficción como T-800, Wall-E o J.A.R.V.I.S. Resolverán problemas abiertos: Deberían poder abarcar múltiples posibles tareas, distintas unas de otras (reparar una puerta, ir a recoger a los niños del colegio, regar las plantas, darte conversación...).
- **Serán proactivas:** En función de la misión u objetivo que tenga, y de las circunstancias, iniciará cualquier tipo de rutina sin esperar a que un humano se lo pida o esté pendiente.
- **Serán flexibles:** Podrán encontrar similitudes entre algo que conocen y algo que se le parezca un poco. Por ejemplo, aunque inicialmente solo haya sido programada para saber andar, será capaz de aprender a correr sin necesidad de intervenir en su programa.
- **Se autoprogramarán:** Serán capaces de detectar sus propios límites y se regularán a sí mismas para no excederlos.
- **Usarán muchas redes neuronales:** Y además podrán entrar en conflicto entre ellas en algunas ocasiones. Esto quiere decir que necesitarán una capacidad de almacenaje de información y cómputo que aún hoy no hemos llegado a alcanzar.
- **Imitarán el comportamiento humano:** Serán capaces de razonar, y por tanto, de alcanzar algún tipo de consciencia.
- **Aprenderán como las personas:** Podrán recordar datos, observar nuevas situaciones y encontrar relaciones entre diferentes acciones. Esto quiere decir que si saben jugar al ajedrez y "observan" el juego de las damas, podrán aprender a jugar a las damas basándose en lo que saben sobre jugar al ajedrez.
- **Serán capaces de aprender nuevas tareas:** Modificarán la tarea o cómo realizan la tarea para adaptarse a las circunstancias.
- **Serán capaces de adaptarse a nuevos escenarios:** Podrán adaptarse a cambios y nuevas situaciones para seguir cumpliendo su objetivo.

Este tipo de IA es la que sería capaz de analizar cualquier situación y deducir el conjunto de acciones más adecuado para dicha situación y contexto. Lo mismo sabría conducir un coche, que resolver una ecuación matemática o mantener una conversación sobre un tema concreto.

Aunque aún no existe este tipo de IA, todas las empresas e instituciones dedicadas a la investigación y desarrollo de IA están buscando formas de avanzar hacia este tipo de Inteligencia Artificial. De momento, al menos, se está trabajando en conseguir una IAF en el campo de los asistentes virtuales.

Sin duda uno de los ámbitos más ambiciosos de aplicar esta IAF (los asistentes virtuales humanoides) necesitan contar también con otras ramas científicas como son la robótica y mecatrónica.

Escuelas de Pensamiento

En el ámbito de la Inteligencia Artificial más moderna podemos encontrar dos escuelas de pensamiento:

- Inteligencia Artificial **Convencional**.
- Inteligencia Artificial **Computacional**.

Estas dos escuelas difieren en la ciencia que hay tras los procesos que siguen para llegar a los resultados esperados. Pero, con los avances que se están dando en los recursos que utiliza la segunda, muchas de las aplicaciones que tenía la primera, están siendo llevadas al campo computacional.

INTELIGENCIA ARTIFICIAL CONVENCIONAL

Se conoce también como Inteligencia Artificial **simbólico-deductiva**. Está basada en el análisis formal y estadístico del comportamiento humano ante diferentes problemas:

- **Razonamiento basado en casos:** Ayuda a tomar decisiones mientras se resuelven ciertos problemas concretos y, aparte de que son muy importantes, requieren de un buen funcionamiento.
- **Sistemas expertos:** Infieren una solución a través del conocimiento previo del contexto en que se aplica y ocupa de ciertas reglas o relaciones.
- **Redes bayesianas:** Propone soluciones mediante inferencia probabilística.
- **Inteligencia artificial basada en comportamientos:** Esta inteligencia contiene autonomía y puede auto-regularse y controlarse para mejorar.
- **Smart process management:** Facilita la toma de decisiones complejas, proponiendo una solución a un determinado problema al igual que lo haría un especialista en dicha actividad.

Esta rama de la Inteligencia Artificial ha sido la que ha proporcionado la mayoría de algoritmos que conocemos como "automatización", y básicamente se sirven de sistemas con reglas condicionales y estadística avanzada.

INTELIGENCIA ARTIFICIAL COMPUTACIONAL

La Inteligencia Computacional (también conocida como IA **subsimbólica-inductiva**) implica desarrollo o aprendizaje interactivo (por ejemplo, modificaciones interactivas de los parámetros en sistemas de conexiones). El aprendizaje se realiza basándose en datos empíricos, utilizando métodos computacionales inspirados en procesos de la naturaleza, que permiten alcanzar soluciones aptas a problemas complejos que los modelos tradicionales no pueden resolver por no existir una solución analítica, por no contar con todos los parámetros necesarios o porque el problema es en sí estocástico y precisa de una aproximación envolvente en vez de convergente.

Esta corriente ha sido la que impulsó hace pocos años lo que conocemos como "Aprendizaje Automático" o "Machine Learning", que es la técnica que más se está utilizando actualmente en desarrollos de IA.

Algunas técnicas de esta escuela son:

- **Máquina de vectores soporte:** sistemas que permiten reconocimiento de patrones genéricos de gran potencia.
- **Redes neuronales:** sistemas basados en redes de unidades de computación lineal para simular computación no lineal
- **Modelos ocultos de Markov:** aprendizaje basado en dependencia temporal de eventos probabilísticos.
- **Sistemas difusos:** técnicas para lograr el razonamiento bajo incertidumbre
- **Computación evolutiva:** también conocidos como **algoritmos genéticos**, aplica conceptos inspirados en la biología, tales como población, mutación y supervivencia del más apto para generar soluciones sucesivamente mejores para un problema.

Clasificación Stuart J. Russell y Peter Norvig

Stuart J. Russell y Peter Norvig, investigadores informáticos, publicaron en **1995** su libro "**Artificial Intelligence: A Modern Approach**", que se ha convertido en el libro de texto fundamental en cientos de universidades a nivel mundial (ya lleva varias ediciones publicadas). Plantean cuatro categorías básicas partiendo de un enfoque de génesis del acto inteligente, del origen y proceso por el cual se llega al comportamiento inteligente.

Sistemas Cognitivos: Piensan como humanos, intentan emular el proceso humano → Proceso de toma de decisiones, resolución de problemas, y el propio paradigma del aprendizaje.

Test de Turing: Actúan como humanos, intentan emular el comportamiento humano (sin pasar por el pensamiento o razonamiento que conduce a dicho comportamiento) → A nivel práctico se aplica en la robótica y sistemas de actuadores en el mundo físico.

Leyes del pensamiento: Piensan con razonamientos. Cumplimiento exacto de las leyes del razonamiento lógico, teniendo en cuenta todos los factores que afectan a la cuestión. No hemos llegado a esto aún. Sería el caso de los sistemas expertos. Solo son posibles aproximaciones para campos de investigación muy especializados y acotados.

Agentes inteligentes: Actúan racionalmente (sin pasar por el proceso de razonamiento lógico).

Los dos últimos requieren una capacidad de cómputo muy importante, a veces, aún, inaccesible.

Clasificación Hintze

En noviembre de **2016**, **Arend Hintze**, profesor de la Universidad de Michigan e investigador en el campo de la Inteligencia Artificial, escribió un artículo titulado: "**Understanding the four types of AI, from reactive robots to self-aware beings**" (Comprendiendo los cuatro tipos de IA, desde los robots reactivos a los seres auto-conscientes), en el que sintetizaba toda la evolución de los últimos desarrollos y avances en materia de Inteligencia Artificial para aportar una clasificación más realista y concreta para los tipos de entidades que existen o que se aspira a crear.

4 TIPOS DE INTELIGENCIA (ARTIFICIAL)



MÁQUINAS REACTIVAS

Los tipos más básicos de sistemas de IA son puramente reactivos. No tienen la capacidad de formar recuerdos. Tampoco pueden utilizar experiencias pasadas en las que basar las decisiones actuales.

Deep Blue (descrita en el tema anterior) fue una supercomputadora creada por IBM. Fue capaz de vencer al ajedrez al gran maestro internacional Garry Kasparov. Ocurrió a fines de la década de 1990 y es el ejemplo perfecto de este tipo de máquina.

Puede identificar las piezas en un tablero de ajedrez y saber cómo se mueve cada una.

Puede realizar predicciones sobre los mejores movimientos y elegir el mejor de todas las posibilidades.

Pero no tiene ningún concepto del pasado. Tampoco posee recuerdos de lo que ha sucedido antes. Aparte de una regla de ajedrez, Deep Blue ignora todo antes del momento presente. Todo lo que hace es enfocar las piezas del tablero en tiempo real y elegir entre los siguientes movimientos posibles.

En el caso de que se trate de una Inteligencia Artificial reactiva aplicada a una máquina capaz de "conversar" es importante que el usuario sepa que está tratando con una máquina, pues de lo contrario se suelen crear falsas expectativas sobre lo que puede esperar de dicha conversación.

MEMORIA LIMITADA

El segundo tipo de Inteligencia Artificial que contempla la clasificación de Hintze se caracteriza por que sí maneja máquinas que pueden mirar hacia el pasado. Los vehículos autónomos ya hacen algo parecido. Por ejemplo, observan la velocidad y dirección de otros autos, y en base a la memorización de dicha información toman decisiones en el futuro inmediato.

Digamos que estas observaciones se agregan a las representaciones preprogramadas para la memoria de estos coches. Se incluyen marcas de carril, semáforos y otros elementos importantes, como curvas en la carretera.

También se añaden experiencias como cuando el automóvil decide en qué momento cambiar de carril para evitar interrumpir a otro conductor o ser embestido por un automóvil cercano.

Pero estas simples piezas de información sobre el pasado son solo transitorias. No se guardan como parte de la biblioteca de experiencias del automóvil. En estos tipos de inteligencia artificial, la máquina no puede compilar la experiencia durante años, como lo hace un humano.

TEORÍA DE LA MENTE

¿Podemos construir sistemas de Inteligencia Artificial que construyan representaciones completas, recordar sus experiencias y aprender cómo manejar situaciones nuevas?

Llegamos a un punto en el que nos acercamos más a los tipos de Inteligencia Artificial que deseamos en un futuro. Las máquinas de esta clase son más avanzadas. No solo forman representaciones sobre el mundo, también sobre otros agentes o entidades.

En psicología, esto se denomina ‘teoría de la mente’. Implica la comprensión de que las personas, las criaturas y los objetos en el mundo pueden tener pensamientos y emociones que afectan a su propio comportamiento. Esto es crucial para la forma en que los humanos formamos sociedades, porque nos permite la interacción social.

Si las máquinas van a andar entre nosotros, deberán tener una comprensión sobre cómo pensamos y cómo sentimos. Además deberán llegar a saber qué esperamos y cómo queremos que nos traten. Tendrán que ajustar su comportamiento en consecuencia.

Como habrás podido intuir, este tipo de máquinas aún no existen. Igual que la IA Fuerte es aún algo que se sabe cómo funcionará pero que no hemos llegado a desarrollar todavía.

AUTOCONCIENCIA

El paso final del desarrollo de la IA es construir sistemas que puedan formar representaciones sobre sí mismos. En última instancia, los investigadores de la Inteligencia Artificial tendrán que comprender no solo la conciencia, sino también construir máquinas que la tengan.

Los seres conscientes son conscientes de sí mismos, conocen sus estados internos y pueden predecir los sentimientos de los demás. Es probable que estemos lejos de crear máquinas que sean conscientes de sí mismas. Sin embargo, los esfuerzos se enfocan hacia la comprensión de la memoria, el aprendizaje y la capacidad de basar las decisiones en experiencias pasadas.

Este es un paso importante para entender la inteligencia humana por sí misma. Es crucial para diseñar o desarrollar máquinas que sean más excepcionales para clasificar lo que ven frente a ellas.

Los cuatro tipos de inteligencia artificial dan una idea sobre las intenciones que el hombre tiene acerca del futuro de la máquina. Puede que estemos muy lejos de la Inteligencia Artificial autoconsciente. No obstante, está claro que eso es lo que se persigue en última instancia.

Utilización de modelos de Inteligencia Artificial

En esta sección, se explorarán los requisitos básicos de un sistema de resolución de problemas y los diferentes modelos de sistemas de Inteligencia Artificial, incluyendo la automatización de tareas, sistemas de razonamiento impreciso y sistemas basados en reglas.

Requisitos básicos de un sistema de resolución de problemas

Los sistemas de resolución de problemas basados en Inteligencia Artificial deben cumplir con ciertos requisitos fundamentales para ser efectivos y proporcionar soluciones precisas y útiles:

1. **Representación del Problema:** La representación adecuada del problema es crucial para la resolución exitosa. Los sistemas de IA deben seleccionar una estructura de datos y un modelo que reflejen de manera fiel el dominio del problema. Por ejemplo, para el procesamiento del lenguaje natural, se pueden utilizar modelos basados en redes neuronales que convierten el texto en representaciones vectoriales.
2. **Razonamiento y Toma de Decisiones:** Los sistemas de IA deben ser capaces de razonar sobre la información disponible y tomar decisiones informadas para llegar a una solución. Esto implica aplicar técnicas lógicas, de aprendizaje automático o de búsqueda heurística, según la naturaleza del problema.
3. **Aprendizaje y Adaptabilidad:** La capacidad de aprender de la experiencia y adaptarse es esencial para mejorar el rendimiento de un sistema de IA con el tiempo. El aprendizaje automático y el aprendizaje por refuerzo son enfoques comunes utilizados para habilitar esta funcionalidad.
4. **Eficiencia Computacional:** Los sistemas de resolución de problemas deben ser eficientes en términos computacionales para proporcionar respuestas rápidas y escalables a problemas complejos. Esto implica el uso óptimo de algoritmos y técnicas de optimización para reducir el tiempo de ejecución y los recursos necesarios.

5. **Interacción con Usuarios:** Los sistemas de Inteligencia Artificial deben permitir la interacción con los usuarios de una manera comprensible y natural. Esto implica el desarrollo de interfaces de usuario amigables que faciliten la comunicación y la comprensión mutua entre humanos y sistemas de IA.

Modelos de sistemas de Inteligencia Artificial

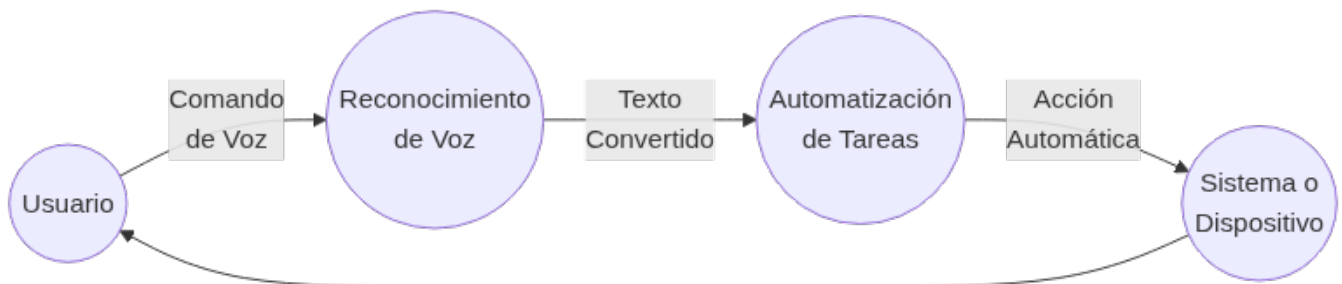
AUTOMATIZACIÓN DE TAREAS

La automatización de tareas es uno de los enfoques más comunes de la Inteligencia Artificial. En este modelo, se desarrollan sistemas que pueden realizar tareas específicas sin intervención humana directa. Algunos ejemplos de automatización de tareas son:

- **Reconocimiento de Voz:** Los sistemas de reconocimiento de voz convierten el habla en texto y pueden automatizar la transcripción de documentos o comandos de voz en dispositivos. Por ejemplo, aplicaciones de reconocimiento de voz como Google Speech-to-Text o Microsoft Azure Speech Service permiten convertir grabaciones de voz en texto escrito de manera automatizada.
- **Detección de Fraude:** Algoritmos de aprendizaje automático pueden analizar patrones de datos financieros y detectar transacciones sospechosas o fraudulentas. Por ejemplo, instituciones financieras utilizan modelos de aprendizaje automático para identificar patrones de comportamiento inusuales que podrían indicar actividades fraudulentas.

La automatización de tareas no solo mejora la eficiencia en diversas industrias, sino que también reduce la carga de trabajo manual y permite a los humanos centrarse en tareas más creativas y estratégicas.

A continuación, un diagrama para ilustrar el proceso de automatización de tareas mediante un sistema de reconocimiento de voz:



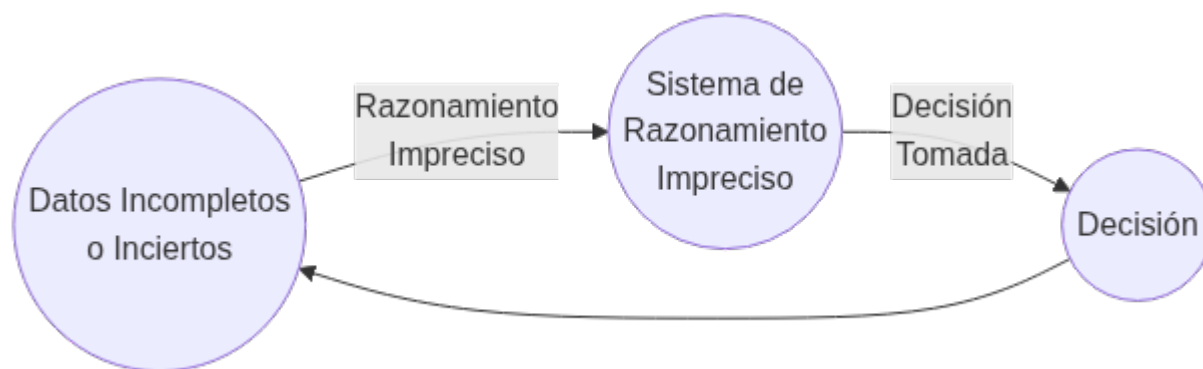
SISTEMAS DE RAZONAMIENTO IMPRECISO

Los sistemas de razonamiento impreciso permiten el manejo de incertidumbre y vaguedad en los datos y la toma de decisiones. Estos sistemas son útiles cuando los datos son incompletos o inciertos y se basan en la lógica difusa y otras técnicas de incertidumbre. Algunos ejemplos son:

- **Controladores de Tráfico:** En el control del tráfico y semáforos, se pueden utilizar sistemas de razonamiento impreciso para optimizar los tiempos de espera de los vehículos y mejorar el flujo del tráfico. La lógica difusa permite ajustar los tiempos de semáforos en función del flujo vehicular en tiempo real.
- **Diagnóstico Médico:** En medicina, se pueden aplicar sistemas de razonamiento impreciso para evaluar síntomas y proporcionar diagnósticos preliminares o sugerencias de tratamiento. Por ejemplo, en el diagnóstico de enfermedades como el cáncer, donde los resultados de las pruebas pueden no ser definitivos, los sistemas de razonamiento impreciso pueden ayudar a proporcionar una evaluación más completa y considerar múltiples factores para el diagnóstico.

El razonamiento impreciso es especialmente valioso cuando se enfrentan problemas en los que la información es vaga o incierta, lo que permite tomar decisiones más robustas y flexibles.

A continuación, un diagrama para ilustrar cómo un sistema de razonamiento impreciso maneja la incertidumbre en la toma de decisiones:



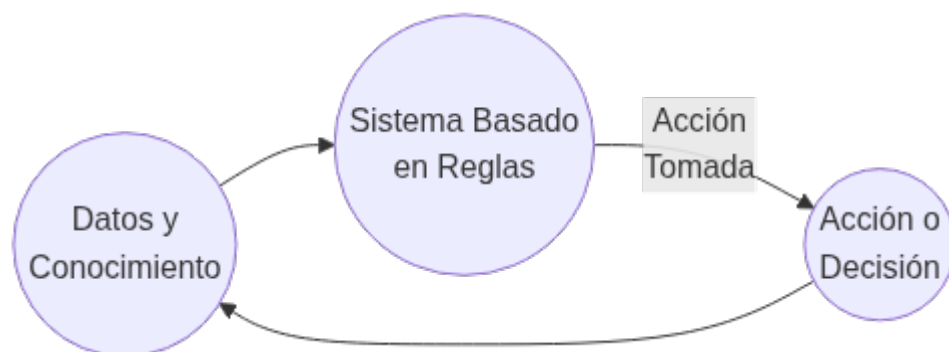
SISTEMAS BASADOS EN REGLAS

Los sistemas basados en reglas son sistemas de Inteligencia Artificial que utilizan reglas lógicas para representar el conocimiento y tomar decisiones. Cada regla consiste en una condición y una acción, y cuando se cumple la condición, se aplica la acción correspondiente. Algunos ejemplos son:

- **Sistemas de Recomendación:** Los sistemas de recomendación utilizan reglas lógicas para sugerir productos, películas, música u otros elementos en función del comportamiento del usuario y otros datos relevantes. Por ejemplo, plataformas de comercio electrónico como Amazon utilizan sistemas de recomendación basados en reglas para ofrecer productos relacionados basados en el historial de compras del usuario.
- **Diagnóstico en Sistemas de Soporte Médico:** En sistemas de soporte médico, las reglas se utilizan para evaluar síntomas y datos médicos y proporcionar diagnósticos preliminares o sugerencias de tratamiento. Por ejemplo, en sistemas de asistencia médica remota, las reglas basadas en síntomas pueden proporcionar recomendaciones iniciales antes de que el paciente sea atendido por un profesional de la salud.

Los sistemas basados en reglas son ampliamente utilizados en aplicaciones donde se requiere un razonamiento transparente y fácil de entender, ya que las reglas lógicas son explícitas y pueden ser interpretadas por humanos.

A continuación, un diagrama para ilustrar cómo un sistema basado en reglas aplica las reglas lógicas para tomar decisiones:



Estos modelos de sistemas de Inteligencia Artificial representan diferentes enfoques para resolver problemas y tomar decisiones en diversas aplicaciones. Cada uno de estos modelos tiene sus propias ventajas y desafíos, y la elección del enfoque adecuado depende del contexto y los requisitos específicos del problema a resolver.

Técnicas de la Inteligencia Artificial

A continuación veremos una taxonomía de técnicas que se usan en IA, algunas se estudiarán a fondo en este módulo (Modelos de Inteligencia Artificial - MIA) y otros en otro módulo del curso (Sistemas de Aprendizaje Automático - SAA).

Modelo Clásico. Sistemas expertos (MIA)

La Inteligencia Artificial, inicialmente, tuvo un desarrollo más teórico que práctico. Los planteamientos originarios de esta Inteligencia Artificial clásica se definieron para un tipo de trabajo informático que ignoraba en buena medida cómo se ha desarrollado en los últimos decenios y que actualmente está establecido como convencional.



Curiosidad

Recuerda que estamos hablando de los años 60 del Siglo XX, y que en esa época apenas existían ordenadores experimentales, con una memoria y capacidad de cómputo que ahora consideraríamos ridículos. Cualquier Smartwatch o controlador de aspiradora inteligente tiene más memoria y velocidad de cálculo que los ordenadores que se utilizaron o se previó que se podrían utilizar para desarrollar Inteligencia Artificial entonces.

Otro aspecto importante a tener en cuenta sobre lo que se entendía por Inteligencia Artificial en esos primeros años es que se preveía que en un plazo de tiempo razonable iba a ser posible que las máquinas "pensaran" como los humanos. Es decir, que los mecanismos de la Inteligencia Artificial imitarían la manera de aprender y reaccionar (actuar) del cerebro humano.

Se dedicó tiempo y esfuerzo, por tanto, a intentar definir de manera matemática y computacional cómo funcionaba el cerebro humano. En última instancia se intentó definir un proceso informático (basado en algoritmos matemáticos) equivalente a lo que haría una neurona humana.

Por tanto, para entender bien qué es la Inteligencia Artificial Clásica, debemos tener en cuenta que:

- Lo que ahora denominamos **Inteligencia Artificial Clásica** fue más bien un **ejercicio de creación de principios generales**, que posteriormente se emplearon para desarrollar los primeros programas informáticos prácticos de Inteligencia Artificial aplicada. Pero esta IA está bastante alejada de lo que hoy por hoy entendemos a nivel práctico como Inteligencia Artificial.
- La Inteligencia Artificial Clásica **quería desarrollar programas informáticos que replicaran el conocimiento humano**, inicialmente en casos particulares y "sencillos", con la intención de ir poco a poco abarcando procesos y casos más complejos. De tal manera que la máquina pudiera "pensar" y actuar como un humano experto en dicho caso particular.

La Inteligencia Artificial Clásica necesitaba que en el proceso de aprendizaje de dicha IA participaran "expertos" en la tarea que se pretendía que la máquina realizara por sí misma.



Ejemplo:

Si se quería que una máquina aprendiera a jugar al ajedrez, en el proceso de aprendizaje era necesario contar con expertos jugadores de ajedrez. De esa necesidad de contar con "expertos" se acabó extendiendo el término "**Sistema Experto**" para designar a los primeros programas de IA que se desarrollaron.

Siendo más concretos, la definición de Sistema Experto es:



Definición

Un sistema experto es un programa informático que se ha desarrollado a partir de nuestro conocimiento sobre una cuestión, y que consigue que el ordenador muestre un comportamiento equivalente al que tendría un experto humano sobre el mismo tema

En esencia se seguía un proceso con cuatro fases:

1. **Localizar al humano experto con conocimiento:** Según el caso particular para el que se quisiera crear esa IA, era necesario incorporar al equipo de desarrollo a una o varias personas expertas en la materia, para que aportaran todo el conocimiento en profundidad.
2. **Definir reglas:** Ese conocimiento humano había que convertirlo en reglas lo más sencillas posible, que relacionaran los diferentes casos y aspectos del conocimiento que se pretendía replicar con la IA.
3. **Informatizar:** Esas reglas había que traducirlas a lenguaje informático.
4. **Iterar:** Probar a ver si realmente la máquina se comportaba de forma "inteligente", buscar fallos, redefinir reglas, o mejorar la programación, y volver a probar. Así tantas veces como fuera necesario hasta que se pudiera considerar que la máquina actuaba igual de bien que el experto humano.

La Inteligencia Artificial Clásica quería "informatizar" modelos de conocimiento. Es decir, lograr convertir en programas informáticos capacidades humanas como "jugar al ajedrez", "detectar faltas de ortografía", "aprender un idioma"...

Pero esta manera de programar Inteligencia Artificial tiene bastantes limitaciones. **Sólo es asequible cuando el conocimiento o "inteligencia" que se quiere informatizar se basa en una relación de causalidad: Causa-Efecto.**

**Ejemplo:**

En el caso del **juego del ajedrez**:

1. Se busca a una o varias personas expertas jugadoras de ajedrez, que conozcan en profundidad el juego, sepan cuáles son las jugadas más características, etc.
2. Con la ayuda de estos expertos jugadores, los científicos definen todos los aspectos del juego de ajedrez, desde cómo es el tablero, las fichas, los movimientos, la jerarquía o relación de importancia entre las fichas, las posibles reacciones a los movimientos del contrario.
3. Los informáticos toman todas esas reglas y las traducen a lenguaje de programación.
4. Se comprueba que el ordenador sea capaz de jugar al ajedrez, sin equivocarse, y priorizando los movimientos que antes le permitan obtener la victoria. Si algo sale mal o se detectan fallos, hay que volver a revisar todo el proceso y mejorarlo (redefinir la forma de algunas normas, o la programación, etc).

Cada posible movimiento del contrario permite que la Inteligencia Artificial reaccione de diferentes maneras (moviendo tal o cual ficha). A su vez, este movimiento de la Inteligencia Artificial permite otras diferentes maneras de reaccionar por parte del contrario... Son lo que hemos mencionado más arriba: relaciones de causalidad. Cada acción tiene una serie de posibles reacciones (y la IA debe elegir una de ellas), que a su vez tienen otra serie de reacciones posibles (el contrario debe elegir una) y así sucesivamente. Gracias a la memoria de la computadora y la capacidad de cómputo es capaz de ver todas las posibles situaciones a 10, 20, 30... movimientos; e ir escogiendo los movimientos que con mayor probabilidad le puedan llevar a la victoria.

Cuando el conocimiento o "inteligencia" que se quiere informatizar se basa en una correlación (relaciones proporcionales entre todas las variables que intervienen) es prácticamente imposible definir y traducir a lenguaje informático todas esas reglas y relaciones. Para estos casos necesitamos otra manera de abordar la Inteligencia Artificial... que es la que se ha desarrollado posteriormente y veremos en los siguientes apartados.

Aprendizaje Automático (Machine Learning) (SAA)

El **Aprendizaje Automático (Machine Learning)** es una rama clave de la Inteligencia Artificial que permite a las máquinas aprender y mejorar su rendimiento en tareas específicas a través de la experiencia. En lugar de ser programadas explícitamente para realizar una tarea, las máquinas utilizan datos para aprender patrones y tomar decisiones informadas. El Aprendizaje Automático se ha convertido en una herramienta poderosa en una variedad de aplicaciones, desde reconocimiento de voz y visión por computadora hasta recomendaciones de productos y diagnósticos médicos.

**Importante**

Es importante entender que el Aprendizaje Automático es una rama de la IA, aunque en la actualidad ha adquirido mucha importancia y se utiliza en prácticamente todos los proyectos de IA. De manera que hoy cuando hablamos de Inteligencia Artificial en realidad estamos hablando de esta rama concreta (**el todo por la parte**).

El Machine Learning o Aprendizaje Autónomo (Automático) a su vez ha evolucionado en estos pocos años que lleva desarrollándose: Inicialmente se focalizaba en lograr que la máquina aprendiera basándose en datos, a través de estudiar el reconocimiento de patrones (casos similares entre el total de elementos del data set o base de datos). Actualmente se centra más bien en "resolver" problemas prácticos que en "aprender", aunque evidentemente "aprende" (pero el aprendizaje como tal ya no es el foco, sino el resultado obtenido). Al reconocimiento de patrones que ya se usaba desde el principio añade ahora lo que conocemos como el razonamiento probabilístico, la estadística y la recuperación de datos.

DEFINICIONES DE APRENDIZAJE AUTOMÁTICO

Arthur Samuel (que trabajó para IBM) en 1959 describía el Aprendizaje Automático como:

**Definición**

El campo del estudio que da a las computadoras la capacidad de aprender sin ser programadas explícitamente

Esta es una definición antigua e informal respecto a lo que hoy en día entendemos por Machine Learning.

Tom Mitchell (profesor en la Universidad de Carnegie Mellon) ha ofrecido una definición más moderna:

Definición

Se dice que un programa de computadora aprende de la experiencia E con respecto a alguna clase de tareas T y medida de rendimiento P, si su desempeño en las tareas en T medido por P mejora con la experiencia E

Ejemplo

Jugar a las damas.

- E es la experiencia de jugar muchas partidas de damas.
- T es la tarea de jugar a las damas.
- P es la probabilidad de que el programa gane la partida actual.

A medida que la máquina "observa" el desarrollo de cada partida gana experiencia. Gracias a esta experiencia acaba siendo capaz de realizar la tarea (jugar a las damas) por sí misma. Y además va comprobando el rendimiento obtenido en cada partida (si gana o no gana, en cuántos movimientos, etc), por lo que va perfeccionando su capacidad de jugar de manera eficaz.

El Aprendizaje Automático consiste en un programa informático que analiza y aprende de los datos que le proporcionamos para decidir qué hacer con ellos y proporcionar respuestas. Genera reglas para, con eso que ha "aprendido", acelerar procesos, reconocer patrones, segmentar grupos (personas, hábitos, etc). Lo fundamental es que el "cómo aprende" es automático; nosotros sólo le tenemos que dar datos o ejemplos de partida.

La definición de Aprendizaje Automático más aproximada a lo que entendemos actualmente sería:

Definición

El Aprendizaje Automático (Machine Learning) es un proceso de adquisición de conocimiento de manera automática mediante la utilización de ejemplos (experiencia) de entrenamiento.

TIPOS DE APRENDIZAJE AUTOMÁTICO

Aprendizaje Supervisado

La característica fundamental del Aprendizaje Automático Supervisado es que dicho aprendizaje se realiza **a partir de datos que ya han sido etiquetados previamente**.

¿Qué queremos decir con datos etiquetados? Pues que al programa que va a "aprender" le proporcionamos los datos indicando sus características (bien las de entrada, bien las de salida). Por ejemplo, si queremos que un programa de IA sea capaz de distinguir en qué fotos aparece un perro, al proporcionarle fotos para el aprendizaje (datos de entrada) ya le decimos en cuáles aparecen gatos, en cuáles perros y en cuáles patos... Podemos decir que "supervisamos" el aprendizaje dándole pistas al programa de Inteligencia Artificial.

En realidad el término correcto que debemos emplear es el de instancias: que son cada uno de los elementos que forman el conjunto de datos (en el ejemplo, cada foto), se componen de una serie de campos de características o atributos (en el ejemplo, aparecer gato, aparecer perro, aparecer pato...) y un campo objetivo (en el ejemplo, aparecer perro), que es el que se encuentra etiquetado en los datos de entrenamiento. El objetivo de este tipo de aprendizaje es extraer un conjunto de reglas que permitan predecir el campo objetivo para nuevos casos de estudio.

Los problemas de Aprendizaje Supervisado **se dividen en dos categorías: Regresión y Clasificación**. La diferencia entre estas dos categorías radica en el tipo de campo objetivo (lo que queremos que la Inteligencia Artificial nos dé como respuesta), que es numérico en el caso de la Regresión y categórico en el caso de la Clasificación.

Aprendizaje no Supervisado

En este tipo de aprendizaje **no se requiere un etiquetado previo** de las instancias, pues **el objetivo es encontrar relaciones de similitud, diferencia o asociación** en el conjunto de datos.

Es decir, que no "le decimos" a la Inteligencia Artificial qué estamos buscando, ni cuál es el dato concreto sobre el que queremos que haga una predicción. Asumimos que hay ciertos tipos de relación y dependencias entre los diversos datos, pero queremos que sea la Inteligencia Artificial la que encuentre esas relaciones. En muchas ocasiones nos llevamos sorpresas, cuando la IA nos muestra semejanzas entre datos que nos han pasado desapercibidas a los humanos.

Como hemos dicho, el objetivo es que la IA encuentre relaciones de tres tipos:

- Similitudes.
- Diferencias.
- Asociaciones.

Aprendizaje por Refuerzo

Existe también el Aprendizaje por Refuerzo, en el que **el objetivo es aprender cómo mapear situaciones o acciones para maximizar una cierta recompensa**. Se trata de programar agentes mediante premio y castigo sin necesidad de especificar cómo realizar la tarea.

Uno de los casos más conocidos de refuerzo automático es el cuando en la empresa Deep Mind lograron "enseñar" a jugar al Arkanoid ¿Te acuerdas? lo vimos en el punto [Historia de la IA](#). Lo hicieron con este modelo de Aprendizaje. La IA solo conocía los parámetros básicos de movimiento, y los "premios" (puntos por romper bloques, puntos por tardar lo menos posible en terminar la partida) y "castigos" (finalizar la partida sin puntos si se perdía la pelota por el extremo inferior de la pantalla).

En este tipo de problemas lo más importante es definir y programar las condiciones que deben cumplirse (las reglas del juego, qué se puede hacer y cómo interactúan unos elementos con otros). Por ejemplo, en el siguiente vídeo, podemos ver cómo una serie de personajes digitales han "aprendido" a caminar, correr y sortear obstáculos. Se ve claramente que los programadores han sido precisos para que "los brazos" se mantengan articulados al cuerpo, igual que las "patas". Pero no parece que hayan especificado mucho sobre la gravedad o sobre "el cansancio" que supone correr con los brazos hacia arriba.



Redes Neuronales Artificiales (SAA y PIA)

CONCEPTOS BÁSICOS Y FUNCIONAMIENTO

Las redes neuronales artificiales están inspiradas en la estructura y funcionamiento del cerebro humano.

i Definición

Consisten en una colección de nodos interconectados (neuronas artificiales) organizados en capas que transmiten señales entre ellas. Cada neurona recibe entradas ponderadas, las procesa mediante una función de activación y produce una salida que se envía a otras neuronas.

El proceso de aprendizaje en las redes neuronales se basa en ajustar los pesos de las conexiones entre las neuronas para que el modelo pueda realizar predicciones precisas y generalizadas en nuevos datos.

APLICACIONES Y ARQUITECTURAS COMUNES

Las redes neuronales artificiales tienen una amplia gama de aplicaciones, algunas de las cuales incluyen:

- **Reconocimiento de Patrones:** Clasificación de datos en diferentes categorías, como en reconocimiento de imágenes y diagnóstico médico.
- **Procesamiento de Lenguaje Natural:** Análisis de texto y generación de texto coherente, como en traducción automática y generación de subtítulos.
- **Juegos y Control de Robots:** Entrenamiento de agentes para jugar juegos y controlar robots mediante técnicas de aprendizaje por refuerzo.

Las arquitecturas comunes de redes neuronales incluyen:

- **Redes Neuronales Feedforward:** Son las más básicas, donde las señales solo se transmiten en una dirección, desde la entrada hasta la salida, sin ciclos.
- **Redes Neuronales Recurrentes:** Tienen conexiones cíclicas que permiten el procesamiento de secuencias de datos, lo que las hace adecuadas para tareas de procesamiento de lenguaje natural y series de tiempo.

Algoritmos Genéticos

PRINCIPIOS BÁSICOS Y FUNCIONAMIENTO

Definición

Los algoritmos genéticos son una clase de algoritmos de computación evolutiva inspirados en el proceso de selección natural. Utilizan principios biológicos como la reproducción, mutación y selección para buscar soluciones óptimas en problemas de optimización y búsqueda heurística.

En un algoritmo genético, se crea una población inicial de soluciones candidatas, y luego se evalúa su aptitud en función de una función objetivo. Las soluciones con mayor aptitud tienen una mayor probabilidad de ser seleccionadas para reproducirse y producir descendencia mediante operaciones de cruce y mutación. Este proceso se repite a lo largo de generaciones, buscando converger hacia una solución óptima.

OPTIMIZACIÓN Y BÚSQUEDA HEURÍSTICA

Los algoritmos genéticos son ampliamente utilizados en problemas de optimización y búsqueda heurística, como:

- **Problemas de Optimización:** Encontrar la mejor solución posible en un espacio de búsqueda grande, como en el diseño de redes de transporte, rutas de vehículos y programación de horarios.
- **Diseño y Aprendizaje de Parámetros:** Optimización de parámetros en modelos de aprendizaje automático y redes neuronales.

Lógica Difusa

FUNDAMENTOS Y USO EN SISTEMAS DE TOMA DE DECISIONES

La lógica difusa es una extensión de la lógica clásica que permite manejar incertidumbre y vaguedad en los datos. A diferencia de la lógica binaria (verdadero/falso), la lógica difusa utiliza grados de verdad entre 0 y 1, lo que permite representar y razonar con conceptos imprecisos.

La lógica difusa es especialmente útil en sistemas de toma de decisiones, donde las condiciones y resultados pueden ser vagos o subjetivos. Los conjuntos difusos y las reglas de inferencia difusa se utilizan para modelar y resolver problemas con datos inciertos.

VENTAJAS Y DESVENTAJAS FRENTE A LA LÓGICA CLÁSICA

Las ventajas de la lógica difusa incluyen:

- **Tratamiento de Incertidumbre:** Permite manejar datos imprecisos o ambiguos en un contexto más cercano a la forma en que los humanos toman decisiones.
- **Adaptabilidad:** Es útil para modelar sistemas complejos y no lineales donde las relaciones son difíciles de expresar con precisión.

Sin embargo, también tiene algunas desventajas:

- **Complejidad Computacional:** El procesamiento de la lógica difusa puede ser más complejo y costoso en términos computacionales en comparación con la lógica clásica.
- **Interpretación de Resultados:** La interpretación de los resultados difusos puede ser subjetiva y depender del contexto, lo que dificulta la comparación entre diferentes sistemas.

Campos de Aplicaciones de la Inteligencia Artificial

Visión por Computadora

La visión por computadora es un campo de la inteligencia artificial que se enfoca en **enseñar a las máquinas a interpretar y comprender el mundo visual**, permitiéndoles analizar y procesar imágenes y videos. Esta área ha experimentado un rápido avance en los últimos años gracias a los avances en técnicas de Aprendizaje Profundo y el aumento de la capacidad computacional. La visión por computadora tiene una amplia gama de aplicaciones en diversos campos, desde la medicina y la industria hasta la seguridad y el entretenimiento.

Los nuevos desarrollos de reconocimiento de imagen y visión artificial no han tenido un origen único y concreto, sino que se han ido conformando por la aportación de investigadores e ingenieros que han compartido sus ideas, como es el caso de **Yann Lecun**, que ideó **LeNet** usando, ya a **finales 90**, redes convolucionales para el reconocimiento de dígitos manuscritos.

El lanzamiento de **ImageNet**, abierto y gratuito, por parte de **Fei Fei Li**, que ya ha alcanzado más de 14 millones de imágenes, supuso un gran impulso al desarrollo de nuevas aplicaciones de visión artificial.



Curiosidad

Tú mismo puedes descargarte un dataset reducido (¡aunque es de 166 GB!) para probar en [Kaggle](#).

Cada vez más y mejores datasets de imágenes etiquetadas, y un mayor conocimiento y dominio de redes convolucionales, han revolucionado el campo de la visión computacional, que ha pasado de ser una cuestión de más resolución o renderizados 3D, a una cuestión más cognitiva. Se ha conseguido crear modelos que realmente entienden qué están viendo.

La visión artificial automatiza la extracción, el análisis, la clasificación y la comprensión de la información útil a partir de los datos de las imágenes. Los datos de la imagen adoptan muchas formas, como las siguientes:

- Imágenes individuales
- Secuencias de video
- Visualizaciones de varias cámaras
- Datos tridimensionales

APLICACIONES DE VISIÓN POR COMPUTADORA

Vigilancia

La utilización de cámaras para sistemas de vigilancia y seguridad, se ha extendido de forma generalizada en nuestra sociedad actual. Pero, en algunos casos, estas cámaras tienen el plus de formar parte de un sistema de inteligencia artificial.

Esta aplicación de los sistemas de reconocimiento de imagen son bastante controvertidos, pues plantean muchas dudas éticas respecto a la libertad fundamental de las personas. Es una herramienta muy útil y potente, y puede servir para beneficiar al ser humano tanto como para perjudicarlo. Lo bueno es que la comunidad en torno a la inteligencia artificial va descubriendo e ideando formas sencillas de evitar o combatir usos deshonestos de este tipo de herramientas.

En la mayoría de los casos, el software hace cálculos agregados de lo que "ve" y devuelve valores de ciertos indicadores, en vez de la imagen o fragmento de grabación con los datos personales. Solo en países donde hay regímenes autoritarios se mantienen prácticas que atentan contra los derechos de los ciudadanos.

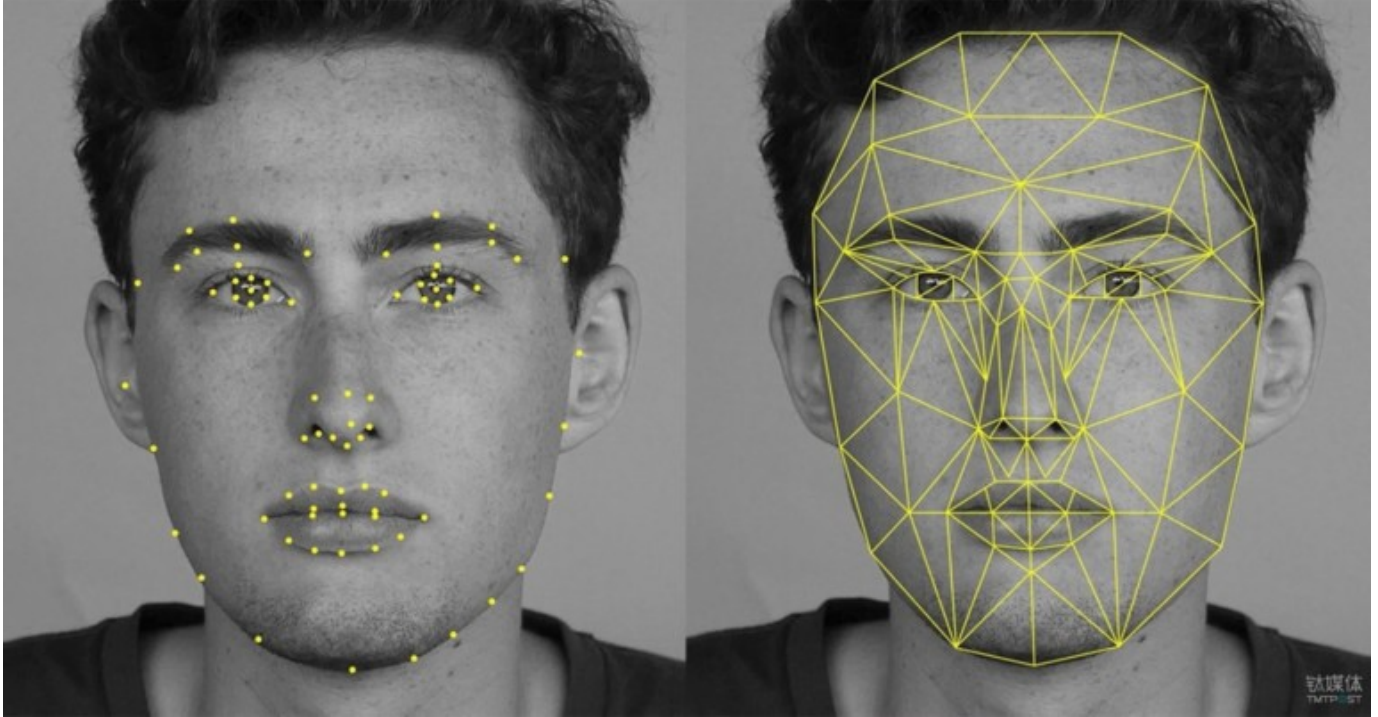
Entre las aplicaciones concretas de este tipo de sistemas, a parte de las obvias por parte de la policía o sistemas de seguridad de organizaciones, están:

- Vigilancia y control del tráfico en las ciudades.
- Cuidado de personas mayores.
- Detección de infracciones de reglas sanitarias en la industria (especialmente en la industria alimentaria).
- Monitorización del uso de infraestructuras críticas o adscritas a normas de utilización.
- Monitorización de funcionamiento y estados en líneas de producción.
- Esto son solo algunos ejemplos, pero cualquier proceso o sistema en el cual se puedan detectar anomalías a través de la imagen, sería un buen candidato para aplicar este tipo de solución.

Incluso la inteligencia artificial puede ayudar a hacer más respetuosas con la privacidad ciertas aplicaciones y herramientas que ya se estaban utilizando, como el caso del software [Cherry Home de la empresa AvantGuard](#).

Reconocimiento Facial

Un analizador facial es un software que identifica o confirma la identidad de una persona a partir del rostro. Funciona mediante la identificación y medición de los rasgos faciales en una imagen. El reconocimiento facial puede identificar rostros humanos en imágenes o videos, determinar si el rostro que aparece en dos imágenes pertenece a la misma persona o buscar un rostro entre una gran colección de imágenes existentes. Los sistemas de seguridad biométricos utilizan el reconocimiento facial para identificar de forma exclusiva a las personas durante la incorporación o el inicio de sesión de los usuarios, así como para reforzar la actividad de autenticación de estos. Los dispositivos móviles y personales también utilizan con frecuencia la tecnología de los analizadores faciales para proteger los dispositivos.



Se pueden detectar los datos faciales tanto en los perfiles frontales como en los laterales del rostro. El sistema de reconocimiento facial analiza la imagen del rostro. Asigna y lee la geometría del rostro y las expresiones faciales. Identifica los puntos de referencia faciales que son clave para distinguir un rostro de otros objetos. La tecnología de reconocimiento facial por lo general busca lo siguiente:

- Distancia entre los ojos
- Distancia de la frente a la barbilla
- Distancia entre la nariz y la boca
- Profundidad de las cuencas oculares
- Forma de los pómulos
- Contorno de los labios, las orejas y la barbilla

El sistema convierte los datos de reconocimiento facial en una cadena de números o puntos denominada huella facial. Cada persona tiene una huella facial única, de forma similar a una huella dactilar. La información utilizada por el reconocimiento facial también se puede utilizar a la inversa para reconstruir digitalmente el rostro de una persona.

El reconocimiento facial puede identificar a una persona al comparar los rostros de dos o más imágenes y evaluar la probabilidad de que coincidan. Por ejemplo, puede verificar que el rostro mostrado en una autofoto tomada con la cámara de un móvil coincide con el rostro de una imagen de un documento de identidad emitido por el gobierno, como un permiso de conducir o un pasaporte, así como verificar que el rostro que aparece en la autofoto no coincide con un rostro de un conjunto de rostros capturados previamente.



Ejemplo

Aplicación de Visión por Computadora:

Un ejemplo práctico de aplicación de visión por computadora es el reconocimiento facial utilizado en aplicaciones de seguridad y desbloqueo de dispositivos. En este caso:

- **Entrada:** La entrada es una imagen o un video que contiene rostros humanos.
- **Procesamiento de Imagen:** El sistema de visión por computadora procesa la imagen para detectar y extraer características clave del rostro, como ojos, nariz, boca, etc.
- **Aprendizaje Automático:** Las características del rostro se utilizan como entrada para un modelo de aprendizaje automático previamente entrenado. El modelo clasifica las características y compara con una base de datos de rostros previamente almacenados.
- **Salida:** Como resultado, el sistema identifica o verifica la identidad del individuo y permite el acceso o desbloqueo según los resultados.

Conducción autónoma.

El sistema de conducción autónoma de vehículos implica varias tareas y subsistemas, pero uno de los más importantes, es el de visión artificial, pues la mayoría de las decisiones de seguridad del coche se basan en lo que captan las cámaras.

La cuestión crítica en estos sistemas, es el reconocimiento de señales de tráfico u objetos/obstáculos alrededor del vehículo a una velocidad relativamente alta. Por ejemplo, si el coche debe parar porque hay una persona cruzando la carretera, el sistema de visión debe captar la imagen con antelación suficiente como para frenar a una distancia también suficiente.



Captar cómo son las líneas de la carretera para ir girando lo que corresponda, también requiere ir captando esas variaciones de trayectoria suficientemente rápido, pues en carretera es muy común ir a velocidades altas. De hecho, hay sistemas que, a partir de cierta velocidad, no permiten usar la función de conducción autónoma.



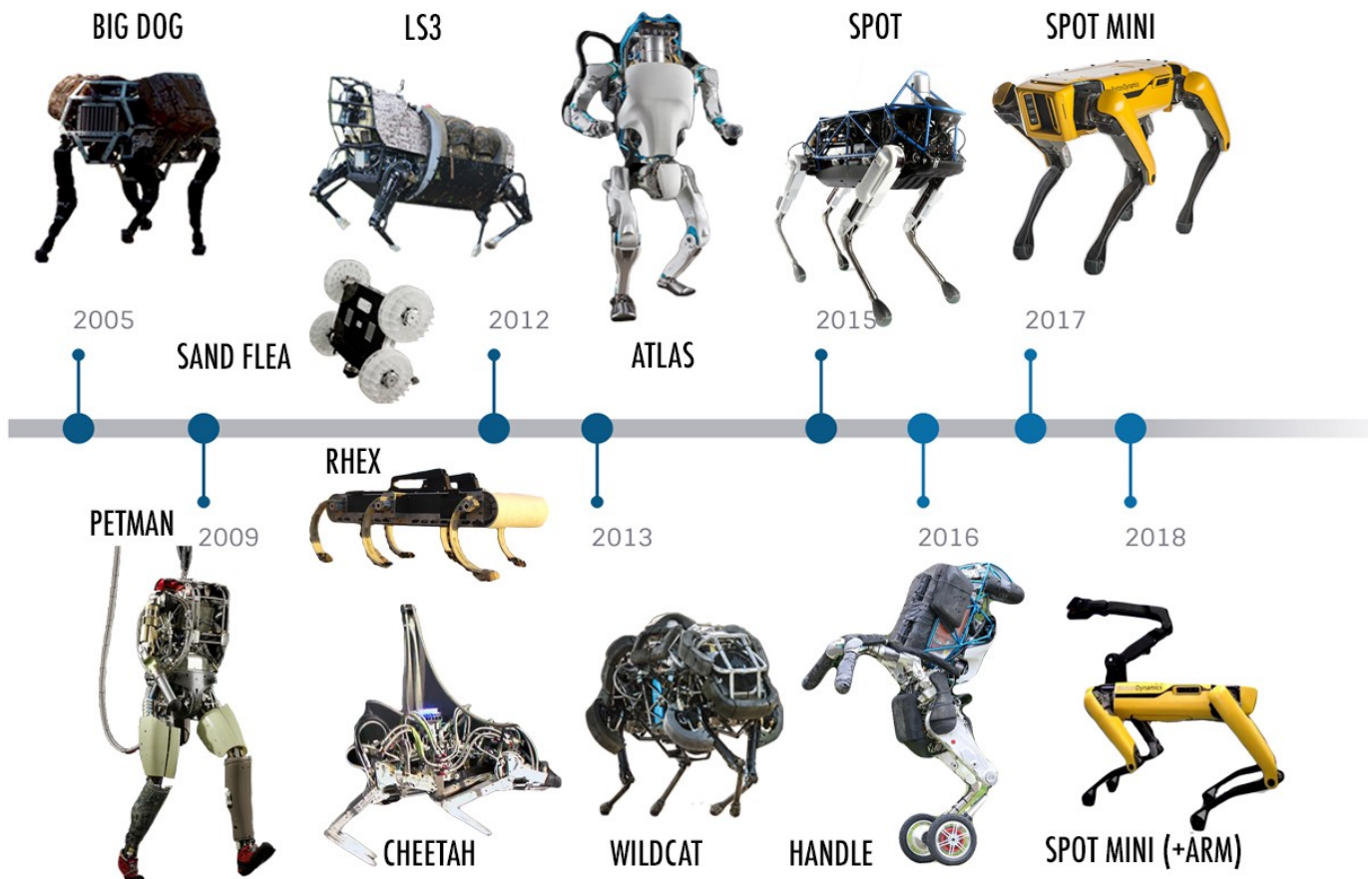
Curiosidad

Si quieres conocer mejor la clasificación de niveles de conducción autónoma y cómo se relaciona el ámbito de la visión artificial con ellos, te recomendamos leer el artículo "[Autonomous Vehicles Are Driving Computer Vision Into the Future](#)".

Sistema auxiliar en robots.

Los robots son sistemas complejos que suelen ejecutar una serie de tareas en el mundo físico en base a una secuencia programada. En la industria, se han estado utilizando sistemas robóticos desde hace muchos años. Pero este campo también ha ido evolucionando, y la inteligencia artificial está aportando grandes avances que causan un importante impacto en el alcance de estos sistemas.

BOSTON DYNAMICS



Un sistema robótico tiene tres partes fundamentales:

- Sensores o entradas.
- Sistema de control.
- Actuadores.

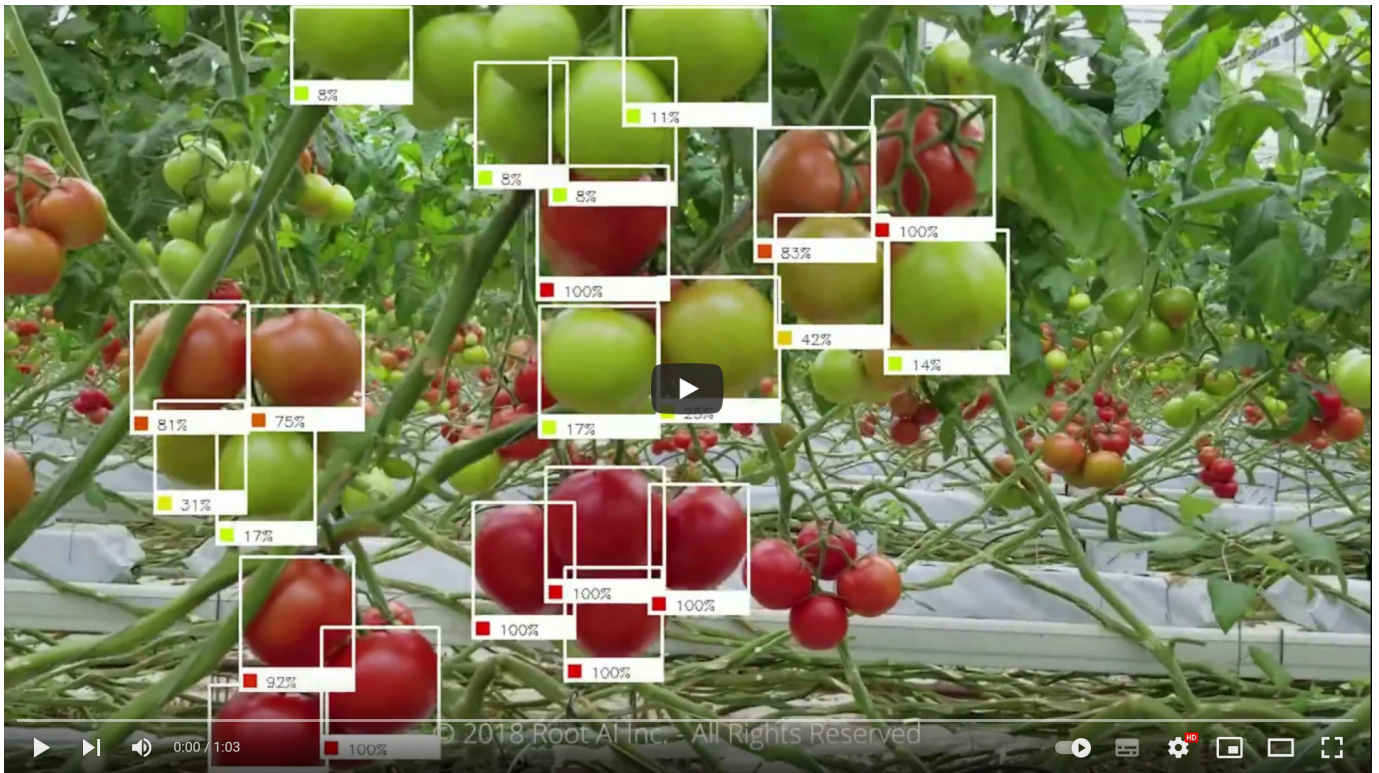
Más adelante hablaremos de cómo los sistemas de control se han beneficiado de la inteligencia artificial, pero aquí nos detenemos en el módulo de visión artificial como parte del conjunto de sensores que aportan los estímulos o la información que el sistema robótico va a necesitar para la toma de decisiones.

El sistema de visión artificial de un robot, le permite detectar objetos y posicionarse a sí mismo o a objetos que transporta en función de lo que está viendo. Esto es un gran avance respecto a otros sistemas de posicionamiento, que exigían constantes tareas de calibración y ralentizaban las tareas del robot.

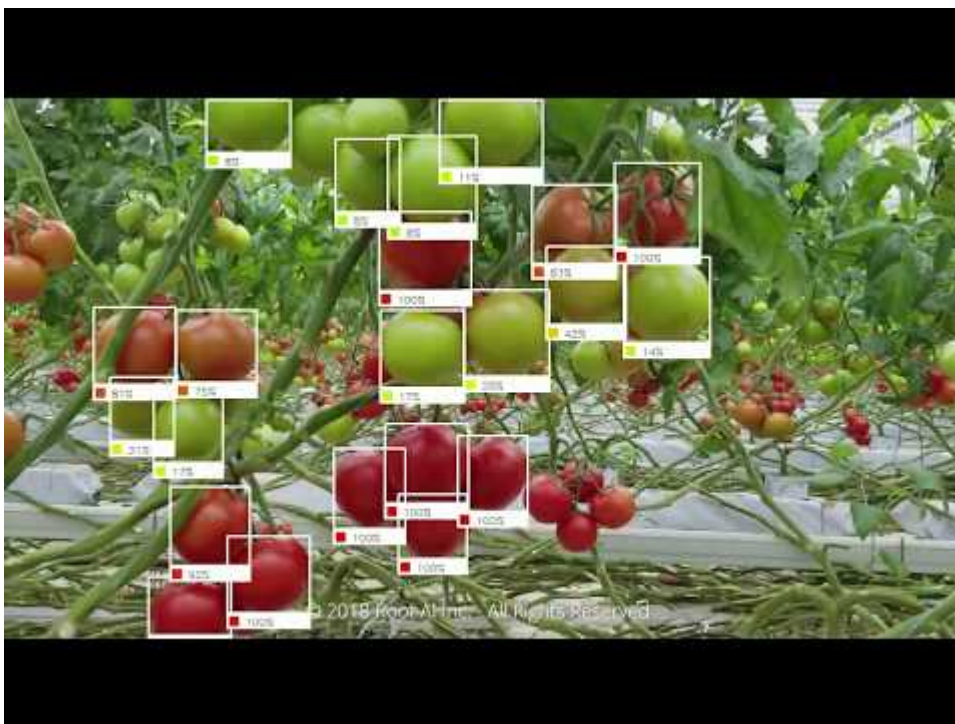
<https://bostondynamics.com/videos/>

Reconocimiento de Objetos

El reconocimiento de objetos implica identificar y localizar objetos específicos en imágenes o videos. Algunos ejemplos incluyen reconocimiento de vehículos en carreteras, detección de peatones en sistemas de asistencia al conductor y clasificación de objetos en aplicaciones de etiquetado automático.



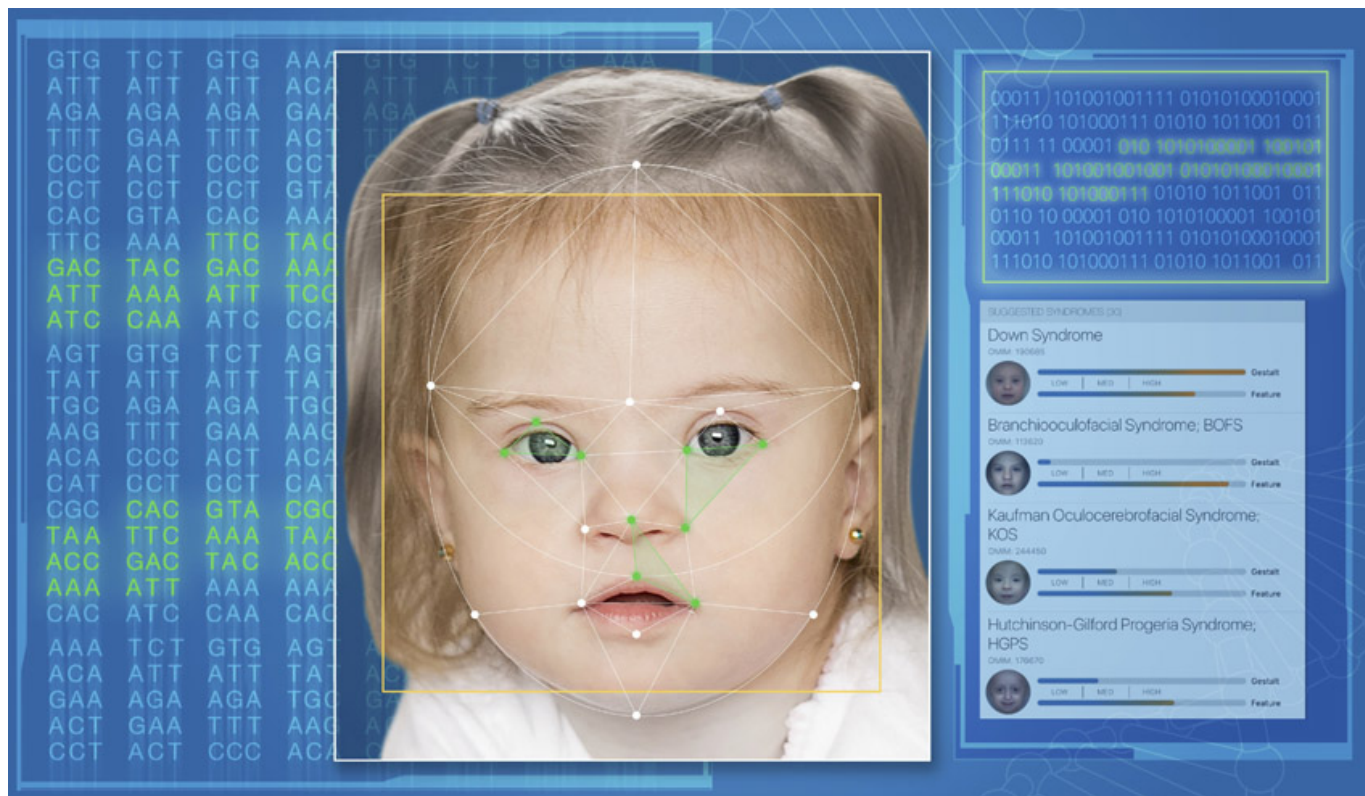
En la industria agroalimentaria, la capacidad de visión inteligente es de vital importancia, porque constituye una parte decisiva de cara a que se obtenga un buen producto o una buena cosecha. En algunos casos, el robot sabe distinguir, mejor que el humano, si una fruta está en su momento óptimo de cosecha.



Detección y diagnóstico

En el campo de la medicina, la inteligencia artificial está teniendo un impacto de muchísimo valor, porque no solo consigue mejorar la vida de las personas, es que, literalmente, en algunos casos consigue salvar vidas. Es el caso de herramientas de inteligencia artificial para la detección y diagnóstico de enfermedades a través de imágenes.

Existen muchos programas informáticos de apoyo y ayuda al diagnóstico que han ido mejorando su aprendizaje a través de su uso repetido y continuado. Actualmente existen diferentes tipos de software que se pueden aplicar a diferentes grupos de enfermedades como MYCIN/MYCIN II para enfermedades infecciosas, CASNET para oftalmología, PIP para enfermedades renales o AI/RHEUM para enfermedades reumatológicas. La empresa FDNA a través de su software de reconocimiento facial Face2Gene® es capaz de apoyar o sospechar el diagnóstico de más de 8.000 enfermedades raras, con un reciente ensayo clínico desarrollado en Japón con buenos resultados.



En el campo del procesamiento y la interpretación de imágenes para el diagnóstico, la IA ofrece algoritmos que mejoran la calidad y la precisión del diagnóstico ya que los métodos de IA son excelentes para reconocer automáticamente patrones complejos en los datos de imágenes, elimina ruido en las imágenes ofreciendo una mayor calidad y permite establecer modelos tridimensionales a partir de imágenes de pacientes concretos.

Investigadores de IBM publicaron una investigación en torno a un nuevo modelo de IA que puede predecir el desarrollo del cáncer de mama maligno, con tasas comparables a las de los radiólogos humanos. Este algoritmo aprende y toma decisiones tanto de datos de imágenes como del historial de la paciente, pudo predecir correctamente el desarrollo del cáncer de mama en el 87% de los casos analizados, y también pudo interpretar el 77% de los casos no cancerosos. Este modelo podría algún día ayudar a los radiólogos a confirmar o negar casos positivos de cáncer de mama. Si bien los falsos positivos pueden causar una enorme cantidad de estrés y ansiedad indebidos, los falsos negativos a menudo pueden obstaculizar la detección temprana y el tratamiento posterior de un cáncer. Cuando se puso a prueba frente a 71 casos diferentes que los radiólogos habían determinado originalmente como «no malignos», pero que finalmente terminaron siendo diagnosticados con cáncer de mama dentro del año, el sistema de IA pudo identificar correctamente el cáncer de mama en el 48% de las personas (48% de los 71 casos), que de lo contrario no se habrían detectado (Fuente: "[La inteligencia artificial y sus aplicaciones en medicina](#)")

Procesos creativos

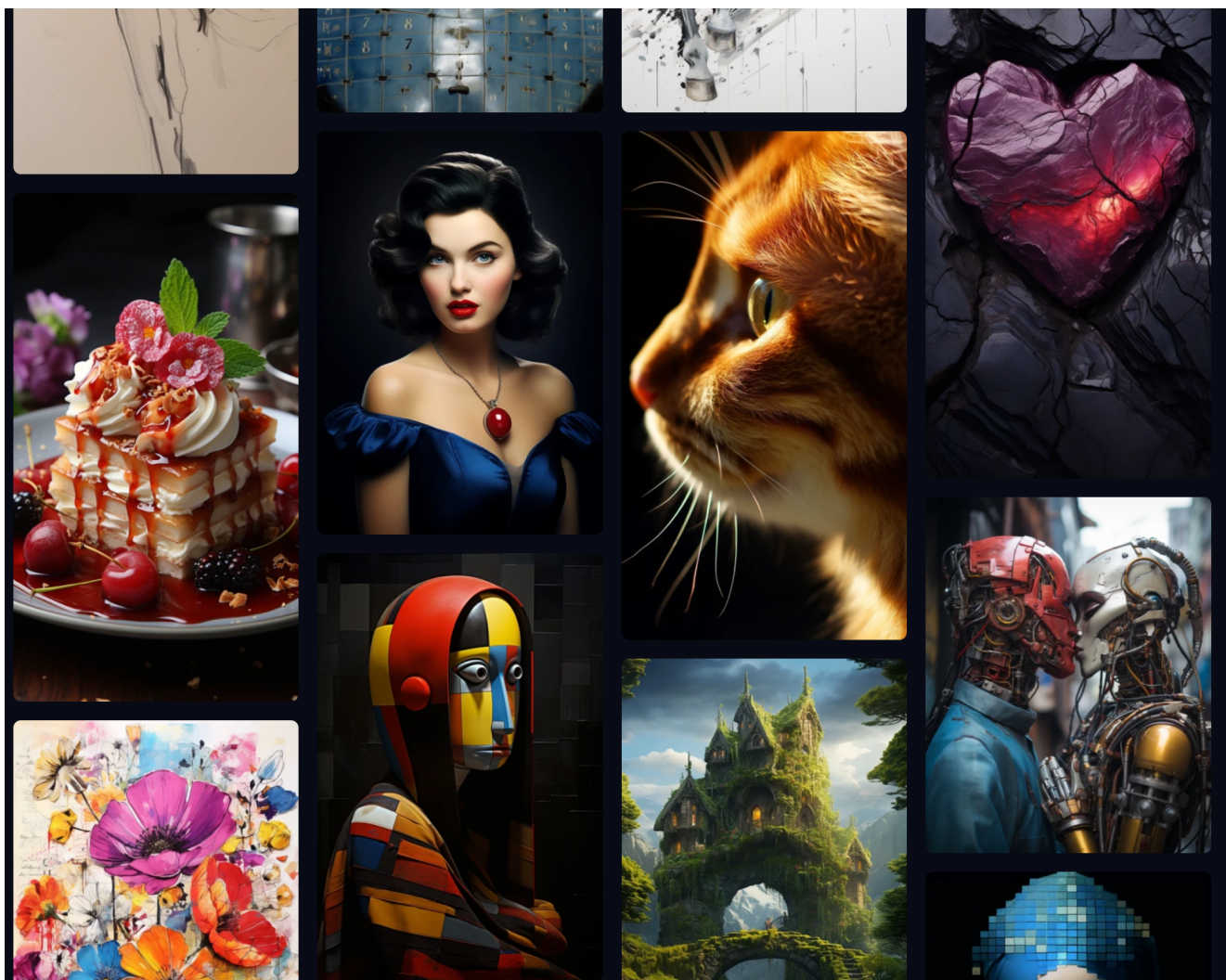
Uno de los grandes e inesperados avances de la computación de la década pasada, ha sido el de los modelos generativos: las redes GAN para el campo de la imagen y los modelos de generación de texto basados en Transformers.

- **Deep Dream:** En 2015 apareció DeepDream, un modelo de generación de imágenes creado por Google. El software Deep Dream fue desarrollado para el imageNet large scale visual recognition challenge (ILSVRC). Este era un desafío reto, propuesto a diferentes equipos de investigación, que consistió en crear un sistema de reconocimiento de objetos y su localización dentro de una misma imagen, aparte de su detección inmediata. En este Desafío se adjudicó a Google el primer premio en el año 2014, logrado gracias al uso del entrenamiento de redes neuronales. En junio de 2015 Google publicó la

investigación, y tras esto hizo su código fuente abierto utilizado para generar las imágenes en un IPython notebook. Con esto se permitió que las imágenes de la red neuronal pudiesen ser creadas por cualquiera. Actualmente, se puede utilizar la aplicación de manera online en la web [DeepDreamGenerator](https://deepdreamgenerator.com/). Básicamente, el algoritmo procesa la imagen dada identificando sus elementos, para utilizar una transferencia de estilos respetando la identidad esencial de la imagen original.

- **Gaugan:** GauGAN es una herramienta con la que se pueden crear paisajes falsos partiendo de un boceto. Este software de Nvidia, hace uso de una red de confrontación generativa (GAN), basado en una técnica denominada "normalización espacialmente adaptativa" que es capaz de generar imágenes realistas a partir de un determinado diseño semántico, controlado por el usuario con el uso de un programa de edición de imágenes, donde cada color actúa como representación de un tipo de objeto, material o ambiente. Se puede utilizar desde su interfaz web abierta.
- **DALL·E:** Una de las más recientes incorporaciones al ámbito de "cosas increíbles que la inteligencia artificial puede hacer ya" es el modelo de generación de imágenes de openAI. Se trata de una implementación multimodal de GPT3. El algoritmo interpreta una descripción escrita que se le proporciona a través de su interfaz, y genera la imagen correspondiente en base a lo que sus 12 mil millones de parámetros del modelo GPT3 han interpretado de la entrada de texto dada. En concreto, se utiliza un proceso llamado "diffusion" que parte de una imagen de ruido aleatoria y va alterando dicho esquema de puntos en función de que vaya reconociendo distintos patrones de objetos cuyas palabras clave se le han dado en la descripción.
- **MidJourney:** Midjourney es un laboratorio independiente de investigación y el nombre de un programa de inteligencia artificial con el cual sus usuarios pueden crear imágenes a partir de descripciones textuales, similar a Dall-e de OpenAI y al Stable Diffusion de código abierto.

La herramienta funcionó bajo versión de beta cerrada hasta que el 13 de julio de 2022 el laboratorio anunció el comienzo de una beta abierta. El equipo de Midjourney está dirigido por David Holz, cofundador de Leap Motion. Midjourney emplea un modelo de negocio *freemium*, con un nivel gratuito limitado y niveles de pago que ofrecen un acceso más rápido, mayor capacidad y funciones adicionales. Los usuarios pueden crear obras de arte con Midjourney dando órdenes a un bot alojado en Discord, ya sea enviando mensajes directos o invitando a dicho bot a un servidor de terceros.



Procesamiento del Lenguaje Natural (PLN)

El Procesamiento del Lenguaje Natural (PLN) es una rama de la Inteligencia Artificial que se enfoca en permitir a las máquinas entender y procesar el lenguaje humano en forma escrita o hablada. El PLN permite que las computadoras analicen, comprendan y generen texto de manera similar a como lo hacen los seres humanos. Esta tecnología ha sido fundamental en el desarrollo de asistentes virtuales, traducción automática, análisis de sentimientos y muchas otras aplicaciones útiles en el ámbito empresarial y cotidiano.

Tratar computacionalmente una lengua implica un proceso de modelización matemática. Los ordenadores solo entienden de bytes y dígitos y los informáticos codifican los programas empleando lenguajes de programación como C, Python o Java.

Los lingüistas computacionales se encargan de la tarea de "preparar" el modelo lingüístico para que los ingenieros informáticos lo implementen en un código eficiente y funcional.

Éstos son algunos de los **componentes** del procesamiento del lenguaje natural. No todos los análisis que se describen se aplican en cualquier tarea de PLN, sino que depende del objetivo de la aplicación.

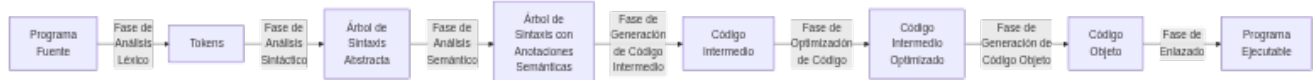
- **Análisis morfológico o léxico.** Consiste en el análisis interno de las palabras que forman oraciones para extraer lemas, rasgos flexivos, unidades léxicas compuestas. Es esencial para la información básica: categoría sintáctica y significado léxico.
- **Análisis sintáctico.** Consiste en el análisis de la estructura de las oraciones de acuerdo con el modelo gramatical empleado (lógico o estadístico).
- **Análisis semántico.** Proporciona la interpretación de las oraciones, una vez eliminadas las ambigüedades morfosintácticas.
- **Análisis pragmático.** Incorpora el análisis del contexto de uso a la interpretación final. Aquí se incluye el tratamiento del lenguaje figurado (metáfora e ironía) como el conocimiento del mundo específico necesario para entender un texto especializado.

Un análisis morfológico, sintáctico, semántico o pragmático se aplicará dependiendo del objetivo de la aplicación. Por ejemplo, un conversor de texto a voz no necesita el análisis semántico o pragmático. Pero un sistema conversacional requiere información muy detallada del contexto y del dominio temático.



Curiosidad

No te recuerda a algo?



Estas son las fases de un Compilador

APLICACIONES DE PROCESAMIENTO DEL LENGUAJE NATURAL

Asistentes Virtuales y Chatbots

Los asistentes virtuales como Siri, Google Assistant y Alexa utilizan PLN para entender y responder a las consultas y comandos de voz de los usuarios. Los chatbots en aplicaciones de servicio al cliente y soporte técnico también emplean PLN para ofrecer respuestas automáticas y contextuales a las preguntas de los usuarios.

Esta generación actual de asistentes están habilitados para llevar a cabo tareas dentro del sistema que las aloja e, incluso, a través de webhooks, en otros sistemas que cuenten con las políticas de acceso correspondientes. De esta forma, los asistentes virtuales más avanzados pueden encargarse de una pizza, comprar online un producto entre varias sugerencias o incluso controlar la domótica de nuestra casa.

Generación de textos

El área del marketing y comunicación ha sido de los primeros que ha abrazado la inteligencia artificial para automatizar y mejorar muchos de sus procesos. Y entre las distintas tareas que puede llevar a cabo la IA, la generación de textos empezó a tener una aplicación comercial clara como herramienta para crear mensajes publicitarios, publicaciones de marketing de contenidos o incluso lemas de producto. Por eso, durante estos años han ido surgiendo una gran cantidad de servicios de este tipo. Siempre para generar breves fragmentos, y con una serie de requerimientos, como incluir las palabras clave.

Pero, recientemente, están surgiendo modelos mucho más generales y con una versatilidad mayor en el tipo de textos, el tema a tratar, el idioma, etc. Es el caso de los modelos BERT, GPT3 y Bloom. El primero, creado por Google en 2018, integrado en el algoritmo de búsqueda de Google y publicado con licencia de código abierto, no tiene una aplicación directa de generación de textos, pero tiene la misma arquitectura que los modelos que se están utilizando en ese campo: los Transformers.

Interpretación de textos

En el campo del análisis de lenguaje natural existen aplicaciones basadas en voz, como los asistentes virtuales que tenemos encima de la mesa, en el móvil o en el ordenador, o las aplicaciones basadas en texto. Ambas utilizan la misma base, y después, para añadir la habilitación oral, se utiliza un módulo de "Voz a Texto" y viceversa.

Las aplicaciones de PLN sirven para extraer información valiosa de los datos sin estructurar basados en textos y para acceder a la información extraída con el objetivo de generar una nueva comprensión de esos datos. Algunos ejemplos de aplicación serían:

- Traducción automática de idiomas.
- Chatbots.
- Opinión de los clientes: Se usa el análisis de entidades para identificar y etiquetar campos en documentos y canales. De esta forma, se pueden conocer mejor las opiniones de los clientes y obtener información valiosa sobre los productos y la experiencia de usuario.
- Comprender los recibos y las facturas: Extrae entidades para identificar las entradas más comunes de los recibos y facturas, como las fechas o los precios, y entiende la relación entre la solicitud y el pago.
- Análisis de documentos: Utiliza la extracción de entidades personalizada para identificar las entidades específicas de cada dominio en los documentos sin tener que invertir tiempo o dinero en análisis manuales.
- Clasificación de contenido general: Clasifica los documentos en función de las entidades más frecuentes, las entidades personalizadas de un dominio concreto o categorías generales disponibles (por ejemplo, deportes y entretenimiento).
- Análisis de tendencias: Agrega noticias con texto que permita a los profesionales del marketing extraer contenido relevante sobre sus marcas de noticias online, artículos y otras fuentes de datos.
- Sanidad: Mejora la documentación clínica, la investigación de minería de datos y los informes de registros automatizados para agilizar los ensayos clínicos.

El campo del procesamiento del lenguaje natural es considerado uno de los grandes retos de la inteligencia artificial ya que es una de las tareas más complicadas y desafiantes: ¿cómo comprender realmente el significado de un texto? ¿cómo intuir neologismos, ironías, chistes o poesía?



Ejemplos

Análisis de Sentimientos

El PLN es utilizado para analizar el contenido de opiniones, comentarios y reseñas de usuarios en línea y determinar si expresan sentimientos positivos, negativos o neutrales. Esto es útil para medir la satisfacción del cliente, realizar estudios de mercado y realizar análisis de reputación de marca.

Ejemplo de Aplicación de Procesamiento del Lenguaje Natural: Análisis de Sentimientos

Un ejemplo práctico de aplicación de Procesamiento del Lenguaje Natural es el análisis de sentimientos en comentarios de productos en línea. En este caso:

- **Entrada:** La entrada es un conjunto de comentarios escritos por usuarios sobre un producto específico.
- **Procesamiento de Lenguaje Natural:** El PLN procesa el texto para tokenizarlo (dividirlo en palabras), eliminar palabras irrelevantes (stopwords) y realizar lematización o extracción de raíces para reducir las palabras a su forma base.
- **Análisis de Sentimientos:** Se utilizan técnicas de análisis de sentimientos para asignar un valor de sentimiento (positivo, negativo o neutral) a cada comentario en función de las palabras y frases utilizadas.
- **Salida:** Como resultado, se obtiene un resumen del sentimiento general de los usuarios hacia el producto, lo que permite a las empresas identificar puntos fuertes y áreas de mejora, así como tomar decisiones basadas en la retroalimentación del cliente.

Análítica avanzada

Los Modelos Predictivos son un grupo de técnicas que, mediante los campos del aprendizaje automático, la recolección de datos históricos, el Big Data y el reconocimiento de patrones, pretende dar una predicción de resultados futuros; con el objetivo de precisar la toma de decisiones mediante técnicas de análisis de datos. En los últimos años el área predictiva ha tomado gran protagonismo en los negocios, la medicina, los servicios financieros, las políticas gubernamentales, la publicidad, la mercadotecnia, las redes sociales y gran cantidad de campos de aplicación.

Se basa, principalmente en datos organizados tabularmente. Es decir, hablamos de datos estructurados y bases de datos relacionales en la mayoría de los casos. Se busca el patrón de comportamiento y la tendencia escondida en las relaciones entre diferentes variables de un sistema, y, a través de aprendizaje supervisado, con modelos de regresión y de clasificación, se obtienen predicciones que ayudan a la toma de decisiones en la organización.

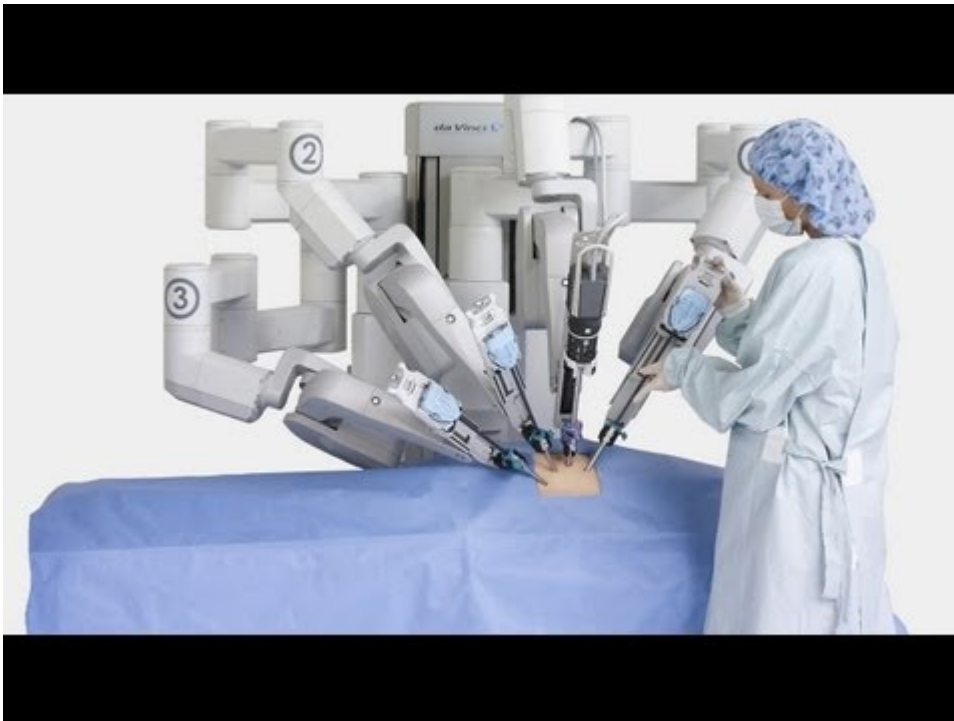
Cada vez van siendo más utilizados también los modelos de aprendizaje automático no supervisado, como el "clustering", que son el alma de sistemas de recomendación en plataformas de contenido online o comercio electrónico.

Los modelos predictivos tienen gran aplicabilidad en todos los sectores comerciales. Son capaces de resolver muchos problemas que antes eran irresolubles.

Robótica e Inteligencia Artificial

La integración de la IA en la robótica ha llevado a la creación de robots cada vez más inteligentes y autónomos, lo que ha revolucionado diversas áreas de aplicación.

Los sistemas robóticos actuales cubren una gran cantidad de campos de aplicación del entorno. En todos ellos, la inteligencia artificial mejora su desempeño y permite acometer nuevas tareas. Entre ellos, destacan muchos robots cuya principal herramienta basada en inteligencia artificial es el módulo de visión artificial, como es el caso de Davinci, el robot cirujano, o agro-bot, el robot que recoge fresas en su punto óptimo de madurez.



La inteligencia artificial tiene un impacto especialmente relevante en el sistema de control del robot.

APLICACIONES DE LA IA EN LA ROBÓTICA

Robots sociales.

Este tipo de robots tienen muy desarrollados los módulos sensoriales, es decir, el de reconocimiento de imagen, procesamiento de lenguaje natural, y su sistema de control es, básicamente, un asistente virtual que tiene ciertas opciones de movilidad y acciones remotas o conectadas, como encender la luz o hacer una llamada de emergencia.

Casas y ciudades inteligentes.

Estos sistemas robóticos cuentan con un elaborado sistema de sensores, y un hardware extendido por diversas localizaciones, lo que hace necesario contar, a menudo con microcontroladores que hagan parte del procesamiento de la información que captan y luego lo envíen al controlador principal. Por ejemplo, un sistema en el que tenemos cámaras que monitorizan las ventanas, en el propio microcontrolador de la cámara se puede hacer la tarea de reconocer la imagen de "ventana abierta", que la unidad principal recibirá junto a otros datos de interés como si está lloviendo o si hace frío, para accionar un actuador que haga saltar una alerta en el móvil de la persona propietaria o incluso que accione un motor para cerrarla.

Conducción autónoma.

La unidad de control de un vehículo autónomo es el paradigma de las técnicas más avanzadas en aprendizaje automático. Se trata de aprendizaje por refuerzo, y se entrena en simuladores virtuales hasta que el sistema tiene un comportamiento más o menos aceptable como para probarlo de forma segura en circuitos de pruebas reales.

Existen vehículos autónomos desde hace bastante tiempo, como es el caso de los drones, e incluso el sistema de control de navegación de los aviones cuenta con muchos automatismos, pero, probablemente, un coche autónomo, hoy por hoy, es el sistema más espectacular en tanto debe lidiar con muchos obstáculos y reglas de circulación.

Robots Colaborativos

Los robots colaborativos, también conocidos como cobots, son robots diseñados para trabajar de forma segura y eficiente junto a los seres humanos. Estos robots se utilizan en la industria para aumentar la productividad y mejorar la seguridad en la colaboración humano-robot. Por ejemplo:

- **Ensamblaje de Productos:** En líneas de ensamblaje de automóviles o electrónicos, los cobots pueden trabajar junto a los operadores humanos para realizar tareas repetitivas y pesadas, mejorando la eficiencia y reduciendo la fatiga de los trabajadores.
- **Embalaje y Logística:** En almacenes y centros de distribución, los cobots pueden colaborar con los trabajadores en la selección, embalaje y envío de productos, agilizando los procesos logísticos.

Robótica Médica

La robótica médica ha revolucionado la cirugía y la asistencia médica, permitiendo procedimientos más precisos y menos invasivos. Algunos ejemplos de aplicaciones de la robótica médica son:

- **Cirugía Asistida por Robot:** Los robots quirúrgicos, controlados por cirujanos, pueden realizar movimientos más precisos y estables durante procedimientos complejos, reduciendo el riesgo de errores y acelerando la recuperación del paciente.
- **Rehabilitación Robótica:** Los robots de rehabilitación se utilizan para asistir en la terapia de pacientes con discapacidades físicas, proporcionando movimientos controlados y repetitivos para mejorar la recuperación.
- **Cuidados de Pacientes:** Los robots asistenciales pueden ayudar a los pacientes en tareas diarias, como levantarse, caminar y tomar medicamentos, brindando una mayor autonomía y apoyo en el cuidado de la salud.

Robots Autónomos

Los robots autónomos son máquinas que pueden operar de manera independiente en entornos desconocidos o adversos sin intervención humana directa. Algunos ejemplos de aplicaciones de robots autónomos incluyen:

- **Exploración Espacial:** Los robots autónomos se utilizan en misiones espaciales para explorar planetas, asteroides y lunas, recopilando datos valiosos sin la necesidad de una comunicación constante con la Tierra.
- **Búsqueda y Rescate:** En situaciones de desastres naturales o emergencias, los robots autónomos pueden buscar y localizar supervivientes en áreas peligrosas o inaccesibles para los equipos de rescate humanos.

Ciencia de datos y Data Mining

La ciencia de datos es una de las disciplinas en las que la inteligencia artificial ha generado un mayor impacto, permitiendo detectar patrones y relaciones mediante métodos no supervisados, y llevar a cabo agrupaciones y heurísticos. Todo ello será visto con detalle en el módulo SAA, donde se encontrarán los algoritmos y aplicaciones más conocidos. En este campo se engloban también los heurísticos y los detectores de anomalías para planes de mantenimiento industrial.



Ejemplo

Minería de datos o Data mining

No es raro ver cómo se usan indiferentemente los conceptos minería de datos y machine learning. Son conceptos "primos hermanos", pero no son lo mismo.

La principal diferencia radica en el objetivo que tiene cada una de las disciplinas. Mientras que la minería de datos descubre patrones anteriormente desconocidos, el Machine Learning se usa para reproducir patrones conocidos y hacer predicciones basadas en los patrones.

En pocas palabras se podría decir que la minería de datos tiene una función exploratoria mientras que el machine learning se focaliza en la predicción.

Ciberseguridad

A día de hoy, la IA juega un papel fundamental en el campo de la ciberseguridad. Algunos de los usos más destacados de la IA en ciberseguridad incluyen:

1. **Detección de amenazas avanzadas:** La IA se utiliza para analizar grandes volúmenes de datos y detectar patrones de comportamiento anómalos que puedan indicar actividades maliciosas. Los sistemas de detección de intrusiones basados en IA pueden identificar amenazas avanzadas y desconocidas que los enfoques tradicionales podrían pasar por alto.
2. **Prevención de ataques de phishing:** Los algoritmos de aprendizaje automático pueden analizar correos electrónicos y sitios web en busca de indicios de phishing y ayudar a bloquear o filtrar contenido malicioso antes de que llegue a los usuarios.
3. **Identificación de malware:** Los sistemas de IA pueden analizar el comportamiento de archivos y aplicaciones para detectar malware y ransomware. Además, la IA puede mejorar la identificación y clasificación de nuevas variantes de malware desconocido.
4. **Autenticación de usuarios:** La IA se utiliza en sistemas de autenticación biométrica y de reconocimiento facial para mejorar la seguridad en el acceso a dispositivos y servicios.
5. **Análisis de logs y eventos de seguridad:** Los sistemas de IA pueden analizar y correlacionar grandes cantidades de registros y eventos de seguridad para identificar patrones de actividad sospechosa y facilitar la respuesta a incidentes.
6. **Predicción y prevención de brechas de seguridad:** Mediante el análisis de datos históricos y la identificación de vulnerabilidades conocidas, la IA puede predecir posibles brechas de seguridad y ayudar a prevenir futuros ataques.
7. **Automatización de tareas de seguridad:** La IA se utiliza para automatizar tareas repetitivas en ciberseguridad, como la gestión de parches, el análisis de vulnerabilidades y la respuesta a incidentes, lo que permite a los profesionales de seguridad centrarse en tareas más complejas.
8. **Protección de redes y sistemas IoT:** La IA puede monitorear y proteger redes empresariales y dispositivos IoT (Internet de las cosas) para detectar y prevenir actividades maliciosas.

Ejemplos de soluciones reales en ciberseguridad basadas en IA incluyen:

- **Cylance:** Una plataforma de prevención de ataques basada en IA que utiliza algoritmos de aprendizaje automático para proteger contra malware y ransomware.
- **Darktrace:** Un sistema de detección de amenazas basado en IA que utiliza algoritmos de inteligencia artificial para identificar y responder a comportamientos anómalos en tiempo real.
- **IBM Watson for Cyber Security:** Una solución de seguridad cibernética que utiliza IA para analizar grandes cantidades de datos y ayudar a identificar y responder a amenazas de manera más rápida y precisa.
- **BioCatch:** Una solución de autenticación biométrica que utiliza IA para analizar el comportamiento del usuario y detectar actividades fraudulentas.



Nuevas Formas de Interacción

Interfaces de Voz



Definición

Las interfaces de voz son una forma de interacción con sistemas de Inteligencia Artificial que permiten a los usuarios comunicarse mediante comandos de voz en lugar de texto o clics.

Estas interfaces han experimentado un crecimiento significativo en popularidad gracias a avances en el procesamiento del lenguaje natural y la tecnología de reconocimiento de voz. A continuación, se desarrollarán algunas aplicaciones y ejemplos de interfaces de voz.

ASISTENTES VIRTUALES

Los asistentes virtuales son aplicaciones de inteligencia artificial que permiten a los usuarios realizar tareas y obtener información a través de comandos de voz. Algunos ejemplos de asistentes virtuales populares incluyen:

- **Siri:** Desarrollado por Apple, Siri es un asistente virtual que se encuentra integrado en dispositivos como iPhones, iPads y Macs. Puede responder preguntas, enviar mensajes, configurar alarmas, hacer llamadas y más.
- **Google Assistant:** Desarrollado por Google, Google Assistant está disponible en dispositivos Android y en otros dispositivos como Google Home. Puede proporcionar información en tiempo real, realizar búsquedas en la web, establecer recordatorios y controlar dispositivos inteligentes del hogar.
- **Amazon Alexa:** Alexa es el asistente virtual de Amazon y está presente en dispositivos como el Amazon Echo. Permite realizar compras en línea, reproducir música, controlar luces y termostatos inteligentes, y realizar muchas otras tareas.

SISTEMAS DE NAVEGACIÓN

Las interfaces de voz también se utilizan en sistemas de navegación para proporcionar instrucciones de conducción en tiempo real. Algunos ejemplos incluyen:

- **Google Maps:** La popular aplicación de mapas y navegación utiliza el reconocimiento de voz para que los conductores reciban instrucciones mientras mantienen la vista en la carretera.

- **Sistemas de Navegación Integrados en Automóviles:** Muchos automóviles modernos están equipados con sistemas de navegación por voz que permiten a los conductores obtener direcciones y encontrar lugares de interés sin quitar las manos del volante.

APLICACIONES DE ACCESIBILIDAD

Las interfaces de voz también juegan un papel crucial en la accesibilidad tecnológica para personas con discapacidades visuales o motrices. Algunos ejemplos de aplicaciones de accesibilidad incluyen:

- **Lectores de Pantalla:** Estas aplicaciones utilizan la voz para leer en voz alta el contenido de la pantalla de un dispositivo, permitiendo que las personas con discapacidades visuales puedan interactuar con la tecnología.
- **Comandos de Voz para Controlar Dispositivos:** Las interfaces de voz permiten a personas con discapacidades motrices controlar dispositivos y realizar tareas sin la necesidad de utilizar las manos.

Interfaces Cerebro-Computadora (BCI)

Definición

Las Interfaces Cerebro-Computadora (Brain-Computer Interface BCI) son tecnologías avanzadas que permiten la comunicación directa entre el cerebro humano y dispositivos tecnológicos. A través del registro y análisis de señales cerebrales, estas interfaces posibilitan que las personas controlen dispositivos y sistemas mediante su actividad cerebral.

Algunas aplicaciones destacadas son:

ASISTENCIA MÉDICA

Las BCI han abierto nuevas posibilidades para asistir a personas con discapacidades motoras. Estas interfaces permiten a individuos con lesiones espinales, amputaciones u otras condiciones que afectan su capacidad de movimiento, controlar prótesis y dispositivos asistenciales mediante señales cerebrales. Por ejemplo, una persona con una extremidad amputada podría usar una prótesis controlada por BCI para realizar movimientos precisos y naturales, restaurando parte de su funcionalidad física.

NEUROFEEDBACK

El neurofeedback es una técnica terapéutica en la que se proporciona a los individuos información en tiempo real sobre su actividad cerebral. Las BCI se utilizan para capturar señales cerebrales y mostrar a los usuarios visualizaciones de sus patrones de actividad cerebral. Esto permite que las personas aprendan a autorregular su actividad cerebral, lo que puede ser beneficioso para tratar problemas de salud mental, como el estrés, la ansiedad y la depresión, y mejorar el rendimiento cognitivo en tareas específicas.

JUEGOS Y ENTRETENIMIENTO

Las BCI también se aplican en el campo de los juegos y el entretenimiento. Al utilizar señales cerebrales para controlar videojuegos y aplicaciones, se crea una experiencia de juego más inmersiva e interactiva. Los jugadores pueden controlar personajes y acciones dentro del juego mediante su actividad cerebral, lo que abre un mundo de posibilidades para nuevas mecánicas de juego y experiencias innovadoras.

Realidad Aumentada y Virtual

Definición

La Realidad Aumentada (AR) y la Realidad Virtual (VR) son tecnologías que mezclan el mundo digital con el mundo real o generan entornos completamente simulados para el usuario. Estas tecnologías ofrecen nuevas formas de interactuar con la IA y el mundo digital, proporcionando experiencias inmersivas y enriquecedoras.

Algunas aplicaciones destacadas son:

EDUCACIÓN Y FORMACIÓN

La AR y la VR se están convirtiendo en herramientas valiosas en el campo de la educación y la formación. Al simular entornos y escenarios de la vida real, estas tecnologías permiten a los estudiantes aprender de manera práctica y vivencial, lo que puede mejorar significativamente la retención de conocimientos y habilidades. Por ejemplo, los estudiantes de medicina pueden realizar simulaciones de cirugías en entornos de realidad virtual, lo que les proporciona una experiencia práctica sin riesgos para los pacientes.

DISEÑO Y VISUALIZACIÓN

Las AR y VR también ofrecen ventajas en el campo del diseño y la visualización. Los arquitectos, ingenieros y diseñadores pueden utilizar estas tecnologías para ver y modificar modelos 3D de edificios, productos o espacios. Esto permite una comprensión más clara y detallada de los proyectos, lo que puede conducir a un diseño más eficiente y una toma de decisiones más informada.

ENTRETENIMIENTO E INMERSIÓN

En el ámbito del entretenimiento, la AR y la VR proporcionan experiencias inmersivas que integran elementos del mundo real y virtual. Los videojuegos de realidad virtual permiten a los jugadores sumergirse por completo en mundos virtuales y participar en experiencias interactivas. Además, la AR se ha utilizado en aplicaciones de entretenimiento móvil, como juegos de realidad aumentada que interactúan con el entorno del usuario.



<https://sataraseguridad.com/2021/03/31/la-realidad-virtual-llega-al-entrenamiento-policial/>

Mapa conceptual



🕒 23 de agosto de 2026

3.2 Diapositivas de la UD01

-  Parte 1 de 5
-  Parte 2 de 5
-  Parte 3 de 5
-  Parte 4 de 5
-  Parte 5 de 5

🕒 17 de septiembre de 2025

3.3 Actividades

Comparativa Java vs Python para Robocode

Robocode Tank Royale (RCTR) ofrece API oficial para múltiples lenguajes. En esta unidad podéis elegir entre **Java (JVM)** y **Python**. A continuación se detallan las diferencias, ventajas e inconvenientes de cada opción.

¿Por qué dos lenguajes?

RCTR v0.34.0 introdujo la API multilingüe. El proyecto Robocode nació en Java, pero la comunidad y la evolución del software han traído soporte para Python, .NET y TypeScript. Esto permite elegir el lenguaje que mejor se adapte a vuestro perfil y a los objetivos del curso.

Tabla comparativa

Aspecto	Java	Python
Entorno requerido	JDK 11-23 + IntelliJ IDEA + Maven	Python 3.10+ + VS Code + pip
Dependencias	<code>pom.xml</code> (Maven) o JAR manual	<code>pip install robocode-tank-royale</code>
Sintaxis API	<code>camelCase()</code> — <code>setTurnLeft()</code> , <code>onScannedBot()</code>	<code>snake_case()</code> — <code>set_turn_left()</code> , <code>on_scanned_bot()</code>
Tipado	Estático (obligatorio)	Dinámico (opcional, con type hints)
Proyecto starter	ZIP preempaquetado (<code>IABDBotMaven.zip</code>)	Manual: 4 ficheros (<code>.py</code> + <code>.json</code> + <code>.cmd</code> + <code>.sh</code>)
Conf. servidor	Variables de entorno en configuración de ejecución de IntelliJ	Variables de entorno <code>SERVER_SECRET</code> y <code>SERVER_URL</code> en terminal
Curva aprendizaje	Más pronunciada (JDK, Maven, IDE pesado)	Más suave (pip, editor ligero)
Rendimiento batalla	Compilado JIT (mayor velocidad)	Interpretado (suficiente para RCTR, pero menor rendimiento)
Ecosistema IA/ML	Limitado para IA aplicada	Rico: scikit-learn, PyTorch, TensorFlow, etc.
Documentación comunitaria	RoboWiki mayoritariamente en Java	RoboWiki en Java (traducción a Python necesaria)
Uso en CE IA&BD	Transversal (programación genérica)	Alineado (Python es el lenguaje de referencia en IA)
API RCTR	Madura (existe desde el origen)	Estable (v0.34.0+, en evolución)

Ejemplos de código comparativos

CLASE BOT E IMPORTS

Java Python

```

1 import dev.robocode.tankroyale.botapi.*;
2 import dev.robocode.tankroyale.botapi.events.*;
3
4 public class MiBot extends Bot {
5     MiBot() {
6         super(BotInfo.fromFile(getConfigPath()));
7     }
8
9     private static String getConfigPath() {
10         String configPath = "src/main/java/MiBot.json";
11         java.io.File configFile = new java.io.File(configPath);
12         if (!configFile.exists()) {
13             configPath = "MiBot.json";
14         }
15         return configPath;
16     }
17 }

```

```
1 from robocode_tank_royale.bot_api import Bot, BotInfo
2
3 class MiBot(Bot):
4     def __init__(self):
5         super().__init__(BotInfo.from_file("MiBot.json"))
```

MÉTODO `run()` Y BUCLE PRINCIPAL

Java Python

```

1 public void run() {
2     setAdjustRadarForBodyTurn(true);
3     setAdjustGunForBodyTurn(true);
4     setAdjustRadarForGunTurn(true);
5
6     while (isRunning()) {
7         setTurnRadarLeft(Double.POSITIVE_INFINITY);
8         forward(100);
9         turnGunLeft(360);
10        back(100);
11        turnGunLeft(360);
12    }
13 }

```

```

1  def run(self):
2      self.set_adjust_radar_for_body_turn(True)
3      self.set_adjust_gun_for_body_turn(True)
4      self.set_adjust_radar_for_gun_turn(True)
5
6      while self.is_running: # type: ignore[attr-defined] # pylint: disable=no-member
7          self.set_turn_radar_left(float("inf"))
8          self.forward(100)
9          self.turn_gun_left(360)
10         self.back(100)
11         self.turn_gun_left(360)

```


EVENT HANDLER `onScannedBot`

Java	Python
------	--------

```
1 public void onScannedBot(ScannedBotEvent e) {
2     fire(1);
3 }
```

```
1 def on_scanned_bot(self, scanned_bot_event: ScannedBotEvent):
2     del scanned_bot_event
3     self.fire(1)
```

DISPARO PREDICTIVO (TRIGONOMÉTRICO)

Java	Python
------	--------

```
1 public void onScannedBot(ScannedBotEvent e) {
2     double enemyDistance = distanceTo(e.getX(), e.getY());
3     double firePower = Math.min(500 / enemyDistance, 3);
4     double bulletSpeed = 20 - firePower * 3;
5     long time = (long) (enemyDistance / bulletSpeed);
6
7     double enemyVelocity = e.getSpeed();
8     double enemyDirection = e.getDirection();
9     double deltaX = enemyVelocity * Math.cos(Math.toRadians(enemyDirection));
10    double deltaY = enemyVelocity * Math.sin(Math.toRadians(enemyDirection));
11    double futureX = e.getX() + (deltaX * time);
12    double futureY = e.getY() + (deltaY * time);
13
14    setTurnGunLeft(gunBearingTo(futureX, futureY));
15    if (getGunTurnRemaining() <= 0 && getGunHeat() == 0) {
16        fire(firePower);
17    }
18 }
```

```
1 def on_scanned_bot(self, e: ScannedBotEvent):
2     enemy_distance = self.distance_to(e.x, e.y)
3     fire_power = min(500 / enemy_distance, 3)
4     bullet_speed = 20 - fire_power * 3
5     time = int(enemy_distance / bullet_speed)
6
7     enemy_velocity = e.speed
8     enemy_direction = e.direction
9     delta_x = enemy_velocity * math.cos(math.radians(enemy_direction))
10    delta_y = enemy_velocity * math.sin(math.radians(enemy_direction))
11    future_x = e.x + (delta_x * time)
12    future_y = e.y + (delta_y * time)
13
14    self.set_turn_gun_left(self.gun_bearing_to(future_x, future_y))
15    if self.gun_turn_remaining <= 0 and self.gun_heat == 0:
16        self.fire(fire_power)
```

Ventajas e inconvenientes

JAVA

Ventajas:

- Documentación histórica amplia (RoboWiki, ejemplos clásicos)
- Proyecto starter preempaquetado (descomprimir y abrir en IntelliJ)
- Rendimiento compilado mayor en cálculos intensivos
- API muy madura y estable

Inconvenientes:

- Requerimientos de entorno pesados (JDK + IntelliJ)
- Curva de aprendizaje mayor si no se conoce Java/Maven

- Menos alineado con el ecosistema de IA y Big Data del curso
- Configuración de ejecución dependiente del IDE

PYTHON

Ventajas:

- Instalación sencilla (`pip install`)
- Editor ligero (VS Code) y rápido de configurar
- Sintaxis limpia y legible
- Alineado con el resto del curso (ecosistema IA/BD)
- Mejor integración con librerías de IA si se quiere ampliar el bot
- Variables de entorno simples (terminal o script)

Inconvenientes:

- Documentación histórica en Java (hay que traducir conceptualmente)
- Proyecto starter manual (4 ficheros) en lugar de ZIP preempaquetado
- Rendimiento interpretado (normalmente no perceptible en RCTR)

Recomendación

Para la naturaleza de este curso de especialización en **Inteligencia Artificial y Big Data**, se recomienda el uso de **Python** como lenguaje principal. Python es la herramienta estándar de la industria en IA/ML, y su sintaxis clara permite centrarse en las estrategias del bot en lugar de la infraestructura del lenguaje.

La opción Java queda disponible para alumnado con experiencia previa en Java o interés particular, pero la vía Python será la que reciba soporte preferente por parte del profesorado.

**Elegid vuestro camino**

- **Noveles en programación o provenientes de Python** → seguid la guía Python
- **Experiencia previa en Java o interés en JVM** → seguid la guía Java
- **Dudas** → consultad al profesorado

🕒 15 de julio de 2026

Robocode TankRoyale

Preparación del entorno

- Al menos java 11 (Yo estoy usando Java 23 (OpenJDK) y la versión 0.34.0 de Tank Royale)

```
1 java -version
```

- Descargar la última versión desde releases: <https://github.com/robocode-dev/tank-royale/releases>
- Ejecutar el `jar` descargado:

```
1 java -jar robocode-tankroyale-gui-x.y.z.jar
```

- Descarga y descomprime los Bots de ejemplo: <https://github.com/robocode-dev/tank-royale/releases>
- Configura la gui para acceder a la carpeta de robots: Config -> Bot Root Directories (del menú)
- [opcional] Añadir sonidos al gui. Descarga y descomprime la carpeta `sounds.zip` de: <https://github.com/robocode-dev/sounds/releases>

```
1 [directorio padre]
2 |─ robocode-tankroyale-gui-x.y.z.jar
3 |─ sounds/ <-- carpeta de sonidos
4   |─ bots_collision.wav
5   ...
6   |─ wall_collision.wav
```

- Necesitaras IntelliJ para poner todo en funcionamiento y para programar tu Bot.



Consejo

Juega un poco por tu cuenta con el entorno GUI, explora las opciones, tipos de batallas, crea alguna ronda de las mismas, inicializa Bots, añádelos, juega con la velocidad de juego, etc.

Robocode con Maven

Robocode tankroyale está disponible como artefacto del repositorio de maven, si estas familiarizado con el uso de maven puede simplificar bastante el trabajo y actualizaciones de dependencias, te dejo el enlace por si quieres investigar por tu cuenta, ya que escapa del objetivo de esta asignatura: <https://central.sonatype.com/search?q=g:dev.robocode.tankroyale&sno=true>

Mi primer Bot

PROYECTO INTELLIJ

Para acelerar el proceso, he preparado un proyecto en IntelliJ con toda la arquitectura básica que necesitará tu Bot. Descarga y descomprime [este paquete](#) y abre el proyecto con el IDE IntelliJ.

Sin maven

Si no quieres usar maven y deseas montar el proyecto por tu cuenta sigue estas indicaciones:

También necesitaras la librería (API) de Robocode Tank Royale que puedes descargar desde aquí: <https://github.com/robocode-dev/tank-royale/releases>. Descarga el archivo: `robocode-tankroyale-bot-api-x.y.z.jar`

La estructura debería quedar de la siguiente manera:

```

1  [directorio padre]
2  |— lib
3  |   └─ robocode-tankroyale-gui-x.y.z.jar
4  └─ IABDBot <-- Proyecto de IntelliJ
5     └─ src
6     ...
7     └─ IABDBot.iml
  
```

Atención

Asegúrate de entender el funcionamiento del Bot IABDBot, tiene comentarios donde se explican partes importante del Bot y que daremos por conocidas en los siguientes puntos.

Es muy importante que entiendas la diferencia entre movimientos bloqueantes y simultáneos, así como las condiciones y los eventos disponibles.

Ojo!

Gracias a este fragmento, tu bot funcionará correctamente desde IntelliJ y desde el gui de RoboCode:

```

1  // Constructor, que carga el fichero de configuración
2  IABDBot() {
3      super(BotInfo.fromFile(getConfigPath()));
4  }
5
6  // Este método obtiene la ruta del fichero de configuración y se asegura que funcione desde IntelliJ y desde la gui de
7  Robocode
8  private static String getConfigPath() {
9      String miBot = "IABDBot";
10     String configPath = "src/main/java/" + miBot + ".json";
11     java.io.File configFile = new java.io.File(configPath);
12     if (!configFile.exists()) {
13         configPath = miBot + ".json";
14     }
15     return configPath;
16 }
  
```

ÚLTIMAS NOVEDADES DEL PROYECTO

- Se pueden grabar los combates en un fichero dentro de la carpeta `recordings` con la extensión `battle.gz` que luego podemos reproducir en diferido. Esto nos permite revisar situaciones y estudiar estrategias de mejora.

CONFIGURACIÓN DEL SERVIDOR (GUI) Y EL PROYECTO INTELIJ PARA EJECUTAR EN LOCAL O REMOTO

El servidor permite conexiones en remoto/local a través de un password que se genera automáticamente la primera vez que lanzas la GUI. Este password lo puedes encontrar/modificar en el archivo `server.properties`, en el ejemplo siguiente la clave de acceso es **CEIABDEPM2025**

```

1  bots-secrets=CEIABDEPM2025
2  [...]
  
```

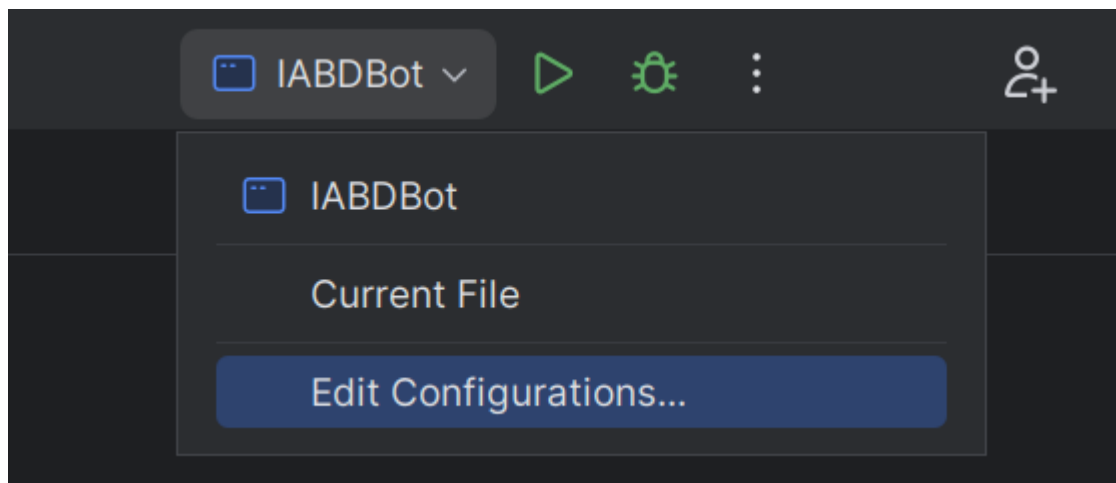
Ojo

Con las últimas versiones de la gui el uso de contraseñas en el servidor está inicialmente deshabilitado, si queremos habilitarlo lo podemos hacer desde el propio interfaz: `Config/Server Options/Enable server secrets` o bien desde el fichero de configuración `server.properties` añadiendo:

```
1 server-secrets-enabled=true
```

Otra información que necesitaras será la IP del servidor al que quieres conectar, normalmente será `localhost` (tu host local), y cuando sea necesario hacer pruebas el profesor te facilitará su IP.

Ahora solo queda configurar nuestro proyecto de IntelliJ para configurar estas variables en el momento de ejecutar el Bot. Accede a la configuración de las ejecuciones en la parte superior derecha (una vez abierta la clase `IABDBot.java`):



Puedes ver que en el apartado `Environment Variables` tienes el siguiente valor:

```
1 SERVER_SECRET=CEIABDEPM2025;SERVER_URL=ws://localhost:7654
```

Como puedes imaginar **SERVER_SECRET** hace referencia a la clave del servidor, y **SERVER_URL** a la dirección del servidor y es donde deberás reemplazar (según el caso) `localhost` por la IP que te indique el profesor. Así podrás ejecutar tu Bot directamente desde IntelliJ y aparecerá en el Servidor para poder añadirlo a las batallas.

Atención

Ojo, el servidor debe estar inicializado antes de lanzar el bot de lo contrario obendrás un error similar a este:

```
1 Exception in thread "main" dev.robocode.tankroyale.botapi.BotException: Could not create web socket for URL: ws://localhost:
2 7654
3     at dev.robocode.tankroyale.botapi.internal.BaseBotInternals.connect(BaseBotInternals.java:268)
4     at dev.robocode.tankroyale.botapi.internal.BaseBotInternals.start(BaseBotInternals.java:254)
5     at dev.robocode.tankroyale.botapi.BaseBot.start(BaseBot.java:114)
6     at IABDBot.main(IABDBot.java:11)
7
Process finished with exit code 1
```

Si el Bot encuentra el servidor y la contraseña es correcta debería aparecer esto al inicializarlo en IntelliJ:

```
1 Connected to: ws://localhost:7654
```

**Consejo**

Prueba por tu cuenta mezclando en las batallas Bots de ejemplo, con tu Bot o incluso el de algún compañero/a.

¿Cómo mejoro mi Bot?

Dividiremos todo lo que debemos conocer en 4 apartados:

CONOCIENDO EL CAMPO DE BATALLA

Sigue atentamente la documentación de RCTR sobre la [anatomía de los Bots](#)

Puntos importantes:

- Si el radar no se mueve, no detecta
- Inicialmente el robot, el cañón y el radar se mueven conjuntamente (pero se puede cambiar)
- El Bot se simplifica a un cuadrado de 36x36 unidades.
- Para las colisiones (con Bots o paredes) se simula como un círculo de 18 unidades de radio.

A continuación revisa las [coordenadas y ángulos](#)

Puntos importantes:

- Este apartado es totalmente diferente al de Robocode Original.

Estudia también las [físicas](#)

Puntos importantes:

- Se frena más rápido que se acelera.
- Cuanto más rápido vas, más lento giras.
- El cañón gira máximo 20º por turno.
- El radar gira máximo 45º por turno.
- La potencia de tiro influye en el daño provocado y los puntos conseguidos, pero también en el calentamiento del cañón.
- Inicialmente el cañón está caliente (3 unidades), al principio has de esperar a que se enfrie para disparar.
- El atropello da más puntos que los disparos (pero te resta energía)
- Si chocas con una pared, pierdes puntos.
- La energía que te sobra en una ronda no da puntos (modo kamikaze con el último robot?)

Por último te en cuenta las [puntuaciones](#)

Puntos importantes:

- Consigues puntos si:
 - tu disparo golpea a otro Bot (sino, te resta)
 - eres el que mata al Bot
 - cada vez que otro Bot muere, pero tu sigues vivo
 - eres el último robot vivo
 - atropellas a otro Bot
 - si matas por atropello a otro Bot
- El último Bot que quede vivo no tiene porqué ser el ganador.

Eventos propios

Definimos una condición, y cuando esta se cumple se dispara un evento:

```

1  // Esta condición se dispara cuando el bot completa su giro
2  public static class TurnCompleteCondition extends Condition {
3
4      private final IBot bot;
5
6      public TurnCompleteCondition(IBot bot) {
7          this.bot = bot;
8      }
9
10     @Override
11     public boolean test() {
12         // El método test() es el que debemos reescribir para definir el resultado de nuestra condición.
13         return bot.getTurnRemaining() == 0;
14     }
15 }

```

Podríamos usar esta condición en un fragmento similar a este:

```

1  waitFor(new TurnCompleteCondition());

```

Podríamos usar funciones Lambda de la siguiente manera:

```

1  waitFor(new Condition() -> getTurnRemaining() == 0);

```

El resultado de los dos fragmentos sería el mismo.

PERSONALIZAR MI BOT

Podemos establecer colores para el cuerpo, torreta de cañón, el radar, el arco de escaneo y de la bala:

```

1  Color verde = Color.fromRgb(0,255,0);
2  setBodyColor(Color.KHAKI);
3  setTurretColor(Color.KHAKI);
4  setRadarColor(Color.KHAKI);
5  setScanColor(Color.KHAKI);
6  setBulletColor(Color.KHAKI);

```

Puedes encontrar la lista completa de colores disponibles en la API de Robocode TankRoyale.

TÉCNICAS DE ESCANEO (RADAR)

Sentidos de nuestro Bot:

Sentido del **tacto**, tu Bot sabe cuando:

- golpea una pared (`onHitWall`),
- es alcanzado por un disparo (`onHitByBullet`),
- es alcanzado por otro Bot (`onHitBot`).

Sentido de la **vista**, tu Bot sabe cuándo ha visto otro robot, pero sólo si lo escanea (`onScannedBot`)

Otros sentidos, tu robot también sabe cuándo ha muerto (`onDeath`), cuándo ha muerto otro robot (`onRobotDeath`).

Además también es consciente de sus balas y sabe cuando una bala ha sido disparada (`onBulletFired`) ha alcanzado a un oponente (`onBulletHit`), cuando una bala golpea una pared (`onBulletWall`) o cuando una bala golpea a otra bala (`onBulletHitBullet`).

Configurar correctamente tu Bot con:

```

1  setAdjustRadarForBodyTurn(true); //true if the radar must adjust/compensate for the body's turn;
2                                     //false if the radar must turn with the body turning (default).
3  setAdjustGunForBodyTurn(true);  //true if the gun must adjust/compensate for the bot's turn;
4                                     //false if the gun must turn with the bot's turn.
5  setAdjustRadarForGunTurn(true); //true if the radar must adjust/compensate for the gun's turn;
6                                     //false if the radar must turn with the gun's turn.

```

Que el radar siempre de vueltas:

```
1 setTurnRadarLeft(Double.POSITIVE_INFINITY); //degrees - is the amount of degrees to turn left. If negative, the radar
   will turn right.
```

Revisa el API (`rescan()` i `setRescan()`)

Fijar el radar en un enemigo (one on one radar):

- Escaneo con multiplicador

```
1 public void onScannedBot(ScannedBotEvent e) {
2     double factor=1.9;
3     setTurnRadarLeft(factor * radarBearingTo(e.getX(), e.getY()));
4 }
```

Segun el factor:

- 1.0 - Bloqueo de radar delgado. Debe llamar a `scan()` para evitar perder el bloqueo. Que Dios te ayude si alguna vez te saltas un turno.
- 1.9 - El arco del radar comienza amplio y se estrecha lentamente tanto como sea posible mientras se mantiene en el objetivo.
- 2.0: el arco del radar recorre un ángulo fijo. El ángulo exacto elegido depende de las posiciones del enemigo y del radar cuando se detecta al enemigo por primera vez. El ángulo se incrementará si es necesario para mantener el bloqueo.
- Arco de radar Ancho

```
1 public void onScannedBot(ScannedBotEvent e) {
2     //set the wide to add to the scan.
3     double wide=36.0;
4     // Absolute angle towards target
5     double angleToEnemy = radarBearingTo(e.getX(), e.getY());
6     // Distance we want to scan from middle of enemy to either side
7     // The 36.0 is how many units from the center of the enemy robot it scans.
8     double extraTurn = Math.min(Math.toDegrees(Math.atan(36.0 / distanceTo(e.getX(), e.getY()))), getMaxRadarTurnRate())
9 );
10
11 // Adjust the radar turn so it goes that much further in the direction it is going to turn
12 // Basically if we were going to turn it left, turn it even more left, if right, turn more right.
13 // This allows us to overshoot our enemy so that we get a good sweep that will not slip.
14 if (angleToEnemy < 0)
15     angleToEnemy -= extraTurn;
16 else
17     angleToEnemy += extraTurn;
18
19 //Turn the radar
20 setTurnRadarLeft(angleToEnemy);
21 }
```

- Radar Oscilante (variable global que sepa la dirección y nos ayude a cambiarla de vez en cuando)
- Escaneo más inteligente (seguir al cercano? según la situación?) ...
- Nos interesa mantener una lista de enemigos? `e.getScannedBotId()` ? y por ejemplo fijar el radar en el más débil?...

TÉCNICAS DE DESPLAZAMIENTO (MOVIMIENTO)

Pensamiento lateral

Si has jugado baloncesto antes, sabes que si quieres defender a alguien que sostiene el balón, debes maximizar tu movimiento lateral enfrentándote siempre a él (de frente). Lo mismo ocurre con tu robot.

Para conseguir esta posición debemos hacer algo similar a esto:

```
1 turnLeft(bearingTo(e.getX(), e.getY()) + 90);
```

que siempre colocará tu robot perpendicular (90 grados) a tu enemigo.

¿Hacia adelante o hacia atrás?

Cuando te enfrentas a un oponente, la idea de "adelante" y "atrás" (del primer Bot) se vuelven algo obsoletas. Probablemente estés pensando más en términos de "atacar a la izquierda" o "atacar a la derecha". Para realizar un seguimiento de la dirección del movimiento, simplemente declare una variable como hicimos para oscilar el radar.

```
1 class MyRobot extends Bot {
2     private byte moveDirection = 1;
```

luego, cuando quieras mover tu robot, simplemente puedes decir:

```
1 setForward(100 * moveDirection);
```

Puedes cambiar de dirección cambiando el valor de moveDirection de 1 a -1 así:

```
1 moveDirection *= -1;
```

Cambiar de dirección

El enfoque más intuitivo para cambiar de dirección es simplemente cambiar la dirección del movimiento cada vez que golpeas una pared o golpeas a otro robot de esta manera:

```
1 public void onHitWall(HitWallEvent e) { moveDirection *= -1; }
2 public void onHitBot(HitBotEvent e) { moveDirection *= -1; }
```

Sin embargo, descubrirás que si haces eso, terminarás presionando obstinadamente contra un robot que te embiste desde un costado (como un perro en celo). Esto se debe a que se llama a `onHitRobot()` tantas veces que `moveDirection` sigue cambiando y nunca te alejas.

Un mejor enfoque es simplemente probar para ver si su robot se ha detenido. Si es así, probablemente significa que has golpeado algo y querrás cambiar de dirección. Puedes hacerlo con el código:

```
1 if (getSpeed() == 0)
2     moveDirection *= -1;
```

Ponlo en tu método `doMove()` (o en cualquier otro lugar donde estés manejando el movimiento) y podrás manejar todos los eventos de impacto con las paredes u otros Bots.

¿Bailamos?

Dando vueltas (Círculos)

Puedes rodear a tu enemigo simplemente usando las técnicas anteriores:

```

1  public class IABDBot extends Bot {
2      double moveDirection=1;
3      private double ultimoEnemigoVisto;
4      ...
5      @Override
6      public void run() {
7          while (isRunning()) {
8              doMove();
9              go();
10         }
11
12         public void onScannedBot(ScannedBotEvent e) {
13             ...
14             ultimoEnemigoVisto=bearingTo(e.getX(), e.getY());
15             ...
16         }
17
18         public void doMove() {
19             // switch directions if we've stopped
20             if (getSpeed() == 0)
21                 moveDirection *= -1;
22             // circle our enemy
23             setTurnLeft(ultimoEnemigoVisto+90);
24             setForward(1000 * moveDirection);
25         }

```

Objetivo: rodea a tu enemigo usando el código de movimiento anterior, como un tiburón rodeando a su presa en el agua.

Evita Ametrallamiento

Un problema que puedes notar con el anterior tipo de movimiento es que es presa fácil para los objetivos predictivos porque sus movimientos son tan... predecibles.

Para evadir las balas de manera más efectiva, debes moverte de lado a lado o "atacar". Una buena forma de hacerlo es cambiar de dirección después de un cierto número de "tics", así:

```

1  public void doMove() {
2      // switch directions if we've stopped
3      if (getSpeed() == 0)
4          moveDirection *= -1;
5      // circle our enemy
6      setTurnLeft(ultimoEnemigoVisto+90);
7      if (getTurnNumber() % 20 == 0) {
8          moveDirection *= -1;
9          setForward(1000 * moveDirection);
10     }
11 }

```

Mira como se balancea hacia adelante y hacia atrás usando el código de movimiento anterior. Observa lo bien que esquivas las balas. Pero ojo, porque puede afectar a tu precisión de disparo.

Cada vez más cerca

Notarás que tanto el primero como este último tipo de movimientos tienen otro problema: se atascan fácilmente en las esquinas y terminan golpeándose contra las paredes. Un problema adicional es que si su enemigo está lejos, disparan mucho pero no aciertan mucho.

Para hacer que tu robot se acerque a tu enemigo, simplemente modifica el código para que se gire ligeramente hacia su enemigo, así:

```

1  setTurnLeft(lastScannedBearing + (90 - (15 * moveDirection)));

```

15 es un factor en grados que puedes modificar y ajustar. Prueba!

Modificando el primero conseguimos una variación que utiliza el código anterior para lanzarse en espiral hacia su enemigo.

Mientras que con el segundo que utiliza el código anterior para acercarse cada vez más. También es bastante bueno para evadir balas.

Fíjate que ninguno de los Bots anteriores queda atrapado en una esquina por mucho tiempo.

Evitando las paredes

Un problema con todos los Bots anteriores es que golpean mucho las paredes y golpearlas agota tu energía. Una mejor estrategia sería detenerse antes de chocar contra las paredes. ¿Pero cómo?

Agregar un evento personalizado

Lo primero que debemos hacer es decidir qué tan cerca permitiremos que nuestro robot llegue a las paredes:

```
1 public class WallAvoider extends Bot {
2     ...
3     private int wallMargin = 60;
```

A continuación, agregamos un evento personalizado que se activará cuando se cumpla una determinada condición dentro del método `run()`:

```
1 public void run() {
2     ...
3     // No te acerques mucho a las paredes
4     addCustomEvent(new Condition("TooCloseToWalls") {
5         public boolean test() {
6             // El método test() es el que debemos reescribir para definir el resultado de nuestra condición.
7             return (
8                 // cerca de la pared izquierda
9                 (getX() <= wallMargin ||
10                // cerca de la pared derecha
11                getX() >= getArenaWidth() - wallMargin ||
12                // cerca de la pared inferior
13                getY() <= wallMargin ||
14                // cerca de la pared superior
15                getY() >= getArenaHeight() - wallMargin)
16            );
17        }
18    });
19    ...
20 }
```

Ten en cuenta que estamos creando una clase interna anónima con esta llamada. Necesitamos hacer override del método `test()` para devolver un valor booleano cuando ocurra nuestro evento personalizado.

Gestionar el evento personalizado

Lo siguiente que debemos hacer es manejar el evento, que se puede hacer así:

```
1 public void onCustomEvent(CustomEvent e) {
2     if (e.getCondition().getName().equals("TooCloseToWalls")) {
3         // switch directions and move away
4         moveDirection *= -1;
5         forward(100 * moveDirection);
6     }
7 }
```

Sin embargo, el problema con ese enfoque es que este evento podría dispararse una y otra vez, provocando que cambiemos rápidamente de un lado a otro, sin alejarnos nunca. O si ya aparecemos cerca de la pared quedamos atrapados.

Para evitar este "sacudida de la muerte" deberíamos tener una variable que indique que estamos manejando el evento. Podemos declarar otro así:

```
1 public class WallAvoider extends Bot {
2     ...
3     private int tooCloseToWall = 0;
```

Luego maneje el evento de manera un poco más inteligente:

```

1 public void onCustomEvent(CustomEvent e) {
2     if (e.getCondition().getName().equals("TooCloseToWalls")) {
3         if (tooCloseToWall <= 0) {
4             // Si no estábamos ya cerca de las paredes, ahora lo estamos
5             tooCloseToWall += wallMargin;
6             setMaxSpeed(0); // Para!!!
7         }
8     }
9 }

```

Manejo de los dos modos

Hay dos últimos problemas que debemos resolver. En primer lugar, tenemos un método `doMove()` donde colocamos todo nuestro código de movimiento normal. Si estamos tratando de alejarnos de una pared, no queremos que se llame a nuestro código de movimiento normal, creando (una vez más) la "sacudida de la muerte". En segundo lugar, queremos volver eventualmente al movimiento "normal", por lo que eventualmente deberíamos tener el "tiempo de espera" de la variable `tooCloseToWall`.

Podemos resolver ambos problemas con la siguiente implementación de `doMove()`:

```

1 public void doMove() {
2     ...
3
4     // if we're close to the wall, eventually, we'll move away
5     if (tooCloseToWall > 0) tooCloseToWall--;
6
7     // switch directions if we've stopped
8     if (getSpeed() == 0) {
9         setMaxSpeed(8);
10        moveDirection *= -1;
11        setForward(1000 * moveDirection);
12    }
13 }

```

Con todo el código anterior evitamos chocar contra las paredes. Observa cómo se desliza suavemente hacia los lados pero nunca (bueno, rara vez) los golpea.

Bot multimodo

Además de los colores que elijas, la mayor parte de la personalidad de tu robot está en su código de movimiento. Por otra parte, situaciones diferentes requieren tácticas diferentes. Usando el ejemplo de evitar paredes, es posible que desees codificar tu bot para que cambie los "modos" según ciertos criterios. Podrías pensar en algo como esto:

```

1  public class MultiModeBot extends Bot {
2      private final static int MODO_RONDAR=0;
3      private final static int MODO_ESQUIVAR=1;
4      private final static int MODO_ASESINO=2;
5      private int modoRobot=MODO_RONDAR;
6      ...
7  }
8
9  public void onBotDeath(BotDeathEvent e) {
10     ...
11     if (getEnemyCount() > 10) {
12         // Un gran número de enemigos sugiere movimientos fluidos
13         modoRobot=MODO_RONDAR;
14     } else if (getEnemyCount() > 1) {
15         // esquivar es la mejor táctica para grupos pequeños
16         modoRobot=MODO_ESQUIVAR;
17     } else if (getEnemyCount() == 1) {
18         // Si solo queda un robot, persíguelo
19         modoRobot=MODO_ASESINO;
20     }
21     ...
22 }
23
24 public void doMove() {
25     switch (modoRobot){
26         case MODO_RONDAR:
27             //ronda
28             break;
29         case MODO_ESQUIVAR:
30             //esquiva
31             break;
32         case MODO_ASESINO:
33             //asesina
34             break;
35     }
36     ...
37 }

```

Los detalles se dejan (como siempre) como ejercicio para el alumnado.

TÉCNICAS DE ATAQUE (DISPARO)

Técnicas básicas

- Separa el movimiento del cañón del movimiento del Bot (`setAdjustGunForBodyTurn`)
- Técnica de apuntado básica: `setTurnGunLeft` o `setTurnGunRight` junto con `gunBearingTo`

Fórmula de cálculo de potencia de fuego

Otro aspecto importante al disparar es calcular la potencia de fuego de tu bala. La documentación del método `fire()` explica que puedes disparar una bala en el rango de 0.1 a 3.0. Como probablemente ya habrás concluido, es una buena idea disparar balas de baja potencia cuando tu enemigo está lejos y balas de alta resistencia cuando está cerca.

```

1  public void smartFire(double distance){
2      if ((distance >200) || (getEnergy() <15)){
3          fire(1);
4      } else if (distance > 50){
5          fire(2);
6      } else {
7          fire(3);
8      }
9  }

```

Podrías usar una serie de declaraciones if-else-if-else para determinar la potencia de fuego, en función de si el enemigo está a 50 unidades, a 200, etc (como en el fragmento anterior). Pero tales construcciones son demasiado rígidas. Después de todo, el rango de posibles valores de potencia de fuego cae a lo largo de un continuo, no de bloques discretos. Un mejor enfoque es utilizar una fórmula. He aquí un ejemplo:

```

1  setFire(400/distanceTo(e.getX(), e.getY()));

```

Con esta fórmula, a medida que aumenta la distancia del enemigo, la potencia de fuego disminuye. Asimismo, a medida que el enemigo se acerca, la potencia de fuego aumenta. Los valores superiores a 3 se reducen a 3, por lo que nunca dispararemos una bala mayor que 3, pero probablemente deberíamos reducir el valor de todos modos (solo para estar seguros) de esta manera:

```
1 setFire(Math.min(500/distanceTo(e.getX(), e.getY()), 3));
```

Evitar disparos prematuros

Una situación que encontraras es que tu Bot dispare antes de haber girado el arma hacia el objetivo. Para evitar disparos prematuros, usa el método `getGunTurnRemaining()` para ver qué tan lejos está tu cañón del objetivo y no dispare hasta que esté cerca.

Además, no puedes disparar si el arma está "caliente" desde el último disparo y llamar a `fire()` o `setFire()` solo desperdiciará un turno. Podemos probar si el arma está fría llamando a `getGunHeat()`.

Apuntando de manera predictiva

O... "Usar la trigonometría para impresionar a tus amigos y destruir a tus enemigos".

Si quisiéramos poder golpear a un robot que recorre las paredes siempre fallaríamos, necesitamos poder predecir dónde estará en el futuro, pero ¿cómo podemos hacerlo?

\$\$ Distancia = Velocidad * Tiempo \$\$ Usando la anterior formula podemos calcular cuánto tiempo tardará una bala en llegar allí.

- **Distancia:** se puede encontrar llamando a `distanceTo(e.getX(), e.getY())`
- **Velocidad:** según la documentación de RCTR, una bala viaja a una velocidad de: $20 - potenciaDeFuego * 3$
- **Tiempo:** podemos calcular el tiempo resolviendo: $Tiempo = \{Distancia \over Velocidad\}$

El siguiente código lo hace:

```
1 // calcular la potencia basado en la distancia
2 double enemyDistance = distanceTo(e.getX(), e.getY());
3 double firePower = Math.min(500 / enemyDistance, 3);
4 // calcular la velocidad de la bala
5 double bulletSpeed = 20 - firePower * 3;
6 // calculamos el tiempo que necesita la bala para impactar al enemigo
7 long time = (long) (enemyDistance / bulletSpeed);
```

Obteniendo futuras coordenadas X,Y

A continuación, debemos calcular la posición futura de nuestro enemigo:

```
1 // Calcular la velocidad actual de tu enemigo
2 double enemyVelocity = e.getSpeed();
3 // Calcular la dirección actual del enemigo
4 double enemyDirection = e.getDirection();
5 // Descomponer el desplazamiento en las coordenadas X e Y
6 // Componente X del desplazamiento COSENO
7 double deltaX = enemyVelocity * Math.cos(Math.toRadians(enemyDirection));
8 // Componente Y del desplazamiento SENO
9 double deltaY = enemyVelocity * Math.sin(Math.toRadians(enemyDirection));
10
11 // Calcular las coordenadas futuras
12 double futureX = e.getX() + (deltaX * time);
13 double futureY = e.getY() + (deltaY * time);
```

Girando el arma al punto previsto

Ahora debemos apuntar nuestro cañón al lugar previsto de impacto:

```
1 setTurnGunLeft(gunBearingTo(futureX, futureY));
```

Disparando!

Por último disparamos nuestro cañón

```
1  if (getGunTurnRemaining() <= 0 && getGunHeat() == 0) {  
2      fire(firePower);  
3  }
```

Mejor aún...!

Aquí te dejo algunas mejoras para que las valores:

- ¿Debemos esperar siempre a que el arma esté completamente en la dirección calculada?
- Cuando nuestro enemigo se acerca a una pared, nuestros disparos impactan en ella.
- ¿Puedo saber a quien disparo si mi previsión de impacto falla mucho?

Investigación y desarrollo propio

A partir de aquí el trabajo será individual de cada alumno (puede solaparse con el paso 3). Deberéis investigar/prever a vuestros adversarios (los conocidos, y los de vuestros compañeros). Aplicar las técnicas que consideréis más útiles para intentar quedar lo más arriba posible en la tabla de clasificación.

 19 de octubre de 2025

Robocode TankRoyale en Python

Antes de empezar

Leed primero la [Comparativa Java vs Python](#) para entender las diferencias y elegir vuestro camino.

Esta guía es la versión **Python** de la actividad Robocode. Si buscáis la versión Java, id a [P01 Robocode \(Java\)](#).

Preparación del entorno

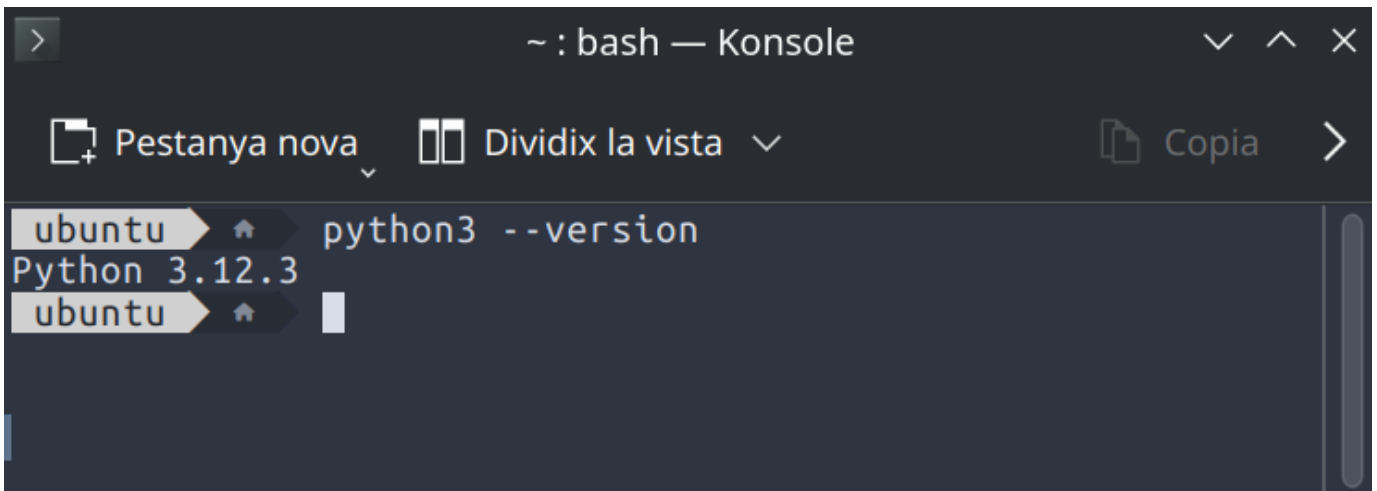
REQUISITOS PREVIOS

- **Python 3.10 o superior** instalado en el sistema
- **VS Code** (Visual Studio Code) u otro editor de código
- Conexión a Internet para instalar dependencias

COMPROBACIÓN DE PYTHON

Abrid un terminal y ejecutad:

```
1 python3 --version
```



Windows

Si usas Windows, el comando es `python --version` en lugar de `python3 --version`.

Si no tenéis Python instalado, descargadlo desde <https://www.python.org/downloads/> o usad el gestor de paquetes de vuestro sistema:

Linux (Debian/Ubuntu) Linux (Fedora) macOS Windows

```
1 sudo apt install python3 python3-pip python3-venv
```

```
1 sudo dnf install python3 python3-pip
```

```
1 brew install python3
```

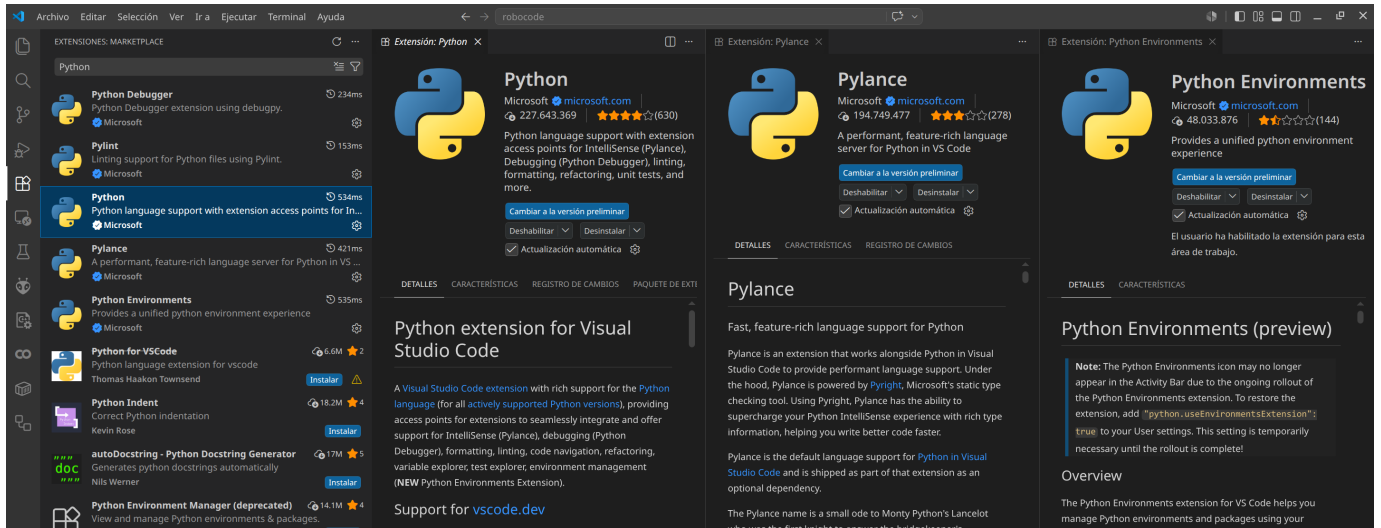
Descargad el instalador desde <https://www.python.org/downloads/> y marcad la opción "Add Python to PATH"

VS CODE Y EXTENSIONES

Si aún no tenéis VS Code, descargadlo desde <https://code.visualstudio.com/>

Una vez instalado, añadid las siguientes extensiones:

1. **Python** (de Microsoft) — todo el soporte para Python: linting, debugging, IntelliSense
2. **Pylance** (de Microsoft) — completado de código y tipado avanzado
3. *Opcional:* **Python Environments** (de Microsoft)



INSTALACIÓN DE LA API DE ROBOCODE

La API de Python para Robocode Tank Royale se distribuye como paquete pip:

```
1 pip install robocode-tank-royale
```

O si trabajáis con entornos virtuales:

```
1 python3 -m venv .venv
2 # Linux/macOS:
3 source .venv/bin/activate
4 # Windows PowerShell:
5 # .venv\Scripts\Activate.ps1
6 # Windows CMD:
7 # .venv\Scripts\activate.bat
8
9 pip install robocode-tank-royale
```

```

ubuntu | robocode | python3 -m venv .venv
ubuntu | robocode | source .venv/bin/activate
ubuntu | .venv | pip install robocode-tank-royale
Collecting robocode-tank-royale
  Using cached robocode-tank-royale-1.0.2-py3-none-any.whl.metadata (6.9 kB)
Requirement already satisfied: pip>=24 in ./.venv/lib/python3.12/site-packages (from robocode-tank-royale) (24.0)
Collecting PyYAML>=6 (from robocode-tank-royale)
  Using cached pyyaml-6.0.3-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (2.4 kB)
Collecting setuptools>=77 (from robocode-tank-royale)
  Using cached setuptools-83.0.0-py3-none-any.whl.metadata (6.6 kB)
Collecting pycountry>=24 (from robocode-tank-royale)
  Using cached pycountry-26.2.16-py3-none-any.whl.metadata (12 kB)
Collecting websockets>=15 (from robocode-tank-royale)
  Using cached websockets-16.1-cp312-cp312-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl.metadata (6.8 kB)
Collecting mpyy (from robocode-tank-royale)
  Using cached mpyy-2.3.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (2.4 kB)
Collecting typing_extensions>=4.6.0 (from mpyy->robocode-tank-royale)
  Using cached typing_extensions-4.16.0-py3-none-any.whl.metadata (3.3 kB)
Collecting mpyy_extensions>=1.0.0 (from mpyy->robocode-tank-royale)
  Using cached mpyy_extensions-1.1.0-py3-none-any.whl.metadata (1.1 kB)
Collecting pathspec>=1.0.0 (from mpyy->robocode-tank-royale)
  Using cached pathspec-1.1.1-py3-none-any.whl.metadata (14 kB)
Collecting librt>=0.13.0 (from mpyy->robocode-tank-royale)
  Using cached librt-0.13.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (1.3 kB)
Collecting ast_serialize>=0.6.0 (from mpyy->robocode-tank-royale)
  Using cached ast_serialize-0.6.0-cp39-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (1.3 kB)
Using cached robocode-tank-royale-1.0.2-py3-none-any.whl (142 kB)
Using cached pycountry-26.2.16-py3-none-any.whl (8.0 MB)
Using cached pyyaml-6.0.3-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (807 kB)
Using cached setuptools-83.0.0-py3-none-any.whl (1.0 MB)
Using cached websockets-16.1-cp312-cp312-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl (187 kB)
Using cached mpyy-2.3.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (15.3 MB)
Using cached ast_serialize-0.6.0-cp39-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (1.3 MB)
Using cached librt-0.13.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (531 kB)
Using cached mpyy_extensions-1.1.0-py3-none-any.whl (5.0 kB)
Using cached pathspec-1.1.1-py3-none-any.whl (57 kB)
Using cached typing_extensions-4.16.0-py3-none-any.whl (45 kB)
Installing collected packages: websockets, typing_extensions, setuptools, PyYAML, pycountry, pathspec, mpyy_extensions, librt, ast_serialize, mpyy, robocode-tank-royale
Successfully installed PyYAML-6.0.3 ast_serialize-0.6.0 librt-0.13.0 mpyy-2.3.0 mpyy_extensions-1.1.0 pathspec-1.1.1 pycountry-26.2.16 robocode-tank-royale-1.0.2 setuptools-83.0.0 typing_extensions-4.16.0 websockets-16.1

```

La GUI de Robocode Tank Royale

DESCARGA Y EJECUCIÓN DE LA GUI

La GUI (interfaz gráfica) de Robocode es **independiente del lenguaje**. El proceso es idéntico tanto si usáis Java como Python:

1. Id a <https://github.com/robocode-dev/tank-royale/releases>
2. Descargad la última versión del fichero `robocode-tankroyale-gui-x.y.z.jar`
3. Ejecutadlo:

```
1 java -jar robocode-tankroyale-gui-x.y.z.jar
```

⚠ Java necesario para la GUI

La GUI requiere Java 11 o superior para ejecutarse, independientemente de que vuestro bot esté escrito en Python. Si no tenéis Java, ved el [Taller T01: Preparar entorno para Java](#) o instalad OpenJDK:

```
1 sudo apt install default-jdk # Linux
```

CONFIGURACIÓN INICIAL DE LA GUI

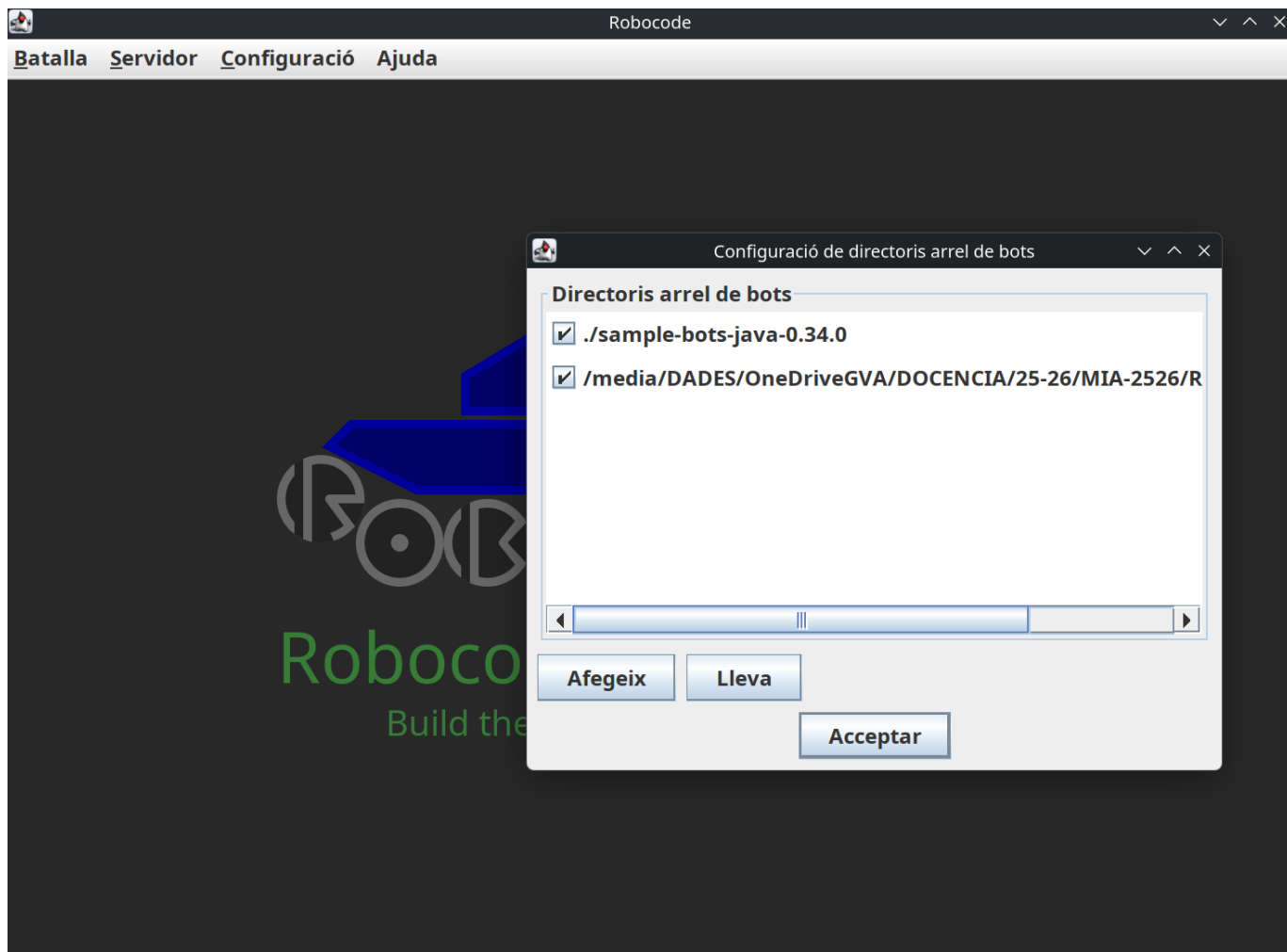
Una vez abierta la GUI:

1. Descargad y descomprimid los **Bots de ejemplo** desde la misma página de releases
2. Id a Config → Bot Root Directories y añadid la carpeta donde habéis descomprimido los bots de ejemplo
3. *Opcional*: descargad y descomprimid `sounds.zip` de los [releases de sonidos](#) en el mismo directorio que el JAR

```

1 [directorio padre]
2 |─ robocode-tankroyale-gui-x.y.z.jar
3 |─ sounds/
4 |   |─ bots_collision.wav
5 |   ...
6 |   |─ wall_collision.wav
7 |─ bots/          <-- carpeta de bots de ejemplo

```



Consejo

Jugad un poco con el entorno GUI, explorad las opciones, tipos de batallas, cread alguna ronda, inicializad bots, añadidlos, ajustad la velocidad de juego, etc.

Vuestro primer Bot en Python

ESTRUCTURA DEL PROYECTO

Cread una carpeta para vuestro bot (por ejemplo, `MiBot`) con la siguiente estructura:

```

1 MiBot/
2 |─ MiBot.py
3 |─ MiBot.json
4 |─ MiBot.cmd      (Windows)
5 |─ MiBot.sh       (Linux/macOS)

```

FICHERO DE CONFIGURACIÓN JSON

El bot necesita un fichero `.json` con la información del bot. Cread `MiBot.json`:

```

1  {
2    "name": "MiBot",
3    "version": "1.0",
4    "authors": ["Vuestro Nombre"],
5    "description": "Mi primer bot con Python",
6    "homepage": "",
7    "countryCodes": ["es"],
8    "platform": "Python",
9    "programmingLang": "Python 3.x"
10 }
```

CÓDIGO BÁSICO DEL BOT

Cread `MiBot.py` con el siguiente contenido:

```

1  import os
2  from robocode_tank_royale.bot_api import Bot, BotInfo
3  from robocode_tank_royale.bot_api.events import ScannedBotEvent, HitByBulletEvent
4
5
6  class MiBot(Bot):
7      def __init__(self):
8          config_path = os.path.join(os.path.dirname(__file__), "MiBot.json")
9          super().__init__(BotInfo.from_file(config_path))
10
11      def run(self):
12          while self.is_running: # type: ignore[attr-defined] # pylint: disable=no-member
13              self.forward(100)
14              self.turn_gun_left(360)
15              self.back(100)
16              self.turn_gun_left(360)
17
18      def on_scanned_bot(self, scanned_bot_event: ScannedBotEvent):
19          del scanned_bot_event
20          self.fire(1)
21
22      def on_hit_by_bullet(self, hit_by_bullet_event: HitByBulletEvent):
23          bearing = self.calc_bearing(hit_by_bullet_event.bullet.direction)
24          self.turn_right(90 - bearing)
25
26
27  def main():
28      bot = MiBot()
29      bot.start()
30
31
32  if __name__ == "__main__":
33      main()
```

**VS Code: Pylint/Pylance no encuentran el paquete**

Si después de seleccionar el intérprete del venv (`Ctrl+Shift+P` → "Python: Select Interpreter" → `./.venv/bin/python`) los errores persisten, es porque Pylint usa su propio intérprete y Pylance necesita el fichero `.vscode/settings.json`.

Cread la carpeta `.vscode/` dentro de la carpeta de vuestro bot (`/home/ubuntu/robocode/MiBot/`) y añadid el siguiente fichero:

```
1 {
2   "python.defaultInterpreterPath": "${workspaceFolder}/.venv/bin/python",
3   "python.linting.pylintEnabled": false,
4   "python.linting.enabled": false,
5   "python.analysis.autoImportCompletions": true
6 }
```

Si no tenéis Pylint instalado en el venv (solo lo tiene el sistema), VS Code puede ejecutar el Pylint del sistema que no ve los paquetes instalados en el venv. Opciones: (1) desactivar Pylint (`"python.linting.enabled": false`) y confiar solo en Pylance, o (2) instalar Pylint en el venv: `pip install pylint`.

Los subrayados rojos desaparecerán al recargar la ventana (`Ctrl+Shift+P` → "Developer: Reload Window").

SCRIPTS DE INICIO**Windows (`MiBot.cmd`)**

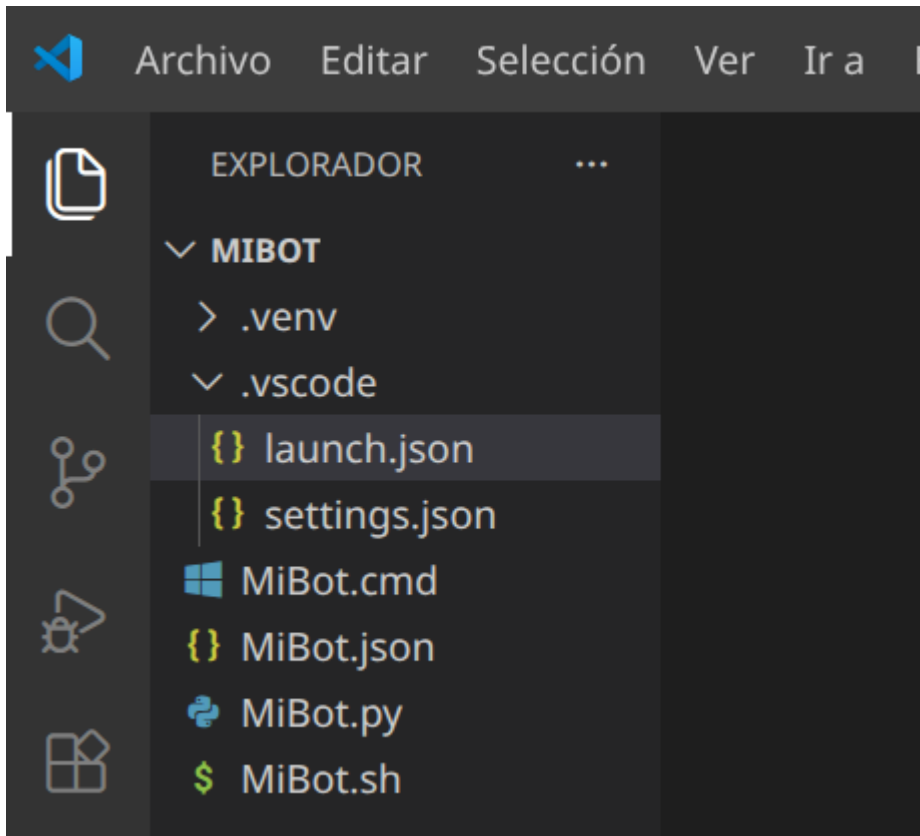
```
1 @echo off
2 python MiBot.py %*
```

Linux/macOS (`MiBot.sh`)

```
1 #!/usr/bin/env sh
2 python3 "${dirname "$0"}/MiBot.py" "$@"
```

Hacedlo ejecutable:

```
1 chmod 755 MiBot.sh
```



PRUEBA DEL BOT

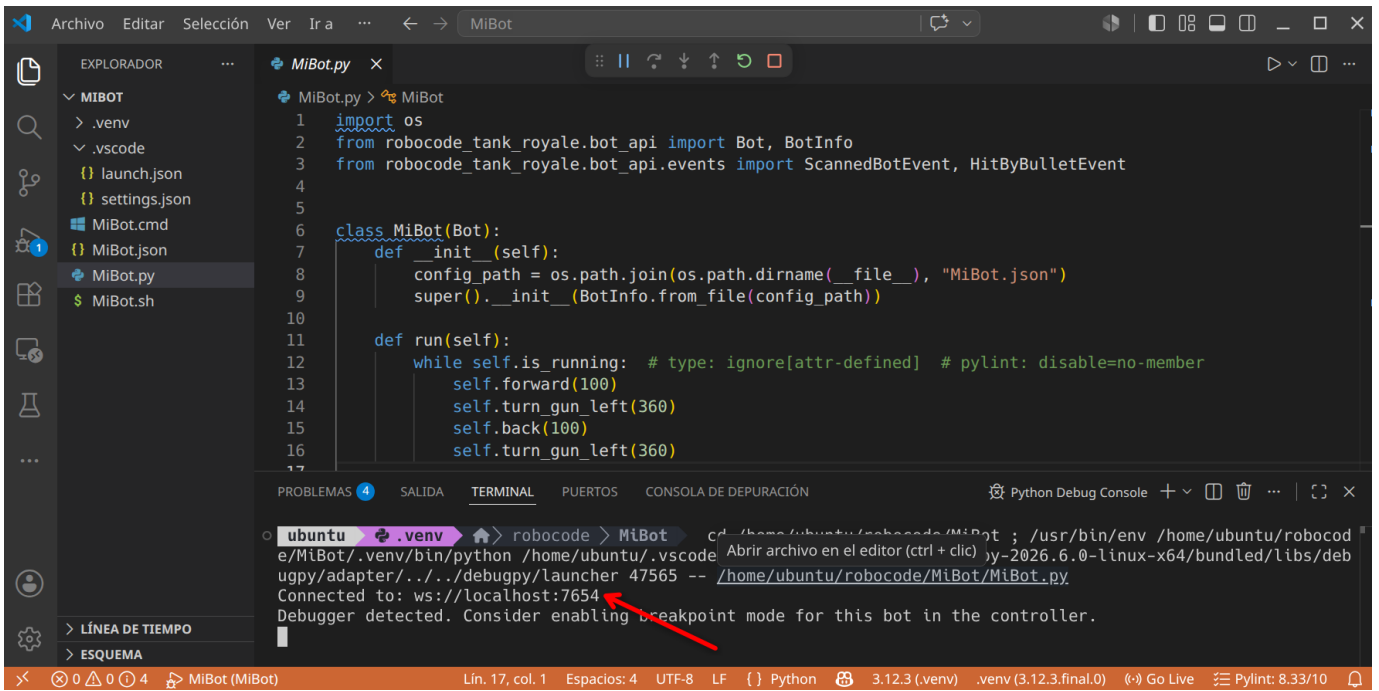
1. Iniciad la GUI de Robocode (`java -jar robocode-tankroyale-gui-x.y.z.jar`)
2. Abrid un terminal en el directorio de vuestro bot
3. Ejecutad:

```
1 python3 MiBot.py
```

Si todo va bien, veréis un mensaje similar a:

```
1 Connected to: ws://localhost:7654
```

Y vuestro bot aparecerá en la lista de bots disponibles en la GUI.



Configuración del servidor y secrets

SECRETS DEL SERVIDOR

El servidor de Robocode puede requerir una contraseña (secret) para conectarse. Esta contraseña se genera automáticamente la primera vez que lanzáis la GUI.

Para habilitar los secrets:

1. Id a Config → Server Options → Enable server secrets en la GUI
2. O editad `server.properties` y añadid:

```

1 server-secrets-enabled=true
2 bots-secrets=CEIABDEPM2025

```

CONECTAR EL BOT CON SECRET

Estableced la variable de entorno `SERVER_SECRET` antes de ejecutar el bot:

Linux/macOS Windows CMD Windows PowerShell

```

1 export SERVER_SECRET=CEIABDEPM2025
2 python3 MiBot.py

```

```

1 set SERVER_SECRET=CEIABDEPM2025
2 python MiBot.py

```

```

1 $Env:SERVER_SECRET = "CEIABDEPM2025"
2 python MiBot.py

```

CONECTARSE A UN SERVIDOR REMOTO

Si el profesor indica una IP remota, podéis especificarla con `SERVER_URL` :

Linux/macOS Windows CMD

```
1 export SERVER_URL=ws://192.168.1.100:7654
2 export SERVER_SECRET=CEIABDEPM2025
3 python3 MiBot.py
```

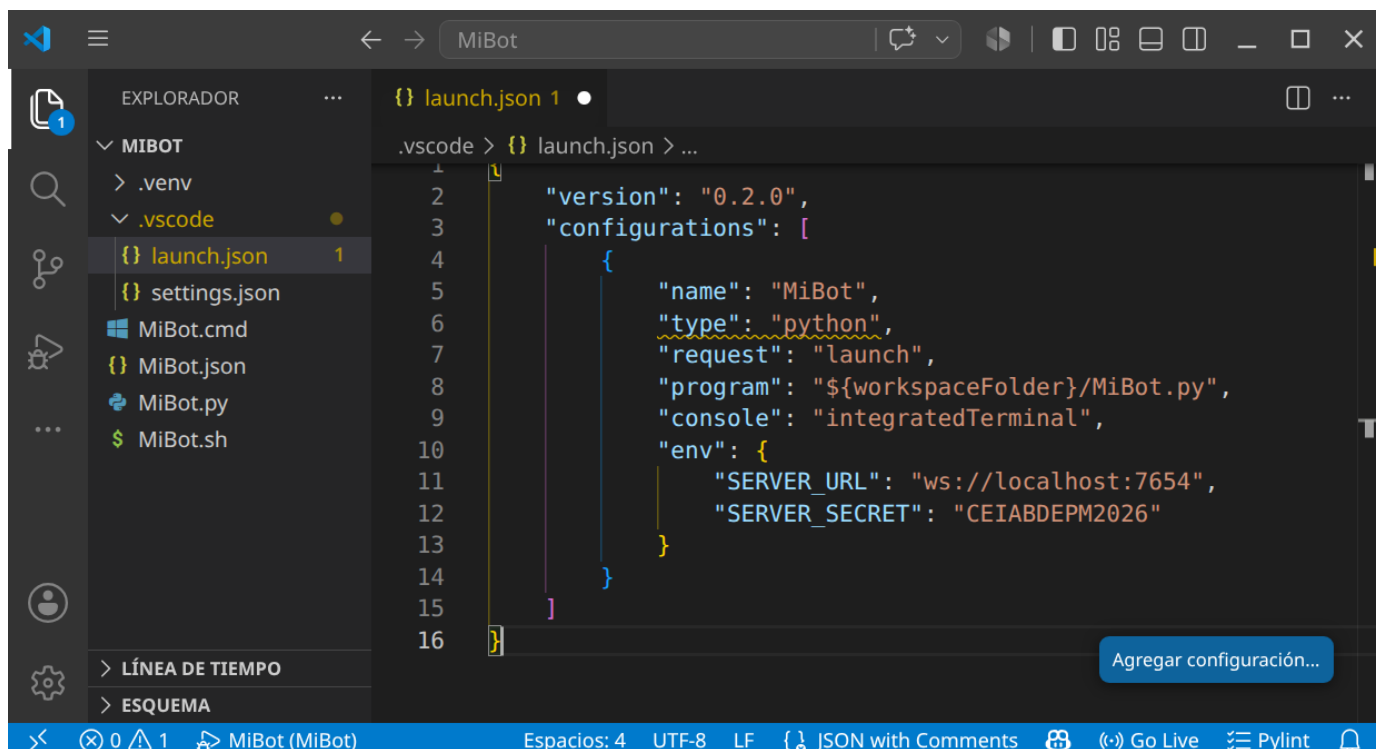
```
1 set SERVER_URL=ws://192.168.1.100:7654
2 set SERVER_SECRET=CEIABDEPM2025
3 python MiBot.py
```

USO CON VS CODE (LAUNCH.JSON)

Para mayor comodidad, podéis configurar VS Code para ejecutar el bot con las variables de entorno definidas en el fichero `.vscode/launch.json` :

Cread la carpeta `.vscode/` dentro del proyecto y el fichero `launch.json` :

```
1 {
2   "version": "0.2.0",
3   "configurations": [
4     {
5       "name": "MiBot",
6       "type": "python",
7       "request": "launch",
8       "program": "${workspaceFolder}/MiBot.py",
9       "console": "integratedTerminal",
10      "env": {
11        "SERVER_URL": "ws://localhost:7654",
12        "SERVER_SECRET": "CEIABDEPM2025"
13      }
14    }
15  ]
16 }
```



Ahora podéis ejecutar el bot directamente desde VS Code pulsando `F5` o yendo a `Run` → `Start Debugging`.

**El servidor debe estar en marcha**

El servidor (la GUI) debe estar inicializado **antes** de lanzar el bot. Si no, obtendréis un error:

```
1 Could not create web socket for URL: ws://localhost:7654
```

API de Python: diferencias clave con Java

La API de Python utiliza **snake_case** en lugar de **camelCase**. Aquí tenéis una tabla de correspondencia:

Java	Python
setTurnLeft(degrees)	set_turn_left(degrees)
setForward(dist)	set_forward(dist)
setTurnRadarLeft(degrees)	set_turn_radar_left(degrees)
setAdjustRadarForBodyTurn(bool)	set_adjust_radar_for_body_turn(bool)
distanceTo(x, y)	distance_to(x, y)
bearingTo(x, y)	bearing_to(x, y)
gunBearingTo(x, y)	gun_bearing_to(x, y)
getX()	self.x (propiedad)
getY()	self.y (propiedad)
getSpeed()	self.speed (propiedad)
getEnergy()	self.energy (propiedad)
getTurnRemaining()	self.turn_remaining (propiedad)
getGunHeat()	self.gun_heat (propiedad)
isRunning()	self.is_running (propiedad)
onScannedBot(e)	on_scanned_bot(self, e)
e.getX()	e.x (propiedad)
e.getY()	e.y (propiedad)
e.getSpeed()	e.speed (propiedad)
e.getDirection()	e.direction (propiedad)
Double.POSITIVE_INFINITY	float("inf")
Math.min(a, b)	min(a, b)
Math.cos(angle)	math.cos(angle)
Math.toRadians(degrees)	math.radians(degrees)

¿Cómo mejoro mi Bot?

Dividiremos todo lo que debemos conocer en 4 apartados:

CONOCIENDO EL CAMPO DE BATALLA

Sigue atentamente la documentación de RCTR sobre la [anatomía de los Bots](#)

Puntos importantes:

- Si el radar no se mueve, no detecta
- Inicialmente el robot, el cañón y el radar se mueven conjuntamente (pero se puede cambiar)

- El Bot se simplifica a un cuadrado de 36x36 unidades.
- Para las colisiones (con Bots o paredes) se simula como un círculo de 18 unidades de radio.

A continuación revisa las **coordenadas y ángulos**

Puntos importantes:

- Este apartado es totalmente diferente al de Robocode Original.

Estudia también las **físicas**

Puntos importantes:

- Se frena más rápido que se acelera.
- Cuanto más rápido vas, más lento giras.
- El cañón gira máximo 20º por turno.
- El radar gira máximo 45º por turno.
- La potencia de tiro influye en el daño provocado y los puntos conseguidos, pero también en el calentamiento del cañón.
- Inicialmente el cañón está caliente (3 unidades), al principio has de esperar a que se enfríe para disparar.
- El atropello da más puntos que los disparos (pero te resta energía)
- Si chocas con una pared, pierdes puntos.
- La energía que te sobra en una ronda no da puntos (modo kamikaze con el último robot?)

Por último ten en cuenta las **puntuaciones**

Puntos importantes:

- Consigues puntos si:
- tu disparo golpea a otro Bot (sino, te resta)
- eres el que mata al Bot
- cada vez que otro Bot muere, pero tu sigues vivo
- eres el último robot vivo
- atropellas a otro Bot
- si matas por atropello a otro Bot
- El último Bot que quede vivo no tiene porqué ser el ganador.

Eventos propios

Definimos una condición, y cuando esta se cumple se dispara un evento:

```
1 from robocode_tank_royale.bot_api.events import Condition
2
3 class TurnCompleteCondition(Condition):
4     def __init__(self, bot):
5         super().__init__()
6         self.bot = bot
7
8     def test(self):
9         return self.bot.turn_remaining == 0
```

Podríamos usar esta condición en un fragmento similar a este:

```
1 self.wait_for(TurnCompleteCondition(self))
```

Podríamos usar funciones lambda de la siguiente manera:

```
1 self.wait_for(Condition(lambda: self.turn_remaining == 0))
```

El resultado de los dos fragmentos sería el mismo.

PERSONALIZAR MI BOT

Podemos establecer colores para el cuerpo, torreta de cañón, el radar, el arco de escaneo y de la bala:

```

1 verde = Color.from_rgb(0, 255, 0)
2 self.set_body_color(Color.from_rgb(0, 0, 0))
3 self.set_turret_color(Color.from_rgb(255, 0, 0))
4 self.set_radar_color(Color.from_rgb(255, 0, 0))
5 self.set_scan_color(Color.from_rgb(255, 255, 0))
6 self.set_bullet_color(Color.from_rgb(255, 255, 255))

```

Puedes encontrar la lista completa de colores disponibles en la API de Robocode TankRoyale.

TÉCNICAS DE ESCANEEO (RADAR)

Sentidos de nuestro Bot:

Sentido del **tacto**, tu Bot sabe cuando:

- golpea una pared (`on_hit_wall`),
- es alcanzado por un disparo (`on_hit_by_bullet`),
- es alcanzado por otro Bot (`on_hit_bot`).

Sentido de la **vista**, tu Bot sabe cuándo ha visto otro robot, pero sólo si lo escanea (`on_scanned_bot`)

Otros sentidos, tu robot también sabe cuándo ha muerto (`on_death`), cuándo ha muerto otro robot (`on_bot_death`).

Además también es consciente de sus balas y sabe cuando una bala ha sido disparada (`on_bullet_fired`) ha alcanzado a un oponente (`on_bullet_hit`), cuando una bala golpea una pared (`on_bullet_wall_hit`) o cuando una bala golpea a otra bala (`on_bullet_hit_bullet`).

Configurar correctamente tu Bot con:

```

1 self.set_adjust_radar_for_body_turn(True) # True si el radar debe ajustar/compensar el giro del cuerpo;
2                                           # False si el radar gira con el cuerpo (por defecto).
3 self.set_adjust_gun_for_body_turn(True)   # True si el cañón debe ajustar/compensar el giro del cuerpo;
4                                           # False si el cañón gira con el cuerpo (por defecto).
5 self.set_adjust_radar_for_gun_turn(True)  # True si el radar debe ajustar/compensar el giro del cañón;
6                                           # False si el radar gira con el cañón (por defecto).

```

Que el radar siempre de vueltas:

```

1 self.set_turn_radar_left(float("inf")) # grados - cantidad de grados a girar a la izquierda. Si es negativo, gira a la
derecha.

```

Revisa el API (`rescan()` y `set_rescan()`)

Fijar el radar en un enemigo (one on one radar):

- Escaneo con multiplicador

```

1 def on_scanned_bot(self, e: ScannedBotEvent):
2     factor = 1.9
3     self.set_turn_radar_left(factor * self.radar_bearing_to(e.x, e.y))

```

Según el factor:

- 1.0 - Bloqueo de radar delgado. Debe llamar a `scan()` para evitar perder el bloqueo. Que Dios te ayude si alguna vez te saltas un turno.
- 1.9 - El arco del radar comienza amplio y se estrecha lentamente tanto como sea posible mientras se mantiene en el objetivo.
- 2.0: el arco del radar recorre un ángulo fijo. El ángulo exacto elegido depende de las posiciones del enemigo y del radar cuando se detecta al enemigo por primera vez. El ángulo se incrementará si es necesario para mantener el bloqueo.
- Arco de radar Ancho

```

1  def on_scanned_bot(self, e: ScannedBotEvent):
2      wide = 36.0
3      # Ángulo absoluto hacia el objetivo
4      angle_to_enemy = self.radar_bearing_to(e.x, e.y)
5      # Distancia que queremos escanear desde el centro del enemigo a cada lado
6      extra_turn = min(
7          math.degrees(math.atan(wide / self.distance_to(e.x, e.y))),
8          self.max_radar_turn_rate
9      )
10     # Ajustamos el giro del radar para que vaya más allá en la dirección que ya gira
11     if angle_to_enemy < 0:
12         angle_to_enemy -= extra_turn
13     else:
14         angle_to_enemy += extra_turn
15     # Giramos el radar
16     self.set_turn_radar_left(angle_to_enemy)

```

- Radar Oscilante (variable global que sepa la dirección y nos ayude a cambiarla de vez en cuando)
- Escaneo más inteligente (seguir al cercano? según la situación?)
- Nos interesa mantener una lista de enemigos? `e.scanned_bot_id`? y por ejemplo fijar el radar en el más débil?

TÉCNICAS DE DESPLAZAMIENTO (MOVIMIENTO)

Pensamiento lateral

Si has jugado baloncesto antes, sabes que si quieres defender a alguien que sostiene el balón, debes maximizar tu movimiento lateral enfrentándote siempre a él (de frente). Lo mismo ocurre con tu robot.

Para conseguir esta posición debemos hacer algo similar a esto:

```

1  self.turn_left(self.bearing_to(e.x, e.y) + 90)

```

que siempre colocará tu robot perpendicular (90 grados) a tu enemigo.

¿Hacia adelante o hacia atrás?

Cuando te enfrentas a un oponente, la idea de "adelante" y "atrás" (del primer Bot) se vuelven algo obsoletas. Probablemente estés pensando más en términos de "atacar a la izquierda" o "atacar a la derecha". Para realizar un seguimiento de la dirección del movimiento, simplemente declara una variable como hicimos para oscilar el radar.

```

1  class MiBot(Bot):
2      def __init__(self):
3          super().__init__(BotInfo.from_file("MiBot.json"))
4          self.move_direction = 1

```

luego, cuando quieras mover tu robot, simplemente puedes decir:

```

1  self.set_forward(100 * self.move_direction)

```

Puedes cambiar de dirección cambiando el valor de `move_direction` de 1 a -1 así:

```

1  self.move_direction *= -1

```

Cambiar de dirección

El enfoque más intuitivo para cambiar de dirección es simplemente cambiar la dirección del movimiento cada vez que golpeas una pared o golpeas a otro robot de esta manera:

```

1  def on_hit_wall(self, e: HitWallEvent):
2      self.move_direction *= -1
3
4  def on_hit_bot(self, e: HitBotEvent):
5      self.move_direction *= -1

```

Sin embargo, descubrirás que si haces eso, terminarás presionando obstinadamente contra un robot que te embiste desde un costado (como un perro en celo). Esto se debe a que se llama a `on_hit_bot` tantas veces que `move_direction` sigue cambiando y nunca te alejas.

Un mejor enfoque es simplemente probar para ver si tu robot se ha detenido. Si es así, probablemente significa que has golpeado algo y querrás cambiar de dirección. Puedes hacerlo con el código:

```
1  if self.speed == 0:
2      self.move_direction *= -1
```

Ponlo en tu método que maneje el movimiento y podrás manejar todos los eventos de impacto con las paredes u otros Bots.

¿Bailamos?

Dando vueltas (Círculos)

Puedes rodear a tu enemigo simplemente usando las técnicas anteriores:

```
1  class MiBot(Bot):
2      def __init__(self):
3          super().__init__(BotInfo.from_file("MiBot.json"))
4          self.move_direction = 1
5          self.ultimo_enemigo_visto = 0
6
7      def run(self):
8          while self.is_running: # type: ignore[attr-defined] # pylint: disable=no-member
9              self.do_move()
10             self.go()
11
12     def on_scanned_bot(self, e: ScannedBotEvent):
13         self.ultimo_enemigo_visto = self.bearing_to(e.x, e.y)
14
15     def do_move(self):
16         if self.speed == 0:
17             self.move_direction *= -1
18         self.set_turn_left(self.ultimo_enemigo_visto + 90)
19         self.set_forward(1000 * self.move_direction)
```

Objetivo: rodea a tu enemigo usando el código de movimiento anterior, como un tiburón rodeando a su presa en el agua.

Evita Ametrallamiento

Un problema que puedes notar con el anterior tipo de movimiento es que es presa fácil para los objetivos predictivos porque sus movimientos son tan... predecibles.

Para evadir las balas de manera más efectiva, debes moverte de lado a lado o "atacar". Una buena forma de hacerlo es cambiar de dirección después de un cierto número de "tics", así:

```
1  def do_move(self):
2      if self.speed == 0:
3          self.move_direction *= -1
4      self.set_turn_left(self.ultimo_enemigo_visto + 90)
5      if self.turn_number % 20 == 0:
6          self.move_direction *= -1
7          self.set_forward(1000 * self.move_direction)
```

Mira como se balancea hacia adelante y hacia atrás usando el código de movimiento anterior. Observa lo bien que esquivo las balas. Pero ojo, porque puede afectar a tu precisión de disparo.

Cada vez más cerca

Notarás que tanto el primero como este último tipo de movimientos tienen otro problema: se atascan fácilmente en las esquinas y terminan golpeándose contra las paredes. Un problema adicional es que si tu enemigo está lejos, disparan mucho pero no aciertan mucho.

Para hacer que tu robot se acerque a tu enemigo, simplemente modifica el código para que se gire ligeramente hacia su enemigo, así:

```
1 self.set_turn_left(self.ultimo_enemigo_visto + (90 - (15 * self.move_direction)))
```

15 es un factor en grados que puedes modificar y ajustar. ¡Prueba!

Modificando el primero conseguimos una variación que utiliza el código anterior para lanzarse en espiral hacia su enemigo.

Mientras que con el segundo que utiliza el código anterior para acercarse cada vez más. También es bastante bueno para evadir balas.

Fíjate que ninguno de los Bots anteriores queda atrapado en una esquina por mucho tiempo.

Evitando las paredes

Un problema con todos los Bots anteriores es que golpean mucho las paredes y golpearlas agota tu energía. Una mejor estrategia sería detenerse antes de chocar contra las paredes. ¿Pero cómo?

Agregar un evento personalizado

Lo primero que debemos hacer es decidir qué tan cerca permitiremos que nuestro robot llegue a las paredes:

```
1 class WallAvoider(Bot):
2     def __init__(self):
3         super().__init__(BotInfo.from_file("WallAvoider.json"))
4         self.wall_margin = 60
5         self.too_close_to_wall = 0
6         self.move_direction = 1
```

A continuación, agregamos un evento personalizado que se activará cuando se cumpla una determinada condición dentro del método `run()`:

```
1 def run(self):
2     # ... inicialización ...
3     # No te acerques mucho a las paredes
4     self.add_custom_event(Condition("TooCloseToWalls", lambda: (
5         self.x <= self.wall_margin or
6         self.x >= self.arena_width - self.wall_margin or
7         self.y <= self.wall_margin or
8         self.y >= self.arena_height - self.wall_margin
9     )))
10    # ... resto del bucle ...
```

Necesitamos hacer override del método `test()` para devolver un valor booleano cuando ocurra nuestro evento personalizado.

Gestionar el evento personalizado

Lo siguiente que debemos hacer es manejar el evento, que se puede hacer así:

```
1 def on_custom_event(self, e: CustomEvent):
2     if e.condition.name == "TooCloseToWalls":
3         self.move_direction *= -1
4         self.set_forward(100 * self.move_direction)
```

Sin embargo, el problema con ese enfoque es que este evento podría dispararse una y otra vez, provocando que cambiemos rápidamente de un lado a otro, sin alejarnos nunca. O si ya aparecemos cerca de la pared quedamos atrapados.

Para evitar este "sacudida de la muerte" deberíamos tener una variable que indique que estamos manejando el evento:

```
1 def on_custom_event(self, e: CustomEvent):
2     if e.condition.name == "TooCloseToWalls":
3         if self.too_close_to_wall <= 0:
4             self.too_close_to_wall += self.wall_margin
5             self.set_max_speed(0)
```

Manejo de los dos modos

Hay dos últimos problemas que debemos resolver. En primer lugar, tenemos un método `do_move()` donde colocamos todo nuestro código de movimiento normal. Si estamos tratando de alejarnos de una pared, no queremos que se llame a nuestro código de

movimiento normal, creando (una vez más) la "sacudida de la muerte". En segundo lugar, queremos volver eventualmente al movimiento "normal", por lo que eventualmente deberíamos tener el "tiempo de espera" de la variable `too_close_to_wall`.

Podemos resolver ambos problemas con la siguiente implementación de `do_move()`:

```
1 def do_move(self):
2     if self.too_close_to_wall > 0:
3         self.too_close_to_wall -= 1
4     if self.speed == 0:
5         self.set_max_speed(8)
6         self.move_direction *= -1
7         self.set_forward(1000 * self.move_direction)
```

Con todo el código anterior evitamos chocar contra las paredes. Observa cómo se desliza suavemente hacia los lados pero nunca (bueno, rara vez) los golpea.

Bot multimodo

Además de los colores que elijas, la mayor parte de la personalidad de tu robot está en su código de movimiento. Por otra parte, situaciones diferentes requieren tácticas diferentes. Usando el ejemplo de evitar paredes, es posible que desees codificar tu bot para que cambie los "modos" según ciertos criterios. Podrías pensar en algo como esto:

```
1 MODO_RONDAR = 0
2 MODO_ESQUIVAR = 1
3 MODO_ASESINO = 2
4
5 class MultiModeBot(Bot):
6     def __init__(self):
7         super().__init__(BotInfo.from_file("MultiModeBot.json"))
8         self.modo_robot = MODO_RONDAR
9
10    def on_bot_death(self, e: BotDeathEvent):
11        if self.enemy_count > 10:
12            self.modo_robot = MODO_RONDAR
13        elif self.enemy_count > 1:
14            self.modo_robot = MODO_ESQUIVAR
15        elif self.enemy_count == 1:
16            self.modo_robot = MODO_ASESINO
17
18    def do_move(self):
19        if self.modo_robot == MODO_RONDAR:
20            # ronda
21            pass
22        elif self.modo_robot == MODO_ESQUIVAR:
23            # esquiva
24            pass
25        elif self.modo_robot == MODO_ASESINO:
26            # asesina
27            pass
```

Los detalles se dejan (como siempre) como ejercicio para el alumnado.

TÉCNICAS DE ATAQUE (DISPARO)

Técnicas básicas

- Separa el movimiento del cañón del movimiento del Bot (`set_adjust_gun_for_body_turn`)
- Técnica de apuntado básica: `set_turn_gun_left` o `set_turn_gun_right` junto con `gun_bearing_to`

Fórmula de cálculo de potencia de fuego

Otro aspecto importante al disparar es calcular la potencia de fuego de tu bala. La documentación del método `fire()` explica que puedes disparar una bala en el rango de `0.1` a `3.0`. Como probablemente ya habrás concluido, es una buena idea disparar balas de baja potencia cuando tu enemigo está lejos y balas de alta resistencia cuando está cerca.

```

1 def smart_fire(self, distance):
2     if distance > 200 or self.energy < 15:
3         self.fire(1)
4     elif distance > 50:
5         self.fire(2)
6     else:
7         self.fire(3)

```

Podrías usar una serie de declaraciones if-elif-else para determinar la potencia de fuego, en función de si el enemigo está a 50 unidades, a 200, etc (como en el fragmento anterior). Pero tales construcciones son demasiado rígidas. Después de todo, el rango de posibles valores de potencia de fuego cae a lo largo de un continuo, no de bloques discretos. Un mejor enfoque es utilizar una fórmula. He aquí un ejemplo:

```

1 self.set_fire(400 / self.distance_to(e.x, e.y))

```

Con esta fórmula, a medida que aumenta la distancia del enemigo, la potencia de fuego disminuye. Asimismo, a medida que el enemigo se acerca, la potencia de fuego aumenta. Los valores superiores a 3 se reducen a 3, por lo que nunca dispararemos una bala mayor que 3, pero probablemente deberíamos reducir el valor de todos modos (solo para estar seguros) de esta manera:

```

1 self.set_fire(min(500 / self.distance_to(e.x, e.y), 3))

```

Evitar disparos prematuros

Una situación que encontraras es que tu Bot dispare antes de haber girado el arma hacia el objetivo. Para evitar disparos prematuros, usa la propiedad `gun_turn_remaining` para ver qué tan lejos está tu cañón del objetivo y no dispare hasta que esté cerca.

Además, no puedes disparar si el arma está "caliente" desde el último disparo y llamar a `fire()` o `set_fire()` solo desperdiciará un turno. Podemos probar si el arma está fría llamando a `gun_heat`.

Apuntando de manera predictiva

O... "Usar la trigonometría para impresionar a tus amigos y destruir a tus enemigos".

Si quisiéramos poder golpear a un robot que recorre las paredes siempre fallaríamos, necesitamos poder predecir dónde estará en el futuro, pero ¿cómo podemos hacerlo?

$\backslash$ Distancia = Velocidad $\times$ Tiempo $\backslash$

Usando la anterior fórmula podemos calcular cuánto tiempo tardará una bala en llegar allí.

- **Distancia:** se puede encontrar llamando a `distance_to(e.x, e.y)`
- **Velocidad:** según la documentación de RCTR, una bala viaja a una velocidad de: $20 - potenciaDeFuego \times 3$
- **Tiempo:** podemos calcular el tiempo resolviendo: $Tiempo = \{Distancia \over Velocidad\}$

El siguiente código lo hace:

```

1 # calcular la potencia basado en la distancia
2 enemy_distance = self.distance_to(e.x, e.y)
3 fire_power = min(500 / enemy_distance, 3)
4 # calcular la velocidad de la bala
5 bullet_speed = 20 - fire_power * 3
6 # calculamos el tiempo que necesita la bala para impactar al enemigo
7 time = int(enemy_distance / bullet_speed)

```

Obteniendo futuras coordenadas X, Y

A continuación, debemos calcular la posición futura de nuestro enemigo:


```

1  import math
2
3  # Calcular la velocidad actual de tu enemigo
4  enemy_velocity = e.speed
5  # Calcular la dirección actual del enemigo
6  enemy_direction = e.direction
7  # Descomponer el desplazamiento en las coordenadas X e Y
8  # Componente X del desplazamiento COSENO
9  delta_x = enemy_velocity * math.cos(math.radians(enemy_direction))
10 # Componente Y del desplazamiento SENO
11 delta_y = enemy_velocity * math.sin(math.radians(enemy_direction))
12
13 # Calcular las coordenadas futuras
14 future_x = e.x + (delta_x * time)
15 future_y = e.y + (delta_y * time)

```

Girando el arma al punto previsto

Ahora debemos apuntar nuestro cañón al lugar previsto de impacto:

```

1  self.set_turn_gun_left(self.gun_bearing_to(future_x, future_y))

```

¡Disparando!

Por último disparamos nuestro cañón:

```

1  if self.gun_turn_remaining <= 0 and self.gun_heat == 0:
2      self.fire(fire_power)

```

Mejor aún...!

Aquí te dejo algunas mejoras para que las valores:

- ¿Debemos esperar siempre a que el arma esté completamente en la dirección calculada?
- Cuando nuestro enemigo se acerca a una pared, nuestros disparos impactan en ella.
- ¿Puedo saber a quién disparo si mi previsión de impacto falla mucho?

Investigación y desarrollo propio

A partir de aquí el trabajo será individual de cada alumno (puede solaparse con el paso 3). Deberéis investigar/prever a vuestros adversarios (los conocidos, y los de vuestros compañeros). Aplicar las técnicas que consideréis más útiles para intentar quedar lo más arriba posible en la tabla de clasificación.

Algunos recursos adicionales:

- [Documentación oficial de la API Python](#)
- [Tutorial oficial "My First Bot for Python"](#)
- [The Book of Robocode](#) — estrategias avanzadas
- [RoboWiki](#) — conocimiento comunitario (código en Java, conceptos universales)

🕒 15 de julio de 2026

Entregable de Robocode Tankroyale

Introducción

Robocode es un juego de programación donde el objetivo es codificar un bot en forma de tanque virtual para competir contra otros bots en un campo de batalla virtual. El jugador es el programador del bot, que no tendrá influencia directa en el juego. En cambio, el jugador debe escribir un programa para el cerebro del robot. El programa dice cómo debe comportarse y reaccionar el robot ante los eventos que ocurren en el campo de batalla.

El nombre Robocode proviene de una versión anterior del juego y es una abreviatura de "Código de robot". Con esta nueva versión, se utiliza el mundo "bot" en lugar de "robot".

El juego está diseñado para ayudarte a aprender a programar y mejorar tus habilidades de programación y divertirtiarte mientras lo haces. Robocode también es útil a la hora de estudiar o mejorar el aprendizaje automático (En nuestro caso solo Inteligencia Artificial) en un juego rápido en tiempo real.

Las batallas de Robocode tienen lugar en un campo de batalla, donde pequeños robots tanque automatizados luchan hasta que solo queda uno, como en un juego Battle Royale. De ahí el nombre Tank Royale.

Robocode no contiene sangre, visceras, personas ni política. Las batallas son simplemente por la emoción de la competición que tanto nos gusta.

Documentación del juego: <https://robocode-dev.github.io/tank-royale/>

Repositorio en GitHub: <https://github.com/robocode-dev/tank-royale>

Documentación de la API:

- **Java (JVM)**
- [API overview](#)
- [Bot API](#)
- **.Net**
- [API overview](#)
- [Bot API](#)



Importante

Ojo! RCTR se basa en una versión más antigua de RoboCode, con un API diferente, y por lo tanto los robots anteriores, y el funcionamiento del campo de batalla han cambiado sustancialmente.

Por lo tanto, mucha de la documentación que encontrareis es sobre el sistema antiguo, en la que la parte de estrategias es válida, pero no el código que la acompaña. La tarea de traducción del antiguo sistema al nuevo se ha llevado a cabo en forma de un "bridge" entre las dos versiones, y está disponible en GitHub: <https://github.com/robocode-dev/robocode-api-bridge>

En cuanto a la documentación y estrategias, la API antigua todavía se puede encontrar en: <https://robocode.sourceforge.io/docs/robocode/>

Y una página que contaba con muchísima información sobre estrategias, robots, código, etc ya solo está disponible en archive.org, la última versión cacheada que he encontrado disponible está en: <https://web.archive.org/web/20200323061702/http://robowiki.net/>

Objetivo de la práctica

Usando RoboCode Tank Royale (RCTR) intentaremos ponernos en el papel de los primeros programadores que dotaban de "inteligencia" a los primeros sistemas. Tal y como hemos visto en el apartado de teoría estos sistemas solo reaccionan ante estímulos que previamente haya previsto su programador, carecen de intuición (a no ser que sea simulada) y el resultado de su inteligencia es tan bueno como lo sea su programación.

El profesor establecerá unos robots (simples) que deberemos derrotar para superar la práctica, dotando de cierta "inteligencia" a nuestro robot. No sirve que sea por suerte o de manera aleatoria, debe estar razonada y documentada.

El siguiente paso, una vez aprobado, será una competición entre todo el alumnado para valorar quien ha sido el que mejor Bot ha diseñado, obteniendo así mayor nota que el resto.

Pasos a seguir

1. Preparación del entorno

En una sesión conjunta prepararemos nuestro entorno de trabajo, instalaremos todo lo necesario y haremos algún combate de pruebas.

2. Mi primer Bot

Siguiendo la guía de la documentación, y con la ayuda del profesor todo el alumnado generará su primer Bot de muestra, aprenderemos a añadirlo a la batalla y a depurar su funcionamiento.

3. ¿Cómo mejoro mi Bot?

Con la ayuda del profesor estudiaremos diferentes estrategias y mejoras que podemos aplicar a nuestro robot para así dotarle de inteligencia.

4. Investigación y desarrollo propio

A partir de aquí el trabajo será individual de cada alumno (puede solaparse con el paso 3). Deberéis investigar/prever a vuestros adversarios (los conocidos, y los de vuestros compañeros). Aplicar las técnicas que consideréis más útiles para intentar quedar lo más arriba posible en la tabla de clasificación.

¿Qué debo entregar?

A través de la plataforma de AULES todo el alumnado deberá entregar **un archivo ZIP** que contenga:

El código fuente de su Bot (el nombre del bot será el nombre de su autor más los 4 últimos dígitos de su DNI o NIE (sin letras)) y la memoria justificativa en formato PDF.

El código fuente del Bot incluye (ejemplo con mi nombre):

Si usas Java: - Archivo .java con la clase del Bot (`David4849.java`) - Archivo .json con la información del autor (`David4849.json`) - Archivos para inicializar el Bot en Windows y Linux (`David4849.cmd` y `David4849.sh`)

Si usas Python: - Archivo .py con la clase del Bot (`David4849.py`) - Archivo .json con la información del autor (`David4849.json`) - Archivos para inicializar el Bot en Windows y Linux (`David4849.cmd` y `David4849.sh`)

La memoria en formato PDF debe contener al menos los siguientes apartados:

- Datos del alumno
- Descripción del funcionamiento: estructura, sistema basado en casos, análisis y evolución de la solución, etc.
- Descripción detallada de los métodos definidos y/o usados
- Conclusiones
- Webgrafía/Bibliografía

Requisitos mínimos

- **Versión 0.34.0 de la API** (Cambiado el 20/10/25, ya tenemos la versión multi-idioma y con escalado. A no ser que se encuentren errores.)
- Modo **classic**
- 10 asaltos
- RamFire, Walls, SpinBot
- Ganar por puntuación acumulada

- Tamaño del campo: 2000 x 2000
- Velocidad de enfriamiento: 0.1
- Máximo tiempo de inactividad: 450
- No llamar a métodos prohibidos
- Demostrar IA (no por azar)
- Los combates serán grabados y devueltos al alumno como feedback por si los quiere revisar.

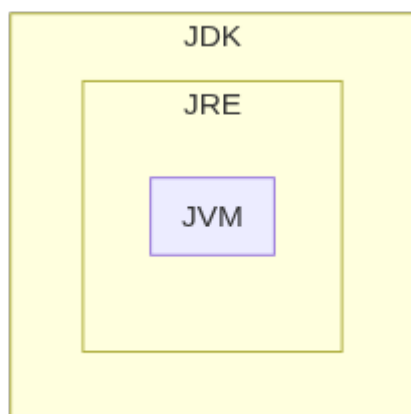
🕒 15 de julio de 2026

3.4 Talleres

Taller UD01_T01: Preparar entorno para Java

Cada software y cada entorno de desarrollo tiene unas características y funcionalidades específicas. Esto también se verá reflejado en la instalación y configuración del software. Dependiendo de la plataforma, entorno o sistema operativo en el que se vaya a instalar el software, se utilizará un paquete de instalación u otro, y habrá que tener en cuenta unas opciones u otras en su configuración. A continuación se muestra cómo instalar una herramienta de desarrollo de software integrada, como Eclipse. Pero también podrás observar los procedimientos para instalar otras herramientas necesarias o recomendadas para trabajar con el lenguaje de programación JAVA, como Tomcat o la Máquina Virtual de Java. Debes tener en cuenta los siguientes conceptos:

- La JVM (Java Virtual Machine, máquina virtual de Java) es la encargada de interpretar el bytecode y generar el código máquina del ordenador (o dispositivo) en el que se ejecuta la aplicación. Esto quiere decir que necesitamos una JVM distinta para cada entorno.
- JRE (Java Runtime Environment) es un conjunto de utilidades Java que incluye la JVM, las bibliotecas y el conjunto de software necesario para ejecutar aplicaciones cliente Java, así como el conector para que los navegadores de Internet ejecuten applets.
- JDK (Java Development Kit) es el conjunto de herramientas para desarrolladores; contiene, entre otras cosas, el JRE y el conjunto de herramientas necesarias para compilar el código, empaquetarlo, generar documentación...



El proceso de instalación consta de los siguientes pasos: 1. Descargue, instale y configure el JDK. 2. Descargue e instale un servidor web o de aplicaciones. 3. Descargue, instale y configure el IDE (Netbeans o Eclipse). 4. Configurar JDK con IDE. 5. Configure el servidor web o de aplicaciones con el IDE instalado. 6. Si es necesario, instalación de conectores. 7. Si es necesario, instale un nuevo software.

Descargue e instale el JDK

Podemos diferenciar entre:

- Java SE (Java Standard Edition): es la versión estándar de la plataforma, siendo esta plataforma la base para todos los entornos de desarrollo Java ya sea de aplicaciones cliente, de escritorio o web.
- Java EE (Java Enterprise Edition): esta es la versión más grande de Java y generalmente se utiliza para crear grandes aplicaciones cliente/servidor y para el desarrollo de servicios web.

En este curso se utilizarán las funcionalidades de Java SE. El archivo es diferente según el sistema operativo donde se tenga que instalar. Así:

- Para los sistemas operativos Windows y Mac OS hay un archivo instalable.
- Para los sistemas operativos GNU/Linux que admiten paquetes .rpm o .deb, también están disponibles paquetes de este tipo.
- Para el resto de sistemas operativos GNU/Linux existe un archivo comprimido (terminado en .tar.gz).

En los dos primeros casos, simplemente hay que seguir el procedimiento de instalación habitual del sistema operativo con el que estamos trabajando. En este último caso, sin embargo, hay que descomprimir el archivo y copiarlo en la carpeta donde se desea instalar. Normalmente, todos los usuarios tendrán permisos de lectura y ejecución en esta carpeta.

A partir de la versión 11 de JDK, Oracle distribuye el software con una licencia significativamente más restrictiva que las versiones anteriores. En particular, solo se puede utilizar para "desarrollar, probar, crear prototipos y demostrar sus aplicaciones". Cualquier uso "para fines comerciales, de producción o empresariales internos" distinto del mencionado anteriormente queda explícitamente excluido.

Si lo necesitas para alguno de estos usos no permitidos en la nueva licencia, además de las versiones anteriores del JDK, existen versiones de referencia de estas versiones licenciadas "GNU General Public License version 2, with the Classpath Exception", que permiten la mayoría de los usos habituales. Estas versiones están enlazadas a la misma página de descarga y también a la dirección jdk.java.net.

Una alternativa es utilizar <https://adoptium.net/> antes conocido como adoptOpenJDK, que ahora se ha integrado en la fundación Eclipse. Desde allí podemos descargar los binarios de la versión openJDK para nuestra plataforma sin restricciones. [Noticia completa] (<https://es.wikipedia.org/wiki/OpenJDK>).

En GNU/Linux podemos utilizar los comandos:

- `sudo apt install default-jdk` para instalar el jdk predeterminado.
- `java --version` para ver las versiones disponibles en nuestro sistema.
- `sudo update-alternatives --config java` para elegir cuál de las versiones instaladas queremos usar por defecto o incluso ver la ruta de las diferentes versiones que tenemos instaladas.

Configurar las variables de entorno "JAVA_HOME" y "PATH"

Una vez descargado e instalado el JDK, debes configurar algunas variables de entorno:

- La variable `JAVA_HOME`: indica la carpeta donde se ha instalado el JDK. No es obligatorio definirla, pero es muy cómodo hacerlo, ya que muchos programas buscan en ella la ubicación del JDK. Además, resulta muy fácil definir las dos variables siguientes.
- La variable `PATH`. Debe apuntar al directorio que contiene el ejecutable de la máquina virtual. Suele ser la subcarpeta `bin` del directorio donde hemos instalado el JDK.

Variable CLASSPATH Otra variable que tiene en cuenta el JDK es la variable `CLASSPATH`, que apunta a las carpetas donde se encuentran las librerías de la aplicación que se quiere ejecutar con el comando `java`. Es preferible, no obstante, indicar la ubicación de estas carpetas con la opción `-cp` del mismo comando `java`, ya que cada aplicación puede tener diferentes librerías y las variables de entorno afectan a todo el sistema. Establecer la variable `PATH` es esencial para que el sistema operativo encuentre los comandos JDK y pueda ejecutarlos.

IntelliJ

IntelliJ IDEA es un entorno de desarrollo integrado (IDE) escrito en Java para desarrollar software informático escrito en Java, Kotlin, Groovy y otros lenguajes basados en JVM. Está desarrollado por JetBrains (antes conocido como IntelliJ) y está disponible como una edición comunitaria con licencia Apache 2 y en una edición comercial propietaria. Ambas se pueden utilizar para el desarrollo comercial.

Nuestra institución dispone de licencias para nuestros alumnos mientras tengáis correo electrónico @ieseduardoprime.es.

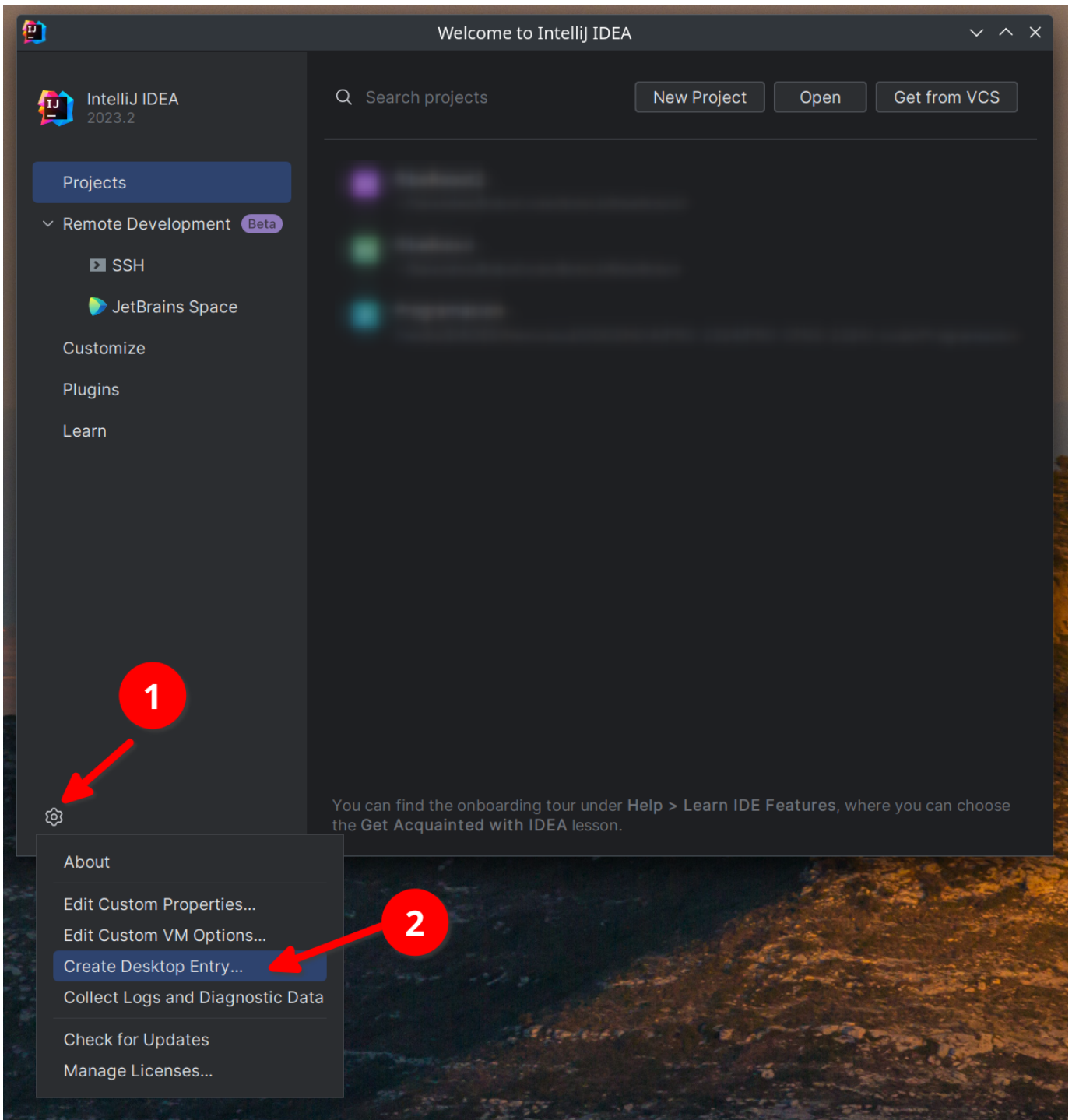
INSTALACIÓN

Descargue desde <https://www.jetbrains.com/idea/> la versión de la herramienta toolbox correspondiente a su sistema operativo.

Siga las instrucciones para su sistema operativo desde <https://www.jetbrains.com/help/idea/installation-guide.html#toolbox>

Una vez instalada la caja de herramientas, puede elegir instalar todos los productos de JetBrains.

Una vez instalada la Idea (IDE) puedes crear una entrada de escritorio desde la pantalla inicial:



Y en la opción Administrar licencias debes seguir estas instrucciones: https://www.jetbrains.com/help/idea/activating_license.html

La dirección del servidor es: <https://iesepm.fls.jetbrains.com/>

AJUSTES

Documentos para configurar su IDE: <https://www.jetbrains.com/help/idea/configuring-project-and-ide-settings.html>

MÓDULOS

Puedes agregar complementos siguiendo estas instrucciones:

<https://www.jetbrains.com/help/idea/managing-plugins.html>

USO BÁSICO ("¡HOLA MUNDO!")

Los documentos te ayudan con tu primer programa en Java: <https://www.jetbrains.com/help/idea/creating-and-running-your-first-java-application.html>

Mucha más información:

- Si vienes de Eclipse: <https://www.jetbrains.com/help/idea/migrating-from-eclipse-to-intellij-idea.html>
- Si estuvieras en NetBeans: <https://www.jetbrains.com/help/idea/netbeans.html>
- Si quieres aprender por tu cuenta: <https://www.jetbrains.com/help/idea/product-educational-tools.html>

Por qué debería elegir IntelliJ en lugar de VsCode para la codificación en Java

IDEA INTELLIJ:

Ventajas:

1. **Entorno integrado completo:** IntelliJ IDEA está diseñado específicamente para el desarrollo de Java y ofrece un conjunto completo de herramientas y características optimizadas para esta tarea.
2. **Análisis estático avanzado:** Proporciona un análisis de código en profundidad que detecta errores y problemas potenciales antes de la compilación.
3. **Depuración avanzada:** ofrece un potente conjunto de herramientas de depuración que ayudan a identificar y resolver problemas en el código.
4. **Refactorización guiada:** Proporciona herramientas para reorganizar y optimizar el código de forma segura, promoviendo buenas prácticas de programación.
5. **Compatibilidad con marcos y tecnologías Java:** Integración nativa con muchos marcos y tecnologías utilizados en el desarrollo Java, lo que facilita la creación de aplicaciones completas.
6. **Generación automática de código:** ayuda a los programadores a generar automáticamente fragmentos de código repetitivos, como captadores y definidores.
7. **Integración con herramientas de compilación:** facilita la integración con herramientas de compilación como Maven y Gradle.
8. **Soporte para pruebas unitarias:** Ofrece integración con marcos de prueba como JUnit para el desarrollo basado en pruebas.
9. **Facilidad de configuración:** Proporciona asistentes guiados para configurar de manera eficiente proyectos Java.

Contras:

1. **Mayor consumo de recursos:** Debido a su naturaleza integral y rica en funciones, IntelliJ IDEA puede consumir más recursos del sistema en comparación con IDE más livianos.
2. **Curva de aprendizaje:** Dado que ofrece una amplia gama de funciones, los principiantes pueden tardar un tiempo en familiarizarse con todas las herramientas disponibles.

VISUAL STUDIO CODE (VSCODE):

Ventajas:

1. **Ligero y rápido:** VSCode es un editor de código liviano y rápido, lo que lo hace ideal para proyectos más pequeños o para aquellos que prefieren una experiencia más ágil.
2. **Amplia gama de extensiones:** Tiene una amplia comunidad que desarrolla extensiones para diversas tecnologías y lenguajes, incluido Java.
3. **Versatilidad:** Si bien no está diseñado específicamente para Java, se puede personalizar para que funcione con Java a través de extensiones.
4. **Integración de control de versiones:** ofrece integración nativa con sistemas de control de versiones como Git.
5. **Curva de aprendizaje rápida:** Debido a su enfoque más ligero, puede resultar más sencillo para los principiantes comenzar a trabajar con él.

Contras:

1. **Funcionalidad limitada de Java:** Aunque existen extensiones de Java, VSCode no ofrece el mismo conjunto completo de herramientas optimizadas para Java que IntelliJ IDEA.

2. **Análisis menos profundo:** Las capacidades de análisis estático y corrección de código podrían no ser tan avanzadas como las de IntelliJ IDEA.
3. **Depuración limitada:** si bien ofrece depuración, es posible que no sea tan avanzada o completa como la de IntelliJ IDEA.
4. **Configuración manual del proyecto:** La configuración de proyectos Java puede requerir más pasos y configuración manual en comparación con IntelliJ IDEA.

Python y VS Code

Si vas a hacer la actividad Robocode en **Python**, necesitarás configurar un entorno alternativo al Java/IntelliJ.

INSTALACIÓN DE PYTHON

Descarga Python desde <https://www.python.org/downloads/> o usa el gestor de paquetes:

Linux (Debian/Ubuntu)	macOS	Windows
1 <code>sudo apt install python3 python3-pip python3-venv</code>		
	1 <code>brew install python3</code>	

Descarga el instalador desde <https://www.python.org/downloads/> y **marca la opción "Add Python to PATH"** durante la instalación.



Windows

Si usas Windows, el comando es `python --version` en lugar de `python3 --version`.

Verifica la instalación:

```
1 python3 --version
```

VISUAL STUDIO CODE

Descarga VS Code desde <https://code.visualstudio.com/>

Instala las siguientes extensiones desde el mercado de extensiones (icono de cuadrícula en la barra lateral izquierda):

1. **Python** (Microsoft) — soporte completo: linting, debugging, IntelliSense
2. **Pylance** (Microsoft) — completado de código y tipado avanzado

ENTORNO VIRTUAL Y PRIMERA PRUEBA

Crea una carpeta para tus proyectos, abre un terminal y ejecuta:

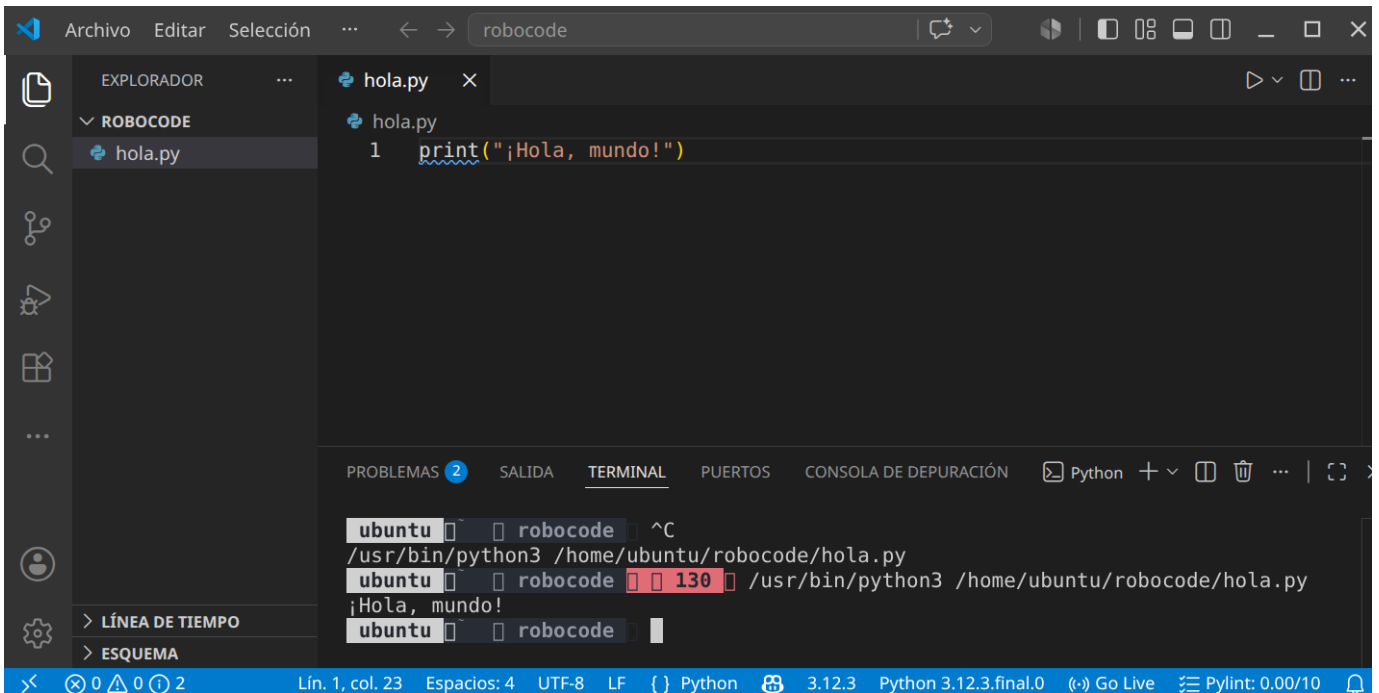
```
1 python3 -m venv .venv
2 # Linux/macOS:
3 source .venv/bin/activate
4 # Windows:
5 # .venv\Scripts\activate
```

Crea un fichero `hola.py`:

```
1 print("¡Hola, mundo!")
```

Ejecútalo:

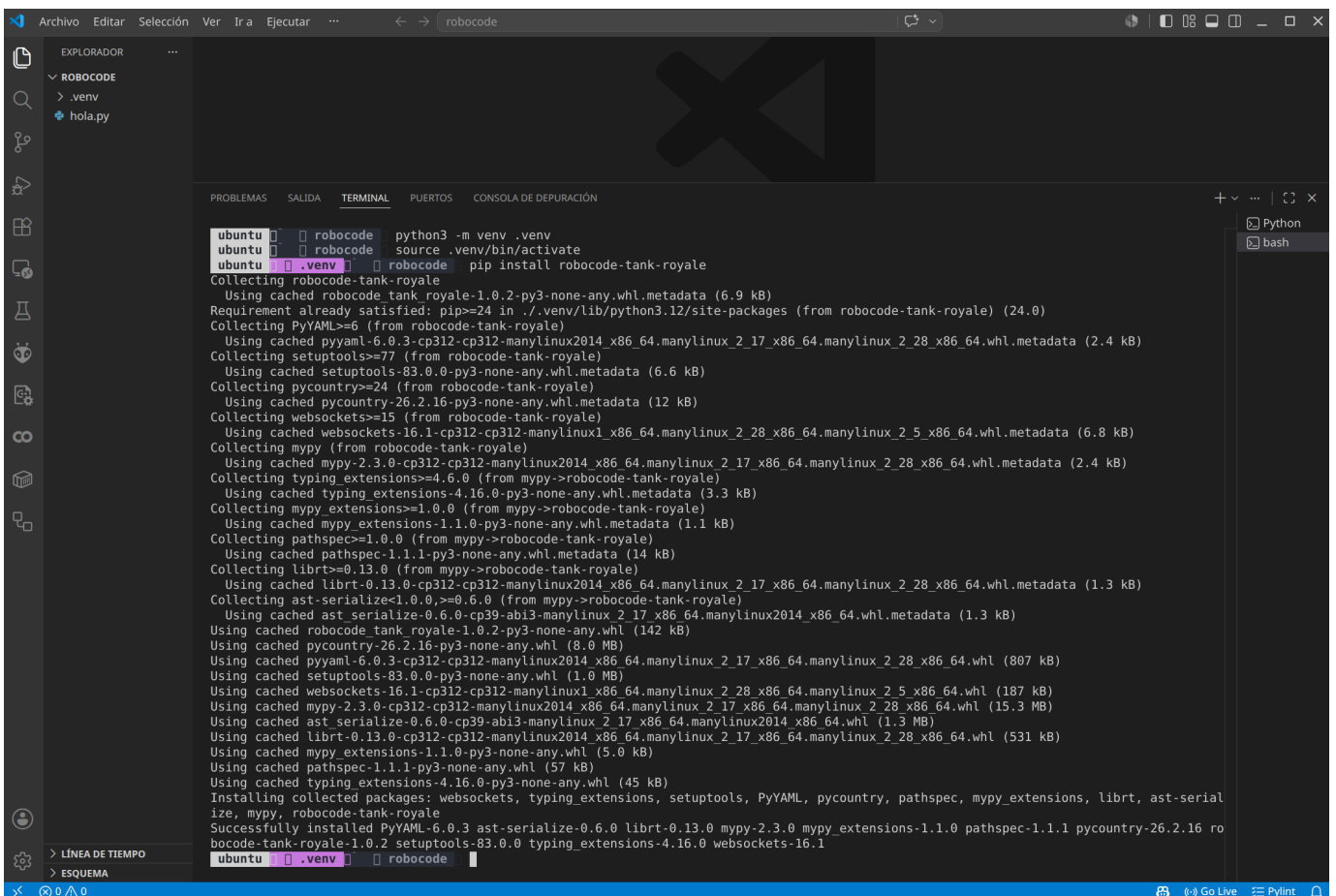
```
1 python3 hola.py
```



INSTALACIÓN DE LA API DE ROBOCODE PARA PYTHON

Cuando llegues a la actividad Robocode, instala la API con:

```
1 pip install robocode-tank-royale
```



Tarea

Debes entregar un documento *.pdf explicando:

• Si eliges Java:

- Captura del comando `java --version`
- Capturas donde se vea que editas `HolaMundo.java`, lo compilas y lo ejecutas en IntelliJ

• Si eliges Python:

- Captura del comando `python3 --version`
- Captura de VS Code con el fichero `hola.py` abierto, con la extensión Python visible y la terminal mostrando la salida

🕒 18 de agosto de 2026

Taller UD01_T02: Crear cuenta en GitHub

Qué es GitHub

GitHub es una plataforma en la nube basada en Git que permite a los desarrolladores almacenar, gestionar y colaborar en proyectos de código. Es el portafolio universal de los programadores.

Crear una cuenta es esencial para quien aprende o busca trabajar en programación porque: sirve como tu currículum técnico, donde muestras tus proyectos y evolución; te permite colaborar en proyectos open source para ganar experiencia real; y es una herramienta fundamental para el control de versiones y trabajo en equipo, usada por prácticamente todas las empresas tech.

Creas tu cuenta

Accede a la plataforma GitHub: <https://github.com/>

Pulsa sobre el botón [Sign Up] y sigue las instrucciones para crear tu cuenta.

Una vez creada tu cuenta, entra en tu página principal, por ejemplo la mía es esta: <https://github.com/martinezpenya> (martinezpenya es mi usuario de github) y realiza una captura de pantalla.

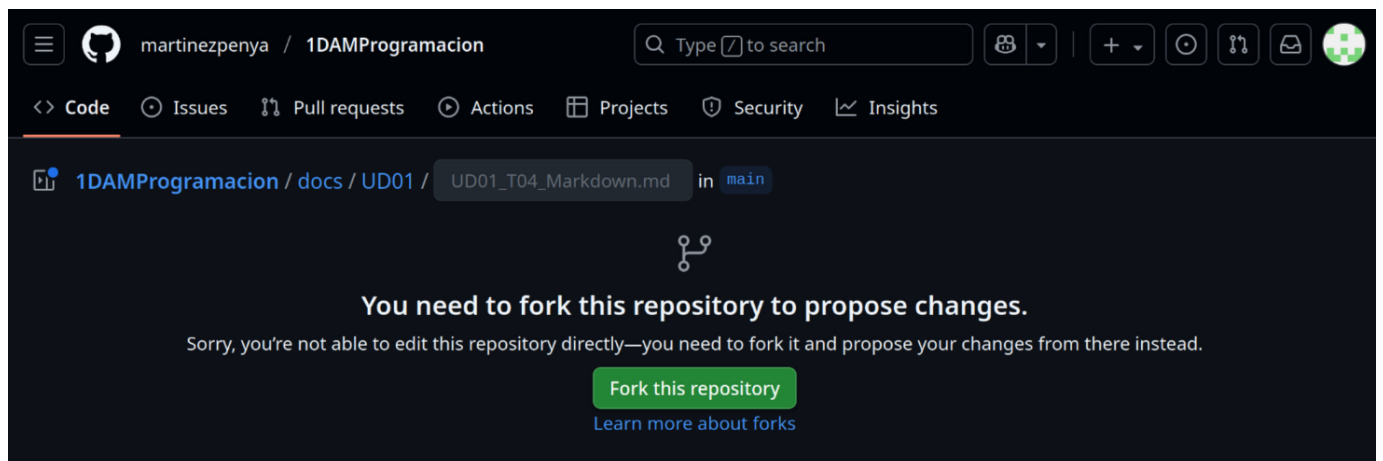
Solicitar corrección de los apuntes

Ahora, para probar nuestra nueva cuenta y colaborar con algún proyecto, no hay nada mejor que ayudar a mejorar los apuntes del profesor de Programación 😊.

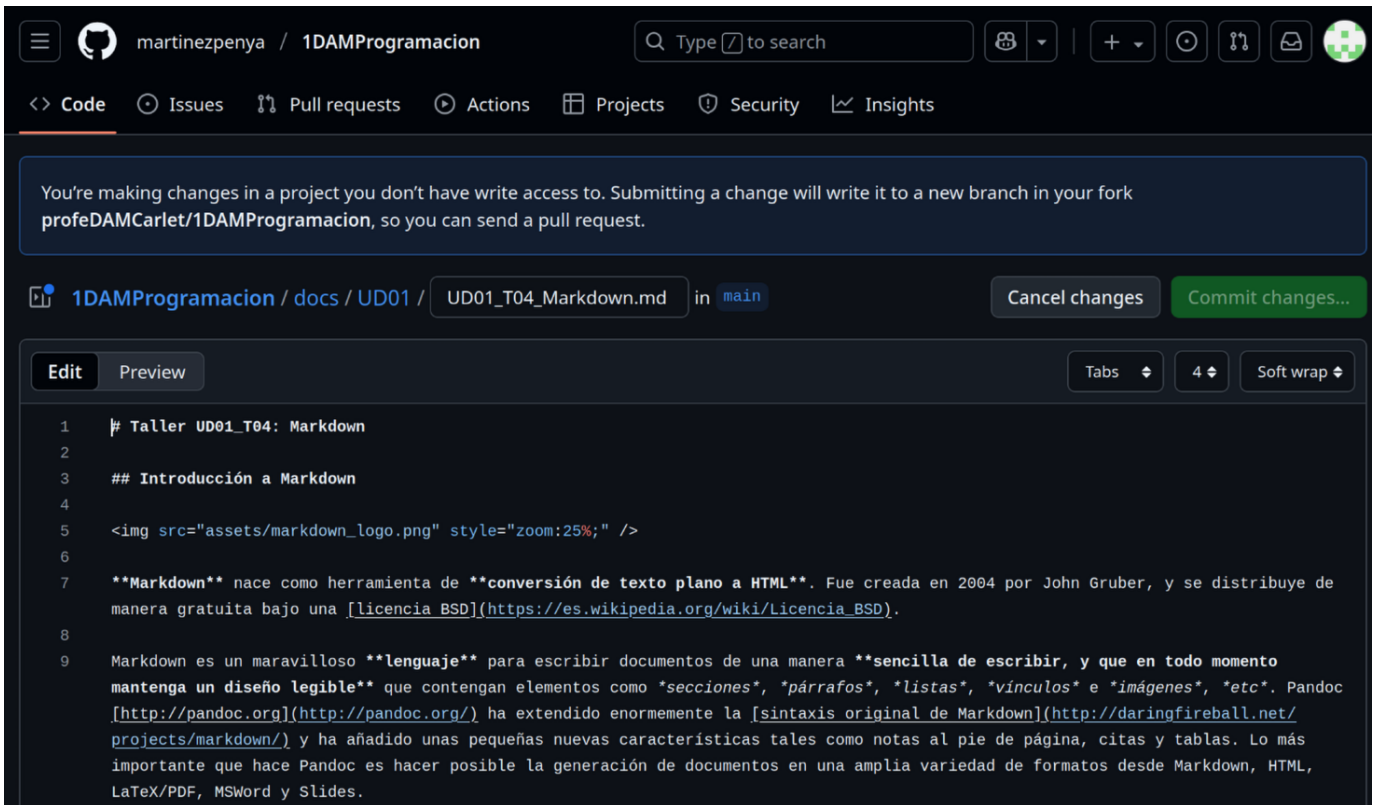
Accedemos a la página de los apuntes en la que hemos detectado el error o queremos sugerir un cambio y en la parte superior derecha debe aparecer el icono:



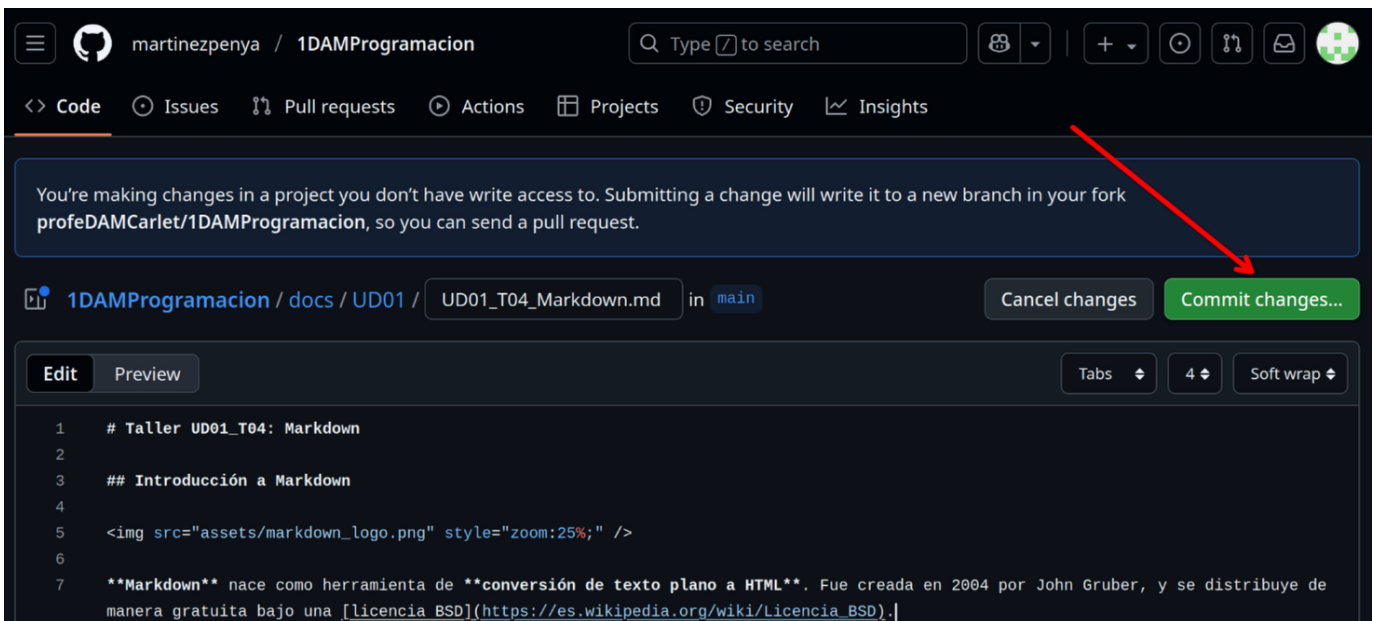
Esto nos llevará a crear un Fork del repositorio (este concepto lo aprenderás más adelante en el módulo de Entornos de Desarrollo):



Ahora debemos pulsar el botón **[Fork this repository]**, y a continuación veremos el código de la página en nuestro fork que es `MarkDown` (Puedes aprender más sobre `MarkDown` en el Taller 4):



Ahora debemos buscar el texto a modificar y una vez hayamos cambiado algo del documento se activará el botón [Commit changes...]:



Ahora debes explicar cual ha sido la modificación que hemos realizado y pulsar el botón [Propose changes]:

Propose changes

Commit message

Update UD01_T04_Markdown.md

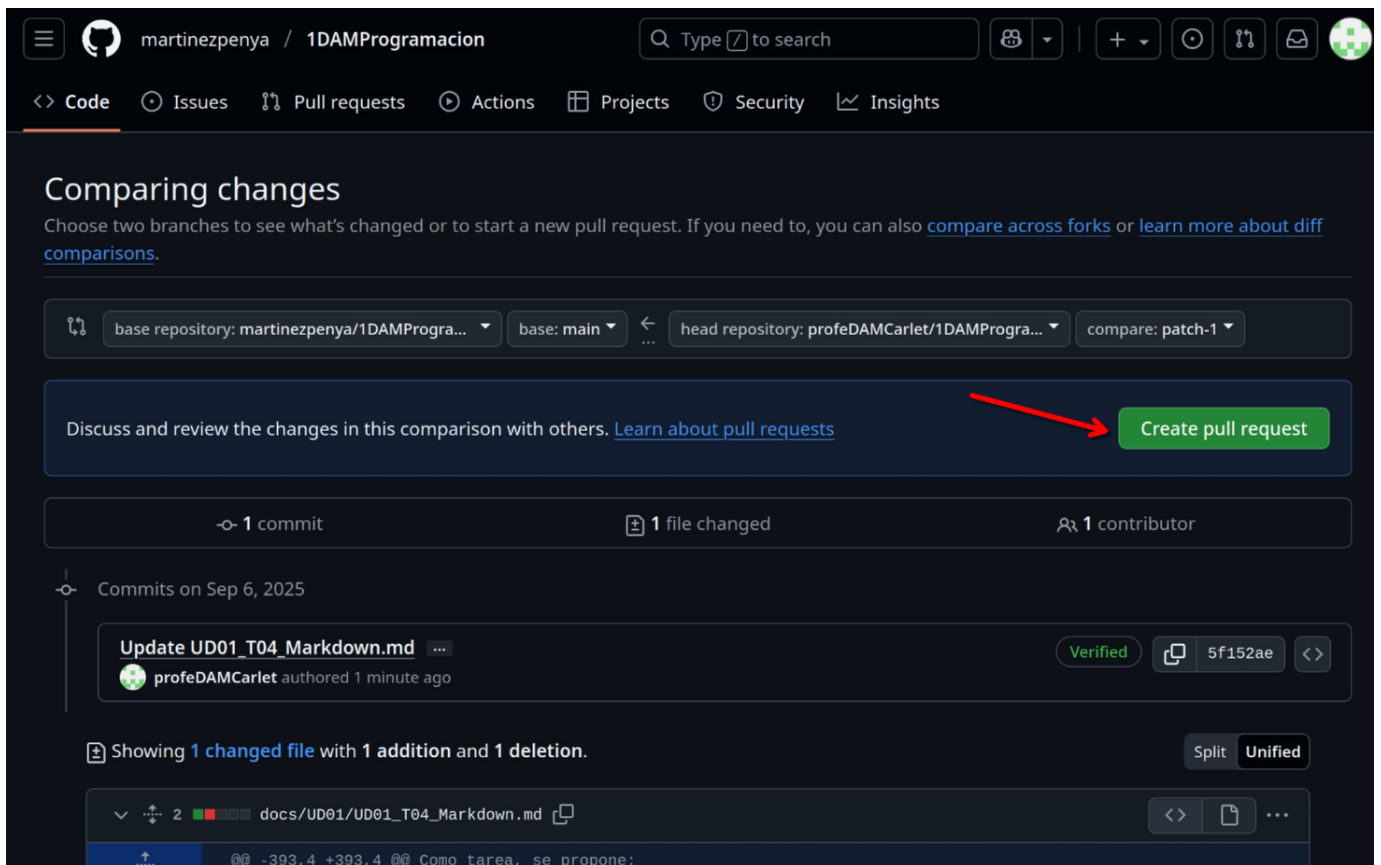
Extended description

He cambiado el formato de las extensiones de los documentos
a entregar.

Cancel

Propose changes

Todavía no hemos terminado! ahora hay que comunicar los cambios propuestos en nuestro Fork al propietario del repositorio, para que los visualice y valore si los quiere incluir en la página de documentación. Para ello debemos pulsar el botón [Create pull request]:



martinezpenya / 1DAMProgramacion

Search: Type to search

Code Issues Pull requests Actions Projects Security Insights

Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).

base repository: martinezpenya/1DAMProgra... base: main head repository: profeDAMCarlet/1DAMProgra... compare: patch-1

Discuss and review the changes in this comparison with others. [Learn about pull requests](#) **Create pull request**

1 commit 1 file changed 1 contributor

Commits on Sep 6, 2025

Update UD01_T04_Markdown.md ... Verified 5f152ae <>
profeDAMCarlet authored 1 minute ago

Showing 1 changed file with 1 addition and 1 deletion. Split Unified

docs/UD01/UD01_T04_Markdown.md <> [icon] ...

-393,4 +393,4 @ @ Como tarea, se propone:

Ahora podemos modificar el mensaje (pero no hace falta), directamente pulsamos sobre el botón [Create pull request]:

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#). [Learn more about diff comparisons here](#).

base repository: martinezpenya/1DAMProgra... base: main head repository: profeDAMCarlet/1DAMProgra... compare: patch-1

Add a title

Update UD01_T04_Markdown.md

Helpful resources

[GitHub Community Guidelines](#)

Add a description

Write Preview H B I ...

He cambiado el formato de las extensiones de los documentos a entregar.

Markdown is supported Paste, drop, or click to add files

☒ Allow edits by maintainers Create pull request

Remember, contributions to this repository should follow our [GitHub Community Guidelines](#).

Ahora si, deberías ver una página similar a la siguiente, de la que también deberás obtener una captura y adjuntarla al `.pdf`, y además explicar los 4 campos que hay redondeados:

1. Hemos creado un fork de un repositorio
2. Hemos modificado un archivo en nuestro fork
3. Hemos comparado nuestro fork con el original y hemos creado un pull request con las diferencias

O bien, que el cambio no sea aceptado.

En este caso concreto se ha aceptado la modificación:

Update UD01_T04_Markdown.md #1

Merged martinezpenya merged 1 commit into `martinezpenya:main` from `profeDAMCarlet:patch-1` 1 minute ago

Conversation 1 Commits 1 Checks 0 Files changed 1

profeDAMCarlet commented 10 minutes ago Contributor ...

He cambiado el formato de las extensiones de los documentos a entregar.

Update UD01_T04_Markdown.md Verified 5f152ae

martinezpenya commented 1 minute ago Owner ...

Perfecto, gracias!

martinezpenya closed this 1 minute ago

martinezpenya merged commit **8052475** into `martinezpenya:main` 1 minute ago Revert

Tarea

Crea un documento `.pdf`

Añade una **captura** de tu perfil de github.

Añade una **captura** de pantalla donde se vea que has solicitado el **pull request** y que estás esperando a que se integre en el repositorio original.

Además, **explica** que significan cada uno de los **4 apartados** señalados en la captura.

Adjunta el documento `.pdf` con las capturas y las explicaciones a la tarea de AULES

🕒 24 de octubre de 2025

Taller UD01_T03: Markdown

Introducción a Markdown



Markdown nace como herramienta de **conversión de texto plano a HTML**. Fue creada en 2004 por John Gruber, y se distribuye de manera gratuita bajo una [licencia BSD](#).

Markdown es un maravilloso **lenguaje** para escribir documentos de una manera **sencilla de escribir, y que en todo momento mantenga un diseño legible** que contengan elementos como *secciones, párrafos, listas, vínculos e imágenes, etc.* Pandoc <http://pandoc.org> ha extendido enormemente la [sintaxis original de Markdown](#) y ha añadido unas pequeñas nuevas características tales como notas al pie de página, citas y tablas. Lo más importante que hace Pandoc es hacer posible la generación de documentos en una amplia variedad de formatos desde Markdown, HTML, LaTeX/PDF, MSWord y Slides.

Este método te permitirá añadir formatos tales como **negritas**, *cursivas* o [enlaces](#), utilizando texto plano, lo que permitirá hacer de tu escritura algo más simple y eficiente al evitar distracciones.

Con Markdown **no vas a reemplazar todo**, sino cubrir las funcionalidades más comunes que se requieren para escribir un documento relativamente complicado.

Para qué sirve Markdown

Markdown será perfecto para ti sobre todo **si publicas de manera constante en Internet**, donde el lenguaje HTML está más que presente: WordPress, Squarespace, Jekyll...

Pero no estoy hablando solo de [blogs](#) o páginas web. **Servicios** como Trello o **foros** como Stackoverflow también soportan este lenguaje, y con el paso del tiempo encontrarás aún más lugares que lo utilicen.

Además, Markdown está cada vez más extendido en el **mundo “offline”**. Nada te impedirá utilizar este lenguaje para **tomar notas y apuntes** de tus clases o reuniones en una determinada [aplicación](#) (incluso podrías **escribir un libro con él**, ya que puedes exportar fácilmente el resultado final a un formato ePub).

Gracias a la simplicidad de su sintaxis podrás utilizarlo siempre que necesites escribir y dar formato rápidamente, sobre todo si quieres hacerlo desde dispositivos móviles.

Por qué utilizar Markdown

VENTAJAS

- **Markdown para todo.** Para crear apuntes, documentos, notas, sitios web, libros, documentación técnica, etc. de forma off-line.
- **Markdown transportable.** Este tipo de formato siempre será **compatible con todas las plataformas** que utilices, así que utilizar Markdown es una manera de mantener todo tu contenido siempre accesible desde cualquier dispositivo (smartphones, ordenadores de escritorio, tablets...), ya que en cualquiera de ellas siempre encontrarás [las aplicaciones adecuadas](#) para leer y editar este tipo de contenido.
- Ideal para escribir un libro, pues permite la exportación fácil en ePub, PDF...

Si en el futuro Microsoft Word desapareciese perderías acceso a todo el contenido que has creado durante años utilizando dicho procesador. Así que lo más inteligente para evitar eso es **generar tu contenido de la manera más sencilla posible**: utilizando texto plano.

DESVENTAJAS

- No tiene muchas funcionalidades (esto es lo que lo hace muy compatible).

Editores para Markdown

OFF-LINE

- **Typora**
- MarkdownPad
- HarooPad
- Markdown Monster
- ...

ONLINE

- Dillinger
- GitHub
- ...

Párrafos y saltos de línea

Si queremos generar un nuevo párrafo en Markdown simplemente separa el texto mediante una línea en blanco (**pulsando dos veces intro**).

Al igual que sucede con HTML, **Markdown no soporta dobles líneas en blanco**, así que si intentas generarlas estas se convertirán en una sola al procesarse.

Para realizar un salto de línea y empezar **una frase en una línea siguiente dentro del mismo párrafo**, tendrás que pulsar **dos veces la barra espaciadora antes de pulsar una vez intro**.

Por ejemplo si quisieses escribir un poema quedaría tal que así:

«La tierra estaba seca, No había ríos ni fuentes. Y brotó de tus ojos.

Donde cada verso tiene **dos espacios en blanco al final**.

Encabezados

Las # **almohadillas** son uno de los métodos utilizados en Markdown para crear encabezados. Debes usarlos añadiendo **uno por cada nivel**.

Es decir,

```
1 # Encabezado 1
2 ## Encabezado 2
3 ### Encabezado 3
4 #### Encabezado 4
5 ##### Encabezado 5
6 ##### Encabezado 6
```

Se corresponde con:

1. Encapçalament 1

1.1. Encapçalament 2

1.1.1. Encapçalament 3

1.1.1.1. Encapçalament 4

1.1.1.1.0.1. Encapçalament 5

1.1.1.1.0.1. Encapçalament 6

También puedes cerrar los encabezados con el mismo número de almohadillas, por ejemplo escribiendo `### Encabezado 3 ###`. Pero la única finalidad de esto es un **motivo estético**.

Texto básico

Un párrafo no requiere sintaxis especial.

Para aplicar **negrita** al texto, se escribe entre dos asteriscos.

Para aplicar *cursiva* al texto, se escribe entre un solo asterisco.

Para tachar el texto, se escribirá dos virgulillas antes y dos después de éste.

```
1 Este texto es en **negrita**.
2 Este texto es en *itálica*.
3 Este texto está ~~tachado~~.
4 Este texto es en ambos ***negrita e itàlica***.
```

Se corresponde a:

Este texto es en ***negrita***.

Este texto es en *itálica*.

Este texto está ~~tachado~~.

Este texto es en ambos ***negrita e itàlica***.

En Markdown no podemos subrayar el texto. Sin embargo, podremos añadir la etiqueta de html underline `\`.

```
1 Este texto está <u>subrayado</u>
```

Este texto está subrayado

Para **ignorar los caracteres** de formato de Markdown, ponga `\` antes del carácter:

Citas

Las citas se generan utilizando el carácter *mayor que* `>` al comienzo del bloque de texto.

```
1 > No hay que ir para atrás, ni para darse impulso. — Lao Tsé.
```

No hay que ir para atrás, ni para darse impulso. — Lao Tsé.

Si la cita en cuestión se compone de **varios párrafos**, deberás añadir el mismo símbolo `>` al comienzo de cada uno de ellos.

Listas

LISTAS ORDENADAS

Para crear **listas numeradas**, empieza una línea con `1.` or `1)`.

No debes mezclar los formatos dentro de la misma lista. No es necesario especificar los números. GitHub lo hace por tí.

```
1 1. Ítem 1 de la lista.
2 1. Siguiendo ítem de la lista.
3 1. Siguiendo ítem, el tercero, de la lista.
```

Se corresponde con:

1. Ítem 1 de la lista.
2. Siguiendo ítem de la lista.
3. Siguiendo ítem, el tercero, de la lista.

LISTAS NO ORDENADAS

Para crear listas no numeradas, o de viñetas, empieza una línea con `*`, `-` o `+`, pero no mezcles los formatos dentro de la misma lista. (No mezclar formatos de viñetas, como `*` y `+` por ejemplo, dentro del mismo documento).

```
1 * Ítem 1 de la lista.
2 * Siguiendo ítem de la lista.
3 * Siguiendo ítem, el tercero, de la lista.
```

Se corresponde con:

- Ítem 1 de la lista.
- Siguiendo ítem de la lista.
- Siguiendo ítem, el tercero, de la lista.

También podremos combinar ambos tipos de listas. Como por ejemplo:

1. element de llista 2 - element de llista 2.2
 - element de llista 2.2.1
 - element de llista 2.2.2

LISTAS DE TAREAS

Para crear listas de tareas basta con que empiece la línea con `- [ ]`, si queremos que no esté el check marcado, y `- [x]`, si queremos que esté el check marcado.

```
1 - [x] regar plantas.
2 - [ ] realizar ejercicios de programación.
```

Se corresponde con:

- ☒ regar plantas.
- ☐ realizar ejercicios de programación.

Tablas

Las tablas no forman parte de la especificación principal de Markdown, pero Adobe, en cierta forma, las admite.

Para generar una tabla utiliza la barra vertical `|` para generar filas y columnas.

Si insertamos guiones `---` dentro de una celda crearemos el encabezado de la tabla.

```
1 | encabezado1 | encabezado2 | encabezado3 |
2 | --- | --- | --- |
3 | celda 1.1 | celda 1.2 | celda 1.3 |
4 | celda 2.1 | celda 2.2 | celda 2.3 |
```

Quedaría:

encabezado1	encabezado2	encabezado3
celda 1.1	celda 1.2	celda 1.3
celda 2.1	celda 2.2	celda 2.3

Si queremos una **celda con más de una línea** de texto podremos insertar `\` (o **Shift+Intro**) al final de ésta.

Enlaces

Para generar un enlace en Markdown se debe poner un código con dos partes:

- `[texto del enlace]`, que es el texto que se va a mostrar,
- Y después `(nombrefichero.md)`, que es la URL o el nombre de archivo al que se va a vincular.

```
1 [link text](file-name.md)
```

Un ejemplo:

[enlace a web del centro](https://iesmre.com)

La visualización del ejemplo anterior:

[enlace a web del centro](https://iesmre.com)

Imágenes

Para insertar una imagen se debe poner un código con dos partes:

- `![texto alternativo]`, que es el texto que se va a mostrar si la imagen no pudiera visualizarse,
- Seguido de `(nombrefichero.extension)`, que es el archivo imagen (con su dirección).

```
1 [texto alternativo](file-name.md)
```

Un ejemplo:

[logo markdown](assets/markdown_logo.png)

La visualización de la imagen anterior:



Código de bloque

Uno de los puntos más útiles de Markdown a la hora de crear un documento con texto específico de informática es que admite la colocación de bloques de código tanto en línea como en un bloque "delimitado" independiente entre frases.

Para ello utilizaremos:

- Dos comillas invertidas ```` si queremos escribir código dentro de la misma línea de texto del párrafo.
- Si queremos crear un bloque de código multilínea, con un lenguaje específico, pondremos ````` seguido del nombre del lenguaje del bloque.

Unos ejemplos:

- En la misma línea:

...estamos escribiendo un párrafo ```insertar el bloque y seguimos escribiendo...`

- Un bloque de código:

````javascript` y escribimos el código.

```
1 function holamundo(){
2 console.log ("hola mundo web");
3 }
```

### Línea horizontal

Para crear una línea horizontal, de separación de contenido por ejemplo, se añaden tres guiones: `---`



Visualización:

---

### Insertar emojis

Para insertar emojis basta con utilizar `:` seguido del nombre del emoji y cerrar con otro `:`.

Podemos observar, que en algunos editores markdown, al escribir, por ejemplo, `:a` nos muestra todos los emojis con la inicial **a**.

Por ejemplo: `: star :`

Visualización: 🌟

### Crear diagrama de flujo

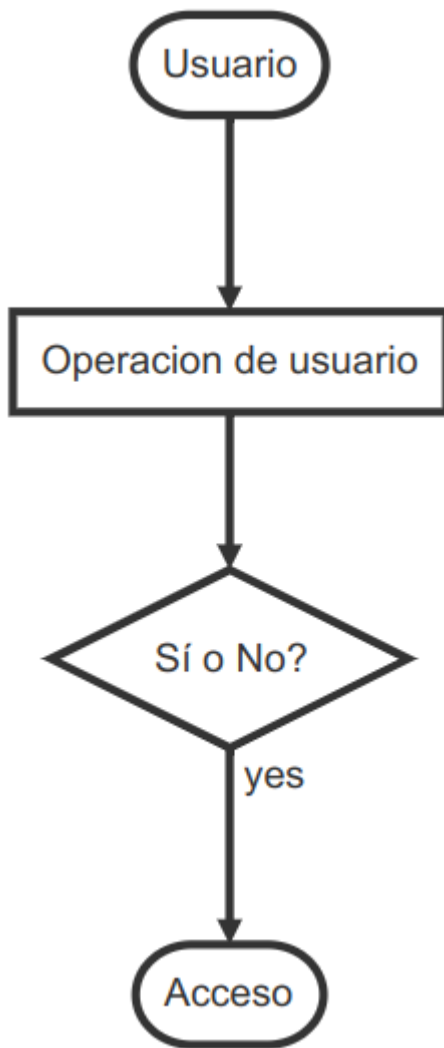
Cuando queremos crear documentos con elementos gráficos como diagramas de flujo, debemos generar una especie de *código* para construirlos.

- Por eso, comenzaremos introduciendo la línea de inicio: ````flow`
- Es conveniente asignar un nombre (por ejemplo: st, op, cond, e...) a cada elemento que conforma el diagrama; así, después podremos unir todos estos.
- Forma de inicio: `st=>start: Nombre`
- Forma de fin: `e=>end: Nombre`
- Rectángulo: `op=>operation: texto de nombre`
- Condición: `cond=>condition: texto de la condición (Si o No?)`
- Subrutina: `subl=>subroutine: nombre subtarea`
- EntradaSalida: `iol=>inputoutput: nombre elemento entrada/salida`
- Líneas: `st->op->cond`
- Caminos de condiciones: `cond(yes)**->e` y `cond(no)->op`
- Línea de cierre: `````

Ejemplo:

```
1 ```flow
2 st=>start: Usuario
3 e=>end: Acceso
4 op=>operation: Operacion de usuario
5 cond=>condition: Sí o No?
6 st->op->cond
7 cond(yes)->e
8 cond(no)->op
9 ```
```

Visualización:



Intenta realizar un diagrama para "programar" un almuerzo. En él, deberás dar los **buenos días**, indicar que **es hora del descanso**, y preguntar si **alguién quiere almorzar**. Si no hay nadie que quiera almorzar contigo, debes **ir a otro grupo de amigos** y volver a indicar que **es hora del descanso**. Si alguien sí quiere almorzar **escribe en la pizarra que os vais a almorzar** y **sal al patio**.

#### Crear secuencias

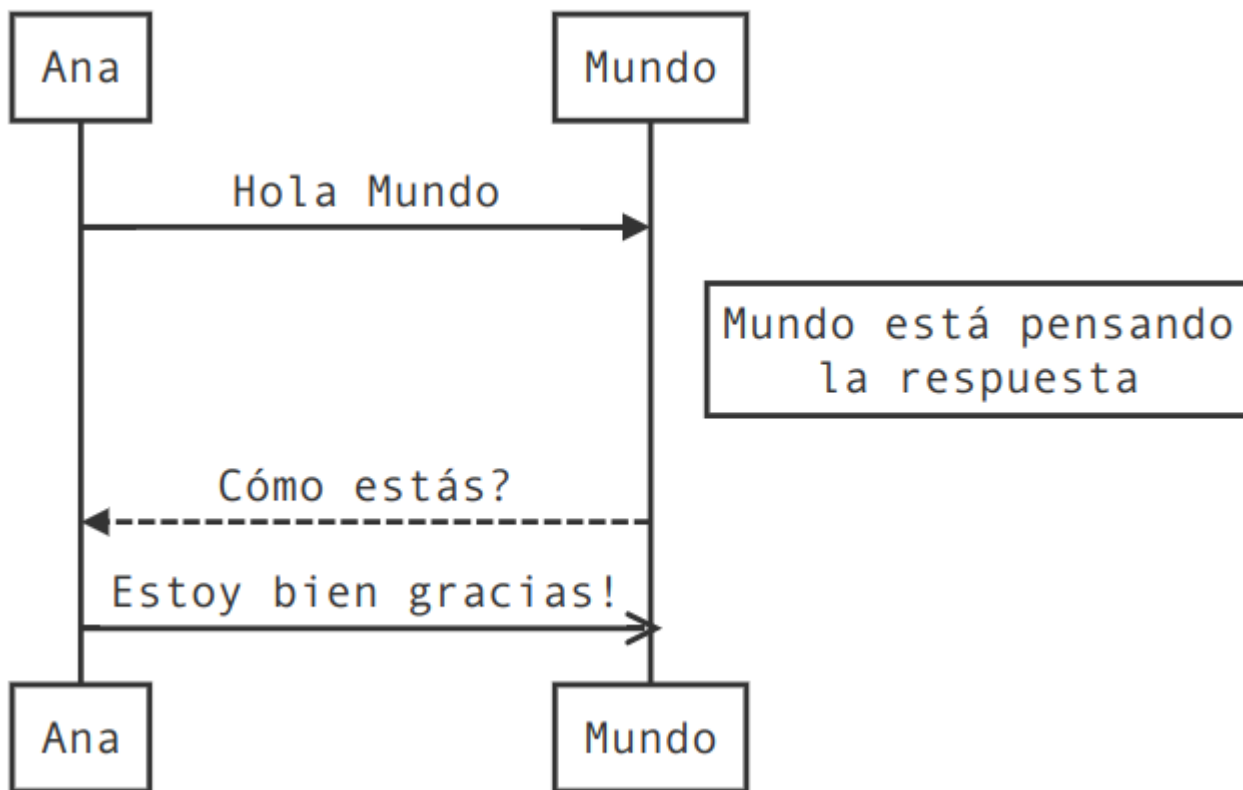
En la secuenciación podemos observar que es bastante parecido a la creación de diagramas; pero la primera línea (crear un bloque de código) no será **flow** sino **sequence**.

```

1 ```sequence
2 Ana->Mundo: Hola Mundo
3 Note right of Mundo: Mundo está pensando\nla respuesta
4 Mundo-->Ana: Cómo estás?
5 Ana->>Mundo: Estoy bien gracias!
6 ```

```

Visualización:



#### Crear índice

Para crear el índice a partir de los encabezados creados debemos insertar `[T0C]`.

#### Tarea

Como tarea, se propone:

- Crear un documento markdown en tu editor markdown favorito (por ejemplo Typora o VSCode) que documente información acerca de tí mismo.
- En dicho documento crear título, índice.
- Añadir 4 encabezados principales (y otros encabezados secundarios dentro de éstos) en el que hables por ejemplo de: *Tus datos, Currículum, Aficiones y Otros datos de interés*. No hace falta que indiques información personal relevante. (O te la puedes inventar)
- Se valorará la inclusión de distintos elementos como: negrita-cursiva-subrayado, listas ordenadas-desordenadas-tareas, enlaces, imágenes, citas, código, etc.
- Si te atreves con ello, crea un diagrama de flujo en el que indiques los pasos que realizas un sábado por la mañana.
- Exporta el documento a pdf.

**Subir a la plataforma *AULES* un documento Markdown (.md) y otro documento PDF (.pdf) que sea la exportación del primero.**

🕒 30 de octubre de 2025

## 4. UD05

### 4.1 UD05 — Sistemas expertos y controladores inteligentes

#### Unidad 5 · 12 h · semanas 11-15 (7 de diciembre al 14 de enero)

Continúa las reglas y la lógica difusa de RA2, justo antes de que la fragmentación de Navidad parta el curso. Se evalúa con **cinco entregables prácticos** y la prueba escrita del RA5.

#### 1. Introducción

Todo el curso ha sido **cómo construir** sistemas de IA. Esta unidad da un paso más: los **sistemas expertos**, que codifican el conocimiento de un experto humano en reglas y lo utilizan para **diagnosticar, asesorar y controlar**. Y lo conectamos con el **control**, porque un sistema experto puede actuar como **controlador inteligente** de un proceso físico, compitiendo con los controladores clásicos (PID).

Antes de construir nada hay que situar **qué es el conocimiento** y **cómo se representa** para que una máquina pueda usarlo — esa es la primera pregunta de la inteligencia artificial simbólica, y la respondemos con la jerarquía DIKW y el continuo de representaciones del §5.

El recorrido de la unidad:

1. **Arquitectura y dinámica** de los sistemas expertos (base de conocimiento, motor de inferencia, ciclo reconocer-actuar).
2. **Estructuras elementales** de representación del conocimiento (reglas, marcos, lógica, ontologías, redes semánticas).
3. **Representar y simular comportamientos** con la librería `experta`, en dominios muy diversos: diagnóstico médico, clasificación, control de procesos.
4. **Sistemas híbridos reglas/datos**: cuando las reglas se combinan o se extraen del aprendizaje automático.
5. **Sistemas de razonamiento impreciso**: la lógica difusa, para trabajar con lo que no es blanco o negro.
6. **Cómo influye la variación de las características** en la dinámica del sistema (sensibilidad y robustez).
7. **Estrategias de control** de un sistema experto.
8. **Controladores inteligentes** (difuso, por reglas, redes neuronales, MPC) y su influencia en el comportamiento del sistema.
9. **Aplicaciones y tendencias** (MYCIN, XCON, BRMS, neuro-simbólico, IA explicable).

#### Hilo conductor de la unidad

Un sistema experto convierte el conocimiento de un experto en reglas que pueden diagnosticar, asesorar y controlar. Primero situamos qué es ese conocimiento y cómo se representa; después lo construimos y simulamos, con reglas puras, híbridas o difusas; luego estudiamos por qué a veces falla; y por último lo usamos como controlador, comparándolo con el PID clásico.

#### 2. Resultado de aprendizaje y criterios de evaluación

**RA5** — Aplica sistemas expertos evaluando la influencia de los controladores inteligentes en el comportamiento del sistema.

CE	Criterio de evaluación	Bloque
RA5-a	Se ha descrito la dinámica y las estructuras elementales de los sistemas expertos.	\$4-5
RA5-b	Se han determinado las destrezas necesarias para representar y simular comportamientos básicos de sistemas de muy diversos ámbitos.	\$6-8
RA5-c	Se ha razonado cómo influye la variación de las características de los sistemas en su dinámica de actuación.	\$9
RA5-d	Se han desarrollado estrategias de control definiendo los objetivos y las especificaciones de la respuesta del sistema.	\$10
RA5-e		\$11

CE	Criterio de evaluación	Bloque
	Se han relacionado los controladores inteligentes con el comportamiento del sistema.	



#### Bloque de contenidos oficial (RD 279/2021, anexo I)

El currículo llama a este bloque, textualmente, «*Sistemas Expertos*», y le asigna estos contenidos:

- *Dinámica de los sistemas expertos.*
- *Estructuras elementales de los sistemas expertos.*
- *Representar y simular comportamientos básicos.*
- *Estrategias de control de un sistema experto.*
- *Aplicaciones de sistemas expertos.*
- *Tendencias en sistemas expertos.*

Son **seis contenidos para cinco criterios de evaluación**, y no encajan uno a uno: los dos últimos (aplicaciones y tendencias) no tienen CE propio y se reparten entre todos, mientras que RA5-c (variación de las características) **no tiene contenido explícito** en el anexo y lo desarrolla el centro (art. 3.3 del Decreto 95/2026).

### 3. Objetivos de la unidad

Objetivo	Al terminar la unidad serás capaz de...
O1	<b>Situ</b> ar el conocimiento en la jerarquía DIKW y <b>describ</b> ir la arquitectura de un sistema experto.
O2	<b>Diferenciar</b> los modos de representación del conocimiento (reglas, frames, lógica, ontologías, redes semánticas).
O3	<b>Representar y simular</b> un comportamiento básico con <i>experta</i> , en un dominio real.
O4	<b>Combinar</b> reglas con datos cuando el conocimiento experto no basta por sí solo.
O5	<b>Aplicar</b> lógica difusa a un problema con incertidumbre.
O6	<b>Explicar</b> cómo influye la variación de reglas, hechos y umbrales en la dinámica (sensibilidad/robustez).
O7	<b>Desarrollar</b> estrategias de control (salience, control de meta) con sus especificaciones.
O8	<b>Relacionar</b> los controladores inteligentes (difuso, por reglas, ANN, MPC) con el comportamiento del sistema y compararlos con un PID.
O9	Conocer aplicaciones reales y tendencias de los sistemas expertos.

### 4. Arquitectura y dinámica de los sistemas expertos (RA5-a)

#### 4.1 Del dato al conocimiento: la jerarquía DIKW

Antes de representar el conocimiento hay que distinguirlo de sus vecinos. La **jerarquía DIKW** (*Data, Information, Knowledge, Wisdom*) los ordena:



Nivel	Qué es	Ejemplo
<b>Datos</b>	Hechos o valores registrados, independientes de quien los lee	«Un reloj inteligente registra la temperatura corporal»
<b>Información</b>	Los datos interpretados por un agente; subjetiva	«La temperatura corporal es 37 °C»
<b>Conocimiento</b>	Información integrada en un modelo del mundo	«Si la temperatura supera 37 °C, la persona tiene fiebre»
<b>Sabiduría</b>	Meta-conocimiento: cuándo y cómo aplicar el conocimiento	«Si tiene fiebre, debe tomar paracetamol»

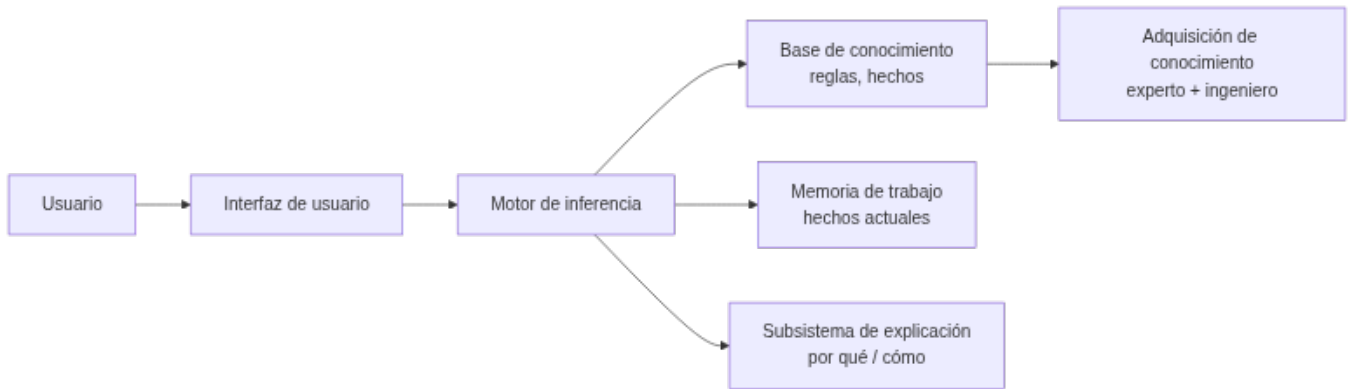


#### Por qué importa esta distinción

Un sistema experto no almacena datos ni información: almacena **conocimiento** (reglas) y, en los más avanzados, un poco de **sabiduría** (metarreglas que deciden cuándo aplicar otras reglas — el «control de meta» del §10.1). Confundir los niveles es el error más común al empezar a diseñar uno: se acumulan datos en vez de codificar reglas.

#### 4.2 ¿Qué es un sistema experto?

Un **sistema experto** es un programa de IA que **emula el razonamiento de un experto humano** en un dominio concreto: codifica su conocimiento en una **base de conocimiento** y utiliza un **motor de inferencia** para aplicar ese conocimiento a los hechos del problema. Comenzaron su desarrollo en los años 70 y fueron muy populares esa década y la siguiente: se consideran los primeros sistemas de IA capaces de obtener resultados con utilidad práctica real.



#### 4.3 Componentes

Componente	Función
<b>Interfaz de usuario</b>	Vía de las consultas: pregunta datos, muestra resultados y alerta de datos erróneos. Incluye un <b>módulo de comunicaciones</b> (con otros sistemas, útil en automatización industrial)
<b>Base de conocimiento (KB)</b>	Reglas y hechos del dominio, formalizados por el <b>ingeniero de conocimiento</b> junto al experto. Puede organizarse como listas, objetos relacionados, cálculo de predicados o redes semánticas
<b>Memoria de trabajo (base de hechos)</b>	Hechos actuales: los que aporta el usuario o los sensores, más los deducidos durante el razonamiento. Guarda el histórico de estados de la consulta
<b>Motor de inferencia</b>	Evalúa qué reglas se cumplen, resuelve conflictos y ejecuta las acciones. Determina el orden de las acciones y la interacción entre las partes del sistema
<b>Subsistema de explicación</b>	Justifica el razonamiento («por qué» y «cómo» llegó a esa conclusión), mostrando las reglas usadas. Ayuda también al ingeniero de conocimiento a <b>depurar</b> el motor y al experto a <b>verificar</b> la coherencia de la base
<b>Adquisición de conocimiento</b>	Herramientas para incorporar nuevo conocimiento sin perfil técnico (el famoso <i>bottleneck</i> : conseguir que el experto vuelque lo que sabe es la parte más lenta de construir el sistema)



#### La explicación marca la diferencia

La capacidad de **explicar** su razonamiento es lo que distingue a un sistema experto de un modelo de ML: en dominios regulados (medicina, finanzas) la justificación es obligatoria. MYCIN fue el pionero de esta *explicabilidad*, mostrando las reglas de inferencia que empleó — aunque, para consultas complejas, enumerar todas las reglas puede resultar tedioso para el usuario.

#### 4.4 El ciclo de ejecución (dinámica)

El motor de inferencia opera en un **bucle reconocer-actuar**:

1. **Reconocer (match)**: se comparan los hechos de la memoria de trabajo con las condiciones (LHS) de las reglas; las reglas activas van a la **agenda**.
2. **Resolver (resolve)**: si hay varias reglas activas, se elige una según la estrategia de control (saliencia, recency, especificidad — ver §10.1).
3. **Actuar (act)**: se ejecuta el consecuente (RHS), que declara/modifica/retira hechos, y el ciclo se repite hasta agotar la agenda.

El objetivo de fondo es hacer que la lógica sea **explícita** en vez de estar enterrada en código que solo revisa un informático: con reglas legibles, el propio experto del dominio puede revisar y mantener el sistema.

#### 4.5 Encadenamiento

- **Hacia delante (forward, data-driven)**: parte de los hechos y deduce conclusiones. Ideal para monitorización, control de procesos en tiempo real y planificación.

- **Hacia atrás (backward, goal-driven):** parte de una **meta** y busca qué hechos la sustentan, preguntando al usuario solo lo necesario. Ideal para diagnóstico (MYCIN, Prolog).
- **Encadenamiento mixto:** combina los dos anteriores.
- Otros mecanismos de razonamiento: **búsqueda heurística** (el proceso de inferencia recorre un árbol) y **herencia** (en entornos orientados a objetos, un objeto hijo hereda propiedades del padre).



#### Las dos reglas de inferencia clásicas

Antes de forward/backward chaining está la lógica que los sustenta: **Modus Ponens** («si  $P$  implica  $Q$ , y  $P$  es verdad, entonces  $Q$  es verdad») y **Modus Tollens** («si  $P$  implica  $Q$ , y  $Q$  no es cierto, entonces  $P$  no es cierto»). El encadenamiento hacia delante aplica Modus Ponens repetidamente; el encadenamiento hacia atrás busca qué  $P$  sustentaría un  $Q$  dado.

#### 4.6 Incertidumbre: factores de certeza

MYCIN introdujo los **factores de certeza (CF)** para manejar la incertidumbre: cada regla tiene un CF y se combinan al encadenar. Alternativas modernas: lógica difusa (§8), teoría de Dempster-Shafer o redes bayesianas (Heckerman & Shortliffe, 1992).



#### Combinar factores de certeza

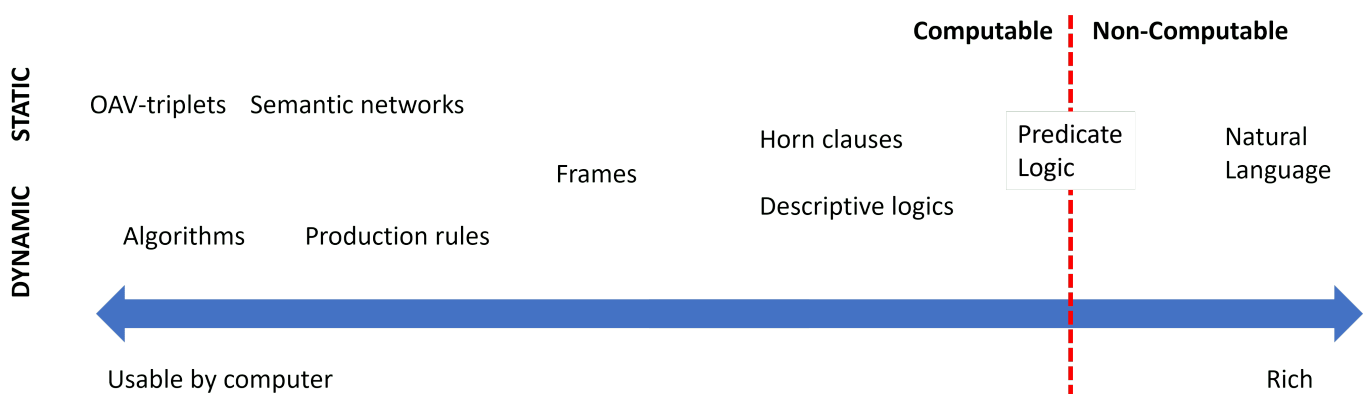
Una regla dice «SI hay fiebre alta ENTONCES sospecha de meningitis **con CF = 0,6**». Otra regla independiente aporta «SI rigidez de nuca ENTONCES sospecha de meningitis **con CF = 0,4**». MYCIN combina ambos apoyos para dar una confianza conjunta mayor que cada una por separado (combinación de evidencia). Si en cambio hubiera evidencia en contra, los CF se descuentan. De ahí nace el concepto de **acumulación de evidencia**, que luego se formalizó con redes bayesianas.

Esta gestión explícita de la **incertidumbre** es una diferencia clave frente a un sistema de reglas «duro» (todo verdadero o falso): permite razonar con conocimiento parcial y explicar el nivel de confianza de una conclusión. La lógica difusa del §8 resuelve el mismo problema de otra forma: en vez de un factor de certeza sobre una regla booleana, deja que el propio hecho sea **parcialmente verdadero**.

### 5. Estructuras elementales de representación (RA5-a/b)

#### 5.1 Un continuo, no una lista cerrada

Representar el conocimiento es uno de los problemas fundamentales de la IA: hay que hacerlo de forma **entendible** para las máquinas, **útil** para resolver problemas y **eficiente** de procesar. Las representaciones forman un **continuo**:



En un extremo están las representaciones **simples** (algoritmos): eficientes para el ordenador pero poco flexibles. En el otro, las **flexibles** (texto natural): muy expresivas pero no directamente utilizables por una máquina. Entre medias viven las que usamos en esta unidad.



## 5.2 Las representaciones, con sus ventajas y límites

Representación	Estructura	Inferencia	Ventajas	Limitaciones
<b>Pares atributo-valor</b> (tripletes objeto-atributo-valor)	Lista de nodos y aristas de un grafo: «el perro es un animal, tiene cuatro patas...»	Recorrido y coincidencia de atributos	Muy simple de construir a partir de descripciones	Poco expresiva para relaciones complejas
<b>Reglas de producción</b>	SI ... ENTONCES ...	Forward/backward + resolución de conflictos	Modular, legible, explicable	Difícil modelar jerarquías; lenta con bases grandes
<b>Representaciones jerárquicas</b>	Árbol: nodos = conceptos, aristas = relaciones (Animales → Vertebrados → Mamíferos → Perros)	Recorrido del árbol, herencia	Natural para taxonomías	Rígida si un concepto pertenece a varias ramas
<b>Marcos (frames)</b>	Registros con ranuras (slots) y valores por defecto	Herencia de clases, disparadores	Taxonomías y conocimiento estructurado	Conflicto con herencia múltiple
<b>Lógica formal</b>	Predicados de primer orden (propuesta por Aristóteles hace más de 2000 años)	Resolución, unificación, deducción	Rigor matemático	Explosión combinatoria; poco utilizable directamente por máquinas (salvo un subconjunto, como en Prolog)
<b>Redes semánticas</b>	Grafos dirigidos con etiquetas	Búsqueda en grafos, propagación	Intuitiva para relaciones	Semántica ambigua
<b>Ontologías</b>	Clases, propiedades, axiomas (OWL)	Razonadores (Pellet, HermiT)	Interoperabilidad, reutilización	Curva de aprendizaje alta



## Lo mismo en cuatro formas distintas

El hecho «si la temperatura supera 37 °C, la persona tiene fiebre» se puede escribir como **regla de producción** (SI temperatura > 37 ENTONCES fiebre), como **lógica proposicional** ( $(p \rightarrow q)$ , con  $(p)$ : «tiene fiebre»,  $(q)$ : «debe tomar paracetamol»), como **jerarquía** (Síntomas → Fiebre → Paracetamol) o como **par atributo-valor** ( persona.temperatura = 38.2 , evaluado por una regla externa). Elegir la representación es elegir **qué es fácil de hacer** con ese conocimiento después.



## Regla práctica de esta unidad

Usaremos sobre todo **reglas de producción** (con `experta`) y, en el §8, **lógica difusa** (con `scikit-fuzzy`). Las ontologías y las redes semánticas se mencionan como alternativas; los **frames** y la **lógica formal** se tratan a nivel conceptual.

## 6. Representar y simular comportamientos con experta (RA5-b)

Para **simular un comportamiento** se escribe la base de conocimiento en `experta` y se ejecuta el ciclo de inferencia. Recordamos el parche obligatorio para Python 3.10+ (la librería es de 2019 y `collections.Mapping` desapareció en esa versión):

```

1 import collections, collections.abc
2 if not hasattr(collections, 'Mapping'):
3 collections.Mapping = collections.abc.Mapping
4 collections.Iterable = collections.abc.Iterable
5 collections.MutableMapping = collections.abc.MutableMapping
6
7 from experta import *
8
9 class DiagnosticoPc(KnowledgeEngine):
10 @DefFacts()
11 def inicio(self):
12 yield Fact(accion="diagnosticar")
13
14 @Rule(Fact(accion="diagnosticar"), salience=10)
15 def arrancar(self):
16 print("Diagnóstico del PC...")
17 self.declare(Fact(luz_encendida=True))
18 self.declare(Fact(sonido="pitidos_cortos"))
19
20 @Rule(Fact(luz_encendida=True), Fact(sonido="pitidos_cortos"))
21 def ram(self):
22 self.declare(Fact(causa="problema_ram"))
23
24 @Rule(Fact(causa="problema_ram"))
25 def resultado(self):
26 print("DIAGNÓSTICO: Fallo de memoria RAM.")
27
28 engine = DiagnosticoPc()
29 engine.reset()
30 engine.run()

```

**Salida esperada:**

```

1 Diagnóstico del PC...
2 DIAGNÓSTICO: Fallo de memoria RAM.

```

**Otro ámbito: clasificación de animales (sistema de producción clásico)**

El mismo motor `experta` simula un sistema de clasificación con reglas de un zoólogo:

```

1 class Animales(KnowledgeEngine):
2 @DefFacts()
3 def inicio(self):
4 yield Fact(analizar=True)
5
6 @Rule(Fact(analizar=True), salience=10)
7 def datos(self):
8 self.declare(Fact(pelo=True), Fact(carnivoro=True), Fact(color="leonado"))
9 self.declare(Fact(manchas="oscuras"))
10
11 @Rule(Fact(pelo=True))
12 def mamifero(self):
13 self.declare(Fact(mamifero=True))
14
15 @Rule(Fact(mamifero=True), Fact(carnivoro=True), Fact(color="leonado"),
16 Fact(manchas="oscuras"))
17 def guepardo(self):
18 print("IDENTIFICADO: guepardo")

```

Así se comprueba que un sistema experto **representa y simula el comportamiento** de un dominio sin programar la lógica imperativamente: el motor aplica las reglas hasta llegar a la conclusión.

**6.1 «Muy diversos ámbitos»: los notebooks de la unidad**

El CE b exige simular comportamientos en **dominios distintos**, no repetir el mismo ejemplo. Esta unidad trae varios notebooks ejecutados de verdad, en dominios reales (la lista completa está en las [actividades guiadas](#) y las [entregables](#)):

Notebook	Dominio	Qué simula
UD05_N01_experta_primeros_pasos.ipynb	Introducción	Primeros pasos con <code>experta</code> : hechos, reglas, <code>DefFacts</code>
UD05_N02_piedra_papel_tijera.ipynb	Juego	Piedra, papel o tijera decidido por reglas
UD05_N03_clasificacion_animales.ipynb	Zoología	El ejemplo de arriba, completo y ejecutado
EX1 · lesión de rodilla	Medicina	Diagnóstico de una lesión a partir de síntomas, con <code>experta</code>
UD05_N04_reglas_desde_datos_titanic.ipynb	Datos históricos	Reglas <b>extraídas</b> de datos, no escritas a mano (§7)
EX2 · valor de mercado	Deporte	Estimación híbrida reglas + aprendizaje automático (§7)
EX3 · centrocampistas · EX4 · quemador de gas	Deporte · industria	Lógica difusa (§8) y control de un proceso real (§10-11)



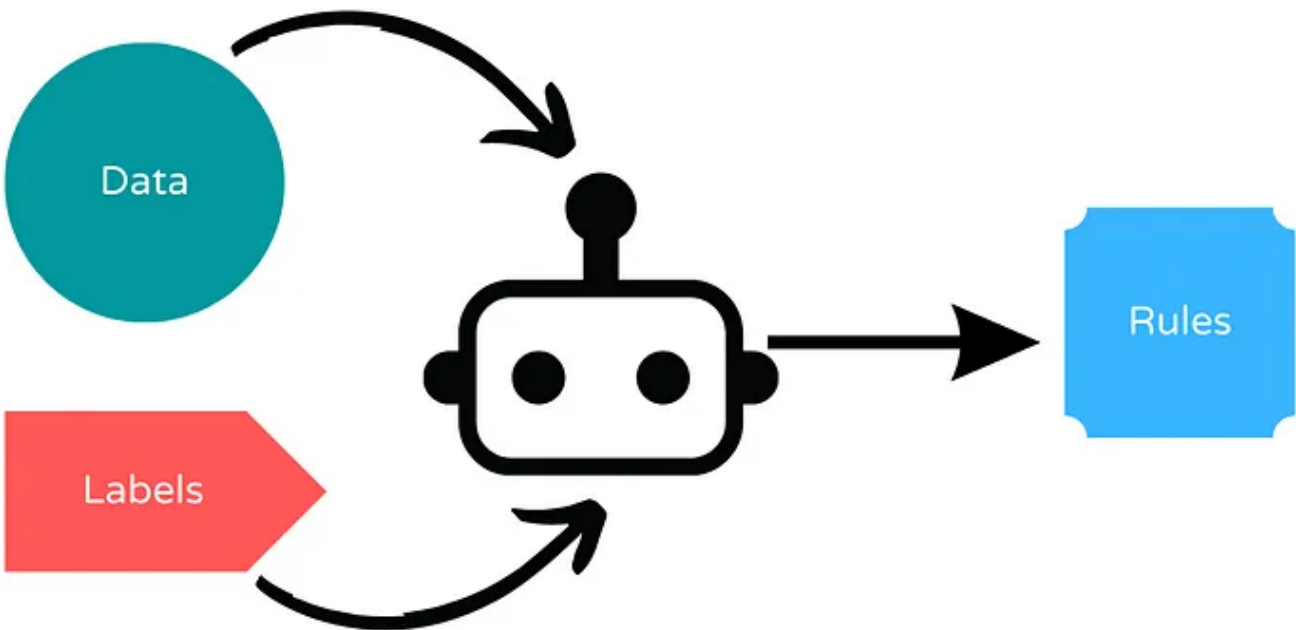
Por qué importa la diversidad

Un sistema experto no es una técnica de un solo dominio: la **misma arquitectura** (base de conocimiento + motor de inferencia) sirve para diagnosticar una rodilla, valorar a un futbolista o regular un quemador de gas. Lo que cambia es el conocimiento que se codifica, no el motor que lo aplica.

7. Sistemas híbridos reglas/datos (RA5-b)

Un sistema experto puro necesita que **alguien escriba las reglas a mano**. Pero a veces el conocimiento del dominio ya está en los datos, o conviene mezclar las dos fuentes. Hay dos enfoques:

- **Deducir reglas a partir de datos:** un algoritmo genera reglas legibles a partir de un conjunto de entrenamiento, en vez de que las escriba una persona. Facilita la **interpretación** del razonamiento, porque el resultado sigue siendo `SI ... ENTONCES ...` en vez de una caja negra.
- **Integrar reglas definidas por el usuario con aprendizaje automático:** el experto pone las reglas de partida y el aprendizaje automático las **mejora** con datos.



## 7.1 Bibliotecas del segundo enfoque

Biblioteca	Qué hace
<b>Human-Learn</b>	Permite <b>definir y dibujar</b> reglas propias como si fueran un clasificador de scikit-learn ( <code>FunctionClassifier</code> ), y combinarlas con aprendizaje automático
<b>skope-rules</b>	Analiza los datos y <b>deduce</b> reglas de clasificación; permite auditarlas e interpretarlas
<b>FIGS</b> ( <code>imodels</code> )	Genera <b>reglas fáciles de interpretar</b> combinando varios árboles pequeños («sumas de árboles»), en vez de un único árbol grande difícil de leer
<b>spaCy</b>	Reglas para <b>extraer información de texto</b> : útil cuando no hay suficientes datos etiquetados o para casos muy específicos



## UD05\_N04\_reglas\_desde\_datos\_titanic.ipynb : reglas que salen de los datos, no de un experto

En vez de que alguien escriba a mano «si viajas en primera clase y eres mujer, sobrevives», **FIGS** analiza los datos históricos del Titanic y **genera esa regla por sí solo**, junto con otras. El resultado sigue siendo legible —un árbol de reglas pequeño—, pero nadie lo escribió a mano: se dedujo de los datos. Es el primer enfoque de la tabla de arriba.



## EX2 : valor de mercado de futbolistas con Human-Learn + FIGS

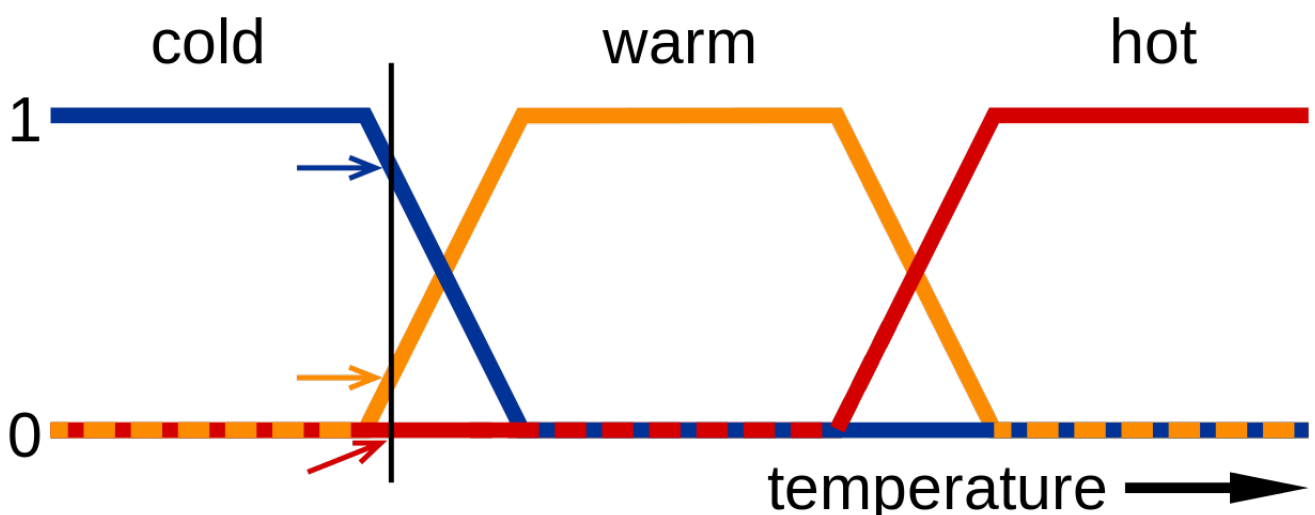
El entregable **EX2** combina las dos ideas sobre los mismos datos (FIFA 22): una `FunctionClassifier` de Human-Learn (reglas que **tú** defines sobre atributos del jugador) y un `FIGSClassifier` que **deduce** sus propias reglas del conjunto de entrenamiento. Comparar los dos resultados sobre el mismo problema es la mejor forma de sentir la diferencia entre los dos enfoques híbridos.



## Esto no es una técnica aislada: es una tendencia

Los sistemas híbridos reglas/datos son la puerta de entrada a los **sistemas neuro-simbólicos** y a las **reglas como guardarraíl del ML** que se tratan en el §12.2. No son un tema aparte de los sistemas expertos: son su evolución natural cuando el conocimiento no cabe entero en la cabeza de un experto.

## 8. Sistemas de razonamiento impreciso: lógica difusa (RA5-b/d)



## 8.1 De lo binario a lo continuo

La lógica proposicional clásica es **binaria**: un enunciado es cierto o falso. Pero los humanos no razonamos así — «hace frío» no es verdadero o falso de forma tajante, es una cuestión de grado. La **lógica difusa** (o borrosa) extiende la lógica proposicional para trabajar con esa incertidumbre:

- Los valores de verdad son **números reales en el intervalo  $[0, 1]$** : 0 es falso, 1 es cierto, 0,5 es «cierto al 50 %».
- La pertenencia de un elemento a un conjunto viene dada por una **función de pertenencia**,  $\mu_A(x)$ : el grado de pertenencia de  $x$  al conjunto  $A$ .
- Conceptos como *húmedo* o *frío* son difíciles de definir con precisión, pero la lógica difusa los define con funciones de pertenencia — lo que facilita crear dispositivos como termostatos: «*si la temperatura es fría, entonces enciende la calefacción*».

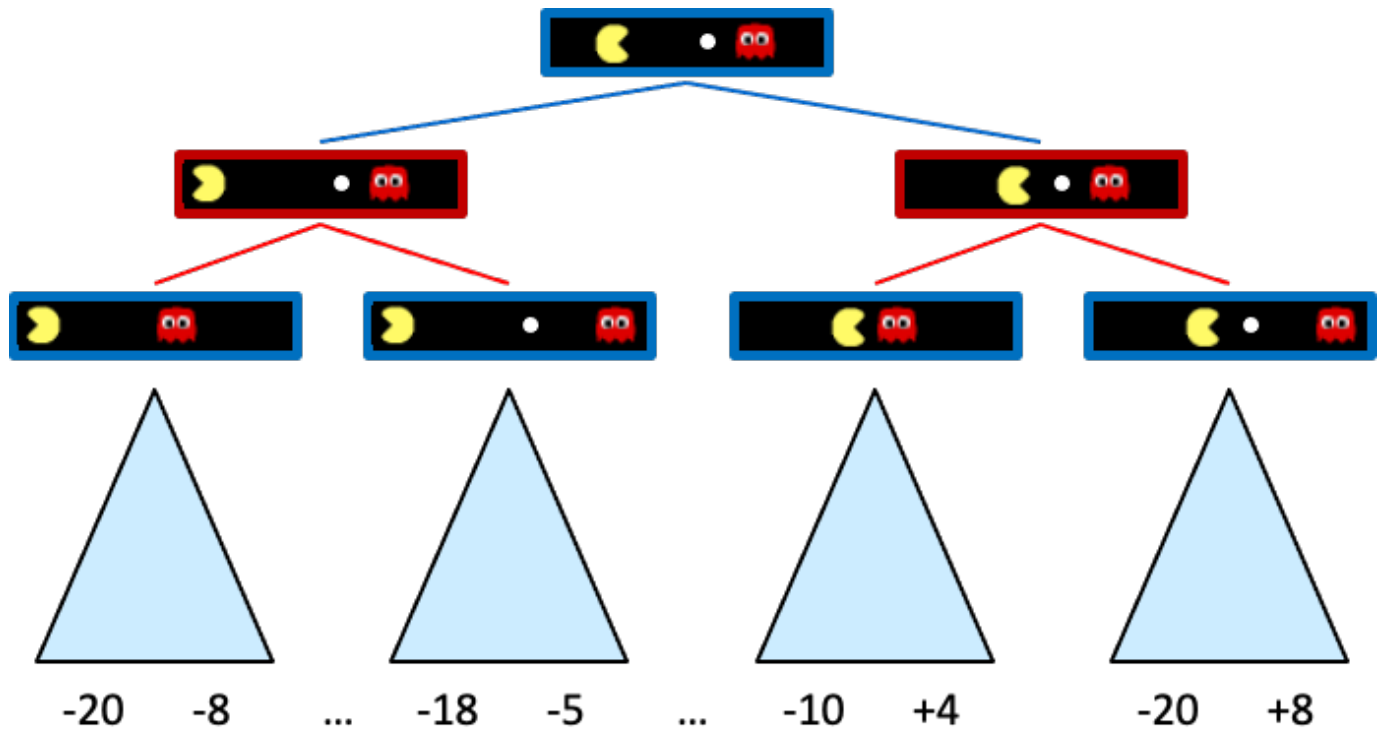
Un **sistema de razonamiento impreciso** es un sistema basado en reglas que usa lógica difusa. Permite trabajar con valores **continuos**, modela mejor el conocimiento humano y es muy apropiado para **sistemas de control** — por eso enlaza directamente con el §10 y el §11.

8.2 Conceptos básicos

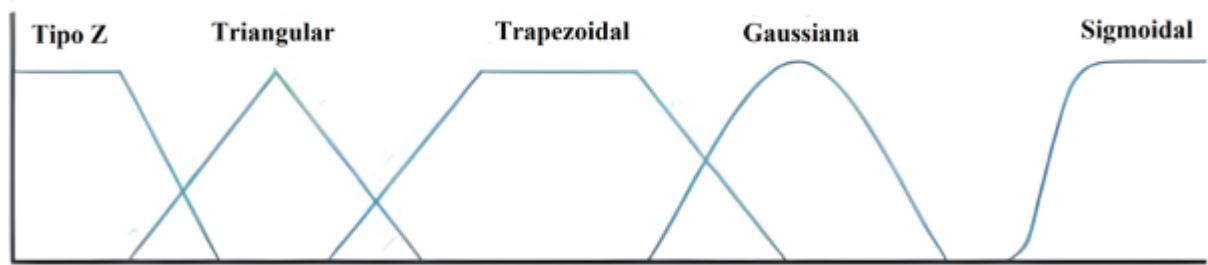
Concepto	Qué es	Ejemplo
Variable lingüística	Variable que toma valores lingüísticos	<i>Temperatura</i>
Valores lingüísticos	Los valores que puede tomar esa variable	<i>Frío, Calor</i>
Función de pertenencia	Asigna a cada valor un grado de pertenencia a un valor lingüístico	$(27^{\circ}\text{C} \rightarrow \text{Calor} = 0{,}8)$
Regla difusa	Regla que usa valores difusos	«Si la temperatura es <b>fría</b> , entonces <b>calefacción alta</b> »
Función de agregación	Combina los valores difusos de varias reglas en una conclusión	$(\text{Calor}=0{,}8, \text{Húmedo}=0{,}7 \rightarrow \text{Sensación desagradable}=0{,}8)$

8.3 El funcionamiento en tres pasos

1. **Fuzzificación**: convierte los datos de entrada precisos en valores difusos, usando las funciones de pertenencia.  $(27^{\circ}\text{C} \rightarrow \text{Calor}=0{,}8, \text{MuchoCalor}=0{,}2)$ .
2. **Evaluación de las reglas**: se aplican las reglas del sistema, combinando las funciones de pertenencia de las variables de entrada para deducir la relevancia de la salida. Por ejemplo: «si la temperatura es **alta** y la humedad es **baja**, la velocidad del ventilador debe ser **alta**».
3. **Desfuzzificación**: convierte la conclusión difusa de vuelta en un valor preciso, con una función de agregación (normalmente **centro de gravedad** o **máximo**).



Las funciones de pertenencia más usadas son **trapezoidales** y **triangulares**; las sinusoidales sirven para representar periodos, y las sigmoidales, probabilidades.



Funciones de pertenencia

#### 8.4 Ejemplo trabajado: la propina del restaurante

Es el ejemplo canónico de `scikit-fuzzy`, y el que verás resuelto paso a paso en el [Taller 2](#).

**Variables de entrada** (funciones triangulares):

- **Calidad del servicio:** baja  $\setminus([0,5])$  · media  $\setminus([0,10])$  · alta  $\setminus([5,10])$
- **Calidad de la comida:** mala  $\setminus([0,5])$  · media  $\setminus([0,10])$  · buena  $\setminus([5,10])$

